

Chương trình đào tạo ABAP 7.7 Hướng tới Dự án LMS - Đại học FPT

Chủ đề dự án: Hệ thống quản lý sinh viên Đại học FPT (LMS)

Mục tiêu: Xây dựng một hệ thống quản lý sinh viên toàn diện, từ quản lý thông tin cơ bản đến các nghiệp vụ phức tạp như đăng ký môn học, điểm danh, vào điểm, báo cáo, và tích hợp các công nghệ hiện đại như GenAI và Fiori.

Đối tượng: Sinh viên ngành Kỹ thuật phần mềm.

Thời lượng: Khoảng 15 buổi. (gồm làm bài tập ở nhà)

Giảng viên: ManhNC5

Buổi 1: Giới thiệu SAP, ABAP và Nền tảng Hệ thống LMS

- **Mục tiêu:** Giúp sinh viên làm quen với môi trường SAP, hiểu về vai trò của ABAP và tổng quan về hệ thống LMS sắp xây dựng.
 - **Nội dung:**
 - Tổng quan về hệ sinh thái SAP và vai trò của ABAP.
 - Cấu trúc cơ bản của một hệ thống SAP (Client, User, Transaction Code, Repository).
 - Giới thiệu về SAP NetWeaver và ABAP Workbench (SE80).
 - Tổng quan về dự án LMS: Các module chức năng chính, mục tiêu kinh doanh.
 - Thiết lập môi trường phát triển ABAP (nếu cần).
 - **Lab 1: Khám phá SAP GUI và SE80**
 - Hướng dẫn truy cập SAP GUI, tạo và chạy một chương trình ABAP đơn giản (ví dụ: WRITE 'Hello, FPT LMS!').
 - Hướng dẫn sử dụng Object Navigator (SE80) để xem các đối tượng chương trình.
 - **Code mẫu ABAP:** Chương trình "Hello World" đơn giản.
 - **Yêu cầu phát triển:** Mở rộng chương trình để hiển thị thêm thông tin về khóa học (tên khóa học, mã khóa học - giả định).
-

Buổi 2: Các Khái niệm Cơ bản trong ABAP (Basic Concepts)

- **Mục tiêu:** Nắm vững các khái niệm lập trình cơ bản trong ABAP, các kiểu dữ liệu, biến, toán tử.
- **Nội dung:**
 - Các kiểu dữ liệu cơ bản (C, N, D, T, I, F, P, STRING, XSTRING).
 - Khai báo biến và hằng số.
 - Toán tử (số học, logic, so sánh).
 - Cấu trúc điều khiển (IF, CASE, DO, WHILE, LOOP).
 - Các kiểu dữ liệu được định nghĩa sẵn (Data Types).
- **Lab 2: Quản lý thông tin Sinh viên cơ bản**
 - Hướng dẫn khai báo các biến để lưu trữ thông tin sinh viên (Mã SV, Tên, Ngày sinh, Giới tính, Email).
 - Thực hiện các phép gán, so sánh và hiển thị thông tin.
 - Sử dụng cấu trúc IF/CASE để kiểm tra điều kiện (ví dụ: kiểm tra độ tuổi sinh viên).
 - **Code mẫu ABAP:** Chương trình nhập và hiển thị thông tin sinh viên cơ bản.
 - **Yêu cầu phát triển:** Thêm chức năng tính tuổi sinh viên dựa trên ngày sinh và hiển thị thông báo "Trưởng thành" hoặc "Chưa trưởng thành" nếu tuổi dưới 18.

Buổi 3: Truy cập Cơ sở dữ liệu (Database Access) - Internal Tables

- **Mục tiêu:** Hiểu cách làm việc với dữ liệu trong ABAP, đặc biệt là Internal Tables, và cách truy vấn dữ liệu từ bảng SAP.
- **Nội dung:**
 - Giới thiệu về Open SQL trong ABAP.
 - Khái niệm và các thao tác cơ bản với Internal Tables (Declaration, Append, Insert, Delete, Modify, Read, Sort, Loop).

- Cú pháp SELECT cơ bản để lấy dữ liệu từ bảng SAP (ví dụ: bảng SSTUDENT, SCourses - giả định).
- Sử dụng INTO TABLE và APPENDING TABLE.

- **Lab 3: Liệt kê Danh sách Sinh viên từ Database**

- Hướng dẫn sử dụng SELECT để lấy danh sách sinh viên từ một bảng dữ liệu giả định trong SAP.
- Lưu trữ dữ liệu vào Internal Table.
- Sử dụng LOOP AT để duyệt qua Internal Table và hiển thị thông tin sinh viên ra màn hình.
- **Code mẫu ABAP:** Chương trình đọc dữ liệu sinh viên từ bảng và hiển thị.
- **Yêu cầu phát triển:** Thêm chức năng lọc sinh viên theo một tiêu chí nhất định (ví dụ: theo mã ngành) và sắp xếp danh sách theo tên.

Buổi 4: Truy cập Cơ sở dữ liệu (Database Access) - Nâng cao

- **Mục tiêu:** Đi sâu vào các kỹ thuật truy vấn dữ liệu phức tạp hơn, xử lý dữ liệu lớn và tối ưu hóa truy vấn.
- **Nội dung:**
 - Các mệnh đề WHERE nâng cao (BETWEEN, LIKE, IN, EXISTS).
 - JOIN các bảng dữ liệu (INNER JOIN, LEFT OUTER JOIN).
 - Sử dụng GROUP BY và các hàm tổng hợp (SUM, COUNT, AVG).
 - Truy vấn dữ liệu với FOR ALL ENTRIES.
 - Sử dụng Field Symbols.
 - Các khái niệm về Performance Tuning cơ bản.
- **Lab 4: Lấy danh sách môn học và hiển thị theo Khóa học**
 - Hướng dẫn JOIN 2 bảng (ví dụ: SCourses và SCourseCatalog) để lấy danh sách môn học thuộc một khóa học cụ thể.

- Sử dụng GROUP BY để đếm số lượng môn học trong mỗi khóa học.
 - **Code mẫu ABAP:** Chương trình JOIN bảng và sử dụng GROUP BY.
 - **Yêu cầu phát triển:** Lấy danh sách sinh viên và môn học họ đã đăng ký, tính tổng số tín chỉ đã đăng ký của mỗi sinh viên.
-

Buổi 5: Modularization - Tái sử dụng Code

- **Mục tiêu:** Hiểu và áp dụng các kỹ thuật Modularization để tổ chức mã nguồn, tăng tính tái sử dụng và dễ bảo trì.
 - **Nội dung:**
 - Subroutines (PERFORM).
 - Functions Modules (RFC và không RFC).
 - Các khái niệm về tham số truyền vào/ra (IMPORTING, EXPORTING, CHANGING, TABLES).
 - Function Groups.
 - **Lab 5: Xây dựng Function Module tính Điểm Trung bình**
 - Hướng dẫn tạo một Function Module nhận vào danh sách điểm của sinh viên (Internal Table) và trả về điểm trung bình.
 - Gọi Function Module này từ một chương trình chính.
 - **Code mẫu ABAP:** Function Module tính điểm TB và chương trình gọi nó.
 - **Yêu cầu phát triển:** Mở rộng Function Module để có thể tính cả điểm trung bình theo môn học và theo học kỳ, yêu cầu thêm các tham số đầu vào phù hợp.
-

Buổi 6: ABAP Objects - Lập trình Hướng đối tượng

- **Mục tiêu:** Làm quen với Lập trình Hướng đối tượng (OOP) trong ABAP, các khái niệm về Class, Object, Inheritance, Polymorphism.

- **Nội dung:**
 - Khái niệm Class, Object, Attribute, Method.
 - Constructor (CONSTRUCTOR).
 - Khai báo và sử dụng Instance và Static Members.
 - Inheritance (Class A INHERITING FROM Class B).
 - Polymorphism và Abstract Classes/Methods.
 - Interfaces.
 - **Lab 6: Mô hình hóa đối tượng Sinh viên và Giảng viên**
 - Hướng dẫn tạo Class ZCL_STUDENT với các thuộc tính (Mã SV, Tên, Email) và Method (ví dụ: display_info).
 - Tạo Class ZCL_LLECTURER tương tự.
 - Sử dụng các đối tượng của các class này.
 - **Code mẫu ABAP:** Định nghĩa Class ZCL_STUDENT và sử dụng đối tượng.
 - **Yêu cầu phát triển:** Tạo một Class cha ZCL_PERSON và cho ZCL_STUDENT, ZCL_LLECTURER kế thừa từ ZCL_PERSON. Triển khai các thuộc tính và phương thức chung.
-

Buổi 7: ABAP Objects - Nâng cao và Design Patterns

- **Mục tiêu:** Nắm vững các kỹ thuật OOP nâng cao và áp dụng các Design Patterns phổ biến trong ABAP.
- **Nội dung:**
 - Concepts: Encapsulation, Abstraction, Inheritance, Polymorphism.
 - Abstract Classes vs. Interfaces.
 - Singleton Pattern, Factory Pattern (sơ lược).

- Error Handling trong ABAP Objects.
- Class Pooling và Global Classes.

- **Lab 7: Quản lý Khóa học bằng ABAP Objects**

- Thiết kế Class ZCL_COURSE với các thuộc tính (Mã Khóa, Tên Khóa, Tín chỉ).
 - Thiết kế Class ZCL_COURSE_MANAGEMENT để quản lý danh sách các khóa học (sử dụng Internal Table trong class).
 - Thực hiện các thao tác Add, Delete, Search Course.
 - **Code mẫu ABAP:** Class quản lý khóa học.
 - **Yêu cầu phát triển:** Áp dụng Singleton Pattern cho ZCL_COURSE_MANAGEMENT để đảm bảo chỉ có một instance duy nhất quản lý danh sách khóa học toàn hệ thống.
-

Buổi 8: Lock Concept & Exceptions

- **Mục tiêu:** Hiểu về cơ chế khóa dữ liệu (Locking) để đảm bảo tính nhất quán dữ liệu khi nhiều người dùng cùng truy cập và cách xử lý lỗi (Exceptions).
- **Nội dung:**
 - Khái niệm Logical Database Locks.
 - Sử dụng Function Modules ENQUEUE_E_<LockObject> và DEQUEUE_E_<LockObject>.
 - Tạo Lock Objects trong SAP (SE11).
 - Các loại Exceptions (Runtime Exceptions, Class-based Exceptions).
 - Sử dụng TRY...CATCH...ENDTRY và RAISE EXCEPTION.
- **Lab 8: Khóa dữ liệu khi đăng ký Môn học**
 - Hướng dẫn tạo Lock Object cho bảng đăng ký môn học (ví dụ: ZDREG_COURSE).

- Thực hiện khóa bản ghi khi một sinh viên bắt đầu đăng ký một môn học.
 - Xử lý trường hợp khóa thất bại và hiển thị thông báo cho người dùng.
 - **Code mẫu ABAP:** Chương trình sử dụng Enqueue/Dequeue và xử lý Exception khi khóa thất bại.
 - **Yêu cầu phát triển:** Mở rộng Logic khóa để cho phép nhiều sinh viên đăng ký cùng một môn học, nhưng khóa bản ghi môn học đó khi có hành động cập nhật thông tin môn học.
-

Buổi 9: Dynamic Programs và ABAP Reporting nâng cao

- **Mục tiêu:** Xây dựng các chương trình ABAP linh hoạt, có khả năng thay đổi hành vi dựa trên input của người dùng hoặc dữ liệu động.
- **Nội dung:**
 - Sử dụng CALL SELECTION-SCREEN DYNAMIC.
 - Truy vấn dữ liệu động (Dynamic SQL: EXEC SQL, cl_sql).
 - Các kỹ thuật tạo báo cáo tương tác (Interactive Reporting).
 - Sử dụng ALV Grid Controls (CL_GUI_ALV_GRID) và ALV Object Model.
- **Lab 9: Báo cáo Điểm thi Học kỳ Tương tác**
 - Xây dựng màn hình Selection Screen động cho phép người dùng chọn năm học, học kỳ, khoa.
 - Sử dụng Dynamic SQL hoặc FOR ALL ENTRIES để lấy dữ liệu điểm thi.
 - Hiển thị kết quả lên ALV Grid.
 - Cấu hình ALV để cho phép người dùng drill-down (xem chi tiết).
 - **Code mẫu ABAP:** Chương trình báo cáo động với ALV Grid.

- **Yêu cầu phát triển:** Thêm chức năng tính toán điểm trung bình chung cho toàn bộ sinh viên trong kết quả báo cáo và cho phép sắp xếp/lọc dữ liệu trực tiếp trên ALV Grid.
-

Buổi 10: HTML5, CSS, JavaScript - Web Fundamentals

- **Mục tiêu:** Cung cấp kiến thức nền tảng về phát triển web để chuẩn bị cho SAPUI5 và Fiori.
 - **Nội dung:**
 - Cấu trúc HTML5 cơ bản (Tags, Elements, Attributes).
 - CSS Styling (Selectors, Properties, Box Model).
 - JavaScript cơ bản (Variables, Data Types, Functions, DOM Manipulation, Event Handling).
 - **Lab 10: Xây dựng giao diện Web đơn giản cho Phiếu Báo Điểm**
 - Tạo một trang HTML đơn giản để hiển thị thông tin Phiếu Báo Điểm (Tên sinh viên, các môn học, điểm, điểm TB).
 - Sử dụng CSS để định dạng giao diện cho đẹp mắt.
 - Sử dụng JavaScript để thêm một nút bấm "In báo cáo" (chỉ hiển thị alert đơn giản).
 - **Code mẫu:** HTML, CSS, JavaScript cho phiếu báo điểm.
 - **Yêu cầu phát triển:** Thêm chức năng tính toán tự động điểm trung bình trên giao diện JavaScript dựa trên điểm nhập thủ công (để mô phỏng dữ liệu).
-

Buổi 11: SAPUI5 - Framework phát triển Frontend hiện đại

- **Mục tiêu:** Giới thiệu và thực hành với SAPUI5, một framework JavaScript mạnh mẽ để xây dựng các ứng dụng doanh nghiệp trên nền tảng web.
- **Nội dung:**

- Kiến trúc MVC (Model-View-Controller) trong SAPUI5.
- Các Controls cơ bản (Text, Input, Button, Table).
- Data Binding (One-Way, Two-Way).
- Views (XML, JSON, JS).
- Controllers và Event Handling.

- **Lab 11: Xây dựng giao diện Đăng ký Môn học bằng SAPUI5**

- Hướng dẫn tạo một ứng dụng SAPUI5 đơn giản.
- Tạo View XML để hiển thị danh sách các môn học có sẵn.
- Tạo Controller để xử lý logic (ví dụ: chọn môn học).
- Sử dụng Data Binding để hiển thị dữ liệu.
- **Code mẫu:** Ứng dụng SAPUI5 cơ bản với danh sách môn học.
- **Yêu cầu phát triển:** Thêm chức năng cho phép sinh viên "chọn" (select) môn học và hiển thị danh sách các môn đã chọn ở một khu vực khác của giao diện.

Buổi 12: OData Services và Kết nối Frontend-Backend

- **Mục tiêu:** Hiểu cách tạo và sử dụng OData Services để kết nối ứng dụng Frontend (SAPUI5) với Backend ABAP.
- **Nội dung:**
 - Khái niệm OData và vai trò của nó.
 - Tạo OData Service trong ABAP (SEGW - Gateway Service Builder).
 - Định nghĩa Entity Types, Entity Sets, Properties, Associations.
 - Implement các Data Provider Class (DPC) cho các nghiệp vụ CRUD (Create, Read, Update, Delete).

- Gọi OData Service từ SAPUI5.
 - **Lab 12: Xây dựng OData Service cho Quản lý Sinh viên**
 - Hướng dẫn sử dụng SEGW để tạo OData Service cho Entity "Student".
 - Implement các phương thức GET_ENTITYSET, GET_ENTITY để lấy danh sách sinh viên và thông tin chi tiết.
 - Gọi OData Service này từ ứng dụng SAPUI5 đã tạo ở buổi trước để hiển thị danh sách sinh viên.
 - **Code mẫu:** OData Service (SEGW) và Client SAPUI5 gọi Service.
 - **Yêu cầu phát triển:** Implement thêm chức năng tạo mới sinh viên thông qua OData Service (POST method) và cập nhật thông tin sinh viên (PUT method).
-

Buổi 13: Fiori - Trải nghiệm Người dùng Hiện đại

- **Mục tiêu:** Hiểu về triết lý Fiori và cách xây dựng các ứng dụng Fiori.
- **Nội dung:**
 - Nguyên tắc thiết kế Fiori.
 - Các loại ứng dụng Fiori (Transactional, Analytical, Factsheet).
 - Giới thiệu về Fiori Launchpad.
 - Sử dụng SAP Business Application Studio (BAS) hoặc Web IDE để phát triển Fiori Apps.
 - Các Fiori Elements (mô hình hóa ứng dụng từ OData).
- **Lab 13: Xây dựng Fiori App cho Quản lý Môn học**
 - Hướng dẫn tạo một ứng dụng Fiori (ví dụ: List Report hoặc Object Page) dựa trên OData Service quản lý môn học (đã có hoặc tạo mới).
 - Cấu hình Fiori Elements để tự động sinh giao diện.

- Triển khai ứng dụng lên Fiori Launchpad (mô phỏng).
 - **Code mẫu:** Dự án Fiori Elements sử dụng OData Service.
 - **Yêu cầu phát triển:** Tùy chỉnh giao diện Fiori App (ví dụ: thêm nút hành động tùy chỉnh, thay đổi layout) để phù hợp hơn với nghiệp vụ quản lý môn học.
-

Buổi 14: Analytical Apps Development & Email Notifications

- **Mục tiêu:** Xây dựng các ứng dụng phân tích dữ liệu và tích hợp chức năng gửi email thông báo.
- **Nội dung:**
 - Các khái niệm về BI (Business Intelligence) và Analytics trong SAP.
 - Sử dụng các Controls phân tích của SAPUI5 (Chart, Table).
 - Tích hợp Email trong ABAP:
 - Sử dụng Transaction SCOT để cấu hình Email Server.
 - Function Module SO_NEW_DOCUMENT_ATT_SEND_API1 hoặc CL_BCS class.
 - Thiết kế luồng gửi email tự động dựa trên các sự kiện trong hệ thống LMS.
- **Lab 14: Báo cáo Phân tích Tình hình Tuyển sinh và Gửi Email Thông báo**
 - Xây dựng một ứng dụng Fiori/SAPUI5 hiển thị biểu đồ (ví dụ: số lượng sinh viên nhập học theo từng kỳ).
 - Thiết lập chức năng gửi email tự động thông báo cho bộ phận tuyển sinh khi có một lượng lớn sinh viên đăng ký thành công trong một ngày.
 - **Code mẫu ABAP:** Chương trình gửi email và cấu trúc dữ liệu cho biểu đồ.
 - **Yêu cầu phát triển:** Cho phép người dùng cấu hình nội dung email gửi đi (ví dụ: thêm các trường thông tin động từ hệ thống) và thiết lập lịch gửi email.

Buổi 15: Upload File và Kết nối GenAI, Tổng kết Dự án

- **Mục tiêu:** Học cách upload/download file trong ABAP và Fiori, giới thiệu về tích hợp GenAI, và tổng kết toàn bộ kiến thức dự án.
- **Nội dung:**
 - Upload/Download file trong ABAP (Sử dụng CL_GUI_FRONTEND_SERVICES).
 - Upload/Download file trong Fiori/SAPUI5.
 - Giới thiệu về API Integration và cách gọi các dịch vụ bên ngoài.
 - Khái niệm cơ bản về Generative AI và các ứng dụng tiềm năng trong LMS (ví dụ: tóm tắt tài liệu, tạo câu hỏi trắc nghiệm).
 - Tổng kết kiến thức đã học, các best practices.
 - Định hướng phát triển tiếp theo.
- **Lab 15: Upload Danh sách Sinh viên và Kết nối GenAI (Demo)**
 - Xây dựng chức năng cho phép upload file Excel chứa danh sách sinh viên mới.
 - *Demo:* Tích hợp với một API GenAI (ví dụ: sử dụng CURL hoặc một thư viện Python đơn giản được gọi từ ABAP) để thử nghiệm việc tóm tắt nội dung một bài giảng hoặc tạo câu hỏi trắc nghiệm dựa trên syllabus môn học.
 - **Code mẫu ABAP:** Chương trình upload file và gọi API demo GenAI.
 - **Yêu cầu phát triển:** Phát triển thêm chức năng xử lý dữ liệu từ file upload (ví dụ: validate, insert vào database). Cho sinh viên ý tưởng về cách áp dụng GenAI vào các bài toán cụ thể của LMS (ví dụ: gợi ý môn học, phân tích xu hướng học tập).

Hướng dẫn cho Bài Lab:

Mỗi bài Lab sẽ đi theo cấu trúc sau:

1. **Mục tiêu của Lab:** Nêu rõ sinh viên sẽ học được gì và đạt được gì sau bài lab.

2. **Mô tả Bài toán:** Giải thích tình huống nghiệp vụ cụ thể mà bài lab sẽ giải quyết trong dự án LMS.
3. **Các Bước Thực hiện (Step-by-Step):**
 - Hướng dẫn chi tiết từng thao tác trên SAP GUI hoặc các công cụ phát triển khác.
 - Kèm theo các đoạn mã ABAP (hoặc SAPUI5/OData) mẫu, được giải thích rõ ràng.
 - Minh họa bằng hình ảnh (nếu có thể).
4. **Code Mẫu ABAP:** Cung cấp toàn bộ đoạn mã nguồn cho bài lab cơ bản.
5. **Yêu cầu Phát triển Thêm:** Đưa ra các yêu cầu mở rộng bài lab theo các hướng khác nhau, khuyến khích sinh viên tự tìm tòi và áp dụng kiến thức đã học.
 - Ví dụ: "Mở rộng chương trình để xử lý thêm trường hợp ngoại lệ...", "Tối ưu hóa truy vấn dữ liệu...", "Thêm tính năng X vào giao diện..."
6. **Gợi ý Giải pháp/Lời giải:** Cung cấp gợi ý hoặc đoạn mã giải pháp cho các yêu cầu phát triển thêm, giúp sinh viên kiểm tra và học hỏi.

Với lộ trình này, sinh viên sẽ được trang bị đầy đủ kiến thức từ cơ bản đến nâng cao, có khả năng xây dựng và phát triển các ứng dụng trên nền tảng SAP S/4HANA, đặc biệt là trong bối cảnh các ứng dụng doanh nghiệp hiện đại với Fiori và các công nghệ mới. Chúc bạn thành công với chương trình đào tạo này!

BUỔI 1: KHỞI TẠO DỰ ÁN LMS & LÀM QUEN MÔI TRƯỜNG ABAP

A. THÔNG TIN QUẢN TRỊ LỚP HỌC (SE1876)

Trước khi bắt đầu, chúng ta xác nhận lại định danh để đảm bảo tính hệ thống cho cả 15 buổi học:

1. Package (Gói chứa code):

- Cấu trúc: ZSPR26_SE1876_[AccountID]
- Ví dụ Giảng viên (DEV006): ZSPR26_SE1876_DEV006
- Ví dụ Sinh viên (DEV331): ZSPR26_SE1876_DEV331
- *Lưu ý: Mọi đối tượng (Table, Program, Class) tạo ra trong khóa học đều phải lưu vào Package này.*

2. Program (Chương trình chạy):

- Cấu trúc: Z[AccountID]_LMS_INTRO
- Ví dụ SV: ZDEV331_LMS_INTRO

3. Tài khoản:

- Giảng viên: DEV006
- Sinh viên: DEV331 - DEV354

B. MỤC TIÊU BÀI HỌC

1. **Thiết lập môi trường:** Tạo thành công Package cá nhân để quản lý mã nguồn dự án LMS.
2. **Tư duy lập trình ABAP:** Hiểu cấu trúc REPORT, sự kiện kích hoạt (Activate), và hệ thống vận chuyển (Transport Request).
3. **Code & Run:** Viết chương trình đầu tiên hiển thị thông tin khởi động hệ thống LMS FPT.

C. NỘI DUNG THỰC HÀNH (LAB 1): KHỞI ĐỘNG HỆ THỐNG LMS

Tình huống: Bạn là lập trình viên backend. Nhiệm vụ đầu tiên của bạn là tạo một chương trình "Dashboard" đơn giản để hiển thị trạng thái hoạt động của hệ thống Quản lý sinh viên (LMS) và thông tin người đang vận hành.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Đăng nhập & Truy cập Object Navigator

- Đăng nhập vào SAP GUI bằng tài khoản được cấp (ví dụ DEV331).
- Tại ô lệnh (Command field), gõ T-Code: SE80 và nhấn Enter.

Bước 2: Tạo Package cá nhân (Quan trọng nhất)

- Ở danh sách thả xuống bên trái, chọn **Package**.
- Gõ tên Package theo quy tắc: ZSPR26_SE1876_DEV331 (Thay DEV331 bằng ID của bạn). Nhấn Enter.
- Hệ thống hỏi "Package ... does not exist. Create object?", chọn **Yes**.
- Cửa sổ *Create Package* hiện ra:
 - **Short Description:** Package cho dự án LMS - SE1876 - [Tên bạn].
 - **Software Component:** HOME (hoặc LOCAL).
 - Nhấn dấu tích xanh (Continue).
- Cửa sổ *Transport Request*: Chọn "Create Request" (tờ giấy trắng), nhập mô tả "SE1876 - LMS Project Setup", nhấn Save (đĩa mềm).

Bước 3: Tạo Program đầu tiên

- Chuột phải vào tên Package vừa tạo -> **Create -> Program**.
- Tên chương trình: ZDEV331_LMS_INTRO (Thay DEV331 bằng ID của bạn).
- Bỏ tích ô "With TOP Include". Nhấn **Save**.

- Lưu vào Transport Request vừa tạo ở bước 2.

Bước 4: Viết Code ABAP

- Màn hình soạn thảo hiện ra. Hãy nhập đoạn code mẫu bên dưới.

Bước 5: Kiểm tra & Kích hoạt

- Nhấn **Check** (Ctrl + F2) để kiểm tra lỗi cú pháp.
- Nhấn **Activate** (Ctrl + F3 - Hình que diêm) để biên dịch code.
- Nhấn **Direct Processing** (F8) để chạy chương trình.

2. Code Mẫu ABAP (Basic)

```
code Abap
*&-----*
*& Report ZDEV331 LMS INTRO
*&-----*
*& Mô tả: Màn hình chào mừng và kiểm tra trạng thái LMS Project
*& Người tạo: DEV331 (Sinh viên SE1876)
*&-----*
REPORT zdev331_lms_intro.

" 1. Hiển thị tiêu đề dự án
WRITE: 'HỆ THỐNG QUẢN LÝ SINH VIÊN - FPT UNIVERSITY (LMS)'.
WRITE: / '-----'.

" 2. Hiển thị thông tin module
WRITE: / 'Module hiện tại: Core System'.
WRITE: / 'Phiên bản: 1.0 (Beta)'.

" 3. Trạng thái kết nối (Giả lập)
WRITE: / 'Kết nối Database: OK'.
```

WRITE: / 'Kết nối GenAI: Đang chờ cấu hình...!'

3. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên cần sửa đổi code mẫu trên để hoàn thành 4 yêu cầu sau nhằm hiểu sâu về cú pháp và biến hệ thống:

Thử thách 1: Formatting (Định dạng giao diện)

- **Yêu cầu:**
 - Dòng tiêu đề "HỆ THỐNG..." phải được tô màu Xanh Dương (Color 1).
 - Dòng "Trạng thái kết nối" nếu OK thì tô màu Xanh Lá (Color 5).
 - Dùng lệnh ULINE để thay thế cho dòng gạch ngang thủ công '----'.
- **Mục tiêu:** Làm quen với từ khóa định dạng hiển thị.

Thử thách 2: System Variables (Biến hệ thống)

- **Yêu cầu:** Thay vì viết code cứng, hãy hiển thị thông tin thực tế của phiên làm việc:
 - "Người vận hành": Hiển thị tài khoản đăng nhập (dùng sy-username).
 - "Ngày hệ thống": Hiển thị ngày hiện tại (dùng sy-datum).
 - "Mã Transaction": Hiển thị mã T-code đang chạy (dùng sy-tcode).
- **Mục tiêu:** Hiểu cách SAP quản lý phiên làm việc (Session).

Thử thách 3: Data Declaration (Khai báo biến)

- **Yêu cầu:**
 - Khai báo một biến gv_campus kiểu chuỗi (String) để lưu tên cơ sở (Ví dụ: 'FPT Hoa Lac').
 - Khai báo một biến gv_semester kiểu số nguyên (Integer) để lưu học kỳ (Ví dụ: 2024).
 - Gán giá trị cho biến và in ra màn hình: "Cơ sở: FPT Hoa Lac - Học kỳ: 2024".

- **Mục tiêu:** Làm quen với từ khóa DATA và phép gán.

Thử thách 4: Chained Statements (Chuỗi lệnh)

- **Yêu cầu:** Viết lại toàn bộ phần hiển thị thông tin (các lệnh WRITE) sử dụng dấu hai chấm : và dấu phẩy , để code ngắn gọn hơn.
- **Mục tiêu:** Tối ưu hóa cú pháp code ABAP.

4. Lời giải tham khảo (Dành cho Giảng viên Review)

Đây là code mong đợi sau khi sinh viên hoàn thành 4 thử thách:

code Abap

```
REPORT zdev331_lms_intro.
```

```
*--- THỬ THÁCH 3: KHAI BÁO BIẾN ---*
```

```
DATA: gv_campus TYPE string,  
      gv_semester TYPE i.
```

```
" Gán dữ liệu
```

```
gv_campus = 'FPT University - Hoa Lac Campus'.  
gv_semester = 2025.
```

```
*--- THỬ THÁCH 4: CHAINED STATEMENTS ---*
```

```
" Sử dụng : để bắt đầu chuỗi lệnh
```

```
WRITE: 'HỆ THỐNG QUẢN LÝ SINH VIÊN (LMS)' COLOR 1, "Thử thách 1: Màu sắc  
      / sy-uline.                "Thử thách 1: ULINE
```

```
*--- THỬ THÁCH 2: BIẾN HỆ THỐNG ---*
```

```
WRITE: / 'Người vận hành (User):', sy-username COLOR 4, "Màu xanh da trời  
      / 'Ngày hệ thống:', sy-datum,  
      / 'Giao dịch (T-Code):', sy-tcode.
```

SKIP 1. "Cách 1 dòng

```
WRITE: / 'Thông tin môi trường:',  
      / 'Cơ sở:', gv_campus,  
      / 'Học kỳ:', gv_semester.
```

SKIP 1.

```
WRITE: / 'Trạng thái hệ thống:',  
      / 'Database:', 'Connected' COLOR 5,    "Thử thách 1: Màu xanh lá  
      / 'GenAI:',   'Disconnected' COLOR 6.  "Màu đỏ
```

PHẦN 1: CÁC BƯỚC CHUẨN BỊ (PRE-REQUISITES)

Trước khi cài đặt, đảm bảo máy tính của sinh viên đã có đủ 2 thứ sau:

1. **Java Development Kit (JDK):** Eclipse cần Java để chạy.
 - **Phiên bản:** Nên dùng JDK 17 (LTS- Long Term Support) hoặc mới hơn.
 - **Cách cài:** Tải về từ trang chủ Oracle hoặc OpenJDK, cài đặt như phần mềm bình thường.
2. **Eclipse IDE:**
 - **Phiên bản:** Tải bản mới nhất từ trang chủ <https://www.eclipse.org/downloads/>.
 - **Gói cài đặt:** Chọn gói "Eclipse IDE for Java Developers" là đủ và nhẹ nhất.
 - **Cài đặt:** Chạy file cài đặt, chọn thư mục cài đặt và thư mục **Workspace** (nơi lưu trữ project).

PHẦN 2: CÀI ĐẶT PLUGIN ADT (CÀI ABAP VÀO ECLIPSE)

Đây là bước quan trọng nhất. Chúng ta sẽ cài đặt "bộ não" ABAP vào Eclipse.

Bước 1: Mở Eclipse.

Bước 2: Vào menu **Help** → **Install New Software...**

Bước 3: Tại cửa sổ "Install", nhấn nút **Add...** để thêm một Repository mới.

Bước 4: Điền thông tin Repository:

- **Name:** SAP ADT (Tên gợi nhớ).
- **Location:** <https://tools.hana.ondemand.com/latest>

- Nhấn **Add**.

Bước 5: Chờ một lát, danh sách các công cụ sẽ hiện ra. Hãy tick chọn vào mục:

- **ABAP Development Tools**

- *Lưu ý:* Sinh viên không cần cài hết (như HANA Tools, UI5 Tools) nếu chỉ tập trung vào ABAP Backend, việc này giúp cài đặt nhanh hơn.

Bước 6: Nhấn **Next**.

Bước 7: Eclipse sẽ kiểm tra các dependency. Nhấn **Next** lần nữa.

Bước 8: Chấp nhận điều khoản bản quyền (Accept the license agreements) và nhấn **Finish**.

Bước 9: Eclipse sẽ tải và cài đặt plugin. Quá trình này có thể mất vài phút.

Bước 10: Khi cài xong, Eclipse sẽ yêu cầu **Restart Now**. Hãy nhấn vào đó để khởi động lại.

PHẦN 3: CẤU HÌNH KẾT NỐI ĐẾN SAP SERVER (TẠO ABAP PROJECT)

Sau khi khởi động lại, chúng ta sẽ kết nối Eclipse với Server SAP của trường.

Bước 1: Mở "ABAP Perspective". Đây là giao diện làm việc chuyên dụng cho ABAP.

- Vào menu **Window** → **Perspective** → **Open Perspective** → **Other...**
- Chọn **ABAP** và nhấn **Open**.

Bước 2: Tạo một Project mới.

- Vào menu **File** → **New** → **ABAP Project**. (Hoặc click chuột phải vào vùng trống của cửa sổ **Project Explorer** → **New** → **ABAP Project**).

- **Bước 3:** Hệ thống sẽ quét các kết nối có sẵn trong SAP Logon.

- Nếu sinh viên đã có SAP Logon và đã cấu hình kết nối, danh sách sẽ hiện ra. Chỉ cần chọn đúng server và nhấn **Next**.

- Nếu không, chọn "**Add System Connection...**" để tạo thủ công.
- **Bước 4:** Điền thông tin kết nối (Nếu tạo thủ công):
 - Application Server: Địa chỉ IP hoặc tên server (VD: sap.university.edu).
 - Instance Number: 2 số cuối (VD: 00).
 - System ID (SID): 3 ký tự (VD: S4H).

(Thông tin này giảng viên cần cung cấp cho sinh viên).

- **Bước 5:** Nhấn **Next**.
- **Bước 6:** Điền thông tin đăng nhập:
 - **Client:** (VD: 100).
 - **User:** Tên tài khoản SAP.
 - **Password:** Mật khẩu.
 - **Language:** EN (Tiếng Anh).
- **Bước 7:** Nhấn **Finish**.

Kết quả:

Nếu thành công, trong cửa sổ **Project Explorer** sẽ xuất hiện một Project mới với tên là [SID]_[Client]_[User].

PHẦN 4: CÁC TIPS TỐI ƯU HÓA MÔI TRƯỜNG LÀM VIỆC

Để sinh viên làm việc hiệu quả hơn, hãy hướng dẫn các em một vài "bí kíp":

1. Thêm Package vào Favorites:

- Trong Project Explorer, mở cây thư mục **System Library**.
- Tìm package của lớp học (ZSPR26_SE1876_SAP).
- Chuột phải → **Add to Favorite Packages**. Lần sau chỉ cần vào mục **Favorite Packages** là thấy ngay.

2. Bật Dark Theme (Chế độ tối):

- Vào **Window** → **Preferences**. Tìm **General** → **Appearance**.
- Theme: Chọn **Dark**.
- Khởi động lại Eclipse.

3. Học thuộc các phím tắt vàng:

- **Ctrl + Space**: Gợi ý code (Content Assist)- Phím quan trọng nhất.
- **Ctrl + 1**: Sửa lỗi nhanh (Quick Fix).
- **Ctrl + F3**: Kích hoạt đối tượng (Activate).
- **F8**: Chạy Data Preview cho CDS View.
- **F2**: Xem thông tin chi tiết (Element Info).

PHẦN 5: XỬ LÝ SỰ CỐ THƯỜNG GẶP (TROUBLESHOOTING)

- Lỗi "Cannot connect to system...":
 - Kiểm tra lại thông tin Application Server, Instance Number.
 - Kiểm tra tường lửa (Firewall) hoặc VPN của trường.

- Đảm bảo dịch vụ ADT đã được kích hoạt trên Server (T-Code SICF, service /sap/bc/adt).
- Eclipse chạy chậm:
 - Tăng RAM cho Eclipse bằng cách sửa file eclipse.ini (thường nằm trong thư mục cài đặt Eclipse). Tìm dòng-Xmx và tăng lên-Xmx2048m (2GB) hoặc cao hơn.

Chúc bạn và lớp học SE1876 có những giờ thực hành hiệu quả trên môi trường Eclipse ADT chuyên nghiệp

QUY CHUẨN TÊN KỸ THUẬT (TECHNICAL NAME)

Trước khi bắt đầu, chúng ta cần chuẩn hóa tên để hệ thống SAP chấp nhận (không dùng dấu gạch ngang -):

- Tên lớp học: SPR26-SE1876-SAP
- **Tên Package (Chuyển đổi)ZSE1876_SP26**

PHẦN 1: TẠO PACKAGE CHA (DÀNH CHO GIẢNG VIÊN)

Chúng ta sẽ sử dụng T-Code **SE80** (Object Navigator). Đây là "đại bản doanh" của ABAPer.

Bước 1: Truy cập SE80

1. Đăng nhập vào SAP GUI.
2. Tại ô nhập lệnh (Command field) ở góc trên bên trái, gõ: SE80.
3. Nhấn **Enter**.

Bước 2: Bắt đầu tạo Package

1. Tại giao diện SE80, nhìn cột bên trái (Navigation Area).

2. Tìm menu xổ xuống (Dropdown list) đang hiển thị (thường là *Program* hoặc *Class/Interface*), hãy đổi chọn thành **Package**.
3. Tại ô nhập tên ngay bên dưới, gõ tên package chaZSE1876_SP26.
4. Nhấn **Enter** (hoặc bấm biểu tượng cái kính mắt kính  - Display).
5. Hệ thống sẽ hiện thông báo ở thanh trạng thái (hoặc popup): "*Package ZSPR26_SE1876_SAP does not exist. Do you want to create it?*" (Package chưa tồn tại, bạn có muốn tạo không?).
6. Nhấn **Yes** (Hoặc nút Create).

Bước 3: Điền thông tin thuộc tính Package (Quan trọng)

Một cửa sổ popup **Create Package** hiện ra. Bạn điền chính xác như sau:

- **Short Description:** Superpackage for Class SPR26-SE1876-SAP (Mô tả rõ ràng để sinh viên nhận diện).
- **Application Component:** Nhập HOME hoặc CA (Cross Application).
 - *Giải thích:* Đây là phân loại module nghiệp vụ. Với lớp học, dùng HOME là an toàn nhất.
- **Software Component:** Nhập HOME.
 - *Lưu ý:* Nếu trường dùng server sandbox riêng biệt hoàn toàn thì dùng LOCAL. Nhưng HOME thường được khuyến dùng để cho phép tạo Transport Request.
- **Package Type:** (Nếu hệ thống yêu cầu) Chọn **Structure Package** hoặc để mặc định (Standard).
 - *Mẹo chuyên gia:* Với lớp học, bạn cứ để mặc định. Chỉ cần các package con khai báo package này là "Cha" là được.

Nhấn dấu tích xanh () hoặc phím **Enter** để tiếp tục.

Bước 4: Tạo Transport Request (TR)

Hệ thống sẽ bật cửa sổ hỏi **Transport Request**. Đây là bước để "đóng gói" việc tạo package này.

1. Tại ô **Request**, đừng chọn cái cũ. Hãy tạo mới để quản lý riêng cho lớp này.

2. Nhấn vào biểu tượng tờ giấy trắng ( - **Create Request**) hoặc nhấn phím tắt **F8**.
3. Cửa sổ nhập thông tin TR hiện ra:
 - **Short Description:** Init Package for Class SPR26-SE1876-SAP
 - Các thông tin khác (Project, Target) để mặc định.
4. Nhấn nút **Save** (Biểu tượng đĩa mềm ).
5. Hệ thống sẽ cấp một mã số, ví dụ: DEVK900123.
6. Nhấn dấu tích xanh () để hoàn tất.

Kết quả: Bạn đã tạo xong "ngôi nhà chung" ZSPR26_SE1876_SAP. Bên trái màn hình SE80 sẽ hiển thị cây thư mục của package này.

BUỔI 2: CÁC KHÁI NIỆM CƠ BẢN & XỬ LÝ LOGIC NGHIỆP VỤ

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Program Name:** Z[AccountID]_LAB2_GRADING (Ví dụ: ZDEV331_LAB2_GRADING)
- **Chủ đề:** Hệ thống tính điểm trung bình và xếp loại sinh viên (GPA Calculation).

B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Phần này đúc kết từ các dự án SAP tỷ đô, sinh viên cần ghi nhớ kỹ:

1. Quy tắc "Vàng" về Kiểu dữ liệu số:

-  **KHÔNG BAO GIỜ** dùng kiểu F (Float) để tính toán Tiền tệ hoặc Điểm số chính xác. Kiểu F gây sai số làm tròn (rounding error) rất nhỏ nhưng nguy hiểm.
-  **LUÔN DÙNG** kiểu P (Packed Number) cho tiền tệ, số lượng, điểm số. Ví dụ: TYPE P DECIMALS 2.

2. Ác mộng "Magic Numbers" (Con số ma thuật):

-  Code tệ: IF gv_score > 5. (Số 5 ở đây là magic number. Nếu năm sau trường đổi điểm đạt là 4.0 thì phải đi sửa code ở 100 chỗ).
-  Code chuẩn: Khai báo hằng số CONSTANTS: gc_pass_score TYPE p VALUE '5.0'. và dùng IF gv_score > gc_pass_score.

3. Clear biến trước khi dùng:

- Trong các vòng lặp hoặc xử lý phức tạp, biến cũ có thể còn lưu giá trị rác. Thói quen tốt là dùng lệnh CLEAR variable_name nếu cần thiết (dù ABAP hiện đại đã tự quản lý khá tốt, nhưng cẩn tắc vô áy náy).

4. Pretty Printer là bạn thân:

- Đừng viết code thụt thò lung tung. Hãy dùng nút **Pretty Printer** (Shift+F1) trên thanh công cụ để code tự động căn lề chuẩn SAP.

C. MỤC TIÊU BÀI HỌC

1. **Input:** Sử dụng PARAMETERS để nhận dữ liệu động từ người dùng.
2. **Logic:** Áp dụng cấu trúc điều kiện IF...ELSEIF...ELSE và CASE.
3. **Validation:** Kiểm tra dữ liệu đầu vào (Data Integrity).

D. NỘI DUNG THỰC HÀNH (LAB 2): TÍNH ĐIỂM & XẾP LOẠI

Tình huống: Giảng viên cần một công cụ nhanh để nhập điểm thành phần (Assignment, Midterm, Final) và hệ thống tự động trả về Điểm Tổng kết (GPA) cùng Xếp loại (Rank) theo quy chế đào tạo.

1. Hướng dẫn chi tiết (Step-by-Step)

- **Bước 1:** Vào SE80, tạo chương trình ZDEV331_LAB2_GRADING.
- **Bước 2:** Khai báo Parameters (Input).
 - Lưu ý đặt tên biến bắt đầu bằng pa_.
- **Bước 3:** Khai báo Variables (Biến nội bộ).
 - Lưu ý đặt tên biến bắt đầu bằng gv_.
- **Bước 4:** Viết Logic tính toán & Xếp loại.
- **Bước 5:** Xử lý hiển thị kết quả.

2. Code Mẫu ABAP (Chuẩn Convention)

```
code Abap
downloadcontent_copy
expand_less
*&-----*
*& Report ZDEV331_LAB2_GRADING
*&-----*
REPORT zdev331_lab2_grading.

*--- 1. INPUT PARAMETERS (Đầu vào) ---*
" OBLIGATORY: Bắt buộc phải nhập, không được để trống
PARAMETERS: pa_sid  TYPE c LENGTH 8 OBLIGATORY,  "Student ID
              pa_name TYPE string,                "Student Name
              pa_ass  TYPE p DECIMALS 2 OBLIGATORY, "Assignment Score
```

```
pa_mid TYPE p DECIMALS 2 OBLIGATORY, "Midterm Score
pa_final TYPE p DECIMALS 2 OBLIGATORY. "Final Score
```

--- 2. DATA DECLARATION (Khai báo biến) ---

```
DATA: gv_gpa TYPE p DECIMALS 2, "Biên chứa điểm trung bình
gv_rank TYPE string. "Biên chứa xếp loại
```

--- 3. CONSTANTS (Hằng số - Best Practice) ---

```
CONSTANTS: gc_weight_ass TYPE p DECIMALS 2 VALUE '0.2', "20%
gc_weight_mid TYPE p DECIMALS 2 VALUE '0.3', "30%
gc_weight_final TYPE p DECIMALS 2 VALUE '0.5'. "50%
```

--- 4. MAIN LOGIC (Xử lý chính) ---

" Tính GPA theo trọng số

```
gv_gpa = ( pa_ass * gc_weight_ass ) +
( pa_mid * gc_weight_mid ) +
( pa_final * gc_weight_final ).
```

" Xếp loại (Business Logic)

```
IF gv_gpa >= 9.
gv_rank = 'Xuất sắc (Excellent)'.
ELSEIF gv_gpa >= 8.
gv_rank = 'Giỏi (Good)'.
ELSEIF gv_gpa >= 5.
gv_rank = 'Đạt (Pass)'.
ELSE.
gv_rank = 'Học lại (Retake)'.
ENDIF.
```

--- 5. OUTPUT (Hiển thị) ---

```
WRITE: / '--- KẾT QUẢ HỌC TẬP ---' COLOR 4.
```

```
WRITE: / 'Sinh viên:', pa_name, '(', pa_sid, ')'.  
WRITE: / 'Điểm thành phần:', pa_ass, '|', pa_mid, '|', pa_final.  
ULINE. "Kẻ đường ngang  
WRITE: / 'GPA TỔNG KẾT:', gv_gpa COLOR 5. "Màu xanh lá  
WRITE: / 'XẾP LOẠI:', gv_rank COLOR 6. "Màu đỏ
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên thực hiện 4 yêu cầu sau để nâng cấp chương trình từ "chạy được" thành "chạy tốt".

Thử thách 1: Data Validation (Quan trọng nhất)

- **Yêu cầu:** Nếu người dùng nhập điểm < 0 hoặc > 10 , hãy dừng chương trình ngay lập tức và báo lỗi.
- **Gợi ý:** Sử dụng IF ... OR ... kiểm tra đầu vào ngay sau phần START-OF-SELECTION.
- **Real-world Note:** Trong dự án thật, nếu không validate, dữ liệu rác sẽ làm hỏng cả hệ thống báo cáo sau này.

Thử thách 2: Text Elements (Thay vì Hardcode Text)

- **Yêu cầu:** Thay thế các chuỗi văn bản cứng như 'Xuất sắc', 'Giỏi' bằng Text Elements.
- **Gợi ý:** Vào menu *Goto -> Text Elements -> Text Symbols*. Tạo ký hiệu 001 là 'Xuất sắc', 002 là 'Giỏi'. Trong code dùng TEXT-001, TEXT-002.
- **Real-world Note:** Đây là bắt buộc để làm **Đa ngôn ngữ (Translation)**. Khách hàng Đức muốn thấy tiếng Đức, khách hàng Việt muốn thấy tiếng Việt.

Thử thách 3: Logic điểm liệt (Business Rule)

- **Yêu cầu:** Kể cả khi $GPA > 5.0$, nhưng nếu điểm thi cuối kỳ (pa_final) < 4.0 , sinh viên vẫn bị coi là "Trượt (Failed by Rule)".
- **Gợi ý:** Sử dụng lồng ghép IF hoặc toán tử AND.

Thử thách 4: Selection Screen Design (Giao diện)

- **Yêu cầu:** Đóng khung các ô nhập liệu điểm số vào một nhóm có tiêu đề là "Academic Scores".

- **Gợi ý:** Sử dụng SELECTION-SCREEN BEGIN OF BLOCK....

4. Lời giải tham khảo cho Thử thách (Snippet)

code Abap

--- Giải pháp Thử thách 4: Đóng khung giao diện ---

```
SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE text-003. "text-003 = 'Nhập điểm'
  PARAMETERS: pa_ass TYPE p DECIMALS 2 OBLIGATORY,
              pa_mid TYPE p DECIMALS 2 OBLIGATORY,
              pa_final TYPE p DECIMALS 2 OBLIGATORY.
SELECTION-SCREEN END OF BLOCK b1.
```

--- Giải pháp Thử thách 1: Validate đầu vào ---

```
IF pa_ass < 0 OR pa_ass > 10 OR pa_mid < 0 OR pa_mid > 10 OR pa_final < 0 OR pa_final > 10.
  MESSAGE 'Dữ liệu điểm không hợp lệ (Phải từ 0-10)!' TYPE 'E'.
  " TYPE 'E' sẽ dừng chương trình và hiển thị màu đỏ
ENDIF.
```

--- Giải pháp Thử thách 3: Logic điểm liệt ---

```
IF gv_gpa >= 5.
  IF pa_final < 4. "Đính điểm liệt
    gv_rank = 'Trượt do điểm liệt (Failed by Final score)'.
  ELSE.
    gv_rank = 'Đạt (Pass)'.
  ENDIF.
ELSE.
  gv_rank = 'Học lại (Retake)'.
ENDIF.
```

Kết thúc Buổi 2.

Sinh viên đã học được cách viết code có cấu trúc, xử lý lỗi (Validation) và tư duy tránh "Hardcode".

BUỔI 2: CÁC KHÁI NIỆM CƠ BẢN & XỬ LÝ LOGIC NGHIỆP VỤ

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Program Name:** Z[AccountID]_LAB2_GRADING (Ví dụ: ZDEV331_LAB2_GRADING)
- **Chủ đề:** Hệ thống tính điểm trung bình và xếp loại sinh viên (GPA Calculation).

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Phần này đúc kết từ các dự án SAP tỷ đô, sinh viên cần ghi nhớ kỹ:

1. Quy tắc "Vàng" về Kiểu dữ liệu số:

- **✗ KHÔNG BAO GIỜ** dùng kiểu F (Float) để tính toán Tiền tệ hoặc Điểm số chính xác. Kiểu F gây sai số làm tròn (rounding error) rất nhỏ nhưng nguy hiểm.
- **✓ LUÔN DÙNG** kiểu P (Packed Number) cho tiền tệ, số lượng, điểm số. Ví dụ: TYPE P DECIMALS 2.

2. Ác mộng "Magic Numbers" (Con số ma thuật):

- **✗** Code tệ: IF gv_score > 5. (Số 5 ở đây là magic number. Nếu năm sau trường đổi điểm đạt là 4.0 thì phải đi sửa code ở 100 chỗ).
- **✓** Code chuẩn: Khai báo hằng số CONSTANTS: gc_pass_score TYPE p VALUE '5.0'. và dùng IF gv_score > gc_pass_score.

3. Clear biến trước khi dùng:

- Trong các vòng lặp hoặc xử lý phức tạp, biến cũ có thể còn lưu giá trị rác. Thói quen tốt là dùng lệnh CLEAR variable_name nếu cần thiết (dù ABAP hiện đại đã tự quản lý khá tốt, nhưng cẩn tắc vô áy náy).

4. Pretty Printer là bạn thân:

- Đừng viết code thụt thò lung tung. Hãy dùng nút **Pretty Printer** (Shift+F1) trên thanh công cụ để code tự động căn lề chuẩn SAP.

C. MỤC TIÊU BÀI HỌC

1. **Input:** Sử dụng PARAMETERS để nhận dữ liệu động từ người dùng.
2. **Logic:** Áp dụng cấu trúc điều kiện IF...ELSEIF...ELSE và CASE.
3. **Validation:** Kiểm tra dữ liệu đầu vào (Data Integrity).

D. NỘI DUNG THỰC HÀNH (LAB 2): TÍNH ĐIỂM & XẾP LOẠI

Tình huống: Giảng viên cần một công cụ nhanh để nhập điểm thành phần (Assignment, Midterm, Final) và hệ thống tự động trả về Điểm Tổng kết (GPA) cùng Xếp loại (Rank) theo quy chế đào tạo.

1. Hướng dẫn chi tiết (Step-by-Step)

- **Bước 1:** Vào SE80, tạo chương trình ZDEV331_LAB2_GRADING.
- **Bước 2:** Khai báo Parameters (Input).
 - Lưu ý đặt tên biến bắt đầu bằng pa_.
- **Bước 3:** Khai báo Variables (Biến nội bộ).
 - Lưu ý đặt tên biến bắt đầu bằng gv_.
- **Bước 4:** Viết Logic tính toán & Xếp loại.
- **Bước 5:** Xử lý hiển thị kết quả.

2. Code Mẫu ABAP (Chuẩn Convention)

```

code Abap
downloadcontent_copy
expand_less
*&-----*
*& Report ZDEV331_LAB2_GRADING
*&-----*
REPORT zdev331_lab2_grading.

*--- 1. INPUT PARAMETERS (Đầu vào) ---*
" OBLIGATORY: Bắt buộc phải nhập, không được để trống
PARAMETERS: pa_sid TYPE c LENGTH 8 OBLIGATORY, "Student ID
             pa_name TYPE string, "Student Name
             pa_ass TYPE p DECIMALS 2 OBLIGATORY, "Assignment Score
             pa_mid TYPE p DECIMALS 2 OBLIGATORY, "Midterm Score
             pa_final TYPE p DECIMALS 2 OBLIGATORY. "Final Score

*--- 2. DATA DECLARATION (Khai báo biến) ---*
DATA: gv_gpa TYPE p DECIMALS 2, "Biến chứa điểm trung bình
      gv_rank TYPE string. "Biến chứa xếp loại

*--- 3. CONSTANTS (Hằng số - Best Practice) ---*
CONSTANTS: gc_weight_ass TYPE p DECIMALS 2 VALUE '0.2', "20%
            gc_weight_mid TYPE p DECIMALS 2 VALUE '0.3', "30%
            gc_weight_final TYPE p DECIMALS 2 VALUE '0.5'. "50%

*--- 4. MAIN LOGIC (Xử lý chính) ---*

" Tính GPA theo trọng số
gv_gpa = ( pa_ass * gc_weight_ass ) +
          ( pa_mid * gc_weight_mid ) +
          ( pa_final * gc_weight_final ).

```

```

" Xếp loại (Business Logic)
IF gv_gpa >= 9.
    gv_rank = 'Xuất sắc (Excellent)'.
ELSEIF gv_gpa >= 8.
    gv_rank = 'Giỏi (Good)'.
ELSEIF gv_gpa >= 5.
    gv_rank = 'Đạt (Pass)'.
ELSE.
    gv_rank = 'Học lại (Retake)'.
ENDIF.

*--- 5. OUTPUT (Hiển thị) ---*
WRITE: / '--- KẾT QUẢ HỌC TẬP ---' COLOR 4.
WRITE: / 'Sinh viên:', pa_name, '(', pa_sid, ')'.
WRITE: / 'Điểm thành phần:', pa_ass, '|', pa_mid, '|', pa_final.
ULINE. "Kẻ đường ngang
WRITE: / 'GPA TỔNG KẾT:', gv_gpa COLOR 5. "Màu xanh lá
WRITE: / 'XẾP LOẠI:', gv_rank COLOR 6. "Màu đỏ

```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên thực hiện 4 yêu cầu sau để nâng cấp chương trình từ "chạy được" thành "chạy tốt".

Thử thách 1: Data Validation (Quan trọng nhất)

- **Yêu cầu:** Nếu người dùng nhập điểm < 0 hoặc > 10 , hãy dừng chương trình ngay lập tức và báo lỗi.
- **Gợi ý:** Sử dụng IF ... OR ... kiểm tra đầu vào ngay sau phần START-OF-SELECTION.
- **Real-world Note:** Trong dự án thật, nếu không validate, dữ liệu rác sẽ làm hỏng cả hệ thống báo cáo sau này.

Thử thách 2: Text Elements (Thay vì Hardcode Text)

- **Yêu cầu:** Thay thế các chuỗi văn bản cứng như 'Xuất sắc', 'Giỏi' bằng Text Elements.

- **Gợi ý:** Vào menu *Goto -> Text Elements -> Text Symbols*. Tạo ký hiệu 001 là 'Xuất sắc', 002 là 'Giỏi'. Trong code dùng TEXT-001, TEXT-002.
- **Real-world Note:** Đây là bắt buộc để làm **Đa ngôn ngữ (Translation)**. Khách hàng Đức muốn thấy tiếng Đức, khách hàng Việt muốn thấy tiếng Việt.

Thử thách 3: Logic điểm liệt (Business Rule)

- **Yêu cầu:** Kể cả khi $GPA > 5.0$, nhưng nếu điểm thi cuối kỳ (pa_final) < 4.0 , sinh viên vẫn bị coi là "Trượt (Failed by Rule)".
- **Gợi ý:** Sử dụng lồng ghép IF hoặc toán tử AND.

Thử thách 4: Selection Screen Design (Giao diện)

- **Yêu cầu:** Đóng khung các ô nhập liệu điểm số vào một nhóm có tiêu đề là "Academic Scores".
- **Gợi ý:** Sử dụng SELECTION-SCREEN BEGIN OF BLOCK....

4. Lời giải tham khảo cho Thử thách (Snippet)

code Abap

--- Giải pháp Thử thách 4: Đóng khung giao diện ---

SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE text-003. "text-003 = 'Nhập điểm'

PARAMETERS: pa_ass TYPE p DECIMALS 2 OBLIGATORY,

pa_mid TYPE p DECIMALS 2 OBLIGATORY,

pa_final TYPE p DECIMALS 2 OBLIGATORY.

SELECTION-SCREEN END OF BLOCK b1.

--- Giải pháp Thử thách 1: Validate đầu vào ---

IF pa_ass < 0 OR pa_ass > 10 OR pa_mid < 0 OR pa_mid > 10 OR pa_final < 0 OR pa_final > 10.

MESSAGE 'Dữ liệu điểm không hợp lệ (Phải từ 0-10)!' TYPE 'E'.

" TYPE 'E' sẽ dừng chương trình và hiển thị màu đỏ

ENDIF.

```
*--- Giải pháp Thử thách 3: Logic điểm liệt ---*
IF gv_gpa >= 5.
    IF pa_final < 4. "Dính điểm liệt
        gv_rank = 'Trượt do điểm liệt (Failed by Final score)'.
    ELSE.
        gv_rank = 'Đạt (Pass)'.
    ENDIF.
ELSE.
    gv_rank = 'Học lại (Retake)'.
ENDIF.
```

Kết thúc Buổi 2.

Sinh viên đã học được cách viết code có cấu trúc, xử lý lỗi (Validation) và tư duy tránh "Hardcode".

Đã rõ. Chúng ta sẽ tuân thủ nghiêm ngặt quy tắc: **Code và Giao diện hiển thị (Output) là Tiếng Anh**, còn **Comment giải thích là Tiếng Việt**.

Chúng ta bước sang **Buổi 3**, một cột mốc quan trọng: Làm việc với Cơ sở dữ liệu (Database).

BUỔI 3: TRUY CẬP CƠ SỞ DỮ LIỆU & INTERNAL TABLES

A. THÔNG TIN & QUY TẮC (LESSON INFO)

- **Package:** ZSPR26_SE1876_[AccountID]
- **Database Table:** ZSTUDENT_[AccountID] (Ví dụ: ZSTUDENT_DEV331)
- **Program Name:** Z[AccountID]_LAB3_DB_ACCESS
- **Chủ đề:** Quản lý Danh sách Sinh viên (Student Master Data).

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Lưu ý sống còn khi làm việc với Database trong SAP:

1. Hạn chế SELECT * (Select All):

-  Tệ: SELECT * FROM zstudent... (Lấy cả 100 cột dù chỉ dùng 2 cột -> Tốn bộ nhớ, chậm hệ thống).
-  Tốt: SELECT id, name, email FROM zstudent... (Chỉ lấy cột cần thiết).

2. Luôn kiểm tra sy-subrc:

- Sau mỗi lệnh SELECT, biến hệ thống sy-subrc sẽ trả về trạng thái.
- 0: Thành công (Có dữ liệu).
- 4: Không tìm thấy dữ liệu.
- Nếu không kiểm tra sy-subrc mà cố tình xử lý dữ liệu, chương trình sẽ bị dump (crash) khi bảng rỗng.

3. Inline Declaration (Khai báo tại chỗ):

- Từ bản ABAP 7.40 trở lên, thay vì khai báo biến dài dòng ở đầu, hãy dùng cú pháp DATA(ten_bien) ngay trong lệnh xử lý. Code gọn và hiện đại hơn rất nhiều.

C. MỤC TIÊU BÀI HỌC

1. **SE11:** Tạo được một bảng dữ liệu (Database Table) đơn giản để lưu trữ hồ sơ sinh viên.
2. **Open SQL:** Viết câu lệnh SELECT để lấy dữ liệu từ Database vào chương trình.
3. **Internal Table:** Hiểu khái niệm bảng nội bộ (biến dạng bảng) để chứa danh sách dữ liệu trong RAM.
4. **Loop Processing:** Duyệt vòng lặp để hiển thị dữ liệu.

D. NỘI DUNG THỰC HÀNH (LAB 3): STUDENT LIST LISTING

Tình huống: Trường Đại học FPT cần một báo cáo danh sách sinh viên hiện có trong hệ thống để gửi cho bộ phận Đào tạo. Dữ liệu cần lấy từ bảng Database chính thức.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo Bảng Database (Table Creation)

- Gõ T-Code SE11. Chọn **Database table**.
- Nhập tên: ZSTUDENT_DEV331 (Thay DEV331 bằng ID của bạn). Nhấn **Create**.
- *Description:* Student Master Data.
- *Delivery Class:* Chọn **A** (Application table).
- *Data Browser/Table View Maint.:* Chọn **Display/Maintenance Allowed**.
- Chuyển sang tab **Fields** (Khai báo cột):
 - MANDT (Type: MANDT, Key: ☒) - *Bắt buộc phải có (Client ID)*.
 - STUDENT_ID (Type: CHAR8, Key: ☒) - Mã SV.
 - FULL_NAME (Type: CHAR40) - Họ tên.

- DOB (Type: DATS) - Ngày sinh.
- COURSE (Type: CHAR10) - Khóa (K17, K18...).
- Nhấn **Technical Settings** (trên thanh công cụ):
 - *Data Class*: USER0
 - *Size Category*: 0
 - Lưu (Save) -> Quay lại (Back).
- Lưu và **Activate** (Ctrl+F3) bảng.

Bước 2: Tạo Dữ liệu giả (Mock Data)

- Vì chưa có giao diện nhập liệu, ta dùng tool hệ thống để tạo dữ liệu test.
- Gõ T-Code SE16N. Nhập bảng ZSTUDENT_DEV331. Enter.
- Nhập vào vài dòng dữ liệu mẫu (Ví dụ: SE180001 - Nguyen Van A, SE180002 - Le Thi B...). Nhấn Save (F8).

Bước 3: Viết Chương trình ABAP

- Vào SE80, tạo program ZDEV331_LAB3_DB_ACCESS.
- Viết logic đọc dữ liệu từ bảng vừa tạo.

2. Code Mẫu ABAP (English UI - Vietnamese Comments)

```
code Abap
*&-----*
*& Report ZDEV331_LAB3_DB_ACCESS
*&-----*
*& Description: Fetch and display student list from Database
*& Author: DEV331 (Student SE1876)
*&-----*
REPORT zdev331_lab3_db_access.
```

--- 1. DATA DECLARATION (Khai báo dữ liệu) ---

" Khai báo Internal Table để chứa danh sách sinh viên lấy từ DB

" Sử dụng cú pháp hiện đại: khai báo inline ngay trong câu lệnh SELECT

" Tuy nhiên, để rõ ràng cho người mới, ta sẽ dùng biến inline bên dưới.

--- 2. MAIN LOGIC (Xử lý chính) ---

START-OF-SELECTION.

" Truy vấn dữ liệu từ bảng ZSTUDENT_DEV331

" SELECT * : Lấy tất cả các cột

" INTO TABLE @DATA(...) : Tự động tạo một bảng nội bộ (lt_students) khớp cấu trúc
SELECT *

FROM zstudent_dev331

INTO TABLE @DATA(lt_students).

" Kiểm tra xem có lấy được dữ liệu không (sy-subrc = 0 là thành công)

IF sy-subrc = 0.

" Sắp xếp danh sách theo Tên sinh viên

SORT lt_students BY full_name ASCENDING.

" In tiêu đề báo cáo

WRITE: / '=== FPT UNIVERSITY STUDENT LIST ===' COLOR 1.

WRITE: / 'Generated Date:', sy-datum.

ULINE. " Kẻ dòng

" Duyệt vòng lặp để in từng sinh viên ra màn hình

LOOP AT lt_students INTO DATA(ls_student).

WRITE: / 'ID:', ls_student-student_id,

 '| Name:', ls_student-full_name,

 '| Course:', ls_student-course.

ENDLOOP.

SKIP 1.

" In tổng số lượng sinh viên tìm thấy

" sy-dbcnt chứa số dòng dữ liệu vừa được tác động bởi câu lệnh SQL/Loop

WRITE: / 'Total Students found:', sy-dbcnt COLOR 4.

ELSE.

" Trường hợp không tìm thấy dữ liệu

WRITE: / 'No data found in database table ZSTUDENT_DEV331.' COLOR 6.

ENDIF.

3. Bốn (4) Thử thách phát triển (Deep Dive Challenges)

Sinh viên thực hiện 4 yêu cầu sau để tối ưu hóa chương trình:

Challenge 1: Data Filtering (Lọc dữ liệu)

- **Requirement:** Thêm một PARAMETERS: pa_course TYPE char10 vào đầu chương trình.
- **Logic:** Chỉ SELECT những sinh viên thuộc khóa học (Course) mà người dùng nhập vào. Nếu người dùng để trống, lấy tất cả.
- **Hint:** Sử dụng WHERE trong câu lệnh SQL. Chú ý xử lý trường hợp tham số rỗng (dùng thẻ wildcards hoặc Dynamic WHERE, nhưng cách đơn giản nhất cho bài này là dùng RANGES hoặc kiểm tra IF trước khi Select - *Cách dễ nhất: Dùng SELECT-OPTIONS nhưng bài này yêu cầu PARAMETERS thì dùng WHERE course = pa_course*).

Challenge 2: Specific Columns Optimization (Tối ưu cột)

- **Requirement:** Thay vì dùng SELECT * (Lấy cả cột DOB - Ngày sinh dù không in ra), hãy chỉ SELECT 3 cột cần thiết: student_id, full_name, course.
- **Hint:** Liệt kê tên cột sau từ khóa SELECT. Lưu ý khi dùng INTO TABLE, cấu trúc bảng nội bộ sẽ thay đổi.

Challenge 3: Age Calculation (Xử lý dữ liệu trong Loop)

- **Requirement:** Trong vòng lặp LOOP, hãy tính tuổi của sinh viên dựa trên cột DOB.
- **Logic:** Age = Current Year - Birth Year. In tuổi ra bên cạnh tên.
- **Hint:** ls_student-dob(4) lấy 4 ký tự đầu (Năm).

Challenge 4: No Data Handling (Trải nghiệm người dùng)

- **Requirement:** Nếu người dùng nhập một mã khóa học không tồn tại (Ví dụ: K99), chương trình hiện tại có thể in ra màn hình trắng trơn hoặc lỗi. Hãy dùng MESSAGE ... TYPE 'I' DISPLAY LIKE 'E' để hiện popup thông báo đẹp mắt "No students found for course K99" thay vì dùng WRITE.

4. Lời giải tham khảo (Solution Snippets)

```
code Abap
*--- Solution for Challenge 1 & 4: Filtering & Message ---*
PARAMETERS: pa_crse TYPE char10. " Input Course

START-OF-SELECTION.

" Select với điều kiện WHERE
" Nếu pa_crse rỗng, logic dưới đây sẽ tìm chính xác những dòng rỗng.
" Để xử lý 'Rỗng = Lấy tất cả', ta cần kỹ thuật Ranges (sẽ học sau)
" hoặc logic IF đơn giản như sau:

IF pa_crse IS NOT INITIAL.
  SELECT student_id, full_name, dob, course " Solution Challenge 2: List columns
    FROM zstudent_dev331
   WHERE course = @pa_crse
   INTO TABLE @DATA(lt_students).
ELSE.
  SELECT student_id, full_name, dob, course
    FROM zstudent_dev331
```

```
    INTO TABLE @lt_students.  
ENDIF.  
  
IF sy-subrc <> 0.  
    " Solution Challenge 4: Popup Message  
    MESSAGE 'No students found matching your criteria.' TYPE 'I' DISPLAY LIKE 'E'.  
    RETURN. " Thoát chương trình  
ENDIF.  
  
*--- Solution for Challenge 3: Age Calculation ---*  
LOOP AT lt_students INTO DATA(ls_student).  
    DATA(lv_age) = sy-datum(4) - ls_student-dob(4). " Tính tuổi  
  
    WRITE: / 'ID:', ls_student-student_id,  
           'Name:', ls_student-full_name,  
           'Age:', lv_age, " In tuổi  
           'Course:', ls_student-course.  
ENDLOOP.
```

Kết thúc Buổi 3.

Sinh viên đã biết cách tạo bảng Database, nhập dữ liệu giả lập và viết code để lấy dữ liệu đó lên báo cáo.

Chào bạn, đây là yêu cầu rất chính xác. Trong các dự án thực tế (Real-world Projects) của SAP, chúng ta **không bao giờ** khai báo trực tiếp

BUỔI 4 (Bổ sung): HƯỚNG DẪN CHI TIẾT TẠO DATABASE & FOREIGN KEY

A. MỤC TIÊU KỸ THUẬT

- Data Element:** Tạo định nghĩa ngữ nghĩa cho dữ liệu (Ví dụ: Nhãn hiển thị là "Course ID" thay vì "CHAR8").
 - Foreign Key (Khóa ngoại):** Thiết lập mối quan hệ ràng buộc.
 - Ràng buộc:* Không thể nhập điểm cho một Sinh viên chưa tồn tại trong hệ thống.
 - Lợi ích:* Khi nhập liệu, nhấn F4 sẽ tự động hiện danh sách để chọn.
-

BUỔI 4: TRUY CẬP DB NÂNG CAO (JOIN, GROUP BY & AGGREGATION)

A. THÔNG TIN & QUY TẮC

- Package:** ZSPR26_SE1876_[AccountID]
- New Tables:**
 - ZCOURSE_[AccountID] (Danh mục môn học).
 - ZENROLL_[AccountID] (Bảng đăng ký học/Điểm số).
- Program Name:** Z[AccountID]_LAB4_ADV_SQL
- Chủ đề:** Báo cáo Tổng hợp Kết quả Đăng ký môn học (Enrollment Report).

B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Chiến thuật tối ưu hóa truy vấn trong các dự án lớn:

1. INNER JOIN vs. LEFT OUTER JOIN:

- Dùng INNER JOIN khi bạn chỉ muốn lấy dữ liệu có ở cả 2 bảng (Sinh viên **đã** đăng ký môn).
- Dùng LEFT OUTER JOIN khi bạn muốn lấy tất cả dữ liệu bảng trái, kể cả khi bảng phải không có (Lấy danh sách **tất cả** sinh viên, kèm môn học nếu có, chưa đăng ký thì để trống).
- *Sai lầm phổ biến:* Dùng Inner Join làm mất dữ liệu của những sinh viên chưa đăng ký môn nào -> Báo cáo sai lệch.

2. Thứ tự JOIN (Database Optimizer):

- Về lý thuyết, SQL Optimizer của SAP S/4HANA (HANA DB) rất thông minh. Tuy nhiên, thói quen tốt là để bảng có lượng dữ liệu nhỏ hơn hoặc bảng được lọc (Filter) nhiều nhất làm bảng chính (Primary table) để thu hẹp phạm vi tìm kiếm sớm.

3. FOR ALL ENTRIES (Vũ khí bí mật của ABAP):

- Khi JOIN quá phức tạp (trên 5-6 bảng) hoặc gây chậm hệ thống, các chuyên gia SAP thường chuyển sang dùng kỹ thuật FOR ALL ENTRIES (Lấy bảng A vào RAM, sau đó dùng dữ liệu bảng A để lấy bảng B). Chúng ta sẽ làm quen kỹ thuật này ở mức nâng cao, nhưng hôm nay hãy tập trung vào JOIN chuẩn mực.

C. MỤC TIÊU BÀI HỌC

1. **Data Modeling:** Tạo thêm bảng ZCOURSE và ZENROLL để hình thành mối quan hệ 1-N (Một sinh viên đăng ký nhiều môn).
2. **Advanced SQL:** Sử dụng INNER JOIN để kết nối 3 bảng (Student - Enrollment - Course).
3. **Aggregation:** Sử dụng COUNT, SUM, AVG kết hợp GROUP BY để làm báo cáo thống kê.

D. NỘI DUNG THỰC HÀNH (LAB 4): STUDENT ENROLLMENT & CREDITS

Tình huống: Phòng đào tạo muốn biết mỗi sinh viên đã đăng ký những môn nào, điểm số bao nhiêu và **tổng số tín chỉ** họ đã tích lũy là bao nhiêu.

B. QUY TRÌNH THỰC HIỆN (STEP-BY-STEP)

Giả định tài khoản: DEV331 (Sinh viên thay bằng ID của mình).

Package: ZSPR26_SE1876_DEV331.

GIAI ĐOẠN 1: TẠO DATA ELEMENTS (Định nghĩa dữ liệu)

Chúng ta cần tạo Data Element để quy định nhãn hiển thị cho Môn học và Tín chỉ.

Bước 1: Tạo Data Element cho Mã Môn học

1. Vào T-Code SE11.
2. Chọn **Data type**, nhập tên: ZDE_COURSE_ID_DEV331. Nhấn **Create**.
3. Chọn **Data element** trong popup. Nhấn Continue.
4. **Tab Data Type:**
 - Short Description: Course ID for LMS.
 - Domain: Để trống (Dùng trực tiếp Built-in type cho nhanh).
 - Chọn tích vào **Predefined Type**.
 - Data Type: CHAR. Length: 8.
5. **Tab Field Label** (Quan trọng để hiển thị đẹp trên báo cáo):
 - Short: Course.
 - Medium: Course ID.
 - Long: Course Identifier.
 - Heading: Course ID.
6. Lưu (Save) vào Package của bạn -> **Activate** (Ctrl+F3).

Bước 2: Tạo Data Element cho Số tín chỉ

1. Tại SE11, nhập tên: ZDE_CREDITS_DEV331. Nhấn **Create** -> **Data element**.

2. **Tab Data Type:**

- Short Description: Course Credits.
- Predefined Type: INT1 (Số nguyên nhỏ, tối đa 255). Length: 3.

3. **Tab Field Label:**

- Short: Credit.
- Medium: Credits.
- Long: Number of Credits.
- Heading: Credits.

4. Lưu -> **Activate** (Ctrl+F3).

GIAI ĐOẠN 2: TẠO BẢNG MÔN HỌC (ZCOURSE)

Bước 1: Khởi tạo bảng

1. Tại SE11, chọn **Database table**.

2. Nhập tên: ZCOURSE_DEV331. Nhấn **Create**.

3. Short Description: Course Master Data.

4. **Tab Delivery and Maintenance:**

- Delivery Class: A (Application table).
- Data Browser/Table View Maint.: Display/Maintenance Allowed.

Bước 2: Khai báo cột (Fields)

Chuyển sang tab **Fields** và nhập chính xác như sau:

Field	Key Data Element	Built-in Type (Tự hiện)	Description
MANDT	✓ MANDT	CLNT 3	Client
COURSE_ID	✓ ZDE_COURSE_ID_DEV331	CHAR 8	Course Identifier
COURSE_NAME	(Để trống, bấm nút Predefined Type) -> Chọn CHAR, Len 40	CHAR 40	Tên môn học
CREDITS	ZDE_CREDITS_DEV331	INT1 3	Course Credits

Lưu ý: Với cột COURSE_NAME, ta dùng trực tiếp Predefined Type cho nhanh, nhưng chuẩn nhất vẫn nên tạo Data Element ZDE_COURSE_NAME.

Bước 3: Technical Settings & Activate

1. Nhấn nút **Technical Settings** (trên thanh công cụ hoặc Ctrl+Shift+F9).
2. Data Class: APPL0 (Master data).
3. Size Category: 0.
4. Nhấn **Save** -> **Back** (F3).
5. Nhấn **Activate** (Ctrl+F3).

GIẢI ĐOẠN 3: TẠO BẢNG ĐĂNG KÝ (ZENROLL) & FOREIGN KEYS (QUAN TRỌNG)

Đây là bảng trung gian kết nối Sinh viên và Môn học.

Bước 1: Khởi tạo bảng

1. Tại SE11, nhập tên: ZENROLL_DEV331. Nhấn **Create**.
2. Description: Student Enrollment Transaction.
3. Delivery Class: A. Data Browser: Allowed.

Bước 2: Khai báo cột (Fields)

Field	Key Data Element	Description
MANDT	☒ MANDT	Client
ENROLL_ID	☒ NUMC10 (Built-in standard)	Mã đăng ký (ID tự tăng)
STUDENT_ID	ZDE_STUDENT_ID_DEV331 (Hoặc dùng lại kiểu của bài 3: CHAR8)	Mã Sinh viên
COURSE_ID	ZDE_COURSE_ID_DEV331	Mã Môn học
SCORE	(Predefined: DEC, Len 4, Decimals 2)	Điểm số

Lưu ý: Cột STUDENT_ID phải có kiểu dữ liệu (Type và Length) giống hệt bảng ZSTUDENT bài trước.

Bước 3: Tạo Foreign Key cho STUDENT_ID (Kết nối về bảng Sinh viên)

1. Đặt con trỏ chuột vào dòng STUDENT_ID.
2. Nhấn nút **Foreign Key** (Biểu tượng chìa khóa/mũi tên) trên thanh công cụ.
3. Popup hiện ra:
 - **Check Table:** Nhập tên bảng sinh viên bài 3 (ZSTUDENT_DEV331). Nhấn Enter.
 - Hệ thống tự động đề xuất mapping:
 - MANDT = ZSTUDENT_DEV331-MANDT
 - STUDENT_ID = ZSTUDENT_DEV331-STUDENT_ID
 - **Foreign Key Field Type:** Chọn Key fields/candidates (Vì Student ID là key của bảng kia).
 - **Cardinality:** Chọn 1 : CN (Một sinh viên có thể xuất hiện nhiều lần trong bảng Enrollment).
4. Nhấn **Copy** (Nút tích xanh).

Bước 4: Tạo Foreign Key cho COURSE_ID (Kết nối về bảng Môn học)

1. Đặt con trỏ chuột vào dòng COURSE_ID.
2. Nhấn nút **Foreign Key**.
3. Popup hiện ra:
 - **Check Table:** Nhập ZCOURSE_DEV331. Nhấn Enter.
 - Kiểm tra mapping trường MANDT và COURSE_ID.
 - **Cardinality:** Chọn 1 : CN (Một môn học có nhiều sinh viên đăng ký).
4. Nhấn **Copy**.

Bước 5: Technical Settings & Activate

1. Nhấn **Technical Settings**.
2. Data Class: APPL1 (Transaction data).
3. Size Category: 1.
4. Save -> Back -> **Activate**.

GIAI ĐOẠN 4: NHẬP DỮ LIỆU & KIỂM TRA (TESTING)

Nhờ có Foreign Key, việc nhập dữ liệu sẽ an toàn hơn rất nhiều.

Bước 1: Nhập Master Data (Môn học)

1. T-Code SE16N. Bảng ZCOURSE_DEV331.
2. Nhập dữ liệu:
 - PRF192 | C Fundamentals | 3

- CEA201 | Computer Org | 3
- JPD113 | Java Programming | 4

3. Lưu (Save).

Bước 2: Nhập Transaction Data (Đăng ký) - Test Foreign Key

1. T-Code SE16N. Bảng ZENROLL_DEV331.
2. Thử nhập một dòng với STUDENT_ID bừa bãi (Ví dụ: '9999').
3. Nhấn Enter. Hệ thống sẽ **Báo Lỗi Đỏ** ngay phía dưới: *"Entry 9999 does not exist in ZSTUDENT_DEV331 (check entry)"*.
 - -> **Kết luận:** Foreign Key hoạt động tốt. Dữ liệu rác bị chặn.
4. Click vào ô STUDENT_ID, nhấn F4. Bạn sẽ thấy danh sách sinh viên từ bảng ZSTUDENT hiện ra để chọn.
5. Nhập dữ liệu hợp lệ:
 - Enroll 0001 | SV001 | PRF192 | 8.00
 - Enroll 0002 | SV001 | JPD113 | 7.50
 - Enroll 0003 | SV002 | PRF192 | 9.00
6. Lưu (Save).

2. Hướng dẫn Code (Step-by-Step)

- **Bước 1:** Tạo chương trình ZDEV331_LAB4_ADV_SQL.
- **Bước 2:** Viết câu lệnh SELECT kết hợp INNER JOIN.
 - Cấu trúc: ZSTUDENT nối với ZENROLL qua Student_ID. ZENROLL nối với ZCOURSE qua Course_ID.
- **Bước 3:** Hiển thị kết quả chi tiết.

3. Code Mẫu ABAP (English UI - Vietnamese Comments)

code Abap

&-----

*& Report ZDEV331_LAB4_ADV_SQL

&-----

REPORT zdev331_lab4_adv_sql.

--- 1. SELECTION SCREEN (Màn hình lọc) ---

PARAMETERS: pa_sid TYPE char8. " Lọc theo mã sinh viên (Tùy chọn)

--- 2. MAIN LOGIC ---

START-OF-SELECTION.

" Thực hiện JOIN 3 bảng để lấy đầy đủ thông tin:

" SV(Tên) -> Đăng ký(Điểm) -> Môn học(Tên môn, Tín chỉ)

" AS a, AS b, AS c là Alias (tên giả) để viết code cho ngắn gọn

SELECT a~student_id, a~full_name,

 b~course_id, b~score,

 c~course_name, c~credits

FROM zstudent dev331 AS a " Bảng Sinh viên (Gốc)

INNER JOIN zenroll dev331 AS b " Nối với bảng Đăng ký

 ON a~student_id = b~student_id

INNER JOIN zcourse_dev331 AS c " Nối tiếp với bảng Môn học

 ON b~course_id = c~course_id

WHERE a~student_id = @pa_sid OR @pa_sid IS INITIAL " Lọc nếu user nhập ID

INTO TABLE @DATA(lt_report).

IF sy-subrc = 0.

 SORT lt_report BY student_id course_id.

WRITE: / '=== STUDENT ACADEMIC TRANSCRIPT ===' COLOR 1.

 ULINE.

```
" Duyệt và in báo cáo
LOOP AT lt_report INTO DATA(ls_row).
" Kỹ thuật in: In Mã SV và Tên 1 lần duy nhất cho mỗi nhóm (ON CHANGE OF)
" Giúp báo cáo dễ nhìn hơn
AT NEW student_id.
    WRITE: / 'Student:', ls_row-student_id, ls_row-full_name COLOR 4.
ENDAT.

WRITE: /15 ls_row-course_id,      " Thụt vào cột 15
        ls_row-course_name,
        'Credits:', ls_row-credits,
        'Score:', ls_row-score COLOR 5.
ENDLOOP.

ELSE.
    MESSAGE 'No enrollment data found.' TYPE 'S' DISPLAY LIKE 'E'.
ENDIF.
```

4. Bốn (4) Thử thách phát triển (Deep Dive Challenges)

Sinh viên cần vận dụng kiến thức SQL nâng cao để giải quyết các bài toán thống kê.

Challenge 1: LEFT OUTER JOIN (Sinh viên chưa học gì)

- **Vấn đề:** Code mẫu dùng INNER JOIN, nên những sinh viên chưa đăng ký môn nào sẽ **biến mất** khỏi báo cáo.
- **Yêu cầu:** Đổi sang LEFT OUTER JOIN (giữa Student và Enroll) để hiện ra cả những sinh viên chưa học môn nào.
- **Output:** Với những dòng này, cột Tên môn/Điểm sẽ rỗng. Hãy xử lý in ra dòng chữ "Not registered yet" màu đỏ.

Challenge 2: Total Credits Calculation (Tính tổng tín chỉ)

- **Yêu cầu:** Tính tổng số tín chỉ (SUM(credits)) mà mỗi sinh viên đã đăng ký.
- **Logic:** Thực hiện việc cộng dồn trong vòng lặp hoặc dùng câu lệnh SELECT SUM(...) GROUP BY... riêng biệt.

- **Output:** In dòng "Total Credits: XX" ở cuối danh sách môn học của mỗi sinh viên.

Challenge 3: Average Score (Điểm trung bình hệ số)

- **Yêu cầu:** Tính điểm trung bình (GPA) của sinh viên đó.
- **Công thức:** $\text{SUM}(\text{Score} * \text{Credits}) / \text{SUM}(\text{Credits})$.
- **Lưu ý:** Cần thận lỗi chia cho 0 (Divide by Zero) nếu sinh viên chưa học môn nào.

Challenge 4: SQL Aggregation (Thống kê cấp cao)

- **Yêu cầu:** Tạo một báo cáo phụ (Block thứ 2) hiển thị "Top Course Statistics" - Môn học nào có nhiều sinh viên đăng ký nhất?
- **Gợi ý:** `SELECT course_id, COUNT( * ) AS total_students FROM zenroll... GROUP BY course_id.`

5. Lời giải tham khảo (Solution Snippets)

```
code Abap
*--- Solution Challenge 1 & 2: Left Join & Total Credits ---*
DATA: lv_total_credits TYPE i,
      lv_total_score  TYPE p DECIMALS 2.

" Đổi INNER thành LEFT OUTER
SELECT a~student_id, a~full_name, b~course_id, b~score, c~credits
FROM zstudent_dev331 AS a
LEFT OUTER JOIN zenroll_dev331 AS b ON a~student_id = b~student_id
LEFT OUTER JOIN zcourse_dev331 AS c ON b~course_id = c~course_id
INTO TABLE @lt_report.

LOOP AT lt_report INTO ls_row.
  AT NEW student_id.
    WRITE: / 'Student:', ls_row-student_id, ls_row-full_name.
    CLEAR: lv_total_credits, lv_total_score. " Reset biến tổng
```

```
ENDAT.
```

```
IF ls_row-course_id IS INITIAL.
```

```
    WRITE: /15 'Not registered any course yet.' COLOR 6.
```

```
ELSE.
```

```
    WRITE: /15 ls_row-course_id, ls_row-score.
```

```
    " Cộng dồn tín chỉ
```

```
    lv_total_credits = lv_total_credits + ls_row-credits.
```

```
ENDIF.
```

```
" Logic xử lý cuối mỗi nhóm sinh viên
```

```
AT END OF student_id.
```

```
    ULINE /15(50).
```

```
    WRITE: /15 'Total Accumulated Credits:', lv_total_credits COLOR 3.
```

```
    SKIP 1.
```

```
ENDAT.
```

```
ENDLOOP.
```

```
*--- Solution Challenge 4: Aggregation ---*
```

```
SELECT course_id, COUNT( * ) AS student_count
```

```
FROM zenroll_dev331
```

```
GROUP BY course_id
```

```
ORDER BY student_count DESCENDING
```

```
INTO TABLE @DATA(lt_stats).
```

```
WRITE: / '=== MOST POPULAR COURSES ==='.
```

```
LOOP AT lt_stats INTO DATA(ls_stat).
```

```
    WRITE: / ls_stat-course_id, 'Students:', ls_stat-student_count.
```

```
ENDLOOP.
```

Kết thúc Buổi 4.

Sinh viên giờ đây đã nắm trong tay sức mạnh của SQL để tổng hợp dữ liệu từ nhiều nguồn.

BUỔI 5: MODULARIZATION (HÀM & TÁI SỬ DỤNG CODE)

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID] (Ví dụ: ZSPR26_SE1876_DEV331)
 - **Function Group (FG):** ZFG_[AccountID] (Ví dụ: ZFG_DEV331)
 - *Lưu ý:* Function Module **bắt buộc** phải nằm trong một Function Group. Không có FG thì không tạo được FM.
 - **Function Module (FM):** ZFM_[AccountID]_CALC_GPA
 - **Program Name:** Z[AccountID]_LAB5_MODULARIZATION
 - **Chủ đề:** Xây dựng "API nội bộ" để tính điểm trung bình (GPA).
-

● B. REAL-WORLD TIPS (KINH NGHIỆM THỰC CHIẾN)

1. Quy tắc "DRY" (Don't Repeat Yourself):

- Trong dự án thực tế, công thức tính điểm có thể thay đổi (năm nay 20/30/50, năm sau 30/30/40).
- Nếu bạn copy công thức này vào 10 báo cáo khác nhau -> Khi sửa luật, bạn phải sửa 10 chỗ (rủi ro sót lỗi cao).
- **Giải pháp:** Viết 1 Function Module. 10 báo cáo cùng gọi nó. Sửa luật chỉ cần sửa 1 chỗ.

2. Function Module (FM) vs. Subroutine (Perform):

- **Subroutine (PERFORM):** Cũ, chỉ có tác dụng *làm gọn code* trong 1 chương trình. Không thể gọi từ chương trình khác.
- **Function Module (CALL FUNCTION):** Hiện đại hơn, là biến toàn cục (Global). Chương trình A, B, C đều gọi được. Đặc biệt hỗ trợ RFC (Remote Function Call) để kết nối với Web/Mobile.

3. Lỗi "Inactive Function Group":

- 90% sinh viên mới học sẽ gặp lỗi: *Tạo FM xong nhưng không Active được.*

- **Lý do:** Khi tạo Function Group mới, hệ thống tạo ra 2 file ngàm (Top Include và Main Program). Bạn phải Active cả cái Function Group trước thì mới Active được Function Module bên trong.
-

C. MỤC TIÊU BÀI HỌC

1. **Function Group:** Tạo và kích hoạt thành công "thùng chứa" hàm.
 2. **Function Module (SE37):**
 - Khai báo tham số nhập (IMPORTING), xuất (EXPORTING).
 - Khai báo ngoại lệ (EXCEPTIONS) để xử lý lỗi logic.
 3. **Integration:** Viết chương trình ABAP gọi hàm vừa tạo và xử lý kết quả trả về (sy-subrc).
-

D. NỘI DUNG THỰC HÀNH (LAB 5): CENTRALIZED GPA CALCULATOR

Tình huống: Phòng đào tạo yêu cầu chuẩn hóa cách tính điểm. Bất kể là màn hình nhập liệu, báo cáo cuối kỳ, hay web portal sinh viên, tất cả đều phải dùng chung một công thức tính GPA và Xếp loại.

1. Hướng dẫn chi tiết (Step-by-Step)

GIẢI ĐOẠN 1: TẠO FUNCTION GROUP (QUAN TRỌNG)

- **Bước 1:** Vào T-Code SE80. Chọn *Repository Browser*. Chọn *Package* của bạn.
- **Bước 2:** Chuột phải vào tên Package -> **Create** -> **Function Group**.
 - Function Group: ZFG_DEV331 (Thay DEV331 bằng ID của bạn).
 - Short text: Utils for Student Management.
 - Nhấn Save.
- **Bước 3 (Bắt buộc):** Kích hoạt Function Group.

- Trong danh sách bên trái, bạn sẽ thấy thư mục **Function Groups** mới xuất hiện.
- Chuột phải vào tên ZFG_DEV331 -> Chọn **Activate** (Cây đũa thần).
- *Lưu ý:* Một popup hiện ra liệt kê các thành phần (Main Program, Top Include). Chọn tất cả và nhấn Enter.

GIẢI ĐOẠN 2: TẠO FUNCTION MODULE (SE37)

- **Bước 4:** Chuột phải vào Function Group ZFG_DEV331 -> **Create -> Function Module**.
 - Function Module: ZFM_DEV331_CALC_GPA.
 - Nhấn Save.
- **Bước 5: Khai báo Tham số (Parameters) - Tab Interface**
 - **Tab Import** (Dữ liệu nhận vào):
 - IV_LAB | Type P | Associated Type (Gõ trực tiếp): P | Pass Value: ☒
 - IV_QUIZ | Type P | Associated Type: P | Pass Value: ☒
 - IV_FINAL | Type P | Associated Type: P | Pass Value: ☒
 - *Mẹo:* Tích vào "Pass Value" để truyền tham trị (an toàn hơn).
 - **Tab Export** (Dữ liệu trả ra):
 - EV_GPA | Type P | Associated Type: P | Pass Value: ☒
 - EV_RANK | Type STRING | Associated Type: STRING | Pass Value: ☒
 - **Tab Exceptions** (Định nghĩa lỗi):
 - INVALID_SCORE | Short text: Score must be between 0 and 10.
- **Bước 6: Viết Logic - Tab Source Code**
 - Nhập đoạn code xử lý logic (Xem mục Code Mẫu bên dưới).

- Nhấn **Activate** (Ctrl+F3).

GIẢI ĐOẠN 3: VIẾT CHƯƠNG TRÌNH GỌI HÀM

- **Bước 7:** Tạo Program mới ZDEV331_LAB5_MODULARIZATION trong Package của bạn.
- **Bước 8:** Sử dụng nút **Pattern** (Cây đưa thần có ô vuông - Nằm trên thanh công cụ ABAP Editor).
 - Chọn "CALL FUNCTION".
 - Nhập tên hàm: ZFM_DEV331_CALC_GPA.
 - Nhấn Enter. Hệ thống sẽ tự sinh ra khung code gọi hàm chuẩn xác 100%.
- **Bước 9:** Điền các biến của bạn vào khung code đó (Xem mục Code Mẫu).

2. Code Mẫu ABAP

A. Logic bên trong Function Module (ZFM_DEV331_CALC_GPA)

code Abap

```
FUNCTION zfm_dev331_calc_gpa.  
*"-----  
*"**"Local Interface:  
*" IMPORTING  
*"   VALUE(IV_LAB) TYPE P  
*"   VALUE(IV_QUIZ) TYPE P  
*"   VALUE(IV_FINAL) TYPE P  
*" EXPORTING  
*"   VALUE(EV_GPA) TYPE P  
*"   VALUE(EV_RANK) TYPE STRING  
*" EXCEPTIONS  
*"   INVALID_SCORE  
*"-----
```

```

" 1. Validation Logic (Kiểm tra đầu vào)
" Nếu điểm nhập vào < 0 hoặc > 10 thì báo lỗi
IF iv_lab < 0 OR iv_lab > 10 OR
  iv_quiz < 0 OR iv_quiz > 10 OR
  iv_final < 0 OR iv_final > 10.

" Kích hoạt Exception và thoát khỏi hàm ngay lập tức
RAISE invalid_score.
ENDIF.

" 2. Calculation Logic (Tính toán)
" Công thức: Lab 20%, Quiz 30%, Final 50%
ev_gpa = ( iv_lab * '0.2' ) + ( iv_quiz * '0.3' ) + ( iv_final * '0.5' ).

" 3. Ranking Logic (Xếp loại)
IF ev_gpa >= 9.
  ev_rank = 'Excellent (Xuất sắc)'.
ELSEIF ev_gpa >= 5.
  ev_rank = 'Pass (Đạt)'.
ELSE.
  ev_rank = 'Fail (Trượt)'.
ENDIF.

```

ENDFUNCTION.

B. Logic Chương trình chính (ZDEV331_LAB5_MODULARIZATION)

code Abap

```

*&-----*
*& Report ZDEV331_LAB5_MODULARIZATION

```

&-----

REPORT zdev331_lab5_modularization.

--- INPUT ---

PARAMETERS: pa_lab TYPE p DECIMALS 2 OBLIGATORY,
pa_quiz TYPE p DECIMALS 2 OBLIGATORY,
pa_final TYPE p DECIMALS 2 OBLIGATORY.

--- OUTPUT VARIABLES ---

DATA: gv_result_gpa TYPE p DECIMALS 2,
gv_result_rank TYPE string.

START-OF-SELECTION.

" Gọi Function Module

" Code này được sinh ra từ nút Pattern, sau đó ta điền biến vào

CALL FUNCTION 'ZFM_DEV331_CALC_GPA'

EXPORTING

iv_lab = pa_lab " Truyền tham số màn hình vào hàm

iv_quiz = pa_quiz

iv_final = pa_final

IMPORTING

ev_gpa = gv_result_gpa " Hứng kết quả từ hàm ra biến

ev_rank = gv_result_rank

EXCEPTIONS

invalid_score = 1 " Nếu hàm RAISE invalid_score, sy-subrc sẽ = 1

OTHERS = 2.

" Kiểm tra kết quả gọi hàm

IF sy-subrc = 0.

" --- Case 1: Thành công ---

WRITE: / '=== FINAL RESULT ===' COLOR 1.

```
WRITE: / 'Input Scores:', pa_lab, '|', pa_quiz, '|', pa_final.  
ULINE.  
WRITE: / 'GPA:', gv_result_gpa COLOR 5.  
WRITE: / 'Rank:', gv_result_rank COLOR 5.  
  
ELSEIF sy-subrc = 1.  
" --- Case 2: Lỗi dữ liệu (Invalid Score) ---  
MESSAGE 'Error: Input scores must be between 0 and 10!' TYPE 'I' DISPLAY LIKE 'E'.  
  
ELSE.  
" --- Case 3: Lỗi không xác định ---  
MESSAGE 'Unknown Error occurred during calculation.' TYPE 'E'.  
ENDIF.
```

3. Bốn (4) Thử thách phát triển (Deep Dive Challenges)

Sinh viên thực hiện 4 yêu cầu sau để hiểu sâu hơn về Modularization:

Challenge 1: Local Modularization (Subroutine)

- **Yêu cầu:** Trong chương trình chính, thay vì viết các lệnh WRITE trực tiếp, hãy tạo một Subroutine tên là FORM display_result.
- **Code:** PERFORM display_result USING gv_result_gpa gv_result_rank.
- **Mục tiêu:** Phân biệt được sự khác nhau giữa CALL FUNCTION (Gọi hàm bên ngoài) và PERFORM (Gọi thủ tục nội bộ).

Challenge 2: Mass Processing (Xử lý hàng loạt)

- **Yêu cầu:**
 1. Select dữ liệu từ bảng ZENROLL_DEV331 (đã tạo ở Buổi 4) vào Internal Table.
 2. Loop qua danh sách đó.
 3. Với mỗi dòng, gọi FM ZFM_DEV331_CALC_GPA để tính lại GPA và Rank (giả sử bảng Enroll có đủ điểm thành phần).

4. Cập nhật lại kết quả vào Internal Table.

- **Real-world:** Đây là mô phỏng quy trình "Batch Job" cuối kỳ để tính điểm cho toàn trường.

Challenge 3: Optional Parameters (Tham số tùy chọn)

- **Yêu cầu:** Sửa FM ZFM_.... Thêm tham số Import IV_STRICT (Type CHAR1, Default ' ', chọn Optional).
- **Logic:** Nếu IV_STRICT = 'X', thì ngưỡng điểm Pass phải là **6.0** thay vì **5.0**.
- **Mục tiêu:** Học cách sử dụng từ khóa IS SUPPLIED hoặc kiểm tra giá trị mặc định trong hàm.

Challenge 4: Changing Parameter (Tham số thay đổi)

- **Yêu cầu:** Tạo một FM mới tên ZFM_..._ROUND_SCORE.
- **Tab Changing:** Tham số CV_SCORE (Type P).
- **Logic:** Hàm nhận vào CV_SCORE (ví dụ 8.25), làm tròn nó (ví dụ thành 8.3 hoặc 8.5 tùy quy tắc) và trả về chính biến CV_SCORE.
- **Mục tiêu:** Hiểu cơ chế **Pass by Reference** (Thay đổi trực tiếp giá trị biến gốc).

4. Lời giải tham khảo (Solution Snippets)

Lời giải Challenge 1: Subroutine

code Abap

```
IF sy-subrc = 0.  
  PERFORM display_result USING gv_result_gpa gv_result_rank.  
ENDIF.
```

```
*--- Định nghĩa Subroutine ở cuối chương trình ---*  
FORM display_result USING pv_gpa TYPE p  
  pv_rank TYPE string.  
  WRITE: / '--- Report generated by Subroutine ---'.
```

```
WRITE: / 'GPA:', pv_gpa COLOR 5.
```

```
WRITE: / 'Rank:', pv_rank.
```

```
ENDFORM.
```

Lời giải Challenge 2: Mass Call (Giả định)

code Abap

```
" Code giả định trong vòng lặp
```

```
LOOP AT lt_enrollment INTO DATA(ls_row).
```

```
" Gọi hàm tính điểm cho từng sinh viên
```

```
CALL FUNCTION 'ZFM_DEV331_CALC_GPA'
```

```
EXPORTING
```

```
iv_lab = ls_row-lab_score
```

```
iv_quiz = ls_row-quiz_score
```

```
iv_final = ls_row-final_score
```

```
IMPORTING
```

```
ev_gpa = ls_row-gpa " Cập nhật GPA mới vào dòng hiện tại
```

```
EXCEPTIONS
```

```
OTHERS = 1.
```

```
MODIFY lt_enrollment FROM ls_row. " Lưu lại thay đổi vào bảng nội bộ  
ENDLOOP.
```

LỜI GIẢI CHI TIẾT CHALLENGE 3 & 4 (BUỔI 5)

Challenge 3: Optional Parameters (Tham số tùy chọn)

Yêu cầu: Sửa hàm tính điểm để thêm chế độ "Chăm gắt" (Strict Mode). Nếu bật chế độ này, điểm đạt (Pass) phải là 6.0 thay vì 5.0. Tham số này không bắt buộc nhập.

Bước 1: Cấu hình SE37 (Function Builder)

1. Vào SE37, mở hàm ZFM_DEV331_CALC_GPA, nhấn **Change** (Cây bút).

2. Chuyển sang tab **Import**.
3. Thêm tham số mới:
 - Parameter Name: IV_STRICT
 - Type: CHAR1
 - **Quan trọng:** Tích vào ô **Optional** (Cột Optional).
 - Default Value: ' ' (Dấu nháy đơn + khoảng trắng + dấu nháy đơn).
4. Lưu và kích hoạt lại.

Bước 2: Viết Logic trong Function Module

```
code Abap
FUNCTION zfm_dev331_calc_gpa.
*"-----
*"**"Local Interface:
*" IMPORTING
*"   VALUE(IV_LAB) TYPE P
*"   VALUE(IV_QUIZ) TYPE P
*"   VALUE(IV_FINAL) TYPE P
*"   VALUE(IV_STRICT) TYPE CHAR1 DEFAULT ' ' "<-- Tham số mới
*" EXPORTING
*"   VALUE(EV_GPA) TYPE P
*"   VALUE(EV_RANK) TYPE STRING
*" EXCEPTIONS
*"   INVALID_SCORE
*"-----

" ... (Phần validation giữ nguyên) ...

" Tính điểm GPA
```

```
ev_gpa = ( iv_lab * '0.2' ) + ( iv_quiz * '0.3' ) + ( iv_final * '0.5' ).
```

```
" --- LOGIC MỚI CHO STRICT MODE ---
```

```
DATA: lv_pass_threshold TYPE p DECIMALS 1.
```

```
" Kiểm tra xem người dùng có bật chế độ Strict không
```

```
IF iv_strict = 'X'.
```

```
    lv_pass_threshold = '6.0'. " Chấm gắt
```

```
ELSE.
```

```
    lv_pass_threshold = '5.0'. " Chấm thường
```

```
ENDIF.
```

```
" Xếp loại dựa trên ngưỡng điểm động
```

```
IF ev_gpa >= 9.
```

```
    ev_rank = 'Excellent'.
```

```
ELSEIF ev_gpa >= lv_pass_threshold. " So sánh với ngưỡng
```

```
    ev_rank = 'Pass'.
```

```
ELSE.
```

```
    ev_rank = 'Fail'.
```

```
ENDIF.
```

```
ENDFUNCTION.
```

Bước 3: Cách gọi trong chương trình chính (Program)

```
code Abap
```

```
downloadcontent_copy
```

```
expand_less
```

```
" Cách 1: Gọi bình thường (Không truyền IV_STRICT -> Mặc định là ' ')
```

```
CALL FUNCTION 'ZFM_DEV331_CALC_GPA'
```

```
EXPORTING
```

```
    iv_lab  = 5
```

```
    iv_quiz = 5
```

```
    iv_final = 5
```

```
IMPORTING
  ev_rank = gv_rank.
" -> Kết quả: Pass (Vì GPA 5.0 >= 5.0)

" Cách 2: Gọi chế độ Strict Mode
CALL FUNCTION 'ZFM_DEV331_CALC_GPA'
EXPORTING
  iv_lab    = 5
  iv_quiz   = 5
  iv_final  = 5
  iv_strict = 'X' " <--- Truyền tham số tùy chọn
IMPORTING
  ev_rank = gv_rank.
" -> Kết quả: Fail (Vì GPA 5.0 < 6.0)
```

Challenge 4: Changing Parameter (Tham số thay đổi)

Yêu cầu: Tạo hàm làm tròn điểm số. Hàm nhận vào một biến, sửa đổi giá trị của chính biến đó (Pass by Reference).

Bước 1: Cấu hình SE37

1. Tạo Function Module mới: ZFM_DEV331_ROUND_SCORE.
2. Chuyển sang tab **Changing**.
3. Thêm tham số:
 - Parameter Name: CV_SCORE
 - Type: P
 - Pass Value: **Bỏ tích** (Để dùng cơ chế tham chiếu Reference - Sửa trực tiếp vùng nhớ).

Bước 2: Viết Logic trong Function Module

code Abap

```

FUNCTION zfm_dev331_round_score.
*"-----
*"**"Local Interface:
*"  CHANGING
*"    REFERENCE(CV_SCORE) TYPE  P
*"-----

" Sử dụng hàm làm tròn có sẵn của ABAP 7.4+
" round( val = ... dec = ... mode = ... )
" Mode 2 = ROUND_UP (Làm tròn lên). Ví dụ: 8.21 -> 8.3

cv_score = round( val = cv_score dec = 1 mode = 2 ).

ENDFUNCTION.

```

Bước 3: Cách gọi trong chương trình chính

```

code Abap

DATA: gv_my_score TYPE p DECIMALS 2 VALUE '8.21'.

WRITE: / 'Before Rounding:', gv_my_score. " In ra 8.21

" Gọi hàm có tham số CHANGING
CALL FUNCTION 'ZFM_DEV331_ROUND_SCORE'
  CHANGING
    cv_score = gv_my_score. " Biến này sẽ bị thay đổi sau khi hàm chạy xong

WRITE: / 'After Rounding:', gv_my_score. " In ra 8.30 (Đã bị thay đổi)

```

Kiến thức bổ sung (Tips)

1. **IS SUPPLIED:**

Trong FM, bạn có thể dùng lệnh IF IS SUPPLIED(IV_STRICT). để kiểm tra xem người dùng có truyền tham số đó hay không (khác với việc kiểm tra giá trị mặc định).

2. **Changing vs Exporting:**

- Dùng EXPORTING khi bạn muốn tạo ra một giá trị mới (Input A -> Output B).
- Dùng CHANGING khi bạn muốn sửa đổi chính giá trị đầu vào (Input A -> Modified A).

Hy vọng phần bổ sung này giúp sinh viên hiểu rõ cơ chế tham số linh hoạt của Function Module!

Kết thúc Buổi 5.

Sinh viên đã nắm vững cách "đóng gói" logic vào Hàm. Code chương trình chính giờ đây rất gọn gàng, chỉ việc gọi hàm và xử lý kết quả.

Đây là nền tảng để bước vào thế giới **Lập trình hướng đối tượng (OOP)** ở buổi sau.

BUỔI 6: ABAP OBJECTS (CLASS & OBJECT)

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Global Class (Mới):** ZCL_[AccountID]_STUDENT (Ví dụ: ZCL_DEV331_STUDENT).
 - *Quy ước:* Class bắt đầu bằng ZCL_ hoặc ZCL_IM_ (Implementation).
- **Program:** Z[AccountID]_LAB6_OOP_INTRO
- **Chủ đề:** Mô hình hóa đối tượng Sinh viên (Data Encapsulation).

● B. REAL-WORLD TIPS (TƯ DUY CHUYÊN GIA)

1. SE24 (Global Class) vs. Local Class:

- Bạn có thể viết Class ngay trong chương trình (Local Class), NHƯNG trong dự án thực tế, 99% chúng ta dùng **T-Code SE24** để tạo **Global Class**.
- *Lý do:* Global Class có thể được gọi bởi bất kỳ chương trình nào, bất kỳ hệ thống nào (qua API), và dễ quản lý phiên bản.

2. Quy tắc đặt tên thuộc tính (Attributes Naming):

- mv_...: Member Variable (Biến instance - Dữ liệu riêng của từng đối tượng). Ví dụ: mv_student_id.
- ms_... / gv_...: Static Variable (Biến tĩnh - Dữ liệu chung cho tất cả đối tượng). Ví dụ: gv_university_name (Ai cũng học FPT).
- mc_...: Constant (Hằng số).

3. Tư duy "Encapsulation" (Đóng gói):

- Không bao giờ để thuộc tính là PUBLIC (Công khai). Hãy để PRIVATE (Riêng tư) để bảo vệ dữ liệu.
- Muốn sửa/đọc dữ liệu, người ngoài phải dùng phương thức Setter và Getter.

- *Ví dụ:* Không cho phép gán Age = -5. Trong method SET_AGE, ta viết code chặn số âm. Nếu để Public, ai cũng có thể gán -5 vào được.

C. MỤC TIÊU BÀI HỌC

1. **Concept:** Hiểu sự khác biệt giữa **Class** (Bản thiết kế/Khuôn mẫu) và **Object** (Ngôi nhà thực tế/Bánh nướng ra từ khuôn).
2. **SE24:** Sử dụng Class Builder để tạo Class, Attribute, Method.
3. **Coding:**
 - Viết code triển khai (Implementation) cho Method.
 - Viết chương trình tạo đối tượng (NEW) và gọi phương thức (->).

D. NỘI DUNG THỰC HÀNH (LAB 6): STUDENT OBJECT MODEL

Tình huống: Thay vì quản lý sinh viên bằng các dòng record vô tri trong bảng Database, ta xây dựng một "Đối tượng Sinh viên" thông minh. Đối tượng này biết tự giữ thông tin của mình và biết tự tính tuổi, tự giới thiệu bản thân.

1. Hướng dẫn chi tiết (Step-by-Step)

GIAI ĐOẠN 1: TẠO GLOBAL CLASS (SE24)

- **Bước 1:** Vào T-Code SE24 (Class Builder).
- **Bước 2:** Nhập tên Class: ZCL_DEV331_STUDENT (Thay ID của bạn). Nhấn **Create**.
 - Chọn *Class*. Nhấn Continue.
 - Description: Student Object for LMS.
 - Instantiation: Public (Cho phép ai cũng tạo được đối tượng).
 - Nhấn Save (Lưu vào Package).

- **Bước 3: Khai báo Thuộc tính (Tab Attributes)**

Khai báo các biến chứa dữ liệu của sinh viên.

- *Level*: Instance Attribute (Dữ liệu riêng).
- *Visibility*: Private (Bảo mật).

Attribute	Level	Visibility	Type	Associated Type	Description
MV_ID	Instance	Private	Type	CHAR8	Student ID
MV_NAME	Instance	Private	Type	STRING	Full Name
MV_DOB	Instance	Private	Type	DATS	Date of Birth

- **Bước 4: Khai báo Phương thức (Tab Methods)**

Khai báo các hành động mà sinh viên có thể làm.

Method	Level	Visibility	Description
SET_INFO	Instance	Public	Nạp thông tin cho SV (Setter)
GET_INFO	Instance	Public	Lấy thông tin ra hiển thị (Getter)
CALC_AGE	Instance	Public	Tính tuổi

- **Bước 5: Định nghĩa Tham số cho Method (Parameters)**

- Click vào tên method **SET_INFO**, nhấn nút **Parameters**:
 - IV_ID (Importing, Type CHAR8).
 - IV_NAME (Importing, Type STRING).
 - IV_DOB (Importing, Type DATS).
- Click vào tên method **GET_INFO**, nhấn nút **Parameters**:
 - EV_RESULT (Exporting, Type STRING).

- Click vào tên method CALC_AGE, nhấn nút **Parameters**:
 - RV_AGE (Returning, Type I). *Lưu ý: Returning chỉ trả về đúng 1 giá trị.*

GIẢI ĐOẠN 2: VIẾT CODE TRIỂN KHAI (IMPLEMENTATION)

- **Bước 6:** Vào tab **Methods**. Double-click vào từng tên Method để viết code ABAP.
 - **Method SET_INFO:**

```
code Abap
METHOD set_info.
" Gán dữ liệu từ tham số nhập vào (IV_) cho thuộc tính nội bộ (MV_)
me->mv_id = iv_id.
me->mv_name = iv_name.
me->mv_dob = iv_dob.
" Từ khóa 'me->' giống như 'this.' trong Java/C#, để chỉ chính đối tượng này
ENDMETHOD.
```

- **Method CALC_AGE:**

```
code Abap
METHOD calc_age.
" Tính tuổi: Năm hiện tại - Năm sinh
IF me->mv_dob IS NOT INITIAL.
    rv_age = sy-datum(4) - me->mv_dob(4).
ELSE.
    rv_age = 0.
ENDIF.
ENDMETHOD.
```

- **Method GET_INFO:**

```
code Abap
METHOD get_info.
DATA: lv_age TYPE i.
```

```
" Gọi method nội bộ để tính tuổi
lv_age = me->calc_age( ).
```

```
" Ghép chuỗi kết quả code ABAP 7.4
"CONCATENATE 'Student:' me->mv_name
"      '(' me->mv_id ')'
"      '- Age:' lv_age
"      INTO ev_result SEPARATED BY space.
```

```
" CODE ABAP 7.7
EV_RESULT = |Student: { ME->MV_NAME } ( { ME->MV_ID } ) - Age: { LV_AGE }|.
ENDMETHOD.
```

- **Bước 7:** Nhấn nút **Activate** (Ctrl+F3) cho toàn bộ Class (Chọn tất cả thành phần).

GIẢI ĐOẠN 3: SỬ DỤNG CLASS TRONG CHƯƠNG TRÌNH

- **Bước 8:** Vào SE80, tạo Program ZDEV331_LAB6_OOP_INTRO.

2. Code Mẫu ABAP (Program)

```
code Abap
*&-----*
*& Report ZDEV331 LAB6 OOP INTRO
*&-----*
REPORT zdev331_lab6_oop_intro.

*--- 1. KHAI BÁO BIẾN THAM CHIẾU (REFERENCE VARIABLES) ---*
" Biến này chỉ là cái 'Remote' điều khiển, chưa có đối tượng thực
DATA: lo_student_1 TYPE REF TO zcl_dev331_student,
      lo_student_2 TYPE REF TO zcl_dev331_student.

DATA: gv_msg TYPE string.

START-OF-SELECTION.
```

"--- 2. KHỞI TẠO ĐỐI TƯỢNG (INSTANTIATION) ---*

" Tạo một sinh viên mới (Object) trong vùng nhớ Heap

" Cú pháp mới (7.40+): NEW #()

lo_student_1 = NEW #(.).

lo_student_2 = NEW #(.).

"--- 3. GỌI PHƯƠNG THỨC (METHOD CALL) ---*

" Nạp thông tin cho SV 1

lo_student_1->set_info(

EXPORTING

iv_id = 'SE18001'

iv_name = 'Nguyen Van A'

iv_dob = '20040101'

).

" Nạp thông tin cho SV 2

lo_student_2->set_info(

EXPORTING

iv_id = 'SE18002'

iv_name = 'Tran Thi B'

iv_dob = '20050505'

).

"--- 4. HIỂN THỊ THÔNG TIN ---*

" Lấy thông tin SV 1

lo_student_1->get_info(

IMPORTING

ev_result = gv_msg

).

WRITE: / gv_msg.

```
" Lấy thông tin SV 2
lo_student_2->get_info(
    IMPORTING
    ev_result = gv_msg
).
WRITE: / gv_msg.
```

3. Bốn (4) Thử thách phát triển (Deep Dive Challenges)

Challenge 1: Constructor Method (Hàm khởi tạo)

- **Vấn đề:** Hiện tại ta phải gọi NEW xong mới gọi SET_INFO. Lỡ quên gọi SET_INFO thì đối tượng rỗng.
- **Yêu cầu:** Hãy dùng Method đặc biệt CONSTRUCTOR.
 - Vào SE24, tạo method tên là CONSTRUCTOR.
 - Chuyển các tham số của SET_INFO vào CONSTRUCTOR.
 - Copy code từ SET_INFO sang CONSTRUCTOR.
 - Xóa method SET_INFO.
- **Kết quả:** Khi code trong program, chỉ cần viết:
lo_student_1 = NEW #(iv_id = '...' iv_name = '...'). (Gọn hơn và bắt buộc phải nhập liệu).

Challenge 2: Static Attribute (Biến tĩnh)

- **Yêu cầu:** Khai báo một Attribute mới GV_UNIVERSITY trong Class.
 - Level: **Static Attribute** (Quan trọng).
 - Visibility: Public.
 - Type: String.
 - Gán giá trị mặc định là 'FPT University'.

- **Code Program:** Thử thay đổi giá trị này: `zcl_dev331_student=>gv_university = 'FPT Polytechnic'`.
- **Quan sát:** Kiểm tra xem *tất cả* các đối tượng sinh viên 1 và 2 có bị đổi tên trường theo không? (Đáp án: Có, vì Static là dùng chung).

Challenge 3: Functional Method (Cú pháp hiện đại)

- **Yêu cầu:** Method `CALC_AGE` đang trả về Returning. Hãy gọi nó trực tiếp trong lệnh `WRITE` mà không cần biến trung gian.
- **Code mẫu:** `WRITE: / 'Tuổi SV1 là:', lo_student_1->calc_age( ).`

Challenge 4: Object Array (Danh sách đối tượng)

- **Yêu cầu:** Thay vì khai báo lẻ `lo_student_1`, `lo_student_2`. Hãy khai báo một Internal Table chứa các đối tượng.
- **Code gợi ý:**

```
code Abap
DATA: lt_students TYPE TABLE OF REF TO zcl_dev331_student.
" Loop danh sách từ DB, với mỗi dòng tạo 1 Object, Append Object vào bảng lt_students.
```

4. Lời giải tham khảo (Solution Snippets)

Lời giải Challenge 1: Constructor (Trong Program)

```
code Abap
" Khởi tạo kèm nạp dữ liệu ngay lập tức
lo_student_1 = NEW #( iv_id = 'SE01' iv_name = 'A' iv_dob = '20000101' ).
```

Lời giải Challenge 2: Static Attribute Usage

```
code Abap
" Truy cập biến tĩnh dùng dấu =>
WRITE: / 'Trường học:', zcl_dev331_student=>gv_university.

" Đổi tên trường
zcl_dev331_student=>gv_university = 'FPT AI Center'.
```

```
" Cả 2 sinh viên đều thuộc trường mới  
WRITE: / 'SV1 học tại:', zcl_dev331_student=>gv_university.
```

Lời giải Challenge 4: Object Array

```
code Abap  
DATA: lt_list TYPE TABLE OF REF TO zcl_dev331_student.  
DATA: lo_temp TYPE REF TO zcl_dev331_student.  
  
" Tạo 10 sinh viên  
DO 10 TIMES.  
  lo_temp = NEW #( iv_id = '...' ... ). " Giả sử dùng Constructor  
  APPEND lo_temp TO lt_list.  
ENDDO.  
  
" Duyệt danh sách để in  
LOOP AT lt_list INTO lo_temp.  
  WRITE: / lo_temp->calc_age( ).  
ENDLOOP.
```

Kết thúc Buổi 6.

Sinh viên đã bước chân vào thế giới OOP. Họ hiểu rằng đối tượng bao gồm Dữ liệu (Attributes) và Hành vi (Methods). Các biến mv_ và iv_ không còn xa lạ nữa.

Sinh viên đã bước chân vào thế giới OOP. Họ đã tạo được "Khuôn mẫu" (Class) và "Sản phẩm" (Object).

BUỔI 7: ABAP OBJECTS NÂNG CAO (INHERITANCE & POLYMORPHISM)

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Interface (Mối):** ZIF_[AccountID]_PRINT (Giao diện in ấn/Log).
- **Parent Class (Lớp Cha):** ZCL_[AccountID]_PERSON (Người).
- **Child Class 1 (Lớp Con):** ZCL_[AccountID]_STUDENT (Sinh viên).
- **Child Class 2 (Lớp Con):** ZCL_[AccountID]_LECTURER (Giảng viên).
- **Program:** Z[AccountID]_LAB7_INHERITANCE.

● B. LOGIC TƯ DUY (QUAN TRỌNG)

1. Inheritance (Kế thừa - "IS-A"):

- Sinh viên **LÀ** một Người. Giảng viên **LÀ** một Người.
- Thay vì khai báo ID, Name ở cả 2 nơi, ta đưa nó lên lớp cha PERSON. Lớp con tự động hưởng thụ.

2. Protected vs Private:

- Private: Chỉ mình dùng. Con không được dùng.
- Protected: Mình dùng và **Con được dùng**. (Đây là mấu chốt của bài này).
- Public: Ai cũng dùng được.

3. Polymorphism (Đa hình):

- Cùng một lệnh get_info(), nhưng:
 - Đối tượng Sinh viên trả về: "Tên + Điểm GPA".

- Đối tượng Giảng viên trả về: "Tên + Lương".
- -> Kỹ thuật này gọi là **Redefinition** (Định nghĩa lại phương thức).

C. NỘI DUNG THỰC HÀNH (LAB 7): HỆ THỐNG QUẢN LÝ NHÂN SỰ LMS

Mục tiêu: Xây dựng cấu trúc Class phân cấp.

- **Person:** ID, Tên (Dùng chung).
- **Student:** Thêm GPA, Xếp loại.
- **Lecturer:** Thêm Salary (Lương), Department (Khoa).

GIAI ĐOẠN 1: TẠO INTERFACE (HỢP ĐỒNG GIAO TIẾP)

Interface trong ABAP giống như một bản cam kết. Class nào dùng Interface này thì bắt buộc phải có tính năng đó.

- **Bước 1:** SE24 -> ZIF_DEV331_PRINT -> Create -> Chọn **Interface**.
- **Bước 2:** Tab **Methods**:
 - Method: PRINT_LOG.
 - Level: Instance.
 - Description: In nhật ký hoạt động ra màn hình.
- **Bước 3:** Save & Activate.

GIAI ĐOẠN 2: TẠO LỚP CHA (ZCL_PERSON)

- **Bước 1:** SE24 -> ZCL_DEV331_PERSON -> Create Class.
- **Bước 2:** Tab **Interfaces**:
 - Nhập ZIF_DEV331_PRINT. Nhấn Enter. (Lớp Cha dùng Interface, các lớp Con sẽ tự động có).

- **Bước 3:** Tab **Attributes** (Lưu ý cột Visibility):

Attribute	Level	Visibility	Type	Type Name	Description
MV_ID	Instance	Protected	Type	CHAR8	ID (Chung)
MV_NAME	Instance	Protected	Type	STRING	Tên (Chung)

- **Bước 4:** Tab **Methods**:

Method	Level	Visibility	Description
SET_CORE_INFO	Instance	Public	Nạp ID và Tên
GET_INFO	Instance	Public	Lấy thông tin (Sẽ bị ghi đè)
ZIF_DEV331_PRINT~PRINT_LOG	Instance	Public	(Tự hiện do add Interface)

- **Bước 5:** Define Parameters & Code Implementation:

- **Method SET_CORE_INFO:**

- Params: IV_ID (Importing ,Char8), IV_NAME (String).
- Code:

```
code Abap
METHOD set_core_info.
  me->mv_id = iv_id.
  me->mv_name = iv_name.
ENDMETHOD.
```

- **Method GET_INFO:**

- Params: RV_TEXT (Returning, String).
- Code (Dùng String Template):

```
code Abap
```

```
METHOD get_info.  
  rv_text = |ID: { me->mv_id } - Name: { me->mv_name }|.  
ENDMETHOD.
```

- **Method ZIF_DEV331_PRINT~PRINT_LOG:**

- Double click vào tên method này trong tab Methods để viết code.
- Code:

```
code Abap  
METHOD zif_dev331_print~print_log.  
  " In ra màn hình console/report  
  WRITE: / 'LOG ENTRY:', me->mv_name, 'accessed system at', sy-uzeit.  
ENDMETHOD.
```

- **Bước 6:** Activate Class Person.

GAIAI ĐOẠN 3: TẠO LỚP CON 1 - SINH VIÊN (INHERITANCE)

- **Bước 1:** SE24 -> ZCL_DEV331_STUDENT -> Create Class.
- **Bước 2 (Quan trọng):** Tại popup tạo, hoặc tab **Properties**, tìm nút **Superclass**.
 - Nhập: ZCL_DEV331_PERSON.
 - Nhấn Save. Bạn sẽ thấy tab Attributes tự động có MV_ID, MV_NAME (màu xanh dương - do kế thừa).
- **Bước 3:** Tab **Attributes** (Thêm cái riêng của SV):
 - MV_GPA | Instance | Private | Type P Length 4 Decimals 2.
- **Bước 4:** Tab **Methods**:
 - Tạo mới: SET_ACADEMIC_INFO (Param: IV_GPA).
 - **Redefine (Ghi đè):**
 - Tìm method GET_INFO (được kế thừa từ Person).

- Đặt con trỏ vào nó. Nhấn nút **Redefine** trên thanh công cụ (Icon tờ giấy có mũi tên đỏ).

- **Bước 5: Code Implementation:**

- **Method SET_ACADEMIC_INFO:**

```
code Abap
METHOD set_academic_info.
    me->mv_gpa = iv_gpa.
ENDMETHOD.
```

- **Method GET_INFO (Redefined):**

```
code Abap
METHOD get_info.
    " 1. Gọi logic cũ của lớp Cha (để lấy ID và Name)
    DATA(lv_base_info) = super->get_info( ).

    " 2. Nối thêm thông tin riêng của Con (GPA)
    rv_text = |{ lv_base_info } - GPA: { me->mv_gpa }|.
ENDMETHOD.
```

- **Bước 6: Activate Class Student.**

GIAI ĐOẠN 4: TẠO LỚP CON 2 - GIẢNG VIÊN

- Làm tương tự bước trên với Class ZCL_DEV331_LLECTURER.
- Superclass: ZCL_DEV331_PERSON.
- Attribute riêng: MV_SALARY (Type P Decimals 2).
- Method mới: SET_JOB_INFO (Param: IV_SALARY).
- **Redefine GET_INFO:**

```
code Abap
METHOD get_info.
    DATA(lv_base_info) = super->get_info( ).
```

```
" Giảng viên hiền thị Lương
rv_text = |{ lv_base_info } - Salary: { me->mv_salary } USD|.
ENDMETHOD.
```

- Activate.

GIẢI ĐOẠN 5: CHƯƠNG TRÌNH KIỂM THỬ (POLYMORPHISM DEMO)

Đây là nơi điều kỳ diệu xảy ra. Chúng ta dùng biến kiểu CHA để chứa đối tượng CON.

- **Program:** ZDEV331_LAB7_INHERITANCE.

```
code Abap
*&-----*
*& Report ZDEV331_LAB7_INHERITANCE
*&-----*
REPORT zdev331_lab7_inheritance.

" 1. Khai báo biến
DATA: lo_person TYPE REF TO zcl_dev331_person, " Biến cha
      lo_student TYPE REF TO zcl_dev331_student, " Biến con SV
      lo_lecturer TYPE REF TO zcl_dev331_lecturer. " Biến con GV

" Bảng chứa danh sách hỗn hợp (Polymorphic Table)
DATA: lt_people TYPE TABLE OF REF TO zcl_dev331_person.

START-OF-SELECTION.

" --- TẠO SINH VIÊN ---
lo_student = NEW #( ).
" Gọi method của Cha
lo_student->set_core_info( iv_id = 'SV01' iv_name = 'Nguyen Van Student' ).
" Gọi method riêng của Con
lo_student->set_academic_info( iv_gpa = '8.5' ).
```

" Thêm vào danh sách (Cha chứa được Con - Upcasting)
APPEND lo_student TO lt_people.

" --- TẠO GIẢNG VIÊN ---
lo_lecturer = NEW #(
lo_lecturer->set_core_info(iv_id = 'GV01' iv_name = 'Thay Le Lecturer').
lo_lecturer->set_job_info(iv_salary = '2000').

" Thêm vào danh sách
APPEND lo_lecturer TO lt_people.

" --- LOOP & ĐA HÌNH (POLYMORPHISM) ---
WRITE: / '--- DANH SÁCH NHÂN SỰ (HỖN HỢP) ---' COLOR 1.

LOOP AT lt_people INTO lo_person.
" Ở đây lo_person là kiểu Person (Cha).
" Nhưng khi chạy, nó tự biết mình đang chứa SV hay GV để gọi hàm GET INFO tương ứng.

WRITE: / lo_person->get_info().

" Gọi Interface Method (Cả 2 đều có)
lo_person->zif_dev331_print~print_log().

ULINE.
ENDLOOP.

D. THỬ THÁCH NÂNG CAO (CHALLENGES)

Sinh viên cần hoàn thành 4 nhiệm vụ sau để thực sự làm chủ Kế thừa và Đa hình.

Challenge 1: Abstract Class (Lớp Trừu tượng)

- **Vấn đề:** Trong thực tế quản lý nhân sự, không có ai chỉ đơn thuần là "Person". Họ phải có vai trò cụ thể (Sinh viên hoặc Giảng viên). Việc cho phép tạo đối tượng NEW zcl_dev331_person() là sai về mặt thiết kế hệ thống.
- **Yêu cầu:**
 1. Vào SE24, mở class ZCL_DEV331_PERSON.
 2. Vào tab **Properties**, tích chọn ô **Abstract**.
 3. Lưu và Active.
- **Kiểm thử:** Quay lại chương trình ZDEV331_LAB7_INHERITANCE. Dòng code lo_person = NEW #() sẽ báo lỗi cú pháp (Syntax Error).
 - *Kết luận:* Class Trừu tượng chỉ dùng để làm cha (khuôn mẫu), không thể tạo đối tượng trực tiếp.

Challenge 2: Constructor Chain (Chuỗi khởi tạo)

- **Vấn đề:** Hiện tại ta phải gọi set_core_info sau khi NEW. Ta muốn bắt buộc nhập ID/Name ngay khi khởi tạo.
- **Yêu cầu:**
 1. **Lớp Cha (Person):** Tạo method CONSTRUCTOR với tham số IV_ID và IV_NAME. Viết code gán dữ liệu.
 2. **Lớp Con (Student):** Tạo method CONSTRUCTOR với tham số IV_ID, IV_NAME và IV_GPA.
 3. **Mẫu chốt:** Trong code Constructor của Student, bạn **BẮT BUỘC** phải gọi super->constructor(...) để đẩy ID và Name lên cho lớp cha xử lý, sau đó mới tự xử lý GPA.
- **Mục tiêu:** Hiểu thứ tự khởi tạo: Cha chạy trước -> Con chạy sau.

Challenge 3: Downcasting & Type Inspection (Ép kiểu xuống)

- **Tình huống:** Bạn có một danh sách hỗn hợp lt_people (Kiểu Person). Bạn muốn tính tổng lương, nhưng chỉ có Giảng viên mới có lương, Sinh viên thì không.

- **Yêu cầu:**

1. Duyệt vòng lặp LOOP AT lt_people INTO lo_person.
2. Kiểm tra: Nếu đối tượng hiện tại **LÀ** Giảng viên (Dùng IS INSTANCE OF).
3. Thì **Ép kiểu** (Cast) biến lo_person sang kiểu ZCL_..._LECTURER để truy cập thuộc tính MV_SALARY.
4. Cộng dồn lương và in ra tổng.

- **Mục tiêu:** Hiểu kỹ thuật RTTI (Run Time Type Identification) và Downcasting (?=).

Challenge 4: Independent Interface Implementation

- **Tình huống:** Interface ZIF_..._PRINT (Ghi log) không chỉ dành cho con người. Một "Máy chủ" (Server) hoặc "Xe cộ" (Car) cũng cần ghi log.

- **Yêu cầu:**

1. Tạo một Class mới hoàn toàn: ZCL_DEV331_SERVER (Không kế thừa Person).
2. Class này cũng Implement interface ZIF_DEV331_PRINT.
3. Trong chương trình chính, tạo một bảng nội bộ **chỉ chứa Interface**: DATA: lt_loggable TYPE TABLE OF REF TO zif_dev331_print.
4. Thêm Student, Lecturer, và Server vào bảng này. Duyệt vòng lặp và gọi print_log().

- **Mục tiêu:** Hiểu bản chất Interface: Cho phép xử lý chung các đối tượng không liên quan gì đến nhau (Khác huyết thống).

💡 E. LỜI GIẢI THAM KHẢO (SOLUTION SNIPPETS)

Dưới đây là gợi ý code (Snippets) để sinh viên tham khảo khi gặp khó khăn.

Solution Challenge 2: Constructor Chain (Trong Class Student)

code Abap

METHOD constructor.

" 1. Gọi constructor của lớp cha để xử lý các phần chung (Bắt buộc phải nằm dòng đầu)

```
super->constructor(  
    EXPORTING  
        iv_id   = iv_id  
        iv_name = iv_name  
    ).
```

" 2. Xử lý phần riêng của con

```
me->mv_gpa = iv_gpa.
```

ENDMETHOD.

Cách gọi trong Program:

```
lo_student = NEW #( iv_id = 'SV1' iv_name = 'Teo' iv_gpa = '9.0' ).
```

Solution Challenge 3: Downcasting (Trong Program)

code Abap

```
DATA: lv_total_salary TYPE p DECIMALS 2.
```

```
DATA: lo_temp_lecturer TYPE REF TO zcl_dev331_lecturer.
```

```
LOOP AT lt_people INTO lo_person.
```

" Kiểm tra xem đối tượng Cha này thực chất có phải là Giảng viên không?

```
IF lo_person IS INSTANCE OF zcl_dev331_lecturer.
```

" Ép kiểu từ Cha (Person) xuống Con (Lecturer) để lấy được Lương

" Cú pháp: ?= (Casting Operator)

```
lo_temp_lecturer ?= lo_person.
```

" Bây giờ mới truy cập được thuộc tính Salary (vốn Person không có)

```
lv_total_salary = lv_total_salary + lo_temp_lecturer->mv_salary.
```

```
ENDIF.
```

ENDLOOP.

WRITE: / 'Tổng lương giảng viên:', lv_total_salary.

Solution Challenge 4: Interface List

code Abap

```
" Khai báo bảng chứa bất kỳ thứ gì biết in (biết Print)
DATA: lt_log_objects TYPE TABLE OF REF TO zif_dev331_print.
DATA: lo_server TYPE REF TO zcl_dev331_server.

" Tạo server
lo_server = NEW #( ).

" Thêm các đối tượng khác nhau vào cùng 1 danh sách
APPEND lo_student TO lt_log_objects. " Student có Interface Print
APPEND lo_server TO lt_log_objects. " Server cũng có Interface Print

" Duyệt và in
LOOP AT lt_log_objects INTO DATA(lo_item).
  lo_item->print_log( ).
  " Student in: Log Entry: Nguyen Van A...
  " Server in: Log Entry: Server Main started...
ENDLOOP.
```

✅ Checklist Verification (Đã kiểm tra mã nguồn)

1. **Protected Attribute:** Đã sử dụng Protected trong ZCL_PERSON. Nếu để Private, phương thức GET_INFO của lớp con sẽ báo lỗi cú pháp khi truy cập me->mv_name.
2. **String Template:** Code sử dụng |...| thay vì CONCATENATE, đảm bảo hoạt động tốt với các kiểu dữ liệu số (GPA, Salary).
3. **Method Redefinition:** Đã hướng dẫn thao tác nút "Redefine" trong SE24. Nếu tạo method mới cùng tên sẽ bị lỗi "Method already exists".

4. **Interface Call:** Cú pháp gọi method interface là ->interface~method(). Code mẫu đã tuân thủ đúng.

Bài Lab này đảm bảo sinh viên hiểu rõ sức mạnh của OOP: Viết code ở lớp Cha 1 lần, tái sử dụng ở N lớp Con, và xử lý đa hình linh hoạt.

Kết thúc Buổi 7.

Sinh viên giờ đã hiểu cách xây dựng cấu trúc hướng đối tượng chặt chẽ.

BUỔI 8: LOCK CONCEPT (KHÓA DỮ LIỆU) & EXCEPTIONS (XỬ LÝ LỖI)

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
 - **Lock Object (Mới):** EZSTUDENT_[AccountID] (Ví dụ: EZSTUDENT_DEV331).
 - Lưu ý: Tên Lock Object trong SE11 bắt buộc phải bắt đầu bằng chữ E (Exclusive).
 - **Exception Class (Mới):** ZCX_LMS_ERROR_[AccountID].
 - **Program:** Z[AccountID]_LAB8_LOCKING
 - **Chủ đề:** Cập nhật điểm số an toàn (Concurrency Control).
-

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Chuyện gì xảy ra ở dự án tỷ đô nếu quên Lock?

1. SAP Locking

2. ≠ Database Locking:

- SAP không dùng khóa của Database (SQL Lock) vì nó quá ngắn (chỉ tồn tại trong 1 câu lệnh).
- SAP dùng **Logical Lock** (Lock Object) quản lý bởi **Enqueue Server**. Nó tồn tại suốt phiên làm việc của người dùng (từ lúc mở màn hình sửa đến lúc bấm Save).
- Quy tắc: Trước khi cho phép user sửa (Edit Mode), phải gọi hàm ENQUEUE. Sau khi Lưu xong, phải gọi DEQUEUE.

3. Đừng dùng CATCH SYSTEM-EXCEPTIONS (Cổ lỗ sĩ):

- ❌ Cũ: CATCH SYSTEM-EXCEPTIONS arithmetic_errors = 1...
- ✅ Mới (7.40+): Dùng **Class-based Exceptions** (TRY... CATCH cx_sy_zerodivide). Nó cho phép bạn bắt lỗi chính xác và truy vết (trace) nguyên nhân sâu xa.

4. Lock Mode E vs. S:

- **E (Exclusive - Write Lock):** Chỉ MỘT người được giữ khóa. Người khác không được đọc hay viết. (Dùng khi sửa dữ liệu).
- **S (Shared - Read Lock):** Nhiều người được đọc, nhưng không ai được sửa.

C. MỤC TIÊU BÀI HỌC

1. **SE11 (Lock Object):** Tạo đối tượng khóa cho bảng Sinh viên.
2. **Enqueue/Dequeue:** Gọi các Function Module sinh ra tự động để khóa/mở khóa.
3. **Class-based Exceptions:** Sử dụng cấu trúc TRY...CATCH để xử lý lỗi Runtime (ví dụ: chia cho 0) và lỗi nghiệp vụ (Custom Exception).

D. NỘI DUNG THỰC HÀNH (LAB 8): SECURE GRADE UPDATE

Tình huống: Xây dựng chức năng cập nhật điểm cho Sinh viên.

1. Người dùng nhập ID Sinh viên và Điểm mới.
2. Hệ thống kiểm tra xem có ai đang sửa sinh viên này không?
 - Nếu có: Báo lỗi "Sinh viên đang bị khóa bởi user X".
 - Nếu không: Khóa sinh viên -> Cập nhật điểm -> Mở khóa.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo Lock Object (SE11)

- Gõ T-Code SE11. Chọn **Lock Object**.
- Tên: EZSTUDENT_DEV331 (Bắt đầu bằng E, Z, Student, ID). Nhấn Create.
- *Short text:* Lock for Student Master Data.

- **Tab Tables:**
 - Name: ZSTUDENT_DEV331 (Bảng chính cần khóa).
 - Lock mode: **Write Lock** (E).
- **Tab Lock parameter:** Hệ thống tự động điền khóa chính (MANDT, STUDENT_ID).
- **Save & Activate.**
- *Lưu ý:* Khi Activate, SAP tự động sinh ra 2 Function Modules:
 - ENQUEUE_EZSTUDENT_DEV331 (Đề khóa).
 - DEQUEUE_EZSTUDENT_DEV331 (Đề mở).

Bước 2: Tạo Program Update Điểm

- Tạo ZDEV331_LAB8_LOCKING.
- Thiết kế màn hình nhập Mã SV và Điểm mới.

Bước 3: Viết Logic (Try-Lock-Update-Unlock)

- Gọi ENQUEUE....
- Kiểm tra sy-subrc.
- Nếu thành công -> UPDATE -> DEQUEUE.

2. Code Mẫu (Chuẩn Convention)

```
code Abap
*&-----*
*& Report ZDEV331_LAB8_LOCKING
*&-----*
REPORT zdev331_lab8_locking.
```

--- 1. INPUT PARAMETERS ---

PARAMETERS: pa_id TYPE char8 OBLIGATORY, " Student ID
pa_score TYPE p DECIMALS 2 OBLIGATORY. " New Score

START-OF-SELECTION.

" 2. Cố gắng khóa đối tượng Sinh viên (Locking)

" Gọi FM ENQUEUE được sinh ra từ SE11

CALL FUNCTION 'ENQUEUE_EZSTUDENT_DEV331'

EXPORTING

mode_zstudent_dev331 = 'E' " E = Exclusive (Khóa độc quyền)

student_id = pa_id " Khóa chính xác sinh viên này

EXCEPTIONS

foreign_lock = 1 " Lỗi: Đã có người khác khóa rồi

system_failure = 2

OTHERS = 3.

IF sy-subrc <> 0.

" 3. Xử lý khi KHÔNG khóa được (Exception Handling cơ bản)

" sy-msgv1 thường chứa tên User đang giữ khóa

MESSAGE ID sy-msgid TYPE 'T' NUMBER sy-msgno

WITH sy-msgv1 DISPLAY LIKE 'E'.

WRITE: / 'Error: Student is currently locked by another user.'.

RETURN. " Dừng chương trình

ENDIF.

" 4. Khu vực an toàn (Critical Section)

" Tại đây, chỉ mình ta được quyền sửa sinh viên pa_id

WRITE: / 'Lock acquired successfully.'.

TRY.

" Giả lập tính toán phức tạp có thể gây lỗi

```
IF pa_score < 0 OR pa_score > 10.  
    " Tự ném ra một lỗi (Business Exception)  
    RAISE EXCEPTION TYPE cx_sy_range_out_of_bounds.  
ENDIF.
```

```
" Thực hiện Update Database (Giả sử bảng ZENROLL có cột SCORE)  
UPDATE zenroll_dev331  
    SET score = @pa_score  
    WHERE student_id = @pa_id.
```

```
IF sy-subrc = 0.  
    WRITE: / 'Database updated successfully with score:', pa_score.  
    COMMIT WORK. " Xác nhận lưu xuống DB  
ELSE.  
    WRITE: / 'Student has not registered any course to update.'  
ENDIF.
```

```
CATCH cx_sy_range_out_of_bounds.  
    " Bắt lỗi điểm số không hợp lệ  
    MESSAGE 'Score must be between 0 and 10!' TYPE 'I' DISPLAY LIKE 'E'.
```

```
CATCH cx_root INTO DATA(lx_root).  
    " Bắt tất cả các lỗi khác (Catch-all)  
    WRITE: / 'Runtime Error:', lx_root->get_text( ).  
ENDTRY.
```

```
" 5. Mở khóa (Unlocking) - BẮT BUỘC PHẢI LÀM  
CALL FUNCTION 'DEQUEUE_EZSTUDENT_DEV331'  
EXPORTING  
    mode_zstudent_dev331 = 'E'  
    student_id            = pa_id.
```

WRITE: / 'Lock released.'.

3. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: Test "Foreign Lock" (Thử nghiệm khóa chéo)

- **Vấn đề:** Làm sao biết code khóa có hoạt động không nếu chỉ có 1 mình mình code?
- **Yêu cầu:**
 1. Mở 2 cửa sổ SAP GUI (Session).
 2. Cửa sổ 1: Chạy chương trình, thêm lệnh WAIT UP TO 15 SECONDS. vào giữa Enqueue và Dequeue.
 3. Cửa sổ 2: Trong lúc cửa sổ 1 đang chạy (đang Wait), chạy chương trình với cùng Mã SV.
- **Kết quả:** Cửa sổ 2 phải báo lỗi "Locked by user...".

Challenge 2: Custom Exception Class

- **Yêu cầu:** Thay vì dùng cx_sy_... có sẵn. Hãy vào SE24 tạo Exception Class riêng: ZCX_LMS_INVALID_SCORE.
 - Kế thừa từ: CX_STATIC_CHECK hoặc CX_NO_CHECK.
 - Trong tab Texts, định nghĩa thông báo: "Điểm số &1 không hợp lệ".
- **Code:** RAISE EXCEPTION TYPE zcx_lms_invalid_score EXPORTING...

Challenge 3: Lock All (Khóa toàn bộ bảng)

- **Tình huống:** Khi bảo trì hệ thống, Admin muốn khóa **tất cả** sinh viên, không cho ai sửa bất cứ ai.
- **Yêu cầu:** Gọi hàm ENQUEUE nhưng không truyền student_id (để trống).
- **Kết quả:** Toàn bộ bảng ZSTUDENT bị khóa (Table-level lock). Thử chạy session khác để sửa một SV bất kỳ xem có được không.

Challenge 4: Error Logging (Ghi log lỗi)

- **Yêu cầu:** Trong khối CATCH, không chỉ in ra màn hình. Hãy viết một đoạn code (hoặc gọi Function) để ghi lỗi vào một bảng Log riêng (ZLOG_ERROR).
 - **Mục tiêu:** Học cách lưu vết lỗi phục vụ debug sau này.
-

4. Lời giải tham khảo (Solution Snippets)

A. Challenge 1 (Test Locking)

```
code Abap
" Thêm vào giữa đoạn Enqueue và Dequeue
WRITE: / 'Processing update (Simulating 15s delay)...'.
WAIT UP TO 15 SECONDS.
" Trong 15s này, hãy nhanh tay mở cửa sổ khác chạy thử.
```

B. Challenge 2 (Custom Exception)

SE24: Tạo ZCX_LMS_ERROR_DEV331. Superclass: CX_STATIC_CHECK.

Code ABAP:

```
code Abap
TRY.
    IF pa_score < 0.
        " Ném lỗi tự định nghĩa
        RAISE EXCEPTION TYPE zcx_lms_error_dev331.
    ENDIF.

    CATCH zcx_lms_error_dev331 INTO DATA(lx_my_error).
        " Hiển thị thông báo lỗi
        MESSAGE lx_my_error TYPE 'I' DISPLAY LIKE 'E'.
    ENDTRY.
```

Kết thúc Buổi 8.

Sinh viên đã nắm được vũ khí quan trọng nhất để bảo vệ dữ liệu: **Lock Objects** và cách xử lý tình huống xấu bằng **Exceptions**.

BUỔI 9: DYNAMIC PROGRAMS & BÁO CÁO NÂNG CAO (ALV)

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Program:** Z[AccountID]_LAB9_DYNAMIC_REP
- **Công nghệ hiển thị:** SALV (SAP List Viewer - Factory Method) - *Chuẩn mực báo cáo hiện đại.*
- **Chủ đề:** Báo cáo Tổng hợp Học vụ (LMS Academic Dashboard).

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Bí kíp làm báo cáo "Sếp gật đầu ngay":

1. Quên lệnh WRITE đi:

- Trong môi trường doanh nghiệp, **không ai dùng WRITE để làm báo cáo nữa** (trừ log đơn giản).
- Hãy dùng **ALV (ABAP List Viewer)**. Nó cung cấp sẵn các nút: Sort, Filter, Export to Excel, Print, Change Layout... mà bạn không cần code một dòng nào.

2. SALV vs. ALV Grid:

- **ALV Grid (CL_GUI_ALV_GRID):** Mạnh mẽ, cho phép sửa dữ liệu trực tiếp (Edit), nhưng code phức tạp (cần tạo Screen, Container).
- **SALV (CL_SALV_TABLE):** Sinh ra sau, code cực ngắn (dùng Factory Method), giao diện đẹp, nhưng mặc định là Read-Only. **Chúng ta sẽ dùng cái này cho Lab hôm nay vì nó nhanh và hiệu quả.**

3. Field Symbols (<FS>) - Con trỏ thần thánh:

- Khi làm việc với Dynamic (không biết trước tên bảng hay tên cột), bạn không thể dùng biến thường. Bạn phải dùng FIELD-SYMBOLS. Nó giống như con trỏ (Pointer) trong C, trỏ trực tiếp vào vùng nhớ. Nhanh hơn và linh hoạt hơn.

C. MỤC TIÊU BÀI HỌC

1. **Selection Screen:** Sử dụng SELECT-OPTIONS để tạo bộ lọc đa năng (Khoảng giá trị, loại trừ...).
2. **Dynamic SQL:** Kỹ thuật SELECT ... FROM (variable_table_name).
3. **SALV Table:** Hiển thị dữ liệu lên lưới ALV chỉ với 3 dòng code.
4. **Field Symbols:** Duyệt bảng nội bộ bằng con trỏ để tăng tốc độ.

D. NỘI DUNG THỰC HÀNH (LAB 9): INTERACTIVE SEMESTER REPORT

Tình huống: Ban Giám hiệu cần một báo cáo liệt kê tất cả sinh viên và điểm số. Yêu cầu:

- Có thể lọc theo nhiều Mã SV cùng lúc.
- Có thể lọc theo khoảng điểm (Ví dụ: từ 0 đến 5).
- Kết quả hiển thị dạng bảng, có thể xuất ra Excel để nộp về Bộ GD&ĐT.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo Program & Selection Screen

- Tạo ZDEV331_LAB9_DYNAMIC_REP.
- Thay vì PARAMETERS, dùng SELECT-OPTIONS.
 - s_std cho Student ID.
 - s_scr cho Score.

Bước 2: Lấy dữ liệu (Data Retrieval)

- Join bảng ZSTUDENT và ZENROLL.

- Sử dụng INTO TABLE kết hợp CORRESPONDING FIELDS.

Bước 3: Hiện thị bằng ALV (SALV Factory)

- Dùng class cl_salv_table=>factory.
- Gọi method display().

2. Code Mẫu (Chuẩn Convention & Modern ABAP)

code Abap

```
*&-----*
*& Report ZDEV331_LAB9_DYNAMIC_REP
*&-----*
```

REPORT zdev331_lab9_dynamic_rep.

--- 1. DATA DECLARATION ---

" Định nghĩa cấu trúc dữ liệu cho báo cáo

```
TYPES: BEGIN OF ty_report,
    student_id TYPE zstudent_dev331-student_id,
    full_name  TYPE zstudent_dev331-full_name,
    course_id  TYPE zenroll_dev331-course_id,
    score      TYPE zenroll_dev331-score,
    status     TYPE string, " Cột tính toán thêm (Đậu/Trượt)
END OF ty_report.
```

DATA: lt_report TYPE TABLE OF ty_report.

--- 2. SELECTION SCREEN (Dynamic Filter) ---

" Tạo các biến giả để tham chiếu kiểu dữ liệu cho Select-Options

```
DATA: gv_std_ref TYPE zstudent_dev331-student_id,
    gv_scr_ref TYPE zenroll_dev331-score.
```

SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE TEXT-001.

```
" Select-Options cho phép nhập: Single value, Ranges, Multiple selection...
SELECT-OPTIONS: s_std FOR gv_std_ref, " Lọc theo Mã SV
                s_scr FOR gv_scr_ref. " Lọc theo Điểm (VD: 0 -> 5)
SELECTION-SCREEN END OF BLOCK b1.
```

```
*--- 3. MAIN LOGIC ---*
START-OF-SELECTION.
```

```
" Lấy dữ liệu từ Database dựa trên bộ lọc nhập vào (IN s_...)
SELECT a~student_id, a~full_name, b~course_id, b~score
FROM zstudent_dev331 AS a
INNER JOIN zenroll_dev331 AS b
ON a~student_id = b~student_id
WHERE a~student_id IN @s_std " Dynamic Where Clause
AND b~score IN @s_scr
INTO CORRESPONDING FIELDS OF TABLE @lt_report.
```

```
IF sy-subrc <> 0.
MESSAGE 'No data found matching criteria.' TYPE 'S' DISPLAY LIKE 'E'.
RETURN.
ENDIF.
```

```
" Xử lý dữ liệu (Tính toán cột Status)
" Sử dụng Field-Symbol để loop nhanh hơn và sửa trực tiếp dữ liệu
FIELD-SYMBOLS: <fs_row> TYPE ty_report.
```

```
LOOP AT lt_report ASSIGNING <fs_row>.
IF <fs_row>-score >= 5.
    <fs_row>-status = 'PASS'.
ELSE.
    <fs_row>-status = 'FAIL'.
ENDIF.
```

ENDLOOP.

" 4. DISPLAY ALV (Hiển thị báo cáo hiện đại)

DATA: lo_alv TYPE REF TO cl_salv_table.

TRY.

" Tạo đối tượng ALV gắn với bảng dữ liệu lt_report

cl_salv_table=>factory(

IMPORTING

r_salv_table = lo_alv

CHANGING

t_table = lt_report).

" Bật các chức năng tiêu chuẩn (Toolbar: Sort, Filter, Excel export)

DATA(lo_functions) = lo_alv->get_functions().

lo_functions->set_all(abap_true).

" Tối ưu độ rộng cột

lo_alv->get_columns()->set_optimize(abap_true).

" Đổi tên tiêu đề cột (Nếu cần)

DATA(lo_columns) = lo_alv->get_columns().

TRY.

DATA(lo_col_score) = lo_columns->get_column('SCORE').

lo_col_score->set_long_text('Final Score').

lo_col_score->set_medium_text('Score').

CATCH cx_salv_not_found.

ENDTRY.

" Hiển thị lên màn hình

lo_alv->display().

```
CATCH cx_salv_msg.  
  MESSAGE 'ALV Display Error' TYPE 'E'.  
ENDTRY.
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: Dynamic SQL (Tên bảng động)

- **Yêu cầu:** Thêm PARAMETERS: pa_table TYPE char30.
- **Nhiệm vụ:** Người dùng nhập tên bảng bất kỳ (VD: ZSTUDENT_DEV331). Chương trình đếm số dòng trong bảng đó.
- **Gợi ý:** SELECT COUNT(*) FROM (pa_table) INTO @DATA(lv_count).
- **Mục tiêu:** Hiểu sức mạnh (và sự nguy hiểm) của Dynamic SQL.

Challenge 2: ALV Color Coding (Tô màu dòng)

- **Yêu cầu:** Trong báo cáo ALV, nếu sinh viên FAIL (Điểm < 5), hãy tô màu đỏ cho **cả dòng** đó.
- **Gợi ý:**
 1. Thêm cột color_code TYPE char4 vào cấu trúc ty_report.
 2. Trong Loop, gán mã màu (VD: 'C610' - Đỏ) vào cột đó.
 3. Cấu hình ALV: lo_columns->set_color_column('COLOR_CODE').

Challenge 3: Drill-down Reporting (Tương tác)

- **Yêu cầu:** Khi người dùng click đúp (Double click) vào một dòng trên ALV -> Hiện thông báo "Bạn vừa chọn sinh viên: [Tên SV]".
- **Gợi ý:** Cần tạo Local Class xử lý sự kiện (Event Handler) và đăng ký sự kiện ON_DOUBLE_CLICK của ALV. (Khá nâng cao, dành cho sinh viên giỏi).

Challenge 4: Aggregation (Tổng hợp trên ALV)

- **Yêu cầu:** Không cần code tính toán. Hãy dùng code ABAP để ra lệnh cho ALV tự động hiển thị dòng **Tổng cộng (Total)** điểm số ở cuối bảng.
 - **Gợi ý:** lo_alv->get_aggregations()->add_aggregation(columnname = 'SCORE').
-

4. Lời giải tham khảo (Solution Snippets)

A. Challenge 1 (Dynamic Count)

```
code Abap
IF pa_table IS NOT INITIAL.
  DATA: lv_count TYPE i.
  TRY.
    " Tên bảng được xác định lúc chạy (Runtime)
    SELECT COUNT( * ) FROM (pa_table) INTO lv_count.
    WRITE: / 'Table', pa_table, 'has', lv_count, 'records.'.
  CATCH cx_sy_dynamic_osql_error.
    WRITE: / 'Error: Table does not exist.'.
  ENDTRY.
  RETURN. " Dừng phần ALV bên dưới
ENDIF.
```

B. Challenge 4 (Aggregation)

```
code Abap
" Thêm logic này trước lệnh display()
TRY.
  DATA(lo_aggrs) = lo_alv->get_aggregations( ).
  " Tính tổng cột SCORE
  lo_aggrs->add_aggregation( columnname = 'SCORE' aggregation = if_salv_c_aggregation=>total ).
  CATCH cx_salv_not_found cx_salv_data_error cx_salv_existing.
ENDTRY.
```

C. Challenge 2 (Color Coding - Logic)

Để làm màu sắc, cấu trúc ty_report cần thêm 1 cột đặc biệt (thường kiểu char4 hoặc cấu trúc màu LVC_S_COLO). Với CL_SALV_TABLE, việc tô màu dòng cần setup cột màu (Color Column).

- *Lưu ý:* SALV tô màu dòng hơi khác ALV Grid một chút, đơn giản nhất là tô màu cột (Cell Level).

Kết thúc Buổi 9.

Sinh viên đã có thể tạo ra các báo cáo chuyên nghiệp mà người dùng doanh nghiệp yêu thích.

BUỔI 9: DYNAMIC PROGRAMS & BÁO CÁO NÂNG CAO (ALV GRID)

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Program:** Z[AccountID]_LAB9_ALV (Ví dụ: ZDEV006_LAB9_ALV)
- **Chủ đề:** Báo cáo Tổng hợp Kết quả Học tập (Student Grade Report).
- **Mục tiêu:** Hiển thị danh sách sinh viên, môn học, điểm số, xếp loại với màu sắc trực quan (Xanh/Đỏ/Vàng) bằng lưới ALV.

● B. LÝ THUYẾT CẦN NẮM (TRƯỚC KHI CODE)

1. ALV là gì?

- ALV = ABAP List Viewer.
- Đây là công cụ tiêu chuẩn của SAP để hiển thị danh sách. Nó cung cấp sẵn các nút: Sắp xếp (Sort), Lọc (Filter), Tính tổng (Sum), Xuất ra Excel... mà lập trình viên không cần code thêm.

2. Cấu trúc 3 bước của báo cáo ALV:

- **Bước 1: Get Data:** Dùng SQL lấy dữ liệu vào Internal Table.
- **Bước 2: Build Field Catalog:** Định nghĩa các cột (Cột A tên là gì? Rộng bao nhiêu? Có phải khóa không?).
- **Bước 3: Call ALV Function:** Gọi hàm REUSE_ALV_GRID_DISPLAY để vẽ bảng.

3. Thư viện SLIS:

- Bắt buộc phải khai báo TYPE-POOLS: SLIS ở đầu chương trình để sử dụng được các kiểu dữ liệu của ALV.

C. NỘI DUNG THỰC HÀNH (LAB 9) - STEP BY STEP

Kịch bản: Phòng Đào tạo muốn một báo cáo nhìn thấy ngay sinh viên nào trượt (màu đỏ), sinh viên nào xuất sắc (màu xanh). Dữ liệu lấy từ 3 bảng: ZSTUDENT, ZCOURSE, ZENROLL.

BUƯỚC 1: KHỞI TẠO CHƯƠNG TRÌNH & KHAI BÁO

- Vào SE80, tạo chương trình ZDEV006_LAB9_ALV.
- Tắt bỏ tùy chọn "With TOP Include" (để code trong 1 file cho dễ nhìn).

```
code Abap
*&-----*
*& Report ZDEV006_LAB9_ALV
*&-----*
REPORT zdev006_lab9_alv.

" [QUAN TRỌNG] Thư viện định nghĩa cho ALV
TYPE-POOLS: slis.

*--- 1. CẤU TRÚC DỮ LIỆU BÁO CÁO ---*
" Chúng ta cần hiển thị thông tin tổng hợp từ 3 bảng khác nhau
" nên phải tạo một Structure riêng (Custom Structure)
TYPES: BEGIN OF ty_report,
    student_id TYPE zstudent_dev006-student_id, " Từ bảng Student
    full_name  TYPE zstudent_dev006-full_name,   " Từ bảng Student
    course_id  TYPE zcourse_dev006-course_id,    " Từ bảng Course
    course_name TYPE zcourse_dev006-course_name, " Từ bảng Course
    score      TYPE zenroll_dev006-score,        " Từ bảng Enroll
    rank       TYPE string,                      " Cột tự tính (Xếp loại)
    row_color  TYPE char4,                      " Cột điều khiển màu sắc dòng
END OF ty_report.

" Bảng nội bộ chứa dữ liệu báo cáo
DATA: lt_report TYPE TABLE OF ty_report,
      ls_report TYPE ty_report.
```

--- 2. KHAI BÁO BIẾN CHO ALV ---

DATA: lt_fieldcat TYPE slis_t_fieldcat_alv, " Bảng chứa định nghĩa các cột
ls_fieldcat TYPE slis_fieldcat_alv, " Dòng định nghĩa 1 cột
ls_layout TYPE slis_layout_alv. " Cấu hình giao diện chung

BƯỚC 2: TẠO MÀN HÌNH NHẬP LIỆU (SELECTION SCREEN)

- Sử dụng SELECT-OPTIONS để tạo bộ lọc động (Dynamic Selection), cho phép chọn nhiều sinh viên hoặc một khoảng mã số.

code Abap

--- 3. MÀN HÌNH CHỌN LỌC ---

SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE text-001.

" Select-Options: Cho phép nhập 1 giá trị, 1 khoảng, hoặc nhiều giá trị

" s_sid sẽ tự động tham chiếu kiểu dữ liệu của student_id

SELECT-OPTIONS: s_sid FOR ls_report-student_id.

" Parameters: Nhập 1 giá trị đơn

PARAMETERS: p_sem TYPE char10 DEFAULT 'FA24'.

SELECTION-SCREEN END OF BLOCK b1.

" Gán nhãn cho text-001 (Vào Goto -> Text Elements -> Selection Texts để đặt tên đẹp hơn)

BƯỚC 3: LẤY DỮ LIỆU & XỬ LÝ LOGIC (START-OF-SELECTION)

- Sử dụng INNER JOIN để lấy dữ liệu.
- Xử lý vòng lặp để tính Xếp loại và Gán màu.

code Abap

START-OF-SELECTION.

" --- A. LẤY DỮ LIỆU TỪ DB ---

" Lấy dữ liệu từ bảng Điểm (Enroll) nối sang SV và Môn học

SELECT t1~student_id, t1~course_id, t1~score,

t2~full_name,

```
t3~course_name
FROM zenroll_dev006 AS t1
INNER JOIN zstudent_dev006 AS t2 ON t1~student_id = t2~student_id
INNER JOIN zcourse_dev006 AS t3 ON t1~course_id = t3~course_id
WHERE t1~student_id IN @s_sid " Lọc theo điều kiện người dùng nhập
INTO CORRESPONDING FIELDS OF TABLE @lt_report.
```

```
" Kiểm tra nếu không có dữ liệu thì dừng
IF lt_report IS INITIAL.
  MESSAGE 'Không tìm thấy dữ liệu phù hợp!' TYPE 'S' DISPLAY LIKE 'E'.
  RETURN.
ENDIF.
```

```
" --- B. XỬ LÝ LOGIC NGHIỆP VỤ ---
LOOP AT lt_report INTO ls_report.
```

```
" 1. Logic Xếp loại
IF ls_report-score >= 9.
  ls_report-rank = 'Xuất sắc'.
  " C500: C = Color, 5 = Xanh lá (Green), 0 = Normal, 0 = Normal
  ls_report-row_color = 'C500'.
ELSEIF ls_report-score >= 5.
  ls_report-rank = 'Đạt'.
  " C300: 3 = Vàng (Yellow)
  ls_report-row_color = 'C300'.
ELSE.
  ls_report-rank = 'Học lại'.
  " C600: 6 = Đỏ (Red)
  ls_report-row_color = 'C600'.
ENDIF.
```

```
" Cập nhật lại vào bảng nội bộ
```

```
MODIFY lt_report FROM ls_report.  
ENDLOOP.
```

BƯỚC 4: CẤU HÌNH CỘT (FIELD CATALOG) - QUAN TRỌNG

- Đây là bước "vẽ" các cột cho báo cáo. Sinh viên cần khai báo từng cột một.
- *Lưu ý:* Tên fieldname phải viết **HOA** và khớp 100% với tên trong TYPES ty_report.

code Abap

```
" --- C. CẤU HÌNH ALV ---
```

```
" 1. Cột Mã Sinh viên
```

```
CLEAR ls_fieldcat.
```

```
ls_fieldcat-fieldname = 'STUDENT_ID'. " Tên trường trong bảng nội bộ
```

```
ls_fieldcat-seltext_m = 'Mã SV'.      " Tiêu đề cột hiển thị
```

```
ls_fieldcat-key       = 'X'.          " Cột khóa (Sẽ được cố định màu xanh lơ)
```

```
ls_fieldcat-outputlen = 10.          " Độ rộng cột
```

```
APPEND ls_fieldcat TO lt_fieldcat.
```

```
" 2. Cột Họ tên
```

```
CLEAR ls_fieldcat.
```

```
ls_fieldcat-fieldname = 'FULL_NAME'.
```

```
ls_fieldcat-seltext_m = 'Họ và Tên'.
```

```
ls_fieldcat-outputlen = 25.
```

```
APPEND ls_fieldcat TO lt_fieldcat.
```

```
" 3. Cột Tên Môn học
```

```
CLEAR ls_fieldcat.
```

```
ls_fieldcat-fieldname = 'COURSE_NAME'.
```

```
ls_fieldcat-seltext_m = 'Môn học'.
```

```
ls_fieldcat-outputlen = 20.
```

```
APPEND ls_fieldcat TO lt_fieldcat.
```

" 4. Cột Điểm số

```
CLEAR ls_fieldcat.  
ls_fieldcat-fieldname = 'SCORE'.  
ls_fieldcat-seltext_m = 'Điểm số'.  
ls_fieldcat-just      = 'C'. " Căn giữa (Center)  
APPEND ls_fieldcat TO lt_fieldcat.
```

" 5. Cột Xếp loại

```
CLEAR ls_fieldcat.  
ls_fieldcat-fieldname = 'RANK'.  
ls_fieldcat-seltext_m = 'Xếp loại'.  
ls_fieldcat-emphasize = 'C710'. " Tô màu riêng cho cột này (Màu tím nhạt)  
APPEND ls_fieldcat TO lt_fieldcat.
```

BƯỚC 5: CẤU HÌNH LAYOUT & HIỂN THỊ

- Gắn mã màu dòng vào layout và gọi hàm hiển thị.

code Abap

" --- D. CẤU HÌNH LAYOUT ---

```
ls_layout-colwidth_optimize = 'X'. " Tự động co giãn cột cho đẹp  
ls_layout-zebra             = 'X'. " Kẻ sọc xen kẽ (Zebra pattern)  
ls_layout-info fieldname    = 'ROW COLOR'. " [QUAN TRỌNG] Báo cho ALV biết cột nào chứa mã màu dòng
```

" --- E. GỌI HÀM HIỂN THỊ ALV GRID ---

```
CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY'  
EXPORTING  
  i_callback_program = sy-repid " Chương trình gọi lại (Callback)  
  is_layout          = ls_layout " Cấu hình giao diện  
  it_fieldcat        = lt_fieldcat " Cấu hình cột  
TABLES  
  t_outtab           = lt_report " Bảng dữ liệu cần in  
EXCEPTIONS  
  program_error      = 1
```

```
others          = 2.
```

```
IF sy-subrc <> 0.
```

```
  MESSAGE 'Lỗi khi hiển thị ALV' TYPE 'E'.
```

```
ENDIF.
```

D. THỬ THÁCH NÂNG CAO (CHALLENGE) CHO BUỔI 9

Chào bạn, việc bổ sung đầy đủ 4 thử thách nâng cao này là rất cần thiết để phân loại sinh viên Giỏi/Xuất sắc. Các kỹ thuật như Dynamic SQL hay Drill-down là những yêu cầu thường xuyên gặp trong các bài Test tuyển dụng của FPT hay các công ty đối tác SAP.

Dưới đây là nội dung chi tiết **4 CHALLENGES NÂNG CAO** cho Buổi 9 kèm lời giải tham khảo.

Challenge 1: Dynamic SQL (Truy vấn Bảng động)

- **Tình huống:** Bạn muốn viết một công cụ quản trị (Admin Tool) nhỏ. Thay vì viết cứng tên bảng ZSTUDENT_DEV006 trong code, bạn muốn người dùng nhập tên bảng vào màn hình, và chương trình sẽ đếm xem bảng đó có bao nhiêu dòng dữ liệu.
- **Yêu cầu:**
 1. Thêm tham số PARAMETERS: p_table TYPE tabname.
 2. Sử dụng cú pháp SELECT COUNT(*) FROM (p_table) (Tên bảng để trong ngoặc đơn).
 3. Hiển thị số lượng bản ghi tìm được.
 4. Xử lý lỗi TRY...CATCH nếu người dùng nhập tên bảng không tồn tại (Lỗi CX_SY_DYNAMIC_OSQL_SEMANTICS).

Lời giải tham khảo (Challenge 1)

code Abap

```
*--- CHALLENGE 1: DYNAMIC SQL ---*
```

```
PARAMETERS: p_tab TYPE tabname DEFAULT 'ZSTUDENT_DEV006'.
```

```
DATA: lv_count TYPE i.
```

```
START-OF-SELECTION.
```

```
TRY.
```

```
" Tên bảng được xác định tại thời điểm chạy (Runtime)
```

```
SELECT COUNT( * ) FROM (p_tab) INTO lv_count.
```

```
WRITE: / 'Bảng', p_tab, 'hiện có', lv_count, 'dòng dữ liệu.'.
```

```
CATCH cx_sy_dynamic_osql_semantics.
```

```
MESSAGE 'Lỗi: Tên bảng không tồn tại trong hệ thống!' TYPE 'I' DISPLAY LIKE 'E'.
```

```
CATCH cx_root.
```

```
MESSAGE 'Lỗi hệ thống không xác định.' TYPE 'E'.
```

```
ENDTRY.
```

Challenge 2: ALV Color Coding (Tô màu dòng theo điều kiện)

- **Tình huống:** Sếp muốn nhìn lướt qua báo cáo là biết ngay ai trượt.
- **Yêu cầu:**
 1. Trong cấu trúc dữ liệu báo cáo, đảm bảo đã có cột ROW_COLOR (Char 4).
 2. Trong cấu hình Layout (is_layout), gán info_fieldname = 'ROW_COLOR'.
 3. Trong vòng lặp xử lý dữ liệu:
 - Nếu Điểm < 5: Gán màu Đỏ (C610).
 - Nếu Điểm >= 9: Gán màu Xanh lá (C510).
 - Còn lại: Không tô màu (hoặc màu mặc định).

Lời giải tham khảo (Challenge 2)

code Abap

--- CHALLENGE 2: COLOR CODING ---

" 1. Khai báo Layout

ls_layout-info_fieldname = 'ROW_COLOR'. " Chỉ định cột chứa mã màu

" 2. Logic tô màu trong vòng lặp

LOOP AT lt_report INTO ls_report.

IF ls_report-score < 5.

ls_report-row_color = 'C610'. " C=Color, 6=Red, 1=Intensified, 0=Normal

ELSEIF ls_report-score >= 9.

ls_report-row_color = 'C510'. " 5=Green

ELSE.

CLEAR ls_report-row_color.

ENDIF.

MODIFY lt_report FROM ls_report.

ENDLOOP.

" 3. Gọi hàm ALV như bình thường

Challenge 3: Drill-down Reporting (Tương tác người dùng)

- **Tình huống:** Báo cáo ALV đang hiển thị danh sách tổng quát. Người dùng muốn **Double-click** vào một dòng (ví dụ vào Mã SV) để xem thông báo chi tiết hoặc mở một màn hình khác.
- **Yêu cầu:**
 1. Tạo một FORM xử lý sự kiện tên là USER_COMMAND.
 2. Truyền tên FORM này vào tham số I_CALLBACK_USER_COMMAND của hàm REUSE_ALV_GRID_DISPLAY.
 3. Trong FORM, kiểm tra mã lệnh &IC1 (Mã mặc định khi Double-click).
 4. Dùng lệnh READ TABLE để lấy dữ liệu dòng vừa click và hiện Popup MESSAGE.

Lời giải tham khảo (Challenge 3)

code Abap

```

*--- Bước 1: Gọi hàm ALV có đăng ký Callback ---*
CALL FUNCTION 'REUSE_ALV_GRID_DISPLAY'
  EXPORTING
    i_callback_program    = sy-repid
    i_callback_user_command = 'USER_COMMAND' " Đăng ký tên Form xử lý
    it_fieldcat           = lt_fieldcat
  TABLES
    t_outtab              = lt_report.

*--- Bước 2: Viết Form xử lý sự kiện ---*
FORM user_command USING r_ucomm    LIKE sy-ucomm
                        rs_selffield TYPE slis_selffield.

  " Kiểm tra xem người dùng có phải vừa Double Click không?
  CASE r_ucomm.
    WHEN '&IC1'.
      " Kiểm tra xem click vào cột nào?
      IF rs_selffield-fieldname = 'STUDENT_ID'.
        " Lấy dòng dữ liệu vừa click
        READ TABLE lt_report INTO ls_report INDEX rs_selffield-tabindex.
        IF sy-subrc = 0.
          " Hiện thị Popup
          MESSAGE |Bạn đang xem chi tiết Sinh viên: { ls_report-full_name }| TYPE 'I'.
        ENDIF.
      ENDIF.
    ENDCASE.
  ENDFORM.

```

+ Challenge 4: Aggregation (Tổng hợp số liệu trên ALV)

- **Tình huống:** Báo cáo cần hiển thị Tổng điểm (Total Score) hoặc Điểm trung bình của cả danh sách ngay chân trang mà không cần tính toán thủ công trong code.

- **Yêu cầu:**

1. Tìm đến đoạn khai báo Field Catalog cho cột SCORE.
2. Gán thuộc tính do_sum = 'X'.
3. Chạy báo cáo và kiểm tra dòng màu vàng xuất hiện cuối cùng (Grand Total).

 Lời giải tham khảo (Challenge 4)

```
code Abap
*--- CHALLENGE 4: AGGREGATION ---*
CLEAR ls_fieldcat.
ls_fieldcat-fieldname = 'SCORE'.
ls_fieldcat-seltext_m = 'Điểm Số'.

" Kích hoạt tính tổng tự động
ls_fieldcat-do_sum = 'X'.

APPEND ls_fieldcat TO lt_fieldcat.
```

 Lời khuyên cho Giảng viên khi chấm bài Lab 9:

1. **Với Challenge 1:** Sinh viên hiểu được khái niệm "Dynamic Token" (var) là đạt.
2. **Với Challenge 2:** Nếu sinh viên quên cấu hình ls_layout-info_fieldname, màu sẽ không hiện -> Nhắc nhở sinh viên check Layout.
3. **Với Challenge 3:** Đây là kỹ thuật khó nhất. Nếu sinh viên làm được Popup là đạt yêu cầu. Ở mức nâng cao hơn, Double click có thể dùng lệnh CALL TRANSACTION để mở màn hình sửa sinh viên (Buổi 5).
4. **Với Challenge 4:** Rất dễ, nhưng sinh viên cần biết nó tồn tại để tận dụng sức mạnh của ALV Standard.

Sinh viên đã có thể tạo ra các báo cáo chuyên nghiệp mà người dùng doanh nghiệp yêu thích.

BUỔI 10: WEB FUNDAMENTALS (HTML5, CSS, JAVASCRIPT)

A. THÔNG TIN & QUY TẮC

- **Công cụ:** Visual Studio Code (Khuyên dùng) hoặc Notepad++. Trình duyệt Chrome/Edge.
 - **File thực hành:** student_report_[AccountID].html
 - **Chủ đề:** Xây dựng giao diện Phiếu báo điểm (Grade Report) chạy trên trình duyệt.
-

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Sự chuyển dịch từ ABAP Backend sang Full-stack SAP:

1. JSON là "Internal Table" mới:

- Trong ABAP, bạn chứa dữ liệu trong Internal Table (itab).
- Trong Web/Fiori, dữ liệu được chứa trong **JSON** (JavaScript Object Notation).
- *Ví dụ:* DATA(lt_data) trong ABAP chính là var aData = [...] trong JS. Hãy tập tư duy chuyển đổi giữa hai thứ này.

2. F12 là /h mới:

- Trong SAP GUI, bạn gõ /h để debug.
- Trên trình duyệt, bạn nhấn **F12** (Developer Tools) -> Tab **Console** và **Sources** để debug code JS.

3. DOM Manipulation (Cẩn thận):

- Trong bài này, chúng ta sẽ dùng JS để can thiệp trực tiếp vào HTML (DOM).
- *Lưu ý:* Khi sang SAPUI5 (Buổi 11), framework sẽ tự làm việc này. Tuy nhiên, hiểu về DOM giúp bạn biết cách SAPUI5 vận hành bên dưới.

C. MỤC TIÊU BÀI HỌC

1. **HTML5:** Tạo khung sườn (Skeleton) cho trang web báo cáo.
2. **CSS3:** Trang trí giao diện (Màu sắc, Font chữ) theo phong cách FPT (Cam/Xanh).
3. **JavaScript (ES6):** Xử lý logic tính toán điểm trung bình ngay trên trình duyệt mà không cần gọi về Server.
4. **JSON:** Làm quen với cấu trúc dữ liệu trao đổi chuẩn.

D. NỘI DUNG THỰC HÀNH (LAB 10): WEB-BASED GRADE CARD

Tình huống: Bạn cần tạo một nguyên mẫu (Prototype) giao diện nhập điểm và xem kết quả cho sinh viên để trình bày cho khách hàng xem trước khi code thật bằng SAPUI5.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo file HTML cơ bản

- Tạo một file text mới trên máy tính, đặt tên dev331_lms.html.
- Mở bằng VS Code.
- Gõ cấu trúc HTML cơ bản (<html>, <head>, <body>).

Bước 2: Xây dựng giao diện (HTML)

- Tạo tiêu đề <h1>.
- Tạo form nhập thông tin Sinh viên.
- Tạo bảng (<table>) chứa các môn học và điểm số.
- Tạo nút bấm (<button>) "Tính điểm TB".

Bước 3: Trang trí (CSS)

- Thêm thẻ <style> trong phần <head>.

- Tô màu nền, kẻ bảng, chỉnh font chữ cho đẹp mắt.

Bước 4: Thêm Logic (JavaScript)

- Thêm thẻ <script> ở cuối <body>.
- Viết hàm calculateGPA() để lấy giá trị từ ô nhập liệu, tính toán và hiện kết quả.

2. Code Mẫu (All-in-One File)

Hãy copy đoạn code này vào file .html và mở bằng Chrome.

code Html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>FPT LMS - Grade Report</title>
  <style>
    /* --- CSS: Phần trang trí giao diện --- */
    body { font-family: Arial, sans-serif; background-color: #f4f4f4; padding: 20px; }

    .container {
      background-color: white;
      width: 600px;
      margin: 0 auto;
      padding: 20px;
      border-radius: 8px;
      box-shadow: 0 0 10px rgba(0,0,0,0.1);
    }

    h1 { color: #F36F21; /* FPT Orange */ text-align: center; }

    .info-group { margin-bottom: 15px; }
```

```
label { font-weight: bold; display: inline-block; width: 100px; }

table { width: 100%; border-collapse: collapse; margin-top: 20px; }
th { background-color: #004070; /* FPT Blue */ color: white; padding: 10px; }
td { border: 1px solid #ddd; padding: 8px; text-align: center; }

input[type="number"] { width: 60px; padding: 5px; }

button {
    background-color: #F36F21; color: white; border: none;
    padding: 10px 20px; margin-top: 20px; cursor: pointer; width: 100%; font-size: 16px;
}
button:hover { background-color: #d35400; }

#result-area { margin-top: 20px; text-align: center; font-size: 18px; font-weight: bold; }
.pass { color: green; }
.fail { color: red; }
</style>
</head>
<body>

<div class="container">
    <h1>FPT UNIVERSITY ACADEMIC REPORT</h1>

    <div class="info-group">
        <label>Student ID:</label> <span>SE1876-DEV331</span>
    </div>
    <div class="info-group">
        <label>Full Name:</label> <span>Nguyen Van A</span>
    </div>

    <table>
```

```
<thead>
  <tr>
    <th>Subject</th>
    <th>Credits</th>
    <th>Score (0-10)</th>
  </tr>
</thead>
<tbody id="grade-table">
  <!-- Dòng 1 -->
  <tr>
    <td>Java Programming</td>
    <td>3</td>
    <td><input type="number" id="score1" value="8.5"></td>
  </tr>
  <!-- Dòng 2 -->
  <tr>
    <td>SAP ABAP</td>
    <td>4</td>
    <td><input type="number" id="score2" value="7.0"></td>
  </tr>
  <!-- Dòng 3 -->
  <tr>
    <td>Web Design</td>
    <td>3</td>
    <td><input type="number" id="score3" value="9.0"></td>
  </tr>
</tbody>
</table>

<button onclick="calculateGPA()">CALCULATE GPA</button>

<div id="result-area">
```

```
GPA: <span id="gpa-value">...</span> <br>
Rank: <span id="rank-value">...</span>
</div>
</div>

<script>
/* --- JAVASCRIPT: Phần xử lý logic --- */
function calculateGPA() {
  // 1. Lấy giá trị từ các ô input (DOM Access)
  // Trong JS, giá trị lấy từ input luôn là String, cần parse sang Float
  let s1 = parseFloat(document.getElementById('score1').value) || 0;
  let s2 = parseFloat(document.getElementById('score2').value) || 0;
  let s3 = parseFloat(document.getElementById('score3').value) || 0;

  // 2. Tính toán (Giả sử tín chỉ là 3, 4, 3)
  let totalCredits = 3 + 4 + 3;
  let totalScore = (s1 * 3) + (s2 * 4) + (s3 * 3);
  let gpa = totalScore / totalCredits;

  // Làm tròn 2 chữ số thập phân
  gpa = gpa.toFixed(2);

  // 3. Xếp loại
  let rank = "";
  let colorClass = "";

  if (gpa >= 8) {
    rank = 'Excellent';
    colorClass = 'pass';
  } else if (gpa >= 5) {
    rank = 'Pass';
    colorClass = 'pass';
  }
}
```

```
    } else {  
        rank = 'Fail';  
        colorClass = 'fail';  
    }  
  
    // 4. Hiển thị kết quả lại màn hình  
    document.getElementById('gpa-value').innerText = gpa;  
  
    let rankSpan = document.getElementById('rank-value');  
    rankSpan.innerText = rank;  
  
    // Đổi class CSS động để đổi màu chữ (Class Manipulation)  
    rankSpan.className = colorClass;  
}  
</script>  
  
</body>  
</html>
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Đây là những kỹ thuật sẽ dùng trực tiếp trong SAPUI5.

Challenge 1: Data Binding Simulation (JSON Array)

- **Vấn đề:** Hiện tại danh sách môn học đang "Code cứng" (Hardcode) bằng HTML.
- **Yêu cầu:**
 1. Trong thẻ <script>, khai báo một mảng JSON:

code JavaScript

```
var subjects = [
```

```
{ name: "Soft Skills", credit: 2, score: 0 },  
{ name: "Database", credit: 3, score: 0 }  
];
```

2. Viết hàm JS để duyệt mảng này và tự động sinh ra các dòng <tr> trong bảng HTML.

- **Mục tiêu:** Hiểu cách SAPUI5 binding dữ liệu từ Model ra View.

Challenge 2: Validation (Kiểm tra lỗi)

- **Yêu cầu:** Trong hàm calculateGPA, trước khi tính, hãy kiểm tra. Nếu có bất kỳ điểm nào < 0 hoặc > 10:
 - Hiện thông báo alert("Điểm không hợp lệ").
 - Đổi màu viền của ô input bị sai sang màu đỏ (style.borderColor = 'red').

Challenge 3: Event Listener (Tự động tính)

- **Yêu cầu:** Bỏ nút "Calculate GPA".
- **Logic:** Khi người dùng nhập xong và **rời chuột** khỏi ô input (Sự kiện onchange hoặc onblur), hệ thống tự động tính lại GPA ngay lập tức.
- **Mục tiêu:** Tăng trải nghiệm người dùng (UX).

Challenge 4: ES6 Arrow Functions (Cú pháp mới)

- **Yêu cầu:** Viết lại hàm tính toán sử dụng cú pháp mũi tên const calculate = () => { ... } thay vì function calculate() { ... }.
- **Mục tiêu:** Làm quen với cú pháp JS hiện đại mà các tài liệu SAPUI5 hay dùng.

4. Lời giải tham khảo (Solution Snippets)

A. Challenge 1 (Render from JSON)

code JavaScript

```
// Data Model (Mô phỏng OData)
var subjects = [
  { id: "s1", name: "Soft Skills", credit: 2 },
  { id: "s2", name: "Database Systems", credit: 3 }
];

// Hàm Render chạy khi trang web tải xong
window.onload = function() {
  var tableBody = document.getElementById('grade-table');
  var htmlContent = "";

  // Duyệt mảng (Loop)
  subjects.forEach(sub => {
    // Template Literals (Dấu backtick `) giúp chèn biến vào chuỗi dễ dàng
    htmlContent += `
      <tr>
        <td>${sub.name}</td>
        <td>${sub.credit}</td>
        <td><input type="number" class="score-input" data-credit="${sub.credit}"></td>
      </tr>
    `;
  });

  // Chèn HTML vào bảng (Tất cả dòng hardcoded cũ sẽ bị thay thế)
  tableBody.innerHTML = htmlContent;
};
```

B. Challenge 2 & 3 (Validation & Auto-calc)

```
code JavaScript
// Gán sự kiện cho tất cả input (Code nâng cao một chút)
// document.querySelectorAll('.score-input').forEach(input => {
//   input.addEventListener('input', function() {
//     // Logic validate và tính toán ở đây
```

```
//    if(this.value < 0 || this.value > 10) {  
//        this.style.borderColor = 'red';  
//    } else {  
//        this.style.borderColor = '#ddd'; // Trả về màu cũ  
//        calculateGPA(); // Gọi hàm tính  
//    }  
//    });  
// });
```

Kết thúc Buổi 10.

Sinh viên đã "nhúng chân" vào thế giới Web.

- Họ hiểu rằng: **HTML là bộ khung, CSS là quần áo, JS là bộ não.**

BUỔI 11: SAPUI5 FRAMEWORK & KIẾN TRÚC MVC

A. THÔNG TIN & QUY TẮC

- **Công cụ:** Visual Studio Code (cài thêm extension "Live Server") hoặc SAP Business Application Studio (BAS).
- **Dự án:** zwebapp_course_reg
- **Chủ đề:** Màn hình Đăng ký môn học (Course Registration App).
- **Kiến trúc:** MVC (Model - View - Controller).

B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Làm sao để không "chết chìm" trong biển tài liệu SAPUI5?

1. SAPUI5 SDK (Demokit) là Kinh thánh:

- Đừng Google lung tung. Hãy vào trang chủ ui5.sap.com.
- Mục **Samples**: Chứa code mẫu của mọi nút bấm, bảng biểu. Copy-Paste từ đây là chuẩn nhất.
- Mục **API Reference**: Tra cứu thuộc tính (Properties) và sự kiện (Events).

2. XML View là Chân ái:

- SAPUI5 cho phép viết giao diện bằng JavaScript hoặc JSON, nhưng **99% dự án thật dùng XML View**.
- XML tách biệt rõ ràng cấu trúc giao diện và code xử lý logic. Dễ đọc, dễ bảo trì.

3. Cài đặt "UI5 Inspector" ngay:

- Đây là Extension trên Chrome dành riêng cho lập trình viên SAPUI5. Nó giúp bạn soi (inspect) cấu trúc UI5, xem dữ liệu Binding, và debug lỗi cực nhanh.

4. Data Binding (Phép thuật):

- Trong JS thường, bạn phải `document.getElementById('input').value = 'abc'`.
- Trong SAPUI5, bạn chỉ cần gán biến vào Model, giao diện tự cập nhật. **Two-way binding** (hai chiều) là sức mạnh lớn nhất của nó.

C. MỤC TIÊU BÀI HỌC

1. **Bootstrapping:** Biết cách nhúng thư viện SAPUI5 từ CDN vào file HTML.
2. **MVC Architecture:**
 - **Model (JSON):** Chứa dữ liệu môn học.
 - **View (XML):** Giao diện bảng danh sách môn học.
 - **Controller (JS):** Xử lý sự kiện khi bấm nút "Đăng ký".
3. **Controls:** Sử dụng các control cơ bản của thư viện sap.m (Mobile) như: App, Page, Table, Button.

D. NỘI DUNG THỰC HÀNH (LAB 11): COURSE REGISTRATION APP

Tình huống: Sinh viên cần một giao diện web hiện đại để xem danh sách các môn học đang mở và chọn đăng ký. Dữ liệu tạm thời chạy mock (JSON) tại client.

1. Hướng dẫn chi tiết (Step-by-Step)

Để đơn giản hóa môi trường (tránh cài đặt Node.js phức tạp trong buổi đầu), chúng ta sẽ viết **Single-File App** (Tất cả trong 1 file HTML, nhưng vẫn chia logic MVC rõ ràng).

Bước 1: Tạo cấu trúc HTML và nạp thư viện (Bootstrap)

- Tạo file `index.html`.

- Thêm thẻ <script id="sap-ui-bootstrap" ...> trỏ tới server của SAP.

Bước 2: Định nghĩa VIEW (XML)

- Sử dụng thẻ <script type="sapui5/xmlview"> để nhúng mã XML.
- Tạo sap.m.Table để hiện danh sách môn học.
- Binding dữ liệu dùng cú pháp {tên_trường}.

Bước 3: Định nghĩa CONTROLLER (JS)

- Sử dụng sap.ui.define để tạo Controller.
- Viết hàm onInit để khởi tạo dữ liệu mẫu (Model).
- Viết hàm onRegister để xử lý nút bấm.

Bước 4: Chạy ứng dụng

- Mở file bằng Chrome (hoặc qua Live Server).

2. Code Mẫu (All-in-One for Learning)

```
code Html
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <title>FPT LMS - Course Registration</title>

  <!-- 1. BOOTSTRAP SAPUI5 FRAMEWORK -->
  <!-- Load thư viện từ CDN của SAP -->
  <script id="sap-ui-bootstrap"
    src="https://sdk.openui5.org/resources/sap-ui-core.js"
    data-sap-ui-theme="sap_horizon"
    data-sap-ui-libs="sap.m">
```

```
data-sap-ui-compatibleVersion="edge"
data-sap-ui-async="true"
data-sap-ui-resourceroots='{ "lms.app": "." }'>
</script>

<!-- 2. DEFINING THE VIEW (XML) -->
<!-- Trong dự án thật, cái này nằm ở file View.view.xml riêng -->
<script id="myXmlView" type="sapui5/xmlview">
  <mvc:View
    controllerName="lms.app.Main"
    xmlns="sap.m"
    xmlns:mvc="sap.ui.core.mvc">

    <App>
      <Page title="Course Registration (Spring 2026)">
        <content>
          <!-- Danh sách môn học -->
          <Table id="idCourseTable"
            items="{/courses}"
            mode="MultiSelect"> <!-- Cho phép chọn nhiều dòng -->

            <headerToolbar>
              <OverflowToolbar>
                <Title text="Available Courses" />
                <ToolbarSpacer />
                <!-- Nút Đăng ký -->
                <Button text="Register Selected"
                  type="Emphasized"
                  icon="sap-icon://edit"
                  press=".onRegisterPress" />
              </OverflowToolbar>
            </headerToolbar>
```

```

<columns>
  <Column><Text text="Course ID" /></Column>
  <Column><Text text="Course Name" /></Column>
  <Column hAlign="End"><Text text="Credits" /></Column>
  <Column hAlign="Center"><Text text="Status" /></Column>
</columns>

<items>
  <!-- Template cho từng dòng dữ liệu (Aggregation Binding) -->
  <ColumnListItem>
    <cells>
      <ObjectIdentifier title="{courseId}" />
      <Text text="{name}" />
      <ObjectNumber number="{credits}" unit="Tin chi" />
      <ObjectStatus text="{status}"
        state="{= ${status} === 'Open' ? 'Success' : 'Error' }" />
    </cells>
  </ColumnListItem>
</items>
</Table>
</content>
</Page>
</App>
</mvc:View>
</script>

<!-- 3. DEFINING THE CONTROLLER (JS) -->
<!-- Trong dự án thật, cái này nằm ở file Main.controller.js riêng -->
<script>
  sap.ui.getCore().attachInit(function () {

```

```
sap.ui.define([
    "sap/ui/core/mvc/Controller",
    "sap/ui/model/json/JSONModel",
    "sap/m/MessageToast"
], function (Controller, JSONModel, MessageToast) {
    "use strict";

    return Controller.extend("lms.app.Main", {

        // Hàm chạy đầu tiên khi Controller khởi tạo
        onInit: function () {
            // Tạo dữ liệu giả (Mock Data)
            var oData = {
                courses: [
                    { courseId: "SWP391", name: "Software Project", credits: 5, status: "Open" },
                    { courseId: "SWR302", name: "Software Requirement", credits: 3, status: "Closed" },
                    { courseId: "PRN231", name: "Fullstack Development", credits: 3, status: "Open" },
                    { courseId: "SAP404", name: "SAP S/4HANA & ABAP", credits: 4, status: "Open" }
                ]
            };

            // Tạo JSON Model và gán vào View
            var oModel = new JSONModel(oData);
            this.getView().setModel(oModel);
        },

        // Hàm xử lý sự kiện bấm nút Register
        onRegisterPress: function () {
            // Lấy tham chiếu đến cái bảng
            var oTable = this.byId("idCourseTable");
            // Lấy các dòng đang được tick chọn
            var aSelectedItems = oTable.getSelectedItems();
        }
    });
});
```

```

        if (aSelectedItems.length === 0) {
            MessageToast.show("Please select at least one course!");
            return;
        }

        // Duyệt qua các dòng đã chọn để lấy tên môn
        var sMsg = "Registered: ";
        aSelectedItems.forEach(function(oItem) {
            // Lấy Binding Context của dòng đó -> trở vào dữ liệu trong Model
            var oContext = oItem.getBindingContext();
            sMsg += oContext.getProperty("courseId") + ", ";
        });

        MessageToast.show(sMsg);
    }
});

// 4. INSTANTIATE THE VIEW (Khởi chạy View)
sap.ui.xmlview({
    viewContent: jQuery("#myXmlView").html()
}).placeAt("content");
});
</script>
</head>
<body class="sapUiBody" id="content">
</body>
</html>

```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Đây là những tính năng bắt buộc phải có trong một ứng dụng Fiori chuẩn.

Challenge 1: Search & Filter (Tìm kiếm)

- **Yêu cầu:** Thêm một thanh tìm kiếm (SearchField) ở trên đầu bảng.
- **Logic:** Khi người dùng gõ text (sự kiện liveChange hoặc search), hãy lọc danh sách môn học theo Tên hoặc Mã môn.
- **Gợi ý:** Sử dụng sap.ui.model.Filter và sap.ui.model.FilterOperator.Contains.

```
code JavaScript
// Snippet gợi ý:
var oFilter = new Filter("name", FilterOperator.Contains, sQuery);
oTable.getBinding("items").filter([oFilter]);
```

Challenge 2: Formatter (Định dạng dữ liệu)

- **Vấn đề:** Hiện tại cột Status đang hiển thị text thô ("Open"/"Closed").
- **Yêu cầu:** Viết một file Formatter riêng (hoặc hàm trong Controller).
 - Nếu status là "Open" -> Trả về text: "Đang mở đăng ký".
 - Nếu status là "Closed" -> Trả về text: "Lớp đã đầy".
- **Gợi ý:** Trong View XML: text="{ path: 'status', formatter: '.myFormatterFunction' }".

Challenge 3: I18n (Đa ngôn ngữ)

- **Mô tả:** Ứng dụng FPT cần chạy cả Tiếng Anh và Tiếng Việt.
- **Yêu cầu:** Thay thế các text cứng (như "Available Courses", "Credits") bằng biến tài nguyên ({i18n>availableCourses}).
- **Hướng dẫn:** Tạo sap.ui.model.resource.ResourceModel, trỏ tới file .properties (hoặc định nghĩa object text ngay trong file html này để demo).

Challenge 4: Dialog (Popup xác nhận)

- **Yêu cầu:** Khi bấm nút "Register", thay vì hiện MessageToast ngay, hãy hiện một sap.m.Dialog (Popup).

- Tiêu đề: "Confirm Registration".
 - Nội dung: "Are you sure you want to register X courses?".
 - Nút: "OK" và "Cancel".
- **Mục tiêu:** Học cách quản lý các control lồng nhau (Dependencies).

4. Lời giải tham khảo (Solution Snippets)

A. Challenge 1: Filter Logic

```
code JavaScript
// Thêm vào Controller
onSearch: function (oEvent) {
    // 1. Lấy giá trị người dùng gõ
    var sQuery = oEvent.getParameter("query"); // hoặc "newValue" nếu dùng liveChange

    // 2. Tạo bộ lọc
    var aFilters = [];
    if (sQuery && sQuery.length > 0) {
        var oFilterName = new sap.ui.model.Filter("name", sap.ui.model.FilterOperator.Contains, sQuery);
        var oFilterId = new sap.ui.model.Filter("courseId", sap.ui.model.FilterOperator.Contains, sQuery);
        // Lọc theo Tên HOẶC ID (OR logic)
        aFilters.push(new sap.ui.model.Filter([oFilterName, oFilterId], false));
    }

    // 3. Áp dụng filter lên danh sách items của bảng
    var oTable = this.byId("idCourseTable");
    var oBinding = oTable.getBinding("items");
    oBinding.filter(aFilters);
}
```

B. Challenge 4: Confirm Dialog

```

code JavaScript
// Thêm vào sap.ui.define: "sap/m/Dialog", "sap/m/Button", "sap/m/Text"
// Trong hàm onRegisterPress:
var that = this; // Lưu ngữ cảnh
var oDialog = new sap.m.Dialog({
    title: "Confirm",
    type: "Message",
    content: new sap.m.Text({ text: "Are you sure?" }),
    beginButton: new sap.m.Button({
        text: "Yes",
        press: function () {
            sap.m.MessageToast.show("Registered!");
            oDialog.close();
        }
    }),
    endButton: new sap.m.Button({
        text: "No",
        press: function () {
            oDialog.close();
        }
    }),
    afterClose: function() {
        oDialog.destroy();
    }
});

oDialog.open();

```

Kết thúc Buổi 11.

Sinh viên đã xây dựng được ứng dụng SAPUI5 đầu tiên theo mô hình MVC.
Tuy nhiên, dữ liệu vẫn là "giả" (Local JSON).

BUỔI 12: ODATA SERVICES (KẾT NỐI FRONTEND & BACKEND)

A. THÔNG TIN & QUY TẮC

- **Transaction Code:** SEGW (SAP Gateway Service Builder) và /IWFND/MAINT_SERVICE (Register Service).
 - **Project Name:** ZGW_LMS_[AccountID] (Ví dụ: ZGW_LMS_DEV331).
 - **Chủ đề:** Tạo API để lấy danh sách sinh viên từ SAP ra ngoài Web.
-

B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Làm chủ OData trong dự án thực tế:

1. Phân biệt DPC và MPC:

- Khi bạn Generate dự án, SAP sinh ra 4 class. Đừng hoảng!
- **MPC (Model Provider Class):** Định nghĩa cấu trúc (Dữ liệu có cột nào, kiểu gì). *Ít khi phải đụng vào code.*
- **DPC (Data Provider Class):** Chứa logic lấy dữ liệu (Code SELECT, INSERT nằm ở đây). *Đây là nơi bạn làm việc chính.*

2. Quy tắc "EXT" (Extension):

- SAP luôn sinh ra 1 class gốc (ví dụ ZCL...DPC) và 1 class mở rộng (ZCL...DPC_EXT).
-  **TUYỆT ĐỐI KHÔNG** sửa code trong class gốc (DPC). Vì khi Regenerate, code của bạn sẽ mất sạch.
-  **LUÔN VIẾT CODE** trong class mở rộng (DPC_EXT).

3. Tư duy RESTful:

- OData dựa trên chuẩn REST. Hãy nhớ 4 phương thức tương ứng với CRUD:
 - **GET:** Lấy dữ liệu (GetEntitySet = List, GetEntity = Detail).
 - **POST:** Tạo mới (CreateEntity).

- **PUT/PATCH:** Cập nhật (UpdateEntity).
- **DELETE:** Xóa (DeleteEntity).

C. MỤC TIÊU BÀI HỌC

1. **SEGW:** Tạo dự án Gateway, định nghĩa Entity Type (Student) và Entity Set (Students).
2. **Mapping:** Map cấu trúc OData với bảng Database ZSTUDENT.
3. **Implementation:** Viết code ABAP trong method STUDENTSET_GET_ENTITYSET để trả về danh sách.
4. **Registration:** Đăng ký Service để nó hoạt động trên trình duyệt.
5. **Testing:** Sử dụng **SAP Gateway Client** để test API.

D. NỘI DUNG THỰC HÀNH (LAB 12): EXPOSING STUDENT DATA

Tình huống: Ứng dụng Fiori cần hiển thị danh sách sinh viên thực tế từ bảng ZSTUDENT_DEV331 (đã tạo ở Buổi 3). Bạn cần tạo một OData Service để cung cấp dữ liệu này.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo Project OData (SEGW)

- Vào T-Code SEGW. Nhấn nút **Create Project** (Tờ giấy trắng).
- Project: ZGW_LMS_DEV331. Description: "LMS OData Service".
- Package: ZSPR26_SE1876_DEV331.
- Nhấn OK.

Bước 2: Định nghĩa Data Model

- Chuột phải vào **Data Model** -> **Import** -> **DDIC Structure**.

- Nhập tên Structure: ZSTUDENT_DEV331 (Tên bảng DB).
- Nhập tên Object mới (Entity Type): Student.
- Nhấn Next -> Chọn các cột muốn hiển thị (MANDT, STUDENT_ID, FULL_NAME, DOB, COURSE).
- **Quan trọng:** Tích chọn ô **Key** cho dòng STUDENT_ID.
- Nhấn Finish.

Bước 3: Generate Runtime Objects

- Nhấn nút **Generate Runtime Objects** (Hình vòng tròn đỏ trắng giống Pokeball).
- Giữ nguyên tên các Class (MPC, DPC). Nhấn Enter (Continue).
- Chọn Package -> Save. (Lúc này SAP đang tự viết hàng nghìn dòng code cho bạn).

Bước 4: Viết Code Logic (ABAP Implementation)

- Mở thư mục **Service Implementation** -> **StudentSet**.
- Chuột phải vào **GetEntitySet (Query)** -> Chọn **Go to ABAP Workbench**.
- Hệ thống sẽ mở Class ZCL_ZGW_LMS_..._DPC_EXT.
- Bạn sẽ thấy method STUDENTSET_GET_ENTITYSET đang ở chế độ hiển thị.
- Nhấn nút **Edit** -> Đặt con trỏ vào method -> Nhấn nút **Redefine** (Mũi tên đỏ).
- Viết code SELECT (xem mẫu dưới).

Bước 5: Đăng ký Service (Register)

- Mở T-Code /IWFND/MAINT_SERVICE (Front-end Server).
- Nhấn nút **Add Service**.
- System Alias: LOCAL (Chọn Local Gateway).

- Nhấn **Get Services**. Tìm ZGW_LMS_DEV331_SRV.
- Click vào nó -> Nhấn **Add Selected Services**.
- Package: \$TMP (hoặc package của bạn). Nhấn OK.

Bước 6: Test Service

- Quay lại màn hình chính /IWFND/MAINT_SERVICE.
- Tìm service vừa thêm. Nhấn nút **SAP Gateway Client** (Góc dưới).
- Nhấn **Execute** (F8). Bạn sẽ thấy dữ liệu dạng XML/JSON trả về.

2. Code Mẫu ABAP (Trong DPC_EXT)

Method: STUDENTSET_GET_ENTITYSET

Mục đích: Trả về danh sách sinh viên.

code Abap

METHOD studentset_get_entityset.

* Method này được gọi khi URL là: ../sap/opu/odata/sap/ZGW.../StudentSet

* Nhiệm vụ: Lấy dữ liệu từ DB bỏ vào tham số xuất ET_ENTITYSET

" 1. Khai báo bảng nội bộ trùng cấu trúc với EntitySet

" et_entityset là tham số có sẵn của method

" 2. Select dữ liệu từ bảng ZSTUDENT

" Lưu ý: Cần chuyển đổi tên cột nếu tên trong OData khác tên trong DB

" Ở bài này ta Import từ DDIC nên tên giống hệt nhau -> Dùng CORRESPONDING

SELECT *

FROM zstudent dev331

INTO CORRESPONDING FIELDS OF TABLE @et_entityset.

```
" 3. Sắp xếp (Optional)
SORT et_entityset BY student_id.
```

```
ENDMETHOD.
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên cần nâng cấp OData Service để hỗ trợ các tính năng mạnh mẽ hơn.

Challenge 1: Get Single Entity (Chi tiết sinh viên)

- **Vấn đề:** Hiện tại chỉ lấy được cả danh sách. Nếu gọi URL .../StudentSet('SE001') sẽ lỗi.
- **Yêu cầu:** Redefine method STUDENTSET_GET_ENTITY (Lưu ý: Không có chữ SET ở cuối).
- **Logic:**
 1. Đọc Key từ tham số it_key_tab (Chứa StudentID người dùng gửi lên).
 2. SELECT SINGLE từ DB theo ID đó.
 3. Gán kết quả vào er_entity.

Challenge 2: Filtering (\$filter)

- **Vấn đề:** Khi Frontend gọi .../StudentSet?\$filter=Course eq 'K17', code hiện tại vẫn trả về tất cả (vì ta dùng SELECT * không điều kiện).
- **Yêu cầu:** Sửa method GET_ENTITYSET.
 - Đọc điều kiện lọc từ tham số io_tech_request_context.
 - Chuyển nó thành WHERE clause trong câu lệnh SQL.
- **Gợi ý:** Code mẫu bên dưới.

Challenge 3: Create Entity (Tạo mới từ Web)

- **Yêu cầu:** Cho phép tạo sinh viên mới từ Postman hoặc Fiori.

- **Nhiệm vụ:** Redefine method STUDENTSET_CREATE_ENTITY.
 - Dùng io_data_provider->read_entry_data để lấy cục JSON người dùng gửi lên.
 - INSERT vào bảng ZSTUDENT.

Challenge 4: Exception Handling (Trả về lỗi chuẩn)

- **Yêu cầu:** Trong method GET_ENTITY (Lấy chi tiết), nếu không tìm thấy sinh viên (sy-subrc <> 0).
- **Nhiệm vụ:** Ném ra lỗi HTTP 404 (Not Found) thay vì để hệ thống crash hoặc trả về rỗng.
- **Gợi ý:** Dùng RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception.

4. Lời giải tham khảo (Solution Snippets)

A. Solution Challenge 1 (Get Entity - Detail)

```
code Abap
METHOD studentset_get_entity.
  DATA: ls_key_tab TYPE /iwbep/s_mgw_name_value_pair,
        lv_id      TYPE zstudent_dev331-student_id.

  " 1. Lấy Key (Mã SV) từ URL
  READ TABLE it_key_tab INTO ls_key_tab WITH KEY name = 'StudentId'.
  IF sy-subrc = 0.
    lv_id = ls_key_tab-value.
  ENDIF.

  " 2. Truy vấn DB
  SELECT SINGLE *
    FROM zstudent_dev331
    INTO CORRESPONDING FIELDS OF @er_entity
    WHERE student_id = @lv_id.
```

```

IF sy-subrc <> 0.
" Challenge 4: Raise Exception 404
RAISE EXCEPTION TYPE /iwbep/cx_mgw_busi_exception
EXPORTING
    textid    = /iwbep/cx_mgw_busi_exception=>resource_not_found
    entity_type = 'Student'.
ENDIF.
ENDMETHOD.

```

B. Solution Challenge 2 (Handle Filter)

```

code Abap
" Trong GET_ENTITYSET
DATA: lt_filter_select_options TYPE /iwbep/t_mgw_select_option,
      lr_course_range         TYPE RANGE OF zstudent_dev331-course.

" 1. Lấy thông tin filter từ Request
lt_filter_select_options = io_tech_request_context->get_filter( )->get_filter_select_options( ).

" 2. Chuyển đổi Filter của OData sang Range Table của ABAP
" Tìm filter cho trường 'Course'
READ TABLE lt_filter_select_options INTO DATA(ls_filter) WITH KEY property = 'Course'.
IF sy-subrc = 0.
    lr_course_range = ls_filter-select_options.
ENDIF.

" 3. Select với Range (Nếu range rỗng -> Lấy hết)
SELECT *
FROM zstudent_dev331
INTO CORRESPONDING FIELDS OF TABLE @et_entityset
WHERE course IN @lr_course_range.

```

Kết thúc Buổi 12.

Giờ đây, chúng ta đã có một **OData Service** (ZGW_LMS_..._SRV) hoạt động hoàn hảo. Nó sẵn sàng phục vụ dữ liệu cho bất kỳ ai: Ứng dụng Fiori, Mobile App, hay thậm chí là Excel.

BUỔI 13: FIORI ELEMENTS & KẾT NỐI ODATA (REAL DATA)

A. THÔNG TIN & QUY TẮC

- **Công cụ:** Visual Studio Code (Đã cài SAP Fiori Tools Extension) hoặc SAP Business Application Studio (BAS).
 - **Project Name:** zfiori_student_[AccountID] (Ví dụ: zfiori_student_dev331).
 - **OData Service:** ZGW_LMS_[AccountID]_SRV (Đã tạo ở Buổi 12).
 - **Chủ đề:** Ứng dụng Quản lý Sinh viên (List Report) với dữ liệu thật.
-

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Chiến lược phát triển Frontend trong dự án S/4HANA:

1. Fiori Elements vs. Freestyle UI5:

- **Freestyle UI5 (Như buổi 11):** Bạn tự code mọi thứ (View, Controller). Dừng khi nghiệp vụ quá phức tạp, giao diện phá cách. Tốn công, khó bảo trì.
- **Fiori Elements (Hôm nay):** Bạn không code JS/XML. Bạn chỉ cấu hình (Annotation). SAP tự sinh ra giao diện chuẩn. **80% báo cáo trong S/4HANA dùng cách này.** Nhanh gấp 10 lần.

2. Manifest.json là trái tim:

- Trong Fiori App, file manifest.json quan trọng hơn cả code. Nó định nghĩa: App này tên gì? Gọi OData nào? Màn hình đầu tiên là gì? Cấu hình điều hướng (Navigation) ra sao?

3. Local vs. Remote:

- Khi phát triển (Dev), chúng ta chạy trên localhost nhưng kết nối dữ liệu thật từ Server SAP. Cơ chế này gọi là **Proxy**. Bạn cần file ui5.yaml để cấu hình việc này (tránh lỗi CORS).

C. MỤC TIÊU BÀI HỌC

1. **Environment:** Thiết lập dự án Fiori trong VS Code.
2. **Connection:** Kết nối ứng dụng với OData Service ZGW_LMS... từ backend.
3. **Fiori Elements:** Tạo ứng dụng dạng **List Report Object Page** (Danh sách -> Chi tiết).
4. **Annotations:** Sử dụng Annotation (Ghi chú dữ liệu) để bảo hệ thống biết cột nào cần hiện, cột nào cần ẩn.

D. NỘI DUNG THỰC HÀNH (LAB 13): STUDENT LIST REPORT

Tình huống: Hiệu trưởng cần xem danh sách sinh viên từ hệ thống thật. Yêu cầu giao diện chuẩn Fiori: Có bộ lọc phía trên, bảng dữ liệu phía dưới, xuất Excel được.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Khởi tạo dự án (Dùng Fiori Generator)

- Mở VS Code. Nhấn F1 (hoặc Ctrl+Shift+P).
- Gõ và chọn: **Fiori: Open Application Generator**.
- **Application Type:** chọn SAP Fiori Elements.
- **Floorplan:** chọn List Report Object Page (Mẫu phổ biến nhất).
- Nhấn Next.

Bước 2: Kết nối Data Source (OData)

- **Data Source:** chọn Connect to a System.
- **System:** Chọn hệ thống SAP lớp học (hoặc nhập URL nếu chưa lưu).
- **Service:** Tìm ZGW_LMS_DEV331_SRV (Service bạn tạo ở Buổi 12).
- Nhấn Next.

Bước 3: Cấu hình Entity

- **Main Entity:** StudentSet (Bảng chính).
- **Navigation Entity:** (Để trống hoặc chọn nếu có bảng con).
- Nhấn Next.

Bước 4: Thông tin Dự án

- **Module Name:** zfiori_student_dev331.
- **App Title:** Student Management.
- **Namespace:** lms.fpt.
- **Project Path:** Chọn thư mục lưu trên máy.
- Nhấn Finish. (Chờ máy tải thư viện Node_modules về, hơi lâu chút).

Bước 5: Chạy ứng dụng

- Sau khi cài xong, chuột phải vào file webapp/index.html -> Chọn **Preview Application**.
- Trình duyệt bật lên. Đăng nhập tài khoản SAP.
- *Kết quả:* Bạn sẽ thấy một khung trang web Fiori, nhưng... **Bảng dữ liệu trống trơn và không có cột nào?**
 - *Lý do:* Fiori Elements cần biết bạn muốn hiện cột nào. Ta cần làm **Annotations**.

Bước 6: Thêm Annotations (Hiển thị cột)

- Trong VS Code, mở folder webapp -> annotations -> annotation.xml (Hoặc dùng **Guided Development** nếu đã cài extension).
- *Cách nhanh:* Dùng **Fiori Tools - XML Annotation Editor**.
 - Chọn Target: Student.
 - Thêm Annotation: UI.LineItem (Cấu hình các cột trên bảng).

- Thêm các DataField: StudentId, FullName, Course.
- Thêm Annotation: UI.SelectionFields (Bộ lọc phía trên).
- Thêm Field: StudentId, Course.

2. Code Mẫu Annotation (Local Annotation)

Nếu bạn không dùng tool đồ họa, hãy paste code này vào file annotation.xml trong thư mục webapp/annotations.

code Xml

```
<edmx:Edmx xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Version="4.0">
  <edmx:Reference Uri="https://sap.github.io/odata-vocabularies/vocabularies/UI.xml">
    <edmx:Include Alias="UI" Namespace="com.sap.vocabularies.UI.v1"/>
  </edmx:Reference>
  <edmx:DataServices>
    <Schema xmlns="http://docs.oasis-open.org/odata/ns/edm" Namespace="local">
      <!-- Target trỏ vào Entity Type 'Student' trong OData -->
      <Annotations Target="ZGW_LMS_DEV331_SRV.Student">

        <!-- 1. UI.LineItem: Định nghĩa các cột hiển thị trên bảng -->
        <Annotation Term="UI.LineItem">
          <Collection>
            <Record Type="UI.DataField">
              <PropertyValue Property="Value" Path="StudentId"/>
              <PropertyValue Property="Label" String="Mã Sinh Viên"/>
            </Record>
            <Record Type="UI.DataField">
              <PropertyValue Property="Value" Path="FullName"/>
              <PropertyValue Property="Label" String="Họ và Tên"/>
            </Record>
            <Record Type="UI.DataField">
              <PropertyValue Property="Value" Path="Course"/>
              <PropertyValue Property="Label" String="Khóa Học"/>
            </Record>
          </Collection>
        </Annotation>
      </Annotations>
    </Schema>
  </edmx:DataServices>
</edmx:Edmx>
```

```
</Record>
<Record Type="UI.DataField">
  <PropertyValue Property="Value" Path="Dob"/>
  <PropertyValue Property="Label" String="Ngày Sinh"/>
</Record>
</Collection>
</Annotation>
```

<!-- 2. UI.SelectionFields: Định nghĩa bộ lọc tìm kiếm -->

```
<Annotation Term="UI.SelectionFields">
  <Collection>
    <PropertyPath>StudentId</PropertyPath>
    <PropertyPath>Course</PropertyPath>
  </Collection>
</Annotation>
```

<!-- 3. UI.HeaderInfo: Tiêu đề khi vào trang chi tiết -->

```
<Annotation Term="UI.HeaderInfo">
  <Record>
    <PropertyValue Property="TypeName" String="Student"/>
    <PropertyValue Property="TypeNamePlural" String="Students"/>
    <PropertyValue Property="Title">
      <Record Type="UI.DataField">
        <PropertyValue Property="Value" Path="FullName"/>
      </Record>
    </PropertyValue>
  </Record>
</Annotation>
```

```
</Annotations>
```

```
</Schema>
```

```
</edmx:DataServices>
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên cần tùy biến ứng dụng Fiori Elements để nó thông minh hơn.

Challenge 1: Value Help (Dropdown chọn khóa học)

- **Vấn đề:** Ở bộ lọc Course, người dùng phải gõ tay "K17", "K18".
- **Yêu cầu:** Biến nó thành một danh sách thả xuống (Dropdown/F4 Help) lấy dữ liệu từ bảng ZCOURSE (OData).
- **Gợi ý:** Cần thêm Annotation @Consumption.valueHelpDefinition trong CDS View (nếu dùng RAP) hoặc cấu hình Common.ValueList trong file annotation trỏ tới Entity CourseSet.

Challenge 2: Criticality (Tô màu trạng thái)

- **Yêu cầu:** Nếu sinh viên thuộc khóa "K15" (Sắp ra trường), hãy tô màu đỏ cột Khóa học. Nếu "K18" (Mới vào), tô màu xanh.
- **Gợi ý:**
 1. Trong OData (Backend), thêm logic trả về cột ColorCode (1=Đỏ, 2=Vàng, 3=Xanh).
 2. Trong Annotation UI.LineItem, thêm thuộc tính Criticality trỏ vào cột ColorCode.

Challenge 3: Default Sort (Sắp xếp mặc định)

- **Yêu cầu:** Khi mở App lên, danh sách phải tự động sắp xếp theo Tên (A->Z).
- **Gợi ý:** Cấu hình trong manifest.json hoặc dùng Annotation UI.PresentationVariant.

Challenge 4: Export to Excel

- **Yêu cầu:** Tìm nút "Export to Spreadsheet" trên giao diện Fiori Elements. (Nó có sẵn!).
- **Nhiệm vụ:** Thử xuất ra file, mở lên xem có đủ tiếng Việt có dấu không. (Fiori Elements xử lý việc này tự động, sinh viên chỉ cần verify).

4. Lời giải tham khảo (Config Snippets)

A. Cấu hình PresentationVariant (Sắp xếp mặc định)

Dán đoạn này vào trong annotation.xml:

```
code Xml
<Annotation Term="UI.PresentationVariant">
  <Record>
    <PropertyValue Property="SortOrder">
      <Collection>
        <Record>
          <PropertyValue Property="Property" PropertyPath="FullName"/>
          <PropertyValue Property="Descending" Bool="false"/>
        </Record>
      </Collection>
    </PropertyValue>
    <PropertyValue Property="Visualizations">
      <Collection>
        <AnnotationPath>@UI.LineItem</AnnotationPath>
      </Collection>
    </PropertyValue>
  </Record>
</Annotation>
```

Kết thúc Buổi 13.

Sinh viên đã tạo được một ứng dụng Fiori "Xin sò", kết nối dữ liệu thật, có bộ lọc, có sắp xếp mà gần như không phải viết dòng code JS nào. Đây là sức mạnh của **Low-code/No-code** trong hệ sinh thái SAP.

BUỔI 14: ANALYTICAL APPS (BIỂU ĐỒ) & GỬI EMAIL TỰ ĐỘNG

A. THÔNG TIN & QUY TẮC

- **Frontend Tech:** `sap.viz.ui5.controls.VizFrame` (Thư viện biểu đồ của SAP).
- **Backend Class:** `CL_BCS` (Business Communication Service) - *Chuẩn gửi mail hướng đối tượng.*
- **Transaction:** SCOT (Cấu hình Mail Server - *Giảng viên sẽ cấu hình, SV chỉ cần dùng*), SOST (Xem hàng đợi gửi mail).
- **Chủ đề:** Dashboard Thông kê Tuyển sinh & Cảnh báo sĩ số.

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Làm sao để làm Analytics và Email chuẩn Enterprise?

1. **Đừng Spam Email từ hệ thống DEV:**

- Trong môi trường Dev/Test, email thường bị chặn không gửi ra ngoài internet (để tránh spam nhầm khách hàng thật).
- Chúng ta kiểm tra mail gửi đi bằng T-Code **SOST** (SAPconnect Send Requests). Nếu thấy trạng thái màu vàng/xanh ở đây nghĩa là code đã chạy đúng.

2. **HTML Body là bắt buộc:**

- Đừng gửi email text trơn tuột. Hãy dùng HTML để kẻ bảng, tô màu. Sếp thích đọc email đẹp.
- *Mẹo:* Tạo một Class `ZCL_HTML_BUILDER` để cộng chuỗi HTML cho gọn code.

3. **Analytics: Client vs. Server aggregation:**

- Nếu dữ liệu ít (< 1000 dòng): Lấy hết về Frontend (UI5) rồi để biểu đồ tự đếm/nhóm (Client-side).

- Nếu dữ liệu lớn (> 1 triệu dòng): Phải dùng SELECT SUM... GROUP BY ở Backend hoặc dùng **CDS Views** để tính toán trước, chỉ gửi kết quả cuối cùng ra ngoài.

C. MỤC TIÊU BÀI HỌC

1. **VizFrame:** Tạo biểu đồ cột (Column Chart) trên ứng dụng SAPUI5 để thống kê sinh viên theo khóa học.
2. **CL_BCS:** Viết chương trình ABAP gửi email có định dạng HTML và tệp đính kèm.
3. **Process Integration:** Tư duy về việc kết hợp: Khi số vượt ngưỡng -> Tự động gửi mail.

D. NỘI DUNG THỰC HÀNH (LAB 14)

Chúng ta chia làm 2 phần nhỏ: Frontend (Biểu đồ) và Backend (Email).

PHẦN 1: FRONTEND - BIỂU ĐỒ THỐNG KÊ (SAPUI5)

Tình huống: Tạo một View mới hiển thị số lượng sinh viên theo từng khóa (K17, K18...).

Code View (XML):

Sử dụng VizFrame để vẽ biểu đồ.

Copy đoạn này vào file View của dự án UI5 (hoặc tạo dự án test mới).

code Xml

```
<mvc:View controllerName="lms.analytics.Main" xmlns="sap.m" xmlns:mvc="sap.ui.core.mvc"
  xmlns:viz="sap.viz.ui5.controls" xmlns:viz.feeds="sap.viz.ui5.controls.common.feeds"
  xmlns:viz.data="sap.viz.ui5.data">
  <Page title="Student Enrollment Analytics">
    <content>
      <viz:VizFrame id="idVizFrame" uiConfig="{applicationSet:'fiori'}"
        height="100%" width="100%" vizType="column">

        <!-- 1. Định nghĩa dữ liệu (Dataset) -->
```

```

<viz:dataset>
  <viz.data:FlattenedDataset data="{/enrollmentStats}">
    <viz.data:dimensions>
      <viz.data:DimensionDefinition name="Course" value="{Course}" />
    </viz.data:dimensions>
    <viz.data:measures>
      <viz.data:MeasureDefinition name="StudentCount" value="{Count}" />
    </viz.data:measures>
  </viz.data:FlattenedDataset>
</viz:dataset>

<!-- 2. Định nghĩa trục (Feeds) -->
<viz:feeds>
  <viz.feeds:FeedItem uid="valueAxis" type="Measure" values="StudentCount" />
  <viz.feeds:FeedItem uid="categoryAxis" type="Dimension" values="Course" />
</viz:feeds>

</viz:VizFrame>
</content>
</Page>
</mvc:View>

```

Code Controller (JS):

Tạo dữ liệu giả (Mock Data) để test biểu đồ.

code JavaScript

downloadcontent_copy

expand_less

```

onInit: function() {
  var oData = {
    enrollmentStats: [
      { Course: "K16", Count: 120 },
      { Course: "K17", Count: 250 },
      { Course: "K18", Count: 300 },
    ]
  };
}

```

```

        { Course: "K19", Count: 150 }
    ]
};
var oModel = new sap.ui.model.json.JSONModel(oData);
this.getView().setModel(oModel);

// Cấu hình thêm cho biểu đồ đẹp hơn (Optional)
var oVizFrame = this.byId("idVizFrame");
oVizFrame.setVizProperties({
    title: { visible: true, text: "Students per Course" },
    plotArea: { dataLabel: { visible: true } }
});
}

```

PHẦN 2: BACKEND - GỬI EMAIL THÔNG BÁO (ABAP)

Tình huống: Viết một chương trình ABAP (ZDEV331_LAB14_EMAIL) gửi email báo cáo tình hình tuyển sinh cho Admin.

Code Mẫu ABAP (Sử dụng CL_BCS):

```

code Abap
*&-----*
*& Report ZDEV331 LAB14 EMAIL
*&-----*
REPORT zdev331_lab14_email.

PARAMETERS: pa_email TYPE ad_smtpadr DEFAULT 'admin@fpt.edu.vn'.

START-OF-SELECTION.

TRY.
    " 1. Tạo Request Gửi Mail (BCS Request)
    DATA(lo_send_request) = cl_bcs=>create_persistent( ).

```

" 2. Tạo nội dung Email (HTML)

DATA: lt_body TYPE bcsy_text.

APPEND '<html><body>' TO lt_body.

APPEND '<h2>LMS Daily Report</h2>' TO lt_body.

APPEND '<p>Xin chào Admin,</p>' TO lt_body.

APPEND '<p>Hệ thống LMS hoạt động ổn định. Số liệu K18 tăng đột biến.</p>' TO lt_body.

APPEND '<p style="color:red">Vui lòng kiểm tra Server.</p>' TO lt_body.

APPEND '</body></html>' TO lt_body.

" Tạo Document từ nội dung trên

DATA(lo_document) = cl_document_bcs=>create_document(

 i_type = 'HTM' " Định dạng HTML

 i_text = lt_body " Nội dung

 i_subject = 'FPT LMS Alert: Daily Statistics'). " Tiêu đề

" Gắn Document vào Request

lo_send_request->set_document(lo_document).

" 3. Thêm người nhận (Recipient)

DATA(lo_recipient) = cl_cam_address_bcs=>create_internet_address(pa_email).

lo_send_request->add_recipient(

 i_recipient = lo_recipient

 i_express = abap_true). " Gửi nhanh (Express)

" 4. Gửi ngay lập tức (Send)

DATA(lv_sent_to_all) = lo_send_request->send(i_with_error_screen = abap_true).

IF lv_sent_to_all = abap_true.

 COMMIT WORK. " Quan trọng: Phải Commit thì mail mới vào hàng đợi

 WRITE: / 'Email sent successfully to', pa_email.

```
WRITE: / 'Check T-Code SOST to verify.'.
ELSE.
WRITE: / 'Failed to send email.'.
ENDIF.

CATCH cx_bcs INTO DATA(lx_bcs).
WRITE: / 'Error:', lx_bcs->get_text( ).
ENDTRY.
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: Integration (Frontend gọi Backend)

- **Vấn đề:** Hiện tại code gửi mail đang chạy độc lập ở Backend.
- **Yêu cầu:** Trên màn hình SAPUI5, thêm một nút "Send Report to Admin". Khi bấm nút này, gọi OData Service (Function Import) để kích hoạt chương trình gửi mail ở Backend.
- **Gợi ý:**
 1. Trong SEGW, tạo Function Import SendReport.
 2. Trong DPC_EXT, method EXECUTE_ACTION, gọi code gửi mail.

Challenge 2: Dynamic Content (Nội dung động)

- **Yêu cầu:** Nội dung email không được ghi cứng ("K18 tăng đột biến").
- **Logic:**
 1. Trong ABAP, SELECT COUNT từ bảng ZSTUDENT.
 2. Ghép con số đó vào chuỗi HTML: APPEND |<p>Tổng số sinh viên hiện tại: { lv_count }</p>| TO lt_body.

Challenge 3: Excel Attachment (Đính kèm file)

- **Yêu cầu:** Gửi email kèm theo một file danh sách sinh viên (Students.xls).

- **Gợi ý:**

1. Chuyển đổi dữ liệu bảng thành định dạng Binary (hoặc String tab-delimited).
2. Dùng method lo_document->add_attachment(... i_attachment_type = 'XLS' ...).

Challenge 4: Background Job (Tự động hóa)

- **Yêu cầu:** Không cần ai bấm nút cả. Thiết lập để hệ thống tự động chạy chương trình gửi mail vào 8:00 sáng mỗi ngày.

- **Hướng dẫn:**

1. Vào T-Code **SM36** (Define Job).
2. Tạo Job tên **Z_LMS_DAILY_REPORT**.
3. Step: Chọn chương trình **ZDEV331_LAB14_EMAIL**.
4. Start Condition: Date/Time (Chọn ngày mai, 08:00, tích Periodic Job -> Daily).

4. Lời giải tham khảo (Solution Snippet - Challenge 3)

code Abap

" --- Logic đính kèm file Excel ---

```
DATA: lt_attachment TYPE solix_tab,  
      lv_string     TYPE string,  
      lv_xstring    TYPE xstring.
```

" 1. Tạo nội dung file (Tab delimited)

```
lv_string = 'StudentID   FullName   Course' && cl_abap_char_utilities=>cr_lf.  
lv_string = lv_string && 'SE001 Nguyen Van A       K17'.
```

" 2. Chuyển String sang Binary (XString rồi sang Binary Tab)

```
lv_xstring = cl_bcs_convert=>string_to_xstring( lv_string ).
```

```
cl_bcs_convert=>xstring_to_solix(  
    EXPORTING  
        iv_xstring = lv_xstring  
    IMPORTING  
        et_solix   = lt_attachment ).
```

" 3. Đính kèm vào mail

```
lo_document->add_attachment(  
    i_attachment_type   = 'xls'  
    i_attachment_subject = 'Student_List.xls'  
    i_att_content_hex   = lt_attachment ).
```

Kết thúc Buổi 14.

Sinh viên đã hoàn thiện các tính năng nâng cao: Báo cáo trực quan và Hệ thống thông báo tự động.

BUỔI 15: UPLOAD FILE, GEN-AI INTEGRATION & TỔNG KẾT

A. THÔNG TIN & QUY TẮC

- **Package:** ZSPR26_SE1876_[AccountID]
- **Program:** Z[AccountID]_LAB15_FINAL
- **File mẫu:** Student_Import.txt (Dạng Tab-delimited hoặc CSV).
- **API:** OpenAI API (Giả lập cấu hình kết nối).

● B. REAL-WORLD TIPS & CHEATS (KINH NGHIỆM THỰC CHIẾN)

Những bài học xương máu khi làm việc với Dữ liệu lớn và API:

1. **Đừng Upload Excel (XLSX) ở Backend nếu không cần thiết:**

- Đọc file .xlsx trong ABAP rất nặng và phức tạp (cần thư viện OLE hoặc ABAP2XLSX).
- **Mẹo:** Yêu cầu người dùng lưu file dưới dạng **Text (Tab delimited)** hoặc **CSV**. Xử lý string trong ABAP cực nhanh (lên tới hàng triệu dòng).

2. **Đừng Hardcode API Key:**

- Khi gọi GenAI, tuyệt đối không viết `v_token = 'sk-...'` trong code. Nếu lộ code, bạn mất tiền.
- **Chuẩn:** Lưu API Key trong **SM59** (RFC Destination) hoặc bảng cấu hình bảo mật.

3. **Timeout khi gọi API:**

- AI trả lời khá chậm (3-10 giây). Đừng để user chờ. Hãy dùng kỹ thuật **Background Task** (gọi AI ngầm, khi nào xong thì báo user).

C. MỤC TIÊU BÀI HỌC

1. **File Interface:** Sử dụng CL_GUI_FRONTEND_SERVICES để upload file từ máy tính lên chương trình ABAP.
2. **Data Parsing:** Tách chuỗi (Split String) để chuyển dữ liệu thô thành bảng nội bộ.
3. **HTTP Client:** Sử dụng CL_HTTP_CLIENT để gọi API của OpenAI (ChatGPT) từ ABAP.
4. **JSON Handling:** Parse kết quả JSON trả về từ AI.

D. NỘI DUNG THỰC HÀNH (LAB 15)

PHẦN 1: UPLOAD DANH SÁCH SINH VIÊN (MASS UPLOAD)

Tình huống: Đầu kỳ học, phòng đào tạo gửi một file danh sách 1000 sinh viên. Bạn cần import vào bảng ZSTUDENT.

File mẫu (students.txt):

code Text

SE001 Nguyen Van A K17

SE002 Tran Thi B K18

Code ABAP:

code Abap

REPORT zdev331_lab15_final.

--- 1. KHAI BÁO DỮ LIỆU ---

TYPES: BEGIN OF ty_upload,
 line TYPE string, " Dòng thô
END OF ty_upload.

DATA: lt_raw_data TYPE TABLE OF ty_upload,
 lt_students TYPE TABLE OF zstudent_dev331,
 ls_student TYPE zstudent_dev331.

PARAMETERS: pa_file TYPE string DEFAULT 'C:\students.txt'.

START-OF-SELECTION.

" 2. Upload file từ máy Client

CALL METHOD cl_gui_frontend_services=>gui_upload

EXPORTING

filename = pa_file

filetype = 'ASC' " ASCII (Text)

CHANGING

data_tab = lt_raw_data

EXCEPTIONS

OTHERS = 1.

IF sy-subrc <> 0.

WRITE: / 'Error uploading file.'

RETURN.

ENDIF.

" 3. Xử lý dữ liệu (Parsing)

LOOP AT lt_raw_data INTO DATA(ls_raw).

" Tách chuỗi dựa trên dấu Tab (hoặc dấu phẩy)

SPLIT ls_raw-line AT cl_abap_char_utilities=>horizontal_tab

INTO ls_student-student_id

ls_student-full_name

ls_student-course.

" Validate cơ bản

IF ls_student-student_id IS NOT INITIAL.

APPEND ls_student TO lt_students.

ENDIF.

```
ENDLOOP.
```

```
" 4. Lưu vào Database (Insert hàng loạt)
```

```
IF lt_students IS NOT INITIAL.
```

```
  MODIFY zstudent_dev331 FROM TABLE lt_students.
```

```
  IF sy-subrc = 0.
```

```
    WRITE: / 'Upload thành công:', lines( lt_students ), 'sinh viên.'.
```

```
  ENDIF.
```

```
ELSE.
```

```
  WRITE: / 'File rỗng hoặc sai định dạng.'.
```

```
ENDIF.
```

PHẦN 2: TÍCH HỢP GEN-AI (DEMO)

Tình huống: Sinh viên muốn hệ thống tự động tạo một đoạn "Giới thiệu bản thân" (Bio) dựa trên thông tin Mã SV và Tên, dùng để hiển thị trên profile LMS.

Code ABAP (Gọi API OpenAI):

```
code Abap
```

```
*--- 1. Cấu hình HTTP Client ---*
```

```
DATA: lo_http_client TYPE REF TO if_http_client,
```

```
      lv_url          TYPE string VALUE 'https://api.openai.com/v1/chat/completions',
```

```
      lv_api_key       TYPE string VALUE 'Bearer sk-proj-12345...', " Ví dụ Key (Không chạy thật)
```

```
      lv_json_body     TYPE string.
```

```
" Tạo Client kết nối tới URL
```

```
cl_http_client=>create_by_url(
```

```
  EXPORTING
```

```
    url          = lv_url
```

```
  IMPORTING
```

```
    client       = lo_http_client
```

```
  EXCEPTIONS
```

```
argument_not_found = 1
OTHERS              = 2 ).
```

IF sy-subrc = 0.

" 2. Thiết lập Header (Authentication)

```
lo_http_client->request->set_header_field(
    name = 'Authorization'
    value = lv_api_key ).
```

```
lo_http_client->request->set_header_field(
    name = 'Content-Type'
    value = 'application/json' ).
```

" 3. Thiết lập Body (Prompt gửi cho AI)

" Giả sử ta muốn viết bio cho Nguyen Van A

```
lv_json_body = '{ "model": "gpt-3.5-turbo", "messages": [ {"role": "user", "content": "Viết một đoạn giới thiệu ngắn (50 từ) cho sinh viên CNTT tên là Nguyen Van A, học tại FPT University."} ] }'.
```

```
lo_http_client->request->set_cdata( data = lv_json_body ).
```

" 4. Gửi Request (SEND)

```
lo_http_client->send(
    EXCEPTIONS
    http_communication_failure = 1
    http_invalid_state        = 2 ).
```

" 5. Nhận Response (RECEIVE)

```
lo_http_client->receive(
    EXCEPTIONS
    http_communication_failure = 1
    http_invalid_state        = 2
    http_processing_failed    = 3 ).
```

```
IF sy-subrc = 0.  
  " Lấy kết quả trả về  
  DATA(lv_response) = lo_http_client->response->get_cdata( ).  
  
  WRITE: / '--- AI RESPONSE ---'.  
  WRITE: / lv_response.  
  " Lưu ý: Response là JSON, cần dùng thư viện /ui2/cl_json để parse lấy nội dung text.  
ELSE.  
  WRITE: / 'Lỗi khi gọi AI API'.  
ENDIF.  
  
ENDIF.
```

3. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: JSON Parsing (Xử lý kết quả AI)

- **Vấn đề:** Kết quả lv_response trả về là một chuỗi JSON loằng ngoằng.
- **Yêu cầu:** Sử dụng class /ui2/cl_json=>deserialize để chuyển chuỗi đó thành cấu trúc ABAP và chỉ in ra phần nội dung text ("content") mà AI viết.
- **Mục tiêu:** Kỹ năng xử lý JSON trong ABAP Backend.

Challenge 2: Duplicate Check (Kiểm tra trùng lặp khi Upload)

- **Vấn đề:** Nếu file upload chứa Mã SV đã tồn tại trong Database, lệnh MODIFY sẽ ghi đè (hoặc INSERT sẽ lỗi).
- **Yêu cầu:** Trước khi MODIFY, hãy SELECT kiểm tra DB.
 - Nếu tồn tại: Bỏ qua và ghi vào log lỗi "Dòng X: Sinh viên đã tồn tại".
 - Nếu chưa: Mới thêm vào bảng insert.

Challenge 3: Prompt Engineering (Tối ưu hóa AI)

- **Yêu cầu:** Sửa biến lv_json_body để prompt thông minh hơn.
 - Input: Danh sách môn học và điểm số (Lấy từ bảng ZENROLL).
 - Prompt: "Dựa trên điểm số sau: Java 8.0, ABAP 9.0. Hãy đưa ra nhận xét ưu/nhược điểm và gợi ý nghề nghiệp cho sinh viên này."

Challenge 4: Transactional Processing (All or Nothing)

- **Yêu cầu:** Khi upload file 1000 dòng. Nếu dòng thứ 999 bị lỗi.
 - Logic hiện tại: 998 dòng đầu vẫn vào DB. Dòng 999 lỗi. Dữ liệu không nhất quán.
 - Nhiệm vụ: Dùng ROLLBACK WORK để hủy toàn bộ quá trình nếu có bất kỳ lỗi nào xảy ra. Chỉ COMMIT WORK khi cả 1000 dòng đều OK.

4. Tổng kết Khóa học & Lộ trình Tiếp theo

Chúc mừng các bạn! Chúng ta đã hoàn thành 15 buổi huấn luyện ABAP S/4HANA Project-based.

Hành trình chúng ta đã đi qua:

1. **Foundation (Buổi 1-3):** Từ "Hello World" đến làm chủ Database và Internal Tables.
2. **Core Skills (Buổi 4-8):** SQL Nâng cao, Modularization, OOP, Locking, Exceptions.
3. **Modern Reporting (Buổi 9-10):** Dynamic Report, Web Fundamentals.
4. **Full-stack Development (Buổi 11-13):** SAPUI5, OData, Fiori Elements.
5. **Intelligence & Integration (Buổi 14-15):** Analytics, Email, Upload, GenAI.

Sản phẩm cuối khóa:

Mỗi bạn hiện đang sở hữu một hệ thống LMS thu nhỏ (Package ZSPR26...) bao gồm:

- Bảng dữ liệu Sinh viên/Điểm số.
- Các API (OData) kết nối ra ngoài.
- Giao diện Web (Fiori) quản lý sinh viên.
- Các tiện ích thông báo và báo cáo.

Lời khuyên cho chặng đường tiếp theo:

- **Chứng chỉ:** Hãy ôn luyện để thi chứng chỉ **C_ABAPD_2309** (Certified Development Associate - ABAP Cloud).
- **Công nghệ mới:** Hãy tìm hiểu thêm về **RAP (RESTful ABAP Programming Model)** và **CAP (Cloud Application Programming)**. Đây là tương lai của SAP BTP.

Bài tập lớn cuối khóa:

Hãy dành 1 tuần để hoàn thiện toàn bộ project LMS, đóng gói code, viết tài liệu hướng dẫn sử dụng (User Guide) và nộp lại Package vào hệ thống.

ĐIỀU CHỈNH: TỪ CỔ ĐIỂN SANG MODERN ABAP (TRÊN ECLIPSE)

Công cụ sử dụng: Eclipse IDE cài đặt plugin ABAP Development Tools (ADT).

Tư duy: Không code Frontend bằng JS, mà dùng ABAP để cấu hình giao diện (Fiori Elements).

Phím Tắt Hữu Ích Khác Trong Eclipse

Phím Tắt	Chức Năng
Ctrl + Shift + F	Format toàn file hoặc đoạn được chọn
Ctrl + I	Chỉ căn lại indent (không đổi gì khác)
Ctrl + /	Comment / Uncomment dòng đang đứng
Ctrl + Shift + /	Comment cả block được bôi đen
Ctrl + D	Xóa cả dòng đang đứng
Alt + ↑ / ↓	Di chuyển cả dòng lên / xuống
Ctrl + Alt + F	Format và Activate cùng lúc
Ctrl + F3	Activate (kiểm tra lỗi và lưu)

BUỔI 9: CDS VIEWS - BÁO CÁO THỂ HỆ MỚI (THAY THẾ ALV)

- **Mục tiêu:** Thay vì dùng ALV cũ kỹ, sinh viên học cách tạo **Core Data Services (CDS) Views**. Đây là nền tảng của mọi báo cáo S/4HANA hiện đại.
- **Môi trường:** Eclipse ADT (Data Definition).
- **Nội dung:**
 1. Giới thiệu Eclipse ADT: Tạo ABAP Project, kết nối hệ thống.
 2. Tạo **Basic Interface View** (ZI_Student): Select dữ liệu từ bảng ZSTUDENT.

3. Tạo **Consumption View** (ZC_Student): View dùng để hiển thị báo cáo.
4. Sử dụng tính năng **Data Preview** (F8) trong Eclipse để xem kết quả như ALV.

- **Lab 9: Báo cáo Danh sách Sinh viên bằng CDS**

- Tạo file Data Definition.
- Viết code CDS (SQL-like syntax) để JOIN bảng Sinh viên và Điểm số.
- Thêm Annotation @EndUserText.label để đặt tiêu đề cột.
- Chạy F8 để xem báo cáo ngay trong Eclipse.

BUỔI 10: WEB CƠ BẢN TRONG ABAP (ICF SERVICES)

- **Mục tiêu:** Hiểu cách ABAP đóng vai trò là Web Server thông qua **ICF (Internet Communication Framework)**. Thay vì viết file HTML rồi, ta tạo bộ xử lý HTTP ngay trong Eclipse.
- **Môi trường:** Eclipse ADT (ABAP Class).
- **Nội dung:**
 1. Tạo một Handler Class implement interface IF_HTTP_EXTENSION.
 2. Viết code HTML trong ABAP String và trả về cho trình duyệt.
 3. Cấu hình T-Code SICF (Transaction này vẫn phải dùng GUI tích hợp trong Eclipse) để tạo đường dẫn Web.
- **Lab 10: Trang Web "Hello LMS" chạy từ ABAP Server**
 - Viết class ZCL_LMS_WEB_HANDLER.
 - Trong method HANDLE_REQUEST, dựng một trang HTML đơn giản hiển thị "Danh sách sinh viên" (Hardcode).
 - Truy cập trang web qua trình duyệt bằng URL của SAP Server.

BUỔI 11: RAP MODEL (BẮT ĐẦU VỚI BUSINESS OBJECT)

- **Mục tiêu:** Bắt đầu xây dựng ứng dụng Fiori theo mô hình RAP. Định nghĩa cấu trúc dữ liệu và hành vi (Behavior) ngay trong Eclipse.
- **Môi trường:** Eclipse ADT.
- **Nội dung:**
 1. **Root View Entity:** Định nghĩa gốc của đối tượng Sinh viên.
 2. **Behavior Definition (BDEF):** Khai báo các hành động (Create, Update, Delete) cho Entity.
 3. **Behavior Implementation (Class):** Viết code ABAP xử lý logic khi user bấm Save (ví dụ: Tự động sinh Mã SV).
- **Lab 11: Xây dựng Backend cho App Quản lý Sinh viên**
 - Tạo CDS View ZI_STUDENT_R (Root).
 - Tạo Behavior Definition ZI_STUDENT_R (Managed Implementation).
 - Kích hoạt tính năng Create/Update/Delete chuẩn.

BUỔI 12: SERVICE DEFINITION & BINDING (THAY THẾ SEGW/ODATA)

- **Mục tiêu:** Tạo OData Service siêu tốc bằng Eclipse (nhanh hơn cách cũ SEGW 10 lần).
- **Môi trường:** Eclipse ADT.
- **Nội dung:**
 1. **Service Definition:** Chọn những bảng/CDS nào muốn đưa ra ngoài Web.
 2. **Service Binding:** Chọn giao thức (OData V2 hoặc V4) và kích hoạt API.
 3. **Preview:** Eclipse tích hợp trình duyệt Fiori Elements Preview để test ngay lập tức.
- **Lab 12: Public OData Service cho LMS**

- Tạo Service Definition ZSD_LMS_STUDENT.
- Expose ZC_Student.
- Tạo Service Binding ZSB_LMS_STUDENT_V2.
- Nhấn nút **Preview** để thấy ứng dụng Fiori dạng danh sách chạy trên trình duyệt.

BUỔI 13: FIORI ELEMENTS VỚI METADATA EXTENSIONS (UI TRONG ECLIPSE)

- **Mục tiêu:** Tùy chỉnh giao diện Fiori (Cột nào hiện, bộ lọc nào hiện, nhãn là gì) hoàn toàn bằng code ABAP Annotation trong Eclipse.
- **Môi trường:** Eclipse ADT (Metadata Extension).
- **Nội dung:**
 1. Tạo **Metadata Extension (MDE)** cho CDS View.
 2. Sử dụng Annotation `@UI.lineItem` để cấu hình cột trong bảng.
 3. Sử dụng Annotation `@UI.selectionField` để cấu hình bộ lọc tìm kiếm.
 4. Sử dụng Annotation `@UI.identification` để cấu hình trang chi tiết (Detail Page).
- **Lab 13: Hoàn thiện giao diện Quản lý Sinh viên**
 - Thêm MDE ZX_STUDENT.
 - Cấu hình để cột "Điểm số" hiện màu đỏ nếu < 5 (Criticality).
 - Cấu hình Dropdown list (Value Help) cho cột "Khóa học".
 - Refresh lại màn hình Preview ở Buổi 12 để thấy giao diện thay đổi đẹp mắt.

BUỔI 14: ANALYTICAL CDS & EMAIL (TRÊN ECLIPSE)

- **Mục tiêu:** Tạo báo cáo phân tích đa chiều (Cube/Dimension) và viết logic gửi mail trong Class.
- **Môi trường:** Eclipse ADT.
- **Nội dung:**
 1. **Analytical CDS:** Tạo CDS View với annotation `@Analytics.query: true`. Định nghĩa Axis (Trục) và Measures (Số liệu).
 2. **ABAP Class for Email:** Viết class gửi mail dùng `CL_BCS`.
- **Lab 14: Dashboard Tuyển sinh & Cảnh báo**
 - Tạo CDS View `ZC_Student_Analytics` đếm số lượng sinh viên theo khóa.
 - Preview kết quả dưới dạng Pivot Table/Chart ngay trong Eclipse (hoặc trình duyệt).
 - Viết Class `ZCL_LMS_MAILER` để gửi email cảnh báo sĩ số. Chạy test bằng Console (F9).

BUỔI 15: ABAP CONSOLE, UPLOAD & GEN-AI INTEGRATION

- **Mục tiêu:** Viết các ứng dụng tiện ích, upload file và gọi AI, sử dụng **ABAP Console** trong Eclipse để hiển thị kết quả (thay vì màn hình GUI cũ).
- **Môi trường:** Eclipse ADT (Console App).
- **Nội dung:**
 1. **ABAP Console:** Sử dụng interface `IF_OO_ADT_CLASSRUN` để viết chương trình xuất ra màn hình Console của Eclipse (giống `System.out.println` của Java).
 2. **HTTP Client:** Gọi OpenAI API từ Class.
 3. **JSON Handling:** Parse JSON trong Class.
- **Lab 15: Tool Upload & AI Chatbot trong Console**
 - Viết Class `ZCL_LMS_AI_TOOL` implement `IF_OO_ADT_CLASSRUN`.

- Trong method MAIN, viết code gọi API ChatGPT.
- Input: Một chuỗi JSON sinh viên.
- Output: out->write(response) hiển thị nhận xét của AI ra màn hình Console Eclipse.
- *(Lưu ý: Upload file từ PC lên Eclipse ADT Class Run hơi hạn chế, nên ta sẽ giả lập dữ liệu string hoặc paste dữ liệu vào code để xử lý).*

TÓM TẮT SỰ KHÁC BIỆT KHI DÙNG ECLIPSE

1. **Không còn SE80/SEGW:** Tất cả tích hợp trong Project Explorer của Eclipse.
2. **Không code JS/XML thủ công:** Dùng **RAP Model** (CDS + Behavior + Service Binding) để sinh Fiori App. Đây là cách làm chuẩn của SAP hiện nay.
3. **Output:** Thay vì màn hình xám (GUI), kết quả là Fiori Web App hiện đại chạy trên trình duyệt (Preview).

BUỔI 9B: BÁO CÁO HIỆN ĐẠI VỚI CDS VIEWS (CORE DATA SERVICES)

A. THÔNG TIN & QUY TẮC (ECLIPSE ADT)

- **Công cụ:** Eclipse IDE (đã cài ABAP Development Tools).
- **Package:** ZSPR26_SE1876_[AccountID] (Add package này vào mục "Favorite Packages" trong Eclipse để dễ tìm).
- **Artifact mới: Data Definition (DDL Source).**
- **Quy tắc đặt tên CDS (Virtual Data Model - VDM):**
 - **Interface View (Tầng dưới):** ZI_[Tên_Entity]_[ID] (Ví dụ: ZI_STUDENT_DEV331).
 - **Consumption View (Tầng báo cáo):** ZC_[Tên_Báo_Cáo]_[ID] (Ví dụ: ZC_STUDENT_REPORT_DEV331).
- **Chủ đề:** Báo cáo Tổng hợp Kết quả Đào tạo (thay thế cho ALV Report).

● B. REAL-WORLD TIPS (TẠI SAO DỪNG ECLIPSE?)

1. Code-to-Data Paradigm:

- ABAP cũ: Lấy 1 triệu dòng từ DB về Server ABAP -> Dừng Loop xử lý -> Chậm.
- CDS View: Đẩy toàn bộ logic (cộng trừ nhân chia, join, case) xuống DB HANA xử lý -> Chỉ trả về kết quả cuối cùng -> Siêu nhanh.

2. Không cần nút Save:

- Eclipse tự động lưu khi bạn gõ. Bạn chỉ cần nhấn **Ctrl + F3** để kích hoạt (Activate).

3. F8 là chân ái:

- Trong GUI, F8 là chạy chương trình.
- Trong Eclipse, mở file CDS View và nhấn **F8**, bạn sẽ thấy dữ liệu hiển thị ngay lập tức (Data Preview) mà không cần viết 1 dòng code hiển thị nào.

4. Phím tắt "Thần thánh":

- Ctrl + Shift + A: Tìm kiếm mọi thứ (Bảng, Class, Program, CDS). Quên mã T-Code đi.
 - Ctrl + Space: Gợi ý code (Auto-complete).
-

C. MỤC TIÊU BÀI HỌC

1. **Eclipse ADT:** Làm quen giao diện, tạo ABAP Project, tạo DDL Source.
 2. **CDS Syntax:** Hiểu cú pháp DEFINE VIEW ENTITY, SELECT FROM, ASSOCIATION.
 3. **Joins & Logic:** Thực hiện kết nối bảng và tính toán cột (Case...When) ngay trong View.
 4. **Data Preview:** Xem báo cáo kết quả ngay lập tức.
-

D. NỘI DUNG THỰC HÀNH (LAB 9)

Mục tiêu: Tạo một báo cáo hiển thị: Mã SV, Tên SV, Tên Môn Học, Điểm Số, Trạng Thái (Đậu/Trượt). Dữ liệu lấy từ các bảng ZSTUDENT, ZENROLL, ZCOURSE đã có (dù tạo bằng GUI thì Eclipse vẫn nhìn thấy).

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 0: Kết nối Eclipse

- Mở Eclipse.
- File -> New -> ABAP Project.
- Chọn hệ thống SAP server -> Next -> Đăng nhập (User/Pass sinh viên) -> Finish.

Bước 1: Tạo Interface View (ZI_STUDENT_...)

- Trong cửa sổ **Project Explorer**, tìm đến Package của bạn (ZSPR26_SE1876...).
- Chuột phải vào Package -> New -> **Other ABAP Repository Object**.

- Gõ vào ô tìm kiếm: Data Definition. Chọn nó và nhấn Next.
- **Name:** ZI_STUDENT_DEV331 (Thay DEV331 bằng ID của bạn).
- **Description:** Basic Interface View for Student.
- Nhấn Next -> Next (Chọn Transport Request cũ hoặc tạo mới).
- **Template:** Chọn Define View Entity -> Finish.

Code Bước 1 (ZI):

Mục đích: Lấy dữ liệu thô từ bảng DB, đổi tên cột cho đẹp (CamelCase).

```
code Abap
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Basic Student Interface View'

define view entity ZI_STUDENT_DEV331
as select from zstudent dev331 as Student
{
  key student_id as StudentId,
  full_name as FullName,
  dob as DateOfBirth,
  course as CourseBatch,

  /* Admin Data (Nếu có) */
  @Semantics.user.createdBy: true
  created_by as CreatedBy
}
```

- Nhấn **Ctrl + F3** để Activate.

Bước 2: Tạo Reporting View (ZC_REPORT_...)

- Tương tự Bước 1, tạo Data Definition mới.
- **Name:** ZC_STUDENT_REPORT_DEV331.

- **Description:** Consumption View for Enrollment Report.

Code Bước 2 (ZC - Logic chính):

Mục đích: Join với bảng Điểm và Môn học, tính toán Đậu/Trượt.

```
code Abap
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Student Enrollment Report'
@Metadata.ignorePropagatedAnnotations: true

define view entity ZC_STUDENT_REPORT_DEV331
  as select from ZI_STUDENT_DEV331 as Student

  /* Join với bảng Đăng ký (Enrollment) */
  inner join   zenroll_dev331   as Enrollment on Student.StudentId = Enrollment.student_id

  /* Join với bảng Môn học (Course) */
  inner join   zcourse_dev331   as Course    on Enrollment.course_id = Course.course_id
{
  key Student.StudentId,
  key Enrollment.course_id as CourseId,

  Student.FullName,
  Course.course_name  as CourseName,
  Course.credits      as Credits,

  /* Cột điểm số */
  @Semantics.amount.currencyCode: 'Currency' -- (Giả lập đơn vị nếu cần)
  Enrollment.score    as Score,

  /* Logic tính toán (Case When) - Thay thế code ABAP IF...ELSE */
  case
    when Enrollment.score >= 5.0 then 'Passed'
```

```
    when Enrollment.score >= 0.0 then 'Failed'
    else 'Not Graded'
end          as Status,

/* Logic màu sắc (Criticality) - 3: Xanh, 1: Đỏ */
case
  when Enrollment.score >= 5.0 then 3
  else 1
end          as StatusColor
}
```

- Nhấn **Ctrl + F3** để Activate.

Bước 3: Xem Báo Cáo (Data Preview)

- Đang mở file ZC_STUDENT_REPORT_DEV331.
- Nhấn phím **F8**.
- Một cửa sổ **Data Preview** sẽ hiện ra bên dưới (hoặc tab mới).
- Bạn sẽ thấy bảng dữ liệu đầy đủ. Bạn có thể click vào tiêu đề cột để **Sort**, hoặc dùng nút **Filter** để lọc dữ liệu.

2. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên thực hiện trực tiếp trên Eclipse.

Challenge 1: Calculated Field (Tính tuổi)

- **Yêu cầu:** Trong view ZI_STUDENT..., hãy tính tuổi của sinh viên.
- **Gợi ý:** CDS hỗ trợ các hàm ngày tháng.
 - Syntax: `days_between( DateOfBirth, $session.system_date ) / 365`
 - Lưu ý: CDS View Entity xử lý tính toán rất mạnh.

Challenge 2: Parameters (Bộ lọc đầu vào)

- **Yêu cầu:** Người dùng muốn nhập vào một "Điểm chuẩn" (Passing Grade) thay vì fix cứng là 5.0.
- **Code:**

```
code Abap
define view entity ...
with parameters p_pass_grade : abap.dec(4,2)
as select ...
case when Enrollment.score >= $parameters.p_pass_grade then ...
```

- **Kết quả:** Khi nhấn F8, một cửa sổ Popup sẽ hiện ra hỏi tham số p_pass_grade.

Challenge 3: String Functions (Ghép chuỗi)

- **Yêu cầu:** Tạo cột DisplayName có định dạng: "Tên (Mã SV)". Ví dụ: "Nguyen Van A (SE001)".
- **Gợi ý:** Sử dụng hàm concat_with_space(arg1, arg2, spaces).
 - concat(FullName, concat('(', concat(StudentId, ')'))) (Hơi dài dòng nhưng cơ bản).

Challenge 4: Currency/Unit Conversion (Nâng cao)

- **Yêu cầu:** Giả sử bảng ZCOURSE có cột FEE (Học phí) đơn vị USD. Hãy hiển thị thêm cột học phí quy đổi ra VND (tỷ giá cứng 25000).
- **Code:** cast(Course.fee * 25000 as abap.curr(15, 2)) as FeeVND.

3. Lời giải tham khảo (Snippet cho Challenge 1 & 3)

Mở view ZI_STUDENT_DEV331 và sửa như sau:

```
code Abap
define view entity ZI_STUDENT_DEV331
as select from zstudent_dev331 as Student
{
```

```
key student_id as StudentId,  
  full_name as FullName,  
  dob      as DateOfBirth,  
  
  /* Challenge 3: Ghép chuỗi */  
  concat( full_name, concat( ' - ', student_id ) ) as DisplayName,  
  
  /* Challenge 1: Tính tuổi (Tương đối) */  
  /* Hàm dats_days_between trả về số ngày */  
  div( dats_days_between( dob, $session.system_date ), 365 ) as AgeInYears  
}
```

TỔNG KẾT BUỔI 9 (ECLIPSE)

Sinh viên đã thấy sức mạnh của CDS Views:

1. Code ngắn hơn ABAP thuần rất nhiều.
2. Hiện thị dữ liệu ngay lập tức (F8).
3. Logic tính toán nằm ngay trong truy vấn database.

BUỔI 10B: WEB HANDLER & HTTP SERVICE (TRÊN ECLIPSE)

A. THÔNG TIN & QUY TẮC

- **Công cụ:** Eclipse ADT.
- **Artifact mới:** ABAP Class (implement interface IF_HTTP_EXTENSION).
- **Transaction (Embedded GUI):** SICF (Để kích hoạt đường dẫn Web).
- **Chủ đề:** Tạo trang web "Bảng điểm cá nhân" chạy trực tiếp từ ABAP Server.

B. REAL-WORLD TIPS (TẠI SAO HỌC CÁI NÀY?)

1. Hiểu về HTTP:

- Trước khi làm OData hay Fiori, bạn cần hiểu HTTP Request (GET, POST) và Response là gì.
- Làm bài này giúp bạn hiểu: Khi Fiori gọi về, SAP nhận được gì và trả lời thế nào.

2. Lightweight Web Tools:

- Đôi khi bạn chỉ cần một trang web cực đơn giản để người dùng upload file hoặc xác nhận phê duyệt (Approval) mà không muốn dựng cả một dự án Fiori đồ sộ. HTTP Handler là giải pháp nhanh nhất.

3. Debug API:

- Biết cách dùng IF_HTTP_EXTENSION giúp bạn viết các Mock API để test hệ thống bên thứ 3.

C. MỤC TIÊU BÀI HỌC

1. **Interface IF_HTTP_EXTENSION:** Cửa ngõ để một Class ABAP xử lý yêu cầu Web.
2. **HTML Generation:** Sử dụng ABAP String Template (|...|) để viết HTML/CSS ngay trong code ABAP.

3. **URL Parameters:** Đọc tham số từ thanh địa chỉ (ví dụ: ?student=SE001).
4. **SICF:** Cấu hình Service Node để public class ra internet.

D. NỘI DUNG THỰC HÀNH (LAB 10)

Mục tiêu: Tạo một đường dẫn URL. Khi truy cập, nó hiển thị bảng danh sách sinh viên với giao diện HTML đẹp mắt (có CSS).

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo Handler Class trong Eclipse

- Chuột phải Package -> New -> ABAP Class.
- Name: ZCL_WEB_LMS_DEV331.
- Description: Web Handler for Student List.
- Interfaces: Add IF_HTTP_EXTENSION.
- Finish.

Bước 2: Viết Code HTML trong Method HANDLE_REQUEST

- Mở Class, chuyển sang tab **Methods**.
- Click đúp vào method IF_HTTP_EXTENSION~HANDLE_REQUEST để implement.
- Copy đoạn code mẫu bên dưới (Sử dụng String Template hiện đại).

Code Mẫu (ABAP + HTML/CSS):

```
code Abap
METHOD if_http_extension~handle_request.

" 1. Lấy dữ liệu từ Database (Sử dụng lại bảng ZSTUDENT từ các bài trước)
SELECT * FROM zstudent_dev331 INTO TABLE @DATA(lt_students).
```

" 2. Chuẩn bị HTML Header & CSS

```
DATA(lv_html) = |<!DOCTYPE html>| &  
|<html><head>| &  
|<style>| &  
| body \{ font-family: Arial; background-color: #f4f4f4; padding: 20px; \}| &  
| h1 \{ color: #F36F21; text-align: center; \}| &  
| table \{ width: 100%; border-collapse: collapse; background: white; \}| &  
| th \{ background: #004070; color: white; padding: 10px; \}| &  
| td \{ border: 1px solid #ddd; padding: 8px; text-align: center; \}| &  
|</style>| &  
|</head><body>| &  
|<h1>FPT UNIVERSITY - STUDENT LIST (ABAP Generated)</h1>| &  
|<table>| &  
| <tr><th>Student ID</th><th>Full Name</th><th>Course</th></tr>|.
```

" 3. Duyệt bảng và sinh dòng HTML động

```
LOOP AT lt_students INTO DATA(ls_student).
```

```
lv_html = lv_html &  
|<tr>| &  
| <td>{ ls_student-student_id }</td>| &  
| <td>{ ls_student-full_name }</td>| &  
| <td>{ ls_student-course }</td>| &  
|</tr>|.
```

```
ENDLOOP.
```

" 4. Đóng thẻ HTML

```
lv_html = lv_html & |</table></body></html>|.
```

" 5. Trả kết quả về trình duyệt

```
server->response->set_cdata( data = lv_html ).
```

```
server->response->set_header_field( name = 'Content-Type' value = 'text/html' ).
```

```
server->response->set_status( code = 200 reason = 'OK' ).
```

ENDMETHOD.

- **Activate (Ctrl + F3).**

Bước 3: Cấu hình SICF (Bắt buộc)

- Trong Eclipse, nhấn Ctrl+Shift+A, gõ SICF. (Nó sẽ mở một tab GUI bên trong Eclipse).
- Nhấn Execute (F8).
- Duyệt cây thư mục: default_host -> sap -> bc.
- Chuột phải vào bc -> **New Sub-Element** (hoặc tạo folder riêng z_lms).
- Service Name: zweb_dev331.
- Trong màn hình cấu hình:
 - Tab **Handler List**: Gõ tên class vừa tạo ZCL_WEB_LMS_DEV331.
 - Nhấn Save.
- Quay lại cây thư mục, chuột phải vào zweb_dev331 -> **Activate Service**.

Bước 4: Test

- Chuột phải vào Service zweb_dev331 -> **Test Service**.
- Trình duyệt sẽ bật lên. Đăng nhập.
- **Kết quả:** Bạn sẽ thấy danh sách sinh viên hiển thị dưới dạng trang web đẹp mắt.

2. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: URL Parameters (Lọc sinh viên)

- **Yêu cầu:** Cho phép lọc theo mã sinh viên qua URL.
 - Ví dụ: .../zweb_dev331?id=SE001.
- **Gợi ý:**
 - Dùng server->request->get_form_field('id') để lấy giá trị SE001.
 - Dùng giá trị này trong câu lệnh SELECT ... WHERE student_id =

Challenge 2: JSON API (Thay vì HTML)

- **Yêu cầu:** Biến Class này thành một API thực thụ trả về JSON (để App mobile dùng được).
- **Gợi ý:**
 - Đổi Content-Type thành application/json.
 - Sử dụng class /ui2/cl_json=>serialize để chuyển bảng lt_students thành chuỗi JSON.
 - Không cộng chuỗi HTML nữa.

Challenge 3: Hyperlink Drill-down

- **Yêu cầu:** Trong bảng HTML, biến Mã Sinh Viên thành một đường link ().
- **Logic:** Khi bấm vào link SE001, trang web tải lại và chỉ hiện chi tiết của sinh viên đó (Kết hợp với Challenge 1).

Challenge 4: Security (Chống XSS)

- **Vấn đề:** Nếu tên sinh viên chứa mã độc (ví dụ <script>alert('hack')</script>), trang web sẽ bị lỗi hoặc bị hack.
- **Yêu cầu:** Sử dụng cl_abap_dyn_prg=>escape_xss_url hoặc các hàm escape tương tự trước khi in Tên sinh viên ra HTML.

3. Lời giải tham khảo (Challenge 1 & 2 Snippet)

code Abap
 " --- 1. Lấy tham số URL ---

```
DATA(lv_id_param) = server->request->get_form_field( 'id' ).
```

```
" --- 2. Select dữ liệu ---
```

```
IF lv_id_param IS NOT INITIAL.
```

```
  SELECT * FROM zstudent_dev331
```

```
    WHERE student_id = @lv_id_param
```

```
    INTO TABLE @lt_students.
```

```
ELSE.
```

```
  SELECT * FROM zstudent_dev331 INTO TABLE @lt_students.
```

```
ENDIF.
```

```
" --- (Optional) Challenge 2: Trả về JSON ---
```

```
" server->response->set_header_field( name = 'Content-Type' value = 'application/json' ).
```

```
" DATA(lv_json) = /ui2/cl_json=>serialize( data = lt_students ).
```

```
" server->response->set_cdata( data = lv_json ).
```

```
" RETURN. " Dừng xử lý HTML nếu đang chạy mode API
```

TỔNG KẾT BUỔI 10 (ECLIPSE)

Chúng ta đã tự tay xây dựng một Web Server nhỏ gọn bằng ABAP Class.

- Sinh viên hiểu bản chất: Web App thực ra chỉ là chuỗi String HTML gửi qua mạng.
- Đây là bước đệm để hiểu **OData** (cũng là HTTP nhưng trả về XML/JSON) ở các buổi sau.

BUỔI 11B: RAP MODEL - BUSINESS OBJECT (BACKEND CỐT LÕI)

A. THÔNG TIN & QUY TẮC (ECLIPSE ADT)

- **Công cụ:** Eclipse ADT.
 - **Artifacts mới:**
 - **Root View Entity:** ZI_STUDENT_R_[ID].
 - **Behavior Definition (BDEF):** ZI_STUDENT_R_[ID].
 - **Behavior Implementation (Class):** ZBP_I_STUDENT_R_[ID].
 - **Chủ đề:** Xây dựng lõi xử lý nghiệp vụ (Business Object) cho Sinh viên.
-

B. REAL-WORLD TIPS (TẠI SAO DỪNG RAP?)

1. Managed vs. Unmanaged:

- **Managed (Dùng cho bài này):** SAP tự lo liệu việc Lưu xuống DB, Khóa dữ liệu (Locking), Quản lý bộ nhớ. Bạn chỉ viết logic nghiệp vụ (Validate). -> *Nhanh gấp 10 lần cách cũ.*
- **Unmanaged:** Bạn tự viết code INSERT/UPDATE (Giống Buổi 8). Dừng khi làm việc với các bảng cũ hoặc API bên ngoài.

2. Strict Mode:

- Trong code BDEF, bạn sẽ thấy strict (2);. Đây là chế độ bắt lỗi cú pháp nghiêm ngặt của SAP để đảm bảo code chuẩn Cloud. Dừng xóa nó.

3. Alias (Biệt danh):

- Trong BDEF, luôn đặt Alias cho Entity (ví dụ Student). Nếu không, khi debug hoặc hiện lỗi, nó sẽ hiện tên dài ngoằng ZI_STUDENT_R_DEV331 rất khó chịu.

C. MỤC TIÊU BÀI HỌC

1. **Root Entity:** Tạo CDS View gốc định nghĩa cấu trúc dữ liệu chuẩn.
2. **Behavior Definition (BDEF):** Kích hoạt các tính năng tiêu chuẩn: create, update, delete.
3. **Mapping:** Ánh xạ tên cột trong CDS (CamelCase) với tên cột trong Database (Snake_case).
4. **Behavior Pool:** Tạo Class ABAP để chứa logic xử lý sau này.

D. NỘI DUNG THỰC HÀNH (LAB 11)

Mục tiêu: Tạo một Business Object "Sinh viên". Kích hoạt khả năng tạo mới, sửa đổi, xóa bỏ mà chưa cần viết logic ABAP phức tạp.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Chuẩn bị Bảng Database (Nếu chưa chuẩn)

- RAP Managed yêu cầu bảng DB nên có các trường quản trị. Hãy kiểm tra bảng ZSTUDENT_DEV331.
- Nếu chưa có, hãy vào Eclipse, mở bảng và thêm các cột sau (hoặc dùng bảng cũ cũng được, nhưng sẽ thiếu tính năng Log):
 - created_by (TYPE syuname)
 - created_at (TYPE timestamp)
 - last_changed_by (TYPE syuname)
 - last_changed_at (TYPE timestamp)

Bước 2: Tạo Root View Entity (ZI_STUDENT_R...)

- Chuột phải Package -> New -> Data Definition.
- Name: ZI_STUDENT_R_DEV331. (Chữ **R** nghĩa là Root/Raw).
- Description: **Root Interface View for Student.**

- **Template:** Define Root View Entity.

Code Bước 2 (CDS View):

```
code Abap
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Root Interface View for Student'

define root view entity ZI_STUDENT_R_DEV331
  as select from zstudent_dev331
{
  key student_id    as StudentId,
  full_name        as FullName,
  dob              as DateOfBirth,
  course           as Course,

  /* Admin Fields (Map với bảng DB) */
  @Semantics.user.createdBy: true
  created_by       as CreatedBy,
  @Semantics.systemDateTime.createdAt: true
  created_at       as CreatedAt,
  @Semantics.user.lastChangedBy: true
  last_changed_by  as LastChangedBy,
  @Semantics.systemDateTime.lastChangedAt: true
  last_changed_at  as LastChangedAt
}
```

- Nhấn **Ctrl + F3** để Activate.

Bước 3: Tạo Behavior Definition (BDEF)

- Chuột phải vào file CDS vừa tạo (ZI_STUDENT_R_DEV331) -> **New Behavior Definition**.
- Giữ nguyên tên. Nhấn Finish.
- Eclipse sẽ sinh ra file BDEF. Hãy sửa code như sau.

Code Bước 3 (BDEF):

```
code Abap
managed implementation in class zbp_i_student_r_dev331 unique;
strict ( 2 );

define behavior for ZI STUDENT R DEV331 alias Student
persistent table zstudent dev331
lock master
authorization master ( instance )
// etag master LastChangedAt // (Bỏ qua nếu bảng chưa có cột timestamp)
{
  create;
  update;
  delete;

  /* Mapping: CDS Name = DB Field Name */
  mapping for zstudent_dev331
  {
    StudentId    = student_id;
    FullName     = full_name;
    DateOfBirth  = dob;
    Course       = course;
    CreatedBy    = created_by;
    CreatedAt    = created_at;
    LastChangedBy = last_changed_by;
    LastChangedAt = last_changed_at;
  }
}
```

- **Lưu ý:** Dòng managed implementation in class... sẽ báo lỗi đỏ vì Class chưa tồn tại. Đừng lo, ta làm Bước 4.

Bước 4: Tạo Behavior Implementation Class

- Đặt con trỏ chuột vào tên class zbp_i_student_r_dev331 (đang bị báo lỗi) trong file BDEF.
- Nhấn **Ctrl + 1** (Quick Fix).
- Chọn **Create global class**
- Nhấn Next -> Finish.
- Eclipse sẽ tạo ra một Class rỗng. Nhấn **Ctrl + F3** để Activate class này.
- Quay lại file BDEF, nhấn **Ctrl + F3** để Activate BDEF. (Hết lỗi đỏ).

Hoặc Tạo Class ABAP thủ công (Vì menu tự động đã bị ẩn)

Vì Eclipse của bạn không hiện menu *New Behavior Implementation*, chúng ta sẽ tạo nó như một Class bình thường.

1. Ở cửa sổ Project Explorer bên trái, chuột phải vào **Package** của bạn (ZSPR26_SE1876_DEV331).
2. Chọn **New -> ABAP Class**.
3. Điền thông tin chính xác như sau (Sai 1 chữ là lỗi):
 - **Name:** ZBP_I_STUDENT_R_DEV331
 - **Description:** Behavior Class for Student
4. Nhấn **Next** -> Nhấn **Finish**.
5. Eclipse mở ra một Class rỗng. **HÃY XÓA SẠCH CODE CŨ** trong đó và **COPY ĐOẠN CODE SAU DÁN VÀO:**

```
codeAbap
" Từ khóa FOR BEHAVIOR OF sẽ biến Class thường thành Behavior Class
CLASS zbp_i_student_r_dev331 DEFINITION PUBLIC ABSTRACT FINAL FOR BEHAVIOR OF zi_student_r_dev331.
ENDCLASS.

CLASS zbp_i_student_r_dev331 IMPLEMENTATION.
ENDCLASS.
```

1. Nhấn **Ctrl + F3** để Activate Class này.
(Lúc này hệ thống đã ghi nhận sự tồn tại của Class ZBP_I...).

2. Bốn (4) Thử thách phát triển (Deep Dive)

Đây là các cấu hình Business Logic cơ bản trong BDEF.

Challenge 1: Read-Only Fields (Khóa trường)

- **Yêu cầu:** Mã sinh viên (StudentId) chỉ được nhập khi Tạo mới (Create). Khi Sửa (Update), không được phép đổi mã. Các trường Admin (CreatedBy...) hệ thống tự điền, User không được sửa.
- **Code (Thêm vào BDEF):**

code Abap

downloadcontent_copy

expand_less

field (readonly : update) StudentId;

field (readonly) CreatedBy, CreatedAt, LastChangedBy, LastChangedAt;

Challenge 2: Mandatory Fields (Bắt buộc nhập)

- **Yêu cầu:** Không được để trống Họ tên.
- **Code (Thêm vào BDEF):**

code Abap

downloadcontent_copy

expand_less

field (mandatory) FullName;

- **Kết quả:** Khi lên giao diện Fiori, ô Họ tên sẽ có dấu sao đỏ (*).

Challenge 3: Default Value (Giá trị mặc định)

- **Yêu cầu:** Nếu không chọn Khóa học, mặc định là "K18".
- **Gợi ý:** Việc này cần viết code ABAP trong Class (determination).
- **Khai báo BDEF:** determination setKeyCourse on save { create; }.
- **Implementation:** Dùng Quick Fix để sinh method trong Class, sau đó viết code gán Course = 'K18'.

Challenge 4: Validation (Kiểm tra dữ liệu)

- **Yêu cầu:** Kiểm tra tuổi sinh viên > 18.
- **Khai báo BDEF:** validation validateAge on save { field DateOfBirth; create; update; }.
- **Implementation:** Viết code trong Class để kiểm tra năm sinh.

3. Lời giải tham khảo (Challenge 1 & 2 Snippet)

Mở file **Behavior Definition** (ZI_STUDENT_R_DEV331) và cập nhật phần body:

```
code Abap
define behavior for ZI_STUDENT_R_DEV331 alias Student
persistent table zstudent_dev331
lock master
authorization master ( instance )
{
  create;
  update;
  delete;
```

```

/* Challenge 1: Chỉ đọc */
field ( readonly : update ) StudentId;
field ( readonly ) CreatedBy, CreatedAt, LastChangedBy, LastChangedAt;

/* Challenge 2: Bắt buộc */
field ( mandatory ) FullName;

mapping for zstudent_dev331
{
  StudentId    = student_id;
  /* ... các trường khác ... */
}
}

```

TỔNG KẾT BUỔI 11 (ECLIPSE)

Chúng ta đã định nghĩa xong "Luật chơi" (Behavior) cho đối tượng Sinh viên.

- Hiện tại, chúng ta đã có Backend sẵn sàng xử lý Thêm/Sửa/Xóa.
- Tuy nhiên, chưa có "Cửa sổ" (API) để thế giới bên ngoài chui vào.

BUỔI 12B: SERVICE DEFINITION & BINDING (CÔNG KHAI API ODATA)

A. THÔNG TIN & QUY TẮC (ECLIPSE ADT)

- **Công cụ:** Eclipse ADT.
- **Artifacts mới:**
 - **Projection View:** ZC_STUDENT_APP_[ID] (View dùng cho App, kế thừa Behavior).
 - **Service Definition (ZSD):** ZSD_LMS_[ID].

- **Service Binding (ZSB):** ZSB_LMS_[ID].
 - **Chủ đề:** Public API OData và chạy thử ứng dụng Fiori đầu tiên (Preview).
-

● B. REAL-WORLD TIPS (TẠI SAO LÀM THẾ NÀY?)

1. Quên SEGW đi (khi làm RAP):

- Trước đây: Vào T-Code SEGW -> Import DDIC -> Generate Classes -> Register Service (Mất 30 phút).
- Bây giờ: Service Definition + Binding (Mất 2 phút). SAP tự động sinh mọi thứ ngầm bên dưới.

2. Projection View (ZC) là bắt buộc:

- Mặc dù bạn có thể expose trực tiếp Interface View (ZI), nhưng **không nên**.
- Lý do: ZI là tầng dữ liệu thô (nội bộ). ZC là tầng giao diện (cho Web). Nếu sau này Web cần thêm cột "Màu sắc", ta sửa ZC mà không ảnh hưởng đến logic tính toán ở ZI.

3. Fiori Elements Preview:

- Eclipse tích hợp sẵn trình duyệt để chạy thử Fiori App. Đây là công cụ debug giao diện mạnh nhất của Developer trước khi bàn giao cho bộ phận triển khai (Basis).
-

C. MỤC TIÊU BÀI HỌC

1. **Projection View:** Tạo view tiêu thụ (ZC) kế thừa khả năng transactional (CRUD) từ ZI.
 2. **Service Definition:** Định nghĩa phạm vi của Service (Bao gồm những Entity nào?).
 3. **Service Binding:** Chọn giao thức (Protocol) - OData V2 (cho Fiori cũ) hoặc V4 (cho Fiori mới).
 4. **Test:** Sử dụng tính năng Preview để thực hiện Thêm/Sửa/Xóa sinh viên trên giao diện Web.
-

D. NỘI DUNG THỰC HÀNH (LAB 12)

Mục tiêu: Chạy được ứng dụng Quản lý Sinh viên trên trình duyệt, thực hiện được thao tác Tạo mới và Xóa.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Tạo Projection View (ZC_STUDENT_APP_...)

- *Lý do:* Ở Buổi 9 ta tạo ZC_REPORT (Read-only). Giờ ta cần ZC_APP (Transactional) để kế thừa Behavior của Buổi 11.
- Chuột phải ZI_STUDENT_R_DEV331 -> New Data Definition.
- Name: ZC_STUDENT_APP_DEV331.
- Description: Projection View for LMS App.
- **Template:** Define Projection View.

Code Bước 1:

```
code Abap
@EndUserText.label: 'Student Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Metadata.allowExtensions: true // Cho phép tách file cấu hình giao diện ra ngoài (Buổi 13)

define root view entity ZC_STUDENT_APP_DEV331
  provider contract transactional_query // Bắt buộc dòng này để hỗ trợ CRUD trên Fiori
  as projection on ZI_STUDENT_R_DEV331
  {
    key StudentId,
      FullName,
      DateOfBirth,
      Course,
      CreatedBy,
      CreatedAt,
      LastChangedBy,
```

```
LastChangedAt  
}
```

- Nhấn **Ctrl + F3** để Activate.
- *Lưu ý:* Cần thêm Behavior Projection nếu muốn kích hoạt nút bấm (sẽ làm ở bước nâng cao, hiện tại Create/Update/Delete tự động có sẵn do provider contract).

Bước 2: Tạo Service Definition (ZSD_...)

- Chuột phải vào ZC_STUDENT_APP_DEV331 -> **New Service Definition**.
- Name: ZSD_LMS_DEV331.
- Description: Service Definition for LMS.
- Nhấn Finish.

Code Bước 2:

```
code Abap  
@EndUserText.label: 'LMS Service Definition'  
define service ZSD_LMS_DEV331 {  
  expose ZC_STUDENT_APP_DEV331 as Student; // 'Student' là tên sẽ hiện trên URL OData  
}
```

- Nhấn **Ctrl + F3**.

Bước 3: Tạo Service Binding (ZSB_...)

- Chuột phải vào Service Definition vừa tạo -> **New Service Binding**.
- Name: ZSB_LMS_DEV331.
- Binding Type: **OData V2 - UI** (Chọn V2 để tương thích tốt nhất với Fiori Elements hiện tại).
- Nhấn Finish.
- Màn hình cấu hình hiện ra. Nhấn nút **Activate** (hoặc **Publish**) ở phía bên phải.

- Chờ vài giây đến khi trạng thái chuyển sang màu xanh **Active**.

Bước 4: Chạy thử (Preview)

- Trong màn hình Service Binding, nhìn sang khung bên phải **Service Version Details**.
 - Bạn sẽ thấy Entity Student. Chọn nó.
 - Nhấn nút **Preview** (Nút này nằm ngay trên danh sách Entity).
 - Trình duyệt sẽ bật lên, yêu cầu đăng nhập.
 - **Kết quả:** Một trang web Fiori hiện ra.
 - Nhấn nút **Go**. Danh sách sinh viên hiện ra.
 - Có nút **Create** (Dấu cộng). Hãy thử tạo một sinh viên mới.
 - Có nút **Delete**. Hãy thử xóa.
 - *Lưu ý:* Cột dữ liệu có thể chưa hiện ra hết hoặc chưa đẹp (Vì ta chưa cấu hình UI Annotation - Bài 13 sẽ làm). Nhưng chức năng nút bấm đã hoạt động!
-

2. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: Multi-Entity Service (Nhiều bảng trong 1 API)

- **Yêu cầu:** Thêm View Báo cáo (Buổi 9 - ZC_STUDENT_REPORT...) vào Service này để Admin vừa quản lý, vừa xem báo cáo.
- **Cách làm:** Mở ZSD_LMS_DEV331. Thêm dòng: expose ZC_STUDENT_REPORT_DEV331 as Report;. Activate. Refresh lại trình duyệt Preview.

Challenge 2: OData V4 (Chuẩn mới)

- **Yêu cầu:** Tạo thêm một Binding mới ZSB_LMS_V4_... sử dụng **OData V4 - UI**.
- **Mục tiêu:** So sánh sự khác biệt (V4 nhanh hơn, payload nhỏ hơn, nhưng Preview hơi khác một chút).

Challenge 3: Aliasing (Đổi tên đẹp)

- **Vấn đề:** Tên Entity mặc định dài (ZC_STUDENT_APP_DEV331).
- **Yêu cầu:** Trong Service Definition, đổi tên hiển thị thành LMS_Student.
- **Code:** expose ZC... as LMS_Student;
- **Kết quả:** URL OData sẽ gọn hơn: .../odata/sap/.../LMS_Student.

Challenge 4: Behavior Projection (Chặn quyền ở tầng Web)

- **Vấn đề:** Mặc dù Backend (Buổi 11) cho phép Delete. Nhưng ở Web App này, tôi muốn cấm nút Delete.
- **Cách làm:**
 1. Tạo file **Behavior Definition** cho Projection View ZC_STUDENT_APP....
 2. Code:

```
code Abap
projection;
define behavior for ZC_STUDENT_APP_DEV331 alias Student
{
  use create;
  use update;
  // Không viết 'use delete' -> Nút Delete sẽ biến mất trên Web
}
```

TỔNG KẾT BUỔI 12 (ECLIPSE)

Chúng ta đã thành công đưa ứng dụng lên trình duyệt!

- Backend: RAP BO (Buổi 11).
- API: Service Definition/Binding (Buổi 12).

- Frontend: Fiori Elements (Preview).

BUỔI 13B: METADATA EXTENSIONS (TÙY BIẾN GIAO DIỆN UI)

A. THÔNG TIN & QUY TẮC (ECLIPSE ADT)

- **Công cụ:** Eclipse ADT.
- **Artifact mới: Metadata Extension (ZX_...).**
- **Annotations chính:**
 - @UI.lineItem: Cấu hình danh sách (Cột nào hiện, thứ tự nào).
 - @UI.selectionField: Cấu hình bộ lọc (Thanh tìm kiếm phía trên).
 - @UI.identification: Cấu hình trang chi tiết (Object Page).
 - @UI.headerInfo: Tiêu đề của trang.
- **Chủ đề:** "Trang điểm" cho ứng dụng Quản lý Sinh viên.

● B. REAL-WORLD TIPS (KINH NGHIỆM THỰC TẾ)

1. Separation of Concerns (Tách biệt mối quan tâm):

- **CDS View (ZC_):** Chỉ chứa logic dữ liệu (Join bảng nào, lấy cột nào).
- **Metadata Extension (ZX_):** Chỉ chứa logic giao diện (Màu gì, nhãn là gì, nằm ở đâu).
- *Lợi ích:* Khi cần sửa màu sắc, bạn không sợ làm hỏng logic dữ liệu.

2. Facets (Nhóm dữ liệu):

- Trên trang chi tiết (Object Page), đừng để 100 trường thông tin tràn lan. Hãy dùng **Facets** để gom nhóm chúng lại thành các khối: "Thông tin chung", "Thông tin Admin", "Điểm số".

3. Labeling:

- Luôn dùng label: 'Tiếng Việt' trong Annotation để giao diện thân thiện. Nếu làm dự án đa ngôn ngữ, ta sẽ trở Label này vào file Text (OTR), nhưng trong Lab này ta hardcode tiếng Việt cho nhanh.

C. MỤC TIÊU BÀI HỌC

1. **Enable MDE:** Cấu hình CDS View để cho phép mở rộng Metadata.
2. **Create MDE:** Tạo file cấu hình UI riêng biệt.
3. **List Report Config:** Sắp xếp cột và tạo bộ lọc.
4. **Object Page Config:** Thiết kế trang chi tiết sinh viên.

D. NỘI DUNG THỰC HÀNH (LAB 13)

Mục tiêu: Biến ứng dụng thô sơ của Buổi 12 thành một ứng dụng Fiori chuyên nghiệp với nhãn tiếng Việt, có bộ lọc Mã SV/Khóa học, và tô màu trạng thái điểm số.

1. Hướng dẫn chi tiết (Step-by-Step)

Bước 1: Kích hoạt Metadata Extension (QUAN TRỌNG)

- Mở file Projection View **ZC_STUDENT_APP_DEV331** (đã tạo ở Buổi 12).
- Thêm dòng annotation sau vào **ngay trước** câu lệnh define root view...:

code Abap

downloadcontent_copy

expand_less

@Metadata.allowExtensions: true

- Nhấn **Ctrl + F3** để Activate lại View này.

- Nếu quên bước này, file MDE tạo ra ở bước sau sẽ bị vô hiệu hóa.

Bước 2: Tạo Metadata Extension

- Chuột phải vào View ZC_STUDENT_APP_DEV331 -> **New Metadata Extension**.
- Name: ZX_STUDENT_APP_DEV331 (Chữ X nghĩa là Extension).
- Description: UI Config for Student App.
- Nhấn Finish.

Bước 3: Viết Code Annotation (Cấu hình UI)

- Eclipse sinh ra file mẫu. Hãy thay thế nội dung bằng code bên dưới.
- *Lưu ý logic*: position càng nhỏ thì càng nằm bên trái (hoặc bên trên).

Code Mẫu (MDE):

```
code Abap
@Metadata.layer: #CORE
@UI: {
  headerInfo: {
    typeName: 'Sinh viên',
    typeNamePlural: 'Danh sách Sinh viên',
    title: { type: #STANDARD, value: 'FullName' }, // Tiêu đề to nhất là Tên SV
    description: { type: #STANDARD, value: 'StudentId' } // Mô tả nhỏ ở dưới là Mã SV
  }
}
annotate view ZC_STUDENT_APP_DEV331 with
{
  /* 1. Cấu hình Facet (Khung chứa thông tin ở trang chi tiết) */
  @UI.facet: [ { id:      'GeneralInfo',
                  purpose:  #STANDARD,
                  type:     #IDENTIFICATION_REFERENCE,
```

```
label:      'Thông tin chung',  
position:   10 } ]
```

```
/* 2. Cấu hình từng trường dữ liệu */
```

```
// --- MÃ SINH VIÊN ---
```

```
@UI: {  
  lineItem:    [ { position: 10, label: 'Mã Sinh Viên', importance: #HIGH } ], // Hiện ở bảng (Cột 1)  
  selectionField: [ { position: 10 } ], // Hiện ở thanh lọc (Ô 1)  
  identification: [ { position: 10, label: 'Mã số' } ] // Hiện ở trang chi tiết (Dòng 1)  
}  
StudentId;
```

```
// --- HỌ VÀ TÊN ---
```

```
@UI: {  
  lineItem:    [ { position: 20, label: 'Họ và Tên', importance: #HIGH } ],  
  selectionField: [ { position: 20 } ],  
  identification: [ { position: 20, label: 'Họ tên đầy đủ' } ]  
}  
FullName;
```

```
// --- NGÀY SINH ---
```

```
@UI: {  
  lineItem:    [ { position: 30, label: 'Ngày sinh' } ],  
  identification: [ { position: 30, label: 'Ngày sinh' } ]  
}  
DateOfBirth;
```

```
// --- KHÓA HỌC ---
```

```
@UI: {  
  lineItem:    [ { position: 40, label: 'Khóa học', importance: #MEDIUM } ],  
  selectionField: [ { position: 30 } ],
```

```
    identification: [ { position: 40, label: 'Khóa niên chế' } ]
  }
  Course;

// --- CÁC TRƯỜNG ADMIN (Ẩn khỏi bảng, chỉ hiện ở chi tiết) ---
@UI.identification: [ { position: 90, label: 'Người tạo' } ]
CreatedBy;

@UI.identification: [ { position: 91, label: 'Ngày tạo' } ]
CreatedAt;
}
```

- Nhấn **Ctrl + F3** để Activate.

Bước 4: Kiểm tra kết quả

- Quay lại file **Service Binding** (ZSB_LMS_DEV331).
- Nhấn lại nút **Preview** (hoặc Refresh trình duyệt nếu đang mở).
- **Kết quả:**
 - Tiêu đề bảng đã đổi thành tiếng Việt.
 - Có thanh lọc (Filter Bar) phía trên.
 - Bấm vào một dòng, trang chi tiết (Object Page) hiện ra ngăn nắp theo nhóm "Thông tin chung".

2. Bốn (4) Thử thách phát triển (Deep Dive)

Sinh viên thực hiện trực tiếp trên file MDE ZX_....

Challenge 1: Criticality (Tô màu trạng thái)

- **Yêu cầu:** Mặc dù trong App này chưa có cột điểm, nhưng giả sử ta muốn tô màu cột **Khóa học**.

- Nếu Khóa là 'K18' -> Màu xanh (Mới).
- Các khóa khác -> Màu thường.

- **Cách làm:**

1. Vào ZC_STUDENT_APP..., thêm logic Case When để tạo cột ảo CourseColor (trả về số 3 nếu K18, số 0 nếu khác).
2. Vào ZX_STUDENT_APP..., sửa annotation của cột Course:

code Abap

```
@UI.lineItem: [ { position: 40, criticality: 'CourseColor', label: 'Khóa học' } ]
```

Challenge 2: Value Help (Dropdown List)

- **Vấn đề:** Ô lọc Khóa học vẫn phải gõ tay.
- **Yêu cầu:** Gắn Dropdown list cho cột Course.
- **Cách làm:**

1. Vào ZC_STUDENT_APP....
2. Thêm annotation trước cột Course:

code Abap

```
@Consumption.valueHelpDefinition: [{ entity: { name: 'ZI_COURSE_DEV331', element: 'course_id' } }]
```

Course,

- (Giả sử bạn đã tạo CDS View ZI_COURSE... từ Buổi 9).

Challenge 3: Hidden Fields (Trường ẩn)

- **Yêu cầu:** Trường LastChangedBy và LastChangedAt là thông tin kỹ thuật, không cần hiện ra cho người dùng nhưng vẫn cần có trong OData để hệ thống xử lý (ETag).
- **Cách làm:** Trong MDE, thêm annotation @UI.hidden: true cho các trường này.

Challenge 4: Header Actions (Nút bấm nhanh)

- **Yêu cầu:** Mặc định chỉ có nút Delete. Tôi muốn nút "Edit" hiển thị nổi bật hơn.
- **Cách làm:** Sử dụng `@UI.identification: [ { type: #FOR_ACTION, dataAction: '...', label: 'Action Name' } ]` (Cái này hơi nâng cao, cần định nghĩa Action trong BDEF trước - dành cho sinh viên giỏi tìm hiểu thêm).

3. Lời giải tham khảo (Snippet cho Challenge 1 & 2)

Trong ZC_STUDENT_APP_DEV331:

```
code Abap
define root view entity ZC_STUDENT_APP_DEV331 ...
{
  ...
  /* Challenge 2: Gắn Value Help */
  @Consumption.valueHelpDefinition: [{ entity: { name: 'ZCOURSE_DEV331', element: 'course_id' } }] // Giả sử view Course đã có Course,

  /* Challenge 1: Logic màu */
  case
    when Course = 'K18' then 3 -- Xanh lá (Success)
    when Course = 'K17' then 2 -- Vàng (Warning)
    else 0 -- Mặc định (Neutral)
  end as CourseCriticality,
  ...
}
```

Trong ZX_STUDENT_APP_DEV331:

```
code Abap
downloadcontent copy
expand_less
@UI: {
  lineItem: [ { position: 40, criticality: 'CourseCriticality' } ],
  // Hệ thống sẽ đọc giá trị từ cột CourseCriticality để tô màu cho cột Course
```

```
selectionField: [ { position: 30 } ]  
}  
Course;
```

TỔNG KẾT BUỔI 13 (ECLIPSE)

Chúng ta đã hoàn thiện giao diện ứng dụng Fiori:

1. Giao diện tiếng Việt thân thiện.
2. Bố cục rõ ràng (List Report & Object Page).
3. Có tính năng tô màu và hỗ trợ tìm kiếm.

BUỔI 14B: ANALYTICAL CDS & EMAIL AUTOMATION (TRÊN ECLIPSE)

A. THÔNG TIN & QUY TẮC (ECLIPSE ADT)

- **Công cụ:** Eclipse ADT.
- **Artifacts mới:**
 - **CDS View (Cube/Query):** ZC_STUDENT_ANALYTICS_[ID].
 - **ABAP Class (Console):** ZCL_LMS_MAILER_[ID].
- **Annotations chính:**
 - `@Analytics.dataCategory`: #CUBE (Khối dữ liệu).
 - `@Analytics.query`: true (Truy vấn báo cáo).
 - `@DefaultAggregation`: #SUM (Tự động cộng dồn).
- **Chủ đề:** Dashboard phân tích tuyển sinh & Class gửi mail báo cáo.

● B. REAL-WORLD TIPS (TẠI SAO LÀM THẾ NÀY?)

1. Embedded Analytics (Phân tích nhúng):

- Sức mạnh thực sự của S/4HANA là HANA DB. Nó tính toán hàng triệu dòng dữ liệu trong tích tắc.
- Chúng ta tạo **Analytical CDS View** để tận dụng sức mạnh này. Fiori tự động hiểu View này và vẽ thành biểu đồ (Chart) hoặc bảng xoay (Pivot Table) mà không cần code JS.

2. IF_OO_ADT_CLASSRUN:

- Khi viết code tiện ích (như gửi mail) trên Eclipse, chúng ta không tạo Report (SE38) nữa.
- Chúng ta tạo Class implement giao diện IF_OO_ADT_CLASSRUN. Nhấn **F9** để chạy và xem kết quả ngay tại Console của Eclipse.

C. MỤC TIÊU BÀI HỌC

1. **Analytical CDS:** Hiểu mô hình Cube/Dimension. Tạo View đếm số lượng sinh viên theo Khóa học.
2. **Data Preview (Analytics):** Sử dụng trình xem biểu đồ tích hợp sẵn trong Eclipse.
3. **ABAP Email:** Viết Class gửi HTML Email dùng CL_BCS.

D. NỘI DUNG THỰC HÀNH (LAB 14)

PHẦN 1: ANALYTICAL CDS VIEW (BIỂU ĐỒ)

Mục tiêu: Tạo một báo cáo tự động đếm số sinh viên trong mỗi khóa học và hiển thị dưới dạng Biểu đồ cột.

Bước 1: Tạo CDS View (ZC_STUDENT_ANALYTICS_...)

- Chuột phải Package -> New Data Definition.
- Name: ZC_STUDENT_ANALYTICS_DEV331.
- Template: Define View Entity.

Code Bước 1 (Analytical Query):

```
code Abap
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Analytics: Students per Course'

/* 1. Định nghĩa đây là một Query (Báo cáo phân tích) */
@Analytics.query: true

define view entity ZC_STUDENT_ANALYTICS_DEV331
  as select from zstudent_dev331
{
```

```

/* 2. Dimension (Trục hoành - Các chiều phân tích) */
@AnalyticsDetails.query.axis: #FREE
course as Course,

/* 3. Measure (Trục tung - Số liệu tính toán) */
/* Đếm số lượng sinh viên: Count distinct hoặc Count all */
@AnalyticsDetails.query.axis: #ROWS
@DefaultAggregation: #SUM
@EndUserText.label: 'Total Students'
1 as NumberOfStudents
}
/* Group by ngầm định: CDS tự động Group by các cột Dimension (Course) */

```

- Nhấn **Ctrl + F3** để Activate.

Bước 2: Xem Biểu đồ (Preview)

- Mở file CDS vừa tạo. Nhấn **F8**.
- Eclipse hiện Data Preview.
- Lúc đầu nó hiện dạng Bảng. Hãy nhìn xuống góc dưới (hoặc thanh công cụ của tab Preview), chuyển sang tab **Analysis** (hoặc biểu tượng Biểu đồ).
- **Kết quả:** Bạn sẽ thấy biểu đồ cột thể hiện số lượng sinh viên từng khóa. Bạn có thể kéo thả, xoay trục (Pivot) ngay tại đây.

PHẦN 2: ABAP EMAIL CLASS (GỬI MAIL)

Mục tiêu: Viết một Class để gửi email cảnh báo. Chạy test bằng Console.

Bước 1: Tạo Class (ZCL_LMS_MAILER_...)

- Chuột phải Package -> New ABAP Class.
- Name: ZCL_LMS_MAILER_DEV331.

- Interfaces: Add IF_OO_ADT_CLASSRUN.

Code Bước 1 (Email Logic):

```
code Abap
CLASS zcl_lms_mailer_dev331 DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_lms_mailer_dev331 IMPLEMENTATION.

  METHOD if_oo_adt_classrun~main.
    TRY.
      " 1. Tạo Send Request
      DATA(lo_send_request) = cl_bcs=>create_persistent( ).

      " 2. Tạo nội dung HTML
      DATA: lt_body TYPE bcsy_text.
      APPEND '<html><body>' TO lt_body.
      APPEND '<h2 style="color:blue">LMS Weekly Report</h2>' TO lt_body.
      APPEND '<p>Hệ thống LMS đang hoạt động tốt trên S/4HANA.</p>' TO lt_body.
      APPEND '</body></html>' TO lt_body.

      DATA(lo_document) = cl_document_bcs=>create_document(
        i_type   = 'HTM'
        i_text   = lt_body
```

```

        i_subject = 'FPT LMS Notification (Test from Eclipse)' ).

lo_send_request->set_document( lo_document ).

" 3. Thêm người nhận (Thay bằng email thật của bạn nếu server cấu hình)
DATA(lo_recipient) = cl_cam_address_bcs=>create_internet_address(
    'admin@fpt.edu.vn' ).

lo_send_request->add_recipient( lo_recipient ).

" 4. Gửi
DATA(lv_sent) = lo_send_request->send( ).

IF lv_sent = abap_true.
    COMMIT WORK. " Quan trọng
    out->write( 'Email sent successfully!' ). " In ra Console Eclipse
ELSE.
    out->write( 'Error sending email.' ).
ENDIF.

CATCH cx_bcs INTO DATA(lx_bcs).
    out->write( |Exception: { lx_bcs->get_text( ) }| ).
ENDTRY.
ENDMETHOD.
ENDCLASS.

```

- Nhấn **Ctrl + F3** Activate.

Bước 2: Chạy thử (Console)

- Đang mở file Class, nhấn **F9**.
- Nhìn xuống tab **Console** bên dưới Eclipse.
- Thông báo: Email sent successfully! nghĩa là code đã chạy xong.

- (Để kiểm tra mail đi chưa, vẫn phải vào GUI T-Code SOST, nhưng code thì viết và chạy hoàn toàn trên Eclipse).

2. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: Data Integration (CDS -> Email)

- **Yêu cầu:** Trong Class gửi mail, không viết nội dung tĩnh.
- **Logic:**
 1. SELECT dữ liệu từ CDS View ZC_STUDENT_ANALYTICS_... (Select từ View cũng giống như Select từ Bảng).
 2. Loop qua kết quả, tạo ra một bảng HTML <table>...</table> chứa số liệu thống kê.
 3. Gửi bảng đó trong email.

Challenge 2: Chart Type (Đổi loại biểu đồ)

- **Yêu cầu:** Trong CDS View, thêm Annotation để đổi loại biểu đồ mặc định từ Cột (Column) sang Tròn (Donut).
- **Gợi ý:** @UI.chart: [{ title: 'Students by Course', chartType: #DONUT ... }]. (Cần thêm MDE hoặc viết trực tiếp).

Challenge 3: Exception Reporting

- **Yêu cầu:** Nếu quá trình SELECT dữ liệu bị lỗi hoặc trả về rỗng, Class Mailer phải bắt lỗi và in ra Console: "No data found for report".

Challenge 4: Attachment (Gửi kèm Excel)

- **Yêu cầu:** Tạo một chuỗi String chứa danh sách sinh viên (cách nhau bởi dấu Tab). Chuyển sang Binary và đính kèm vào mail dưới dạng file .xls.
- **Gợi ý:** Dùng `cl_bcs_convert=>string_to_xstring` và `lo_document->add_attachment`.

3. Lời giải tham khảo (Challenge 1 - Integration)

Trong method main của Class Mailer:

code Abap

" --- Challenge 1: Lấy số liệu từ CDS View ---

```
SELECT * FROM ZC_STUDENT_ANALYTICS_DEV331 INTO TABLE @DATA(lt_stats).
```

```
APPEND '<table border="1"><tr><th>Khoa</th><th>So Luong</th></tr>' TO lt_body.
```

```
LOOP AT lt_stats INTO DATA(ls_stat).
```

```
  APPEND |<tr><td>{ ls_stat-Course }</td><td>{ ls_stat-NumberOfStudents }</td></tr>| TO lt_body.
```

```
ENDLOOP.
```

```
APPEND '</table>' TO lt_body.
```

```
" --- Sau đó tiếp tục logic tạo document và gửi ---
```

TỔNG KẾT BUỔI 14 (ECLIPSE)

Chúng ta đã khai thác được sức mạnh phân tích của S/4HANA (Analytics) và khả năng tương tác hệ thống (Email) hoàn toàn trên môi trường Eclipse.

BUỔI 15B: DATA PARSING & GEN-AI INTEGRATION (ECLIPSE CONSOLE)

A. THÔNG TIN & QUY TẮC (ECLIPSE ADT)

- **Công cụ:** Eclipse ADT.
- **Artifact:** ABAP Class (IF_OO_ADT_CLASSRUN).
- **Thư viện:**
 - CL_HTTP_CLIENT: Để gọi API ra ngoài internet.
 - /UI2/CL_JSON (hoặc XCO_CP_JSON trên S/4HANA Cloud): Để xử lý JSON.
- **Chủ đề:** Import dữ liệu sinh viên và Chat với AI về kết quả học tập.

B. REAL-WORLD TIPS (TẠI SAO HỌC CÁI NÀY?)

1. ABAP không còn đơn độc:

- Ngày nay, ABAP phải nói chuyện với thế giới (GitHub, Jira, ChatGPT, Teams). Kỹ năng dùng HTTP Client và xử lý JSON là bắt buộc.

2. Xử lý dữ liệu lớn (Mass Processing):

- Khi upload 1 file Excel 10.000 dòng, đừng bao giờ Loop và Insert từng dòng vào DB.
- Hãy đưa vào một bảng nội bộ (Internal Table) và dùng INSERT ztable FROM TABLE itab (Array Insert). Tốc độ nhanh gấp hàng trăm lần.

3. AI trong ERP:

- Không chỉ để chat vui. AI trong SAP dùng để: Tự động điền mã kế toán, Tóm tắt hồ sơ nhân sự, Gợi ý sửa lỗi code ABAP.

C. MỤC TIÊU BÀI HỌC

1. **String Processing:** Kỹ thuật tách chuỗi (Split), xử lý dữ liệu thô (Raw Data) thành dữ liệu có cấu trúc.
2. **HTTP Request:** Cách tạo kết nối API từ ABAP tới OpenAI.
3. **JSON Parsing:** Đọc kết quả trả về từ AI.
4. **Tổng kết:** Nhìn lại hành trình 15 buổi.

D. NỘI DUNG THỰC HÀNH (LAB 15)

PHẦN 1: IMPORT DỮ LIỆU (SIMULATED UPLOAD)

Tình huống: Do Eclipse Console không thể mở cửa sổ chọn file của Windows, ta sẽ giả lập việc nhận được một chuỗi dữ liệu thô (CSV/Tab-delimited) và viết logic để parse nó vào bảng Database ZSTUDENT.

Bước 1: Tạo Class ZCL_LMS_IMPORT_DEV331

- Implement IF_OO_ADT_CLASSRUN.

Code Bước 1 (Logic Parse & Insert):

```
code Abap
CLASS zcl_lms_import_dev331 DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_lms_import_dev331 IMPLEMENTATION.
```

METHOD if_oo_adt_classrun~main.

" 1. Giả lập dữ liệu nhận được từ File (Dạng CSV: ID,Tên,Khóa)

" Trong thực tế, chuỗi này đến từ Fiori App gửi xuống

DATA(lv_raw_file) = |SE901,Le Van A,K18\n| &&

 |SE902,Tran Thi B,K17\n| &&

 |SE903,Pham Van C,K18|.

DATA: lt_students TYPE TABLE OF zstudent_dev331,

 ls_student TYPE zstudent_dev331.

" 2. Tách dòng (Split by Newline)

SPLIT lv_raw_file AT '\n' INTO TABLE DATA(lt_lines).

" 3. Duyệt từng dòng và tách cột

LOOP AT lt_lines INTO DATA(lv_line).

 SPLIT lv_line AT ',' INTO ls_student-student_id

 ls_student-full_name

 ls_student-course.

" Logic làm sạch dữ liệu (Data Cleansing)

ls_student-student_id = to_upper(ls_student-student_id).

" Gán các trường Admin (vì RAP không chạy ở đây nên phải gán tay)

GET TIME STAMP FIELD ls_student-created_at.

ls_student-created_by = sy-uname.

APPEND ls_student TO lt_students.

ENDLOOP.

" 4. Insert vào DB (Mass Insert)

IF lt_students IS NOT INITIAL.

```
" Dùng MODIFY để nếu trùng ID thì nó Update, chưa có thì Insert  
MODIFY zstudent_dev331 FROM TABLE @lt_students.
```

```
IF sy-subrc = 0.  
  out->write( |Imported { lines( lt_students ) } students successfully.| ).
```

```
" In thử kết quả ra màn hình console  
out->write( lt_students ).  
ELSE.  
  out->write( 'Error importing data.' ).  
ENDIF.  
ENDIF.
```

```
ENDMETHOD.  
ENDCLASS.
```

- Nhấn **F9** để chạy. Kiểm tra bảng ZSTUDENT... xem có dữ liệu mới chưa.

PHẦN 2: TÍCH HỢP GEN-AI (CALL CHATGPT API)

Tình huống: Viết một Class gọi API của OpenAI để xin lời khuyên học tập cho sinh viên.

(Lưu ý: Bạn cần có API Key thật để chạy, nếu không có, code sẽ trả về lỗi 401/404 nhưng logic vẫn đúng).

Bước 1: Tạo Class ZCL_LMS_GENAI_DEV331

Code Bước 1 (HTTP Client Logic):

```
code Abap  
CLASS zcl_lms_genai_dev331 DEFINITION  
  PUBLIC  
  FINAL  
  CREATE PUBLIC .  
  
  PUBLIC SECTION.
```

```
INTERFACES if_oo_adt_classrun .  
PROTECTED SECTION.  
PRIVATE SECTION.  
ENDCLASS.
```

```
CLASS zcl_lms_genai_dev331 IMPLEMENTATION.
```

```
METHOD if_oo_adt_classrun~main.
```

```
" --- CẤU HÌNH ---
```

```
DATA(lv_url) = 'https://api.openai.com/v1/chat/completions'.
```

```
DATA(lv_api_key) = 'Bearer sk-proj-xxxxx...'. " <--- Điền API Key của bạn vào đây
```

```
TRY.
```

```
" 1. Tạo HTTP Client
```

```
cl_http_client=>create_by_url(  
    EXPORTING url    = lv_url  
    IMPORTING client = DATA(lo_client) ).
```

```
" 2. Set Headers
```

```
lo_client->request->set_header_field( name = 'Content-Type' value = 'application/json' ).  
lo_client->request->set_header_field( name = 'Authorization' value = lv_api_key ).
```

```
" 3. Set Body (JSON Payload)
```

```
" Prompt: Hỏi AI về lộ trình học ABAP
```

```
DATA(lv_body) = |\{ "model": "gpt-3.5-turbo", | &&  
    | "messages": [\{ "role": "user", "content": "Cho tôi lộ trình học SAP ABAP trong 3 bước ngắn gọn." \}] \}|.
```

```
lo_client->request->set_cdata( lv_body ).
```

```
" 4. Gửi Request
```

```
lo_client->send( ).
```

```
lo_client->receive( ).
```

```

" 5. Nhận kết quả
DATA(lv_status) = lo_client->response->get_status_code( ).
DATA(lv_json_response) = lo_client->response->get_cdata( ).

out->write( |Status: { lv_status }| ).

IF lv_status = 200.
  out->write( '--- AI RESPONSE (RAW JSON) ---' ).
  out->write( lv_json_response ).
  " (Ở phần Challenge sẽ học cách làm đẹp JSON này)
ELSE.
  out->write( 'Error calling API. Check API Key.' ).
ENDIF.

lo_client->close( ).

CATCH cx_root INTO DATA(lx_root).
  out->write( lx_root->get_text( ) ).
ENDTRY.
ENDMETHOD.
ENDCLASS.

```

- Nhấn **F9** để chạy.

2. Bốn (4) Thử thách phát triển (Deep Dive)

Challenge 1: Parse JSON Response (Làm đẹp kết quả AI)

- **Vấn đề:** Kết quả trả về là chuỗi JSON dài. Ta chỉ cần nội dung câu trả lời.
- **Yêu cầu:** Sử dụng class `/ui2/cl_json` để chuyển chuỗi JSON thành Structure ABAP và in ra đúng phần content.
- **Gợi ý:** Tạo structure ABAP khớp với JSON của OpenAI (Choices -> Message -> Content) hoặc dùng kiểu DATA động.

Challenge 2: Dynamic Prompt (Prompt động từ DB)

- **Yêu cầu:**
 1. Select ngẫu nhiên một sinh viên từ bảng ZSTUDENT.
 2. Tạo prompt: "Hãy đặt một biệt danh vui nhộn cho sinh viên tên [FullName] học khóa [Course]".
 3. Gửi cho AI và in kết quả.

Challenge 3: Transactional AI (Lưu lời khuyên vào DB)

- **Yêu cầu:** Thêm một cột AI_NOTE (String/Char 255) vào bảng ZSTUDENT.
- **Logic:** Sau khi AI trả lời, hãy dùng lệnh UPDATE zstudent... để lưu lời khuyên đó vào DB cho sinh viên tương ứng.

Challenge 4: XCO Library (Modern S/4HANA Cloud)

- **Yêu cầu:** (Dành cho sinh viên tìm hiểu thêm). Nếu hệ thống hỗ trợ thư viện **XCO** (Extension Components), hãy viết lại code tạo JSON Body bằng XCO_CP_JSON thay vì cộng chuỗi thủ công. Đây là chuẩn code mới nhất của SAP.

3. TỔNG KẾT KHÓA HỌC (ECLIPSE ADT TRACK)

Chúc mừng các bạn! Chúng ta đã hoàn thành hành trình 15 buổi chuyển đổi từ tư duy ABAP Cổ điển sang **Modern ABAP trên Eclipse**.

Những gì bạn đã làm được:

1. **Eclipse Mastery:** Từ bỏ SE80, làm chủ công cụ phát triển số 1 hiện nay.
2. **Modern Syntax:** Code ABAP 7.5+ (Inline declaration, String template, Constructor).
3. **Core Data Services (CDS):** Tạo báo cáo không cần code, tận dụng sức mạnh HANA.
4. **RAP Model:** Xây dựng ứng dụng Fiori (Backend) theo chuẩn S/4HANA.
5. **Integration:** Kết nối HTTP, OData, Email và AI.

Dự án cuối khóa (Assignment):

Gói gọn toàn bộ các artifact (ZI_..., ZC_..., ZBP_..., ZCL_...) vào Package ZSPR26_SE1876_[ID].

Quay một video ngắn (2 phút) demo:

1. Mở Eclipse, show cấu trúc Project.
2. Chạy Preview Fiori App (Tạo/Sửa/Xóa sinh viên).
3. Chạy Class Console gọi AI.

PHẦN 1: THIẾT KẾ CSDL (BẢNG, KHÓA CHÍNH, KHÓA NGOẠI)

Tạo 4 bảng cốt lõi. Hãy nhớ thay [ID] bằng mã số của bạn (ví dụ DEV331).

Mô hình quan hệ:

- **ZSTUDENT:** Bảng Sinh viên (Master Data).
- **ZCOURSE:** Bảng Môn học (Master Data).
- **ZSEMESTER:** Bảng Học kỳ (Master Data).
- **ZENROLL:** Bảng Đăng ký & Điểm (Transaction Data) - Bảng này sẽ chứa Khóa ngoại trỏ về 3 bảng trên.

Bước 1.1: Tạo bảng Sinh viên (ZSTUDENT_[ID])

1. Trong Eclipse, chuột phải Package -> **New** -> **Other ABAP Repository Object**.
2. Tìm **Database Table**. Next.
3. Name: ZSTUDENT_DEV331. Description: Student Master Data.
4. Paste code sau:

```
code Abap
@EndUserText.label : 'Bảng Sinh viên FPT LMS'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #ALLOWED

define table zstudent_dev331 {
  key client   : abap.clnt not null;
  @EndUserText.label : 'Mã Sinh viên'
  key student_id : abap.char(8) not null;

  @EndUserText.label : 'Họ và tên'
```

```

full_name    : abap.char(50);

@EndUserText.label : 'Email'
email        : abap.char(100);

@EndUserText.label : 'Ngày sinh'
dob          : abap.dats;

@EndUserText.label : 'Khóa (K17, K18...)'
course_batch : abap.char(10);
}

```

Bước 1.2: Tạo bảng Môn học (ZCOURSE_[ID])

Làm tương tự bước trên với tên ZCOURSE_DEV331.

```

code Abap
@EndUserText.label : 'Bảng Môn học'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #ALLOWED

define table zcourse_dev331 {
  key client    : abap.clnt not null;
  @EndUserText.label : 'Mã Môn học'
  key course_id : abap.char(8) not null;

  @EndUserText.label : 'Tên Môn học'
  course_name    : abap.char(50);

  @EndUserText.label : 'Số tín chỉ'
  credits        : abap.int1;
}

```

Bước 1.3: Tạo bảng Học kỳ (ZSEMESTER_[ID])

Tên bảng: ZSEMESTER_DEV331.

expand_less

@EndUserText.label : 'Bảng Học kỳ'

@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE

@AbapCatalog.tableCategory : #TRANSPARENT

@AbapCatalog.deliveryClass : #A

@AbapCatalog.dataMaintenance : #ALLOWED

```
define table zsemester_dev331 {  
  key client      : abap.clnt not null;  
  @EndUserText.label : 'Mã Học kỳ (VD: SP24)'  
  key semester_id : abap.char(5) not null;  
  
  @EndUserText.label : 'Mô tả (Spring 2024)'  
  description      : abap.char(20);  
}
```

Bước 1.4: Tạo bảng Đăng ký & Điểm (ZENROLL_[ID]) - CÓ KHÓA NGOẠI

Đây là bảng quan trọng nhất.

- **Khóa chính:** enroll_id (Tự tăng hoặc UUID).
- **Khóa ngoại (Foreign Keys):** Liên kết trường student_id, course_id, semester_id về các bảng cha.

Tên bảng: ZENROLL_DEV331.

code Abap

@EndUserText.label : 'Bảng Đăng ký và Điểm thi'

@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE

@AbapCatalog.tableCategory : #TRANSPARENT

@AbapCatalog.deliveryClass : #A

@AbapCatalog.dataMaintenance : #ALLOWED

```

define table zenroll_dev331 {
  key client      : abap.clnt not null;
  @EndUserText.label : 'ID Đăng ký'
  key enroll_id   : abap.numc(10) not null;

  @EndUserText.label : 'Mã Sinh viên'
  @AbapCatalog.foreignKey.label : 'Liên kết bảng Sinh viên'
  @AbapCatalog.foreignKey.screenCheck : true
  key student_id  : abap.char(8) not null
  with foreign key zstudent_dev331
    where client = zenroll_dev331.client
    and student_id = zenroll_dev331.student_id;

  @EndUserText.label : 'Mã Môn học'
  @AbapCatalog.foreignKey.label : 'Liên kết bảng Môn học'
  key course_id   : abap.char(8) not null
  with foreign key zcourse_dev331
    where client = zenroll_dev331.client
    and course_id = zenroll_dev331.course_id;

  @EndUserText.label : 'Mã Học kỳ'
  @AbapCatalog.foreignKey.label : 'Liên kết bảng Học kỳ'
  key semester_id : abap.char(5) not null
  with foreign key zsemester_dev331
    where client = zenroll_dev331.client
    and semester_id = zenroll_dev331.semester_id;

  @EndUserText.label : 'Điểm số (0-10)'
  score               : abap.dec(4,2);
}

```

PHẦN 2: CHƯƠNG TRÌNH TẠO DỮ LIỆU MẪU (MOCK DATA)

Chúng ta sẽ viết một **ABAP Class (Console App)** để sinh tự động 30 dòng dữ liệu cho mỗi bảng.

1. Chuột phải Package -> **New -> ABAP Class**.
2. Name: ZCL_LMS MOCK_DATA_DEV331.
3. Interfaces: IF_OO_ADT_CLASSRUN.

Paste code sau vào:

```
code Abap
CLASS zcl_lms_mock_data_dev331 DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun .
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS zcl_lms_mock_data_dev331 IMPLEMENTATION.
  METHOD if_oo_adt_classrun~main.

    " --- 1. XÓA DỮ LIỆU CŨ ---
    DELETE FROM zenroll_dev331.
    DELETE FROM zstudent_dev331.
    DELETE FROM zcourse_dev331.
    DELETE FROM zsemester_dev331.
    out->write( 'Old data deleted.' ).

    " --- 2. TẠO DỮ LIỆU SINH VIÊN (30 SV) ---
    DATA: lt_students TYPE TABLE OF zstudent_dev331,
```

```
ls_student TYPE zstudent_dev331.
```

```
DO 30 TIMES.
```

```
DATA(lv_index) = sy-index.
```

```
" Tạo Mã SV: SE180001, SE180002...
```

```
ls_student-student_id = |SE18{ lv_index WIDTH = 4 PAD = '0' }|.
```

```
ls_student-full_name = |Sinh Vien FPT { lv_index }|.
```

```
ls_student-email = |sv{ lv_index }@fpt.edu.vn|.
```

```
ls_student-dob = '20040101'.
```

```
ls_student-course_batch = 'K18'.
```

```
APPEND ls_student TO lt_students.
```

```
ENDDO.
```

```
INSERT zstudent_dev331 FROM TABLE @lt_students.
```

```
out->write( |Inserted { lines( lt_students ) } students.| ).
```

```
" --- 3. TẠO DỮ LIỆU HỌC KỲ (3 Học kỳ) ---
```

```
DATA: lt_semesters TYPE TABLE OF zsemester_dev331.
```

```
lt_semesters = VALUE #(
```

```
( semester_id = 'SP24' description = 'Spring 2024' )
```

```
( semester_id = 'SU24' description = 'Summer 2024' )
```

```
( semester_id = 'FA24' description = 'Fall 2024' )
```

```
).
```

```
INSERT zsemester_dev331 FROM TABLE @lt_semesters.
```

```
" --- 4. TẠO DỮ LIỆU MÔN HỌC (5 Môn) ---
```

```
DATA: lt_courses TYPE TABLE OF zcourse_dev331.
```

```
lt_courses = VALUE #(
```

```
( course_id = 'PRF192' course_name = 'C Fundamentals' credits = 3 )
```

```
( course_id = 'PRO192' course_name = 'Java OOP' credits = 3 )
```

```
( course_id = 'DBI202' course_name = 'Database SQL' credits = 3 )
```

```
( course_id = 'SWE102' course_name = 'Intro to SE' credits = 3 )
```

```
( course_id = 'SAP401' course_name = 'SAP S/4HANA' credits = 4 )
```

```

).
INSERT zcourse_dev331 FROM TABLE @lt_courses.

" --- 5. TẠO DỮ LIỆU ĐĂNG KÝ (Random điểm cho 30 SV) ---
DATA: lt_enrolls TYPE TABLE OF zenroll_dev331,
      ls_enroll TYPE zenroll_dev331,
      lv_enroll_id TYPE i.

DATA(lo_rand) = cl_abap_random_int=>create( min = 0 max = 10 ).

LOOP AT lt_students INTO ls_student.
  " Mỗi sinh viên đăng ký học môn SAP401 vào kỳ Spring 2024
  lv_enroll_id = lv_enroll_id + 1.
  ls_enroll-enroll_id = lv_enroll_id.
  ls_enroll-student_id = ls_student-student_id.
  ls_enroll-course_id = 'SAP401'.
  ls_enroll-semester_id = 'SP24'.

  " Random điểm số (Cần chia 10 để ra số thập phân nếu muốn, ở đây lấy chẵn cho nhanh)
  ls_enroll-score = lo_rand->get_next( ).

  APPEND ls_enroll TO lt_enrolls.
ENDLOOP.

INSERT zenroll_dev331 FROM TABLE @lt_enrolls.
out->write( |Inserted { lines( lt_enrolls ) } enrollment records.| ).

ENDMETHOD.
ENDCLASS.

```

- **Chạy:** Nhấn **F9**. Kiểm tra Console để thấy kết quả.
-

PHẦN 3: CHƯƠNG TRÌNH UPLOAD TỪ EXCEL (FILE THẬT)

Vì upload file Excel yêu cầu giao diện (SAP GUI) để chọn file từ máy tính Windows, nên chúng ta sẽ tạo một **ABAP Program (Report)** truyền thống, không dùng Class Console.

Dù viết trên Eclipse, nhưng khi chạy (F8), nó sẽ mở cửa sổ SAP GUI để bạn chọn file.

Yêu cầu: Chuẩn bị 1 file Excel MockData.xls (Lưu dạng Excel 97-2003 .xls để tương thích tốt nhất với hàm chuẩn cũ, hoặc .xlsx nếu hệ thống mới hỗ trợ).

Cấu trúc file Excel (Sheet 1):

- Cột A: Mã SV (SE...)
- Cột B: Họ tên
- Cột C: Email
- Cột D: Khóa (K18...)

Bước 3.1: Viết chương trình Upload

1. Chuột phải Package -> **New** -> **ABAP Program**.
2. Name: Z_LMS_UPLOAD_EXCEL_DEV331.
3. Paste code sau:

```
code Abap
REPORT z_lms_upload_excel_dev331.

*--- 1. KHAI BÁO THAM SỐ CHỌN FILE ---*
PARAMETERS: p_file TYPE rlgrap-filename DEFAULT 'C:\Temp\StudentData.xls'.

*--- 2. KHAI BÁO CẤU TRÚC DỮ LIỆU ---*
TYPES: BEGIN OF ty_excel,
        col1 TYPE string, " Mã SV
        col2 TYPE string, " Tên
```

```
col3 TYPE string, " Email
col4 TYPE string, " Khóa
END OF ty_excel.
```

```
DATA: lt_excel  TYPE TABLE OF ty_excel,
      lt_students TYPE TABLE OF zstudent_dev331,
      ls_student TYPE zstudent_dev331.
```

--- 3. SỰ KIỆN MỞ CỬA SỔ CHỌN FILE (F4) ---

AT SELECTION-SCREEN ON VALUE-REQUEST FOR p_file.

```
CALL FUNCTION 'F4_FILENAME'
```

```
EXPORTING
```

```
field_name = 'P_FILE'
```

```
IMPORTING
```

```
file_name = p_file.
```

--- 4. LOGIC CHÍNH (UPLOAD & INSERT) ---

START-OF-SELECTION.

" Chuyển dữ liệu Excel vào bảng nội bộ

" Hàm này rất phổ biến để đọc Excel. Yêu cầu file phải đang đóng.

```
CALL FUNCTION 'TEXT_CONVERT_XLS_TO_SAP'
```

```
EXPORTING
```

```
i_line_header      = 'X' " Bỏ qua dòng tiêu đề đầu tiên
```

```
i_tab_raw_data      = lt_excel " Bảng trung gian (tự hàm xử lý)
```

```
i_filename          = p_file
```

```
TABLES
```

```
i_tab_converted_data = lt_excel
```

```
EXCEPTIONS
```

```
conversion_failed  = 1
```

```
OTHERS              = 2.
```

```
IF sy-subrc <> 0.  
  WRITE: / 'Lỗi đọc file Excel. Hãy chắc chắn file là dạng .xls và đang đóng.'  
  RETURN.  
ENDIF.
```

```
" Map dữ liệu từ Excel sang cấu trúc bảng DB  
LOOP AT lt_excel INTO DATA(ls_row).  
  CLEAR ls_student.  
  ls_student-student_id = ls_row-col1.  
  ls_student-full_name = ls_row-col2.  
  ls_student-email = ls_row-col3.  
  ls_student-course_batch = ls_row-col4.  
  ls_student-dob = '20000101'. " Mặc định ngày sinh nếu Excel không có  
  
  APPEND ls_student TO lt_students.  
ENDLOOP.
```

```
" Insert vào Database  
IF lt_students IS NOT INITIAL.  
  MODIFY zstudent dev331 FROM TABLE lt_students.  
  IF sy-subrc = 0.  
    WRITE: / 'Upload thành công', lines( lt_students ), 'sinh viên từ Excel.'
```

```
" Hiển thị kết quả kiểm tra  
  SELECT * FROM zstudent_dev331 INTO TABLE @DATA(lt_check).  
  cl_demo_output=>display( lt_check ).  
ELSE.  
  WRITE: / 'Lỗi khi lưu vào Database.'  
ENDIF.  
ELSE.  
  WRITE: / 'File Excel rỗng.'  
ENDIF.
```

Bước 3.2: Cách chạy

1. Nhấn **Ctrl + F3** để Activate chương trình.
 2. Nhấn **F8** trong Eclipse.
 3. Một cửa sổ SAP GUI sẽ hiện lên.
 4. Bấm vào ô P_FILE, chọn file Excel trên máy tính của bạn.
 5. Bấm **Execute (F8)** (hình đồng hồ).
 6. Kết quả sẽ hiển thị danh sách sinh viên đã upload.
-

Tổng kết:

Bạn đã hoàn thành 3 nhiệm vụ:

1. **Thiết kế DB:** 4 bảng chuẩn hóa có quan hệ khóa ngoại.
2. **Mock Data:** Dùng Class Console để sinh dữ liệu test nhanh.
3. **Excel Import:** Dùng Report truyền thông để người dùng cuối upload file thật.

PHẦN 2: THIẾT KẾ CSDL CÁCH TRUYỀN THÔNG VỚI TA-CODE

- **T-Code sử dụng:** SE11 (ABAP Dictionary).
- **Tên bảng:** Thay thế [ID] bằng mã sinh viên của bạn (Ví dụ: DEV331).
 - ZSTUDENT_DEV331
 - ZCOURSE_DEV331
 - ZSEMESTER_DEV331

- ZENROLL_DEV331
- **MANDT:** Tất cả các bảng bắt buộc phải có trường MANDT (Client) làm khóa chính đầu tiên.

PHẦN 1: TẠO CÁC BẢNG DANH MỤC (MASTER DATA)

Chúng ta sẽ tạo 3 bảng độc lập trước: Sinh viên, Môn học, Học kỳ.

Bảng 1: Bảng Sinh viên (ZSTUDENT_[ID])

Bước 1: Khởi tạo

1. Đăng nhập SAP GUI. Gõ SE11 -> Enter.
2. Chọn ô **Database table**, nhập tên: ZSTUDENT_DEV331.
3. Nhấn **Create** (F5).
4. **Short Description:** Nhập "Student Master Data".

Bước 2: Cấu hình chung (Delivery and Maintenance)

1. Tab **Delivery and Maintenance**:
 - **Delivery Class:** Chọn A (Application table).
 - **Data Browser/Table View Maint.:** Chọn Display/Maintenance Allowed (Để sau này nhập liệu được bằng SE16N).

Bước 3: Khai báo trường (Fields)

1. Chuyển sang tab **Fields**. Nhập danh sách sau (Sử dụng *Predefined Type* cho nhanh):

Field	Key	Init	Data Type	Length	Decimals	Short Description
MANDT	☒	☒	CLNT	3	0	Client
STUDENT_ID	☒	☒	CHAR	8	0	Mã Sinh viên

FULL_NAME			CHAR	50	0	Họ và tên
EMAIL			CHAR	100	0	Email FPT
DOB			DATS	8	0	Ngày sinh
COURSE			CHAR	10	0	Khóa (K17, K18)

*Lưu ý: Để nhập Data Type trực tiếp, bạn có thể cần nhấn nút **Predefined Type** trên thanh công cụ của tab Fields.*

Bước 4: Cài đặt kỹ thuật (Technical Settings)

1. Nhấn nút **Technical Settings** (hoặc Ctrl+Shift+F9) trên thanh công cụ.
2. **Data Class:** Chọn USER0 (User data).
3. **Size Category:** Chọn 0 (Dữ liệu nhỏ).
4. Nhấn **Save** (Ctrl+S) -> Chọn Package của bạn (ZSPR26...) -> Save.
5. Nhấn **Back** (F3) để quay lại màn hình Fields.

Bước 5: Kích hoạt

1. Nhấn nút **Activate** (Hình cây diêm/Ctrl+F3).
2. Nếu không có lỗi, thanh trạng thái báo "Object activated".

Bảng 2: Bảng Môn học (ZCOURSE_[ID])

Làm tương tự Bảng 1 với thông số sau:

- **Tên bảng:** ZCOURSE_DEV331
- **Mô tả:** Course Master Data
- **Fields:**

Field	Key	Init	Data Type	Length	Decimals	Short Description
MANDT	☒	☒	CLNT	3	0	Client
COURSE_ID	☒	☒	CHAR	8	0	Mã Môn học
COURSE_NAME			CHAR	50	0	Tên Môn học
CREDITS			INT1	3	0	Số tín chỉ

Nhớ thực hiện Technical Settings và Activate.

Bảng 3: Bảng Học kỳ (ZSEMESTER_[ID])

Làm tương tự Bảng 1 với thông số sau:

- Tên bảng: ZSEMESTER_DEV331
- Mô tả: Semester Master Data
- Fields:

Field	Key	Init	Data Type	Length	Decimals	Short Description
MANDT	☒	☒	CLNT	3	0	Client
SEMESTER_ID	☒	☒	CHAR	5	0	Mã Học kỳ (SP24)
DESCRIPT			CHAR	20	0	Mô tả (Spring 2024)

Nhớ thực hiện Technical Settings và Activate.

PHẦN 2: TẠO BẢNG GIAO DỊCH & KHÓA NGOẠI (TRANSACTION DATA)

Đây là phần quan trọng nhất để liên kết các bảng với nhau.

Bảng 4: Bảng Đăng ký & Điểm (ZENROLL_[ID])

Bước 1: Khai báo Fields

- **Tên bảng:** ZENROLL_DEV331
- **Fields:**

Field	Key	Init	Data Type	Length	Decimals	Short Description
MANDT	☒	☒	CLNT	3	0	Client
ENROLL_ID	☒	☒	NUMC	10	0	ID Đăng ký (Tự tăng)
STUDENT_ID			CHAR	8	0	Mã SV (FK)
COURSE_ID			CHAR	8	0	Mã Môn (FK)
SEMESTER_ID			CHAR	5	0	Mã Kỳ (FK)
SCORE			DEC	4	2	Điểm số (0-10)

Bước 2: Tạo Khóa Ngoại (Foreign Keys)

Chúng ta sẽ tạo 3 khóa ngoại cho 3 trường STUDENT_ID, COURSE_ID, và SEMESTER_ID.

2.1. Tạo Foreign Key cho STUDENT_ID

1. Chọn (bôi đen) dòng STUDENT_ID.
2. Nhấn nút **Foreign Keys** (Hình chìa khóa/mũi tên) trên thanh công cụ.
3. Cửa sổ Popup hiện ra. Tại ô **Check Table**, nhập tên bảng sinh viên: ZSTUDENT_DEV331.
4. Nhấn nút **Generate proposal** (Hình 2 tờ giấy). Hệ thống sẽ tự động ghép:
 - MANDT = ZSTUDENT_DEV331-MANDT
 - STUDENT_ID = ZSTUDENT_DEV331-STUDENT_ID

5. **Short text:** Nhập "Link to Student Table".
6. **Cardinality:** Chọn 1 : CN (Một sinh viên có thể đăng ký nhiều dòng Enrollment).
7. Nhấn **Copy** (F5).

2.2. Tạo Foreign Key cho COURSE_ID

1. Chọn dòng COURSE_ID.
2. Nhấn nút **Foreign Keys**.
3. **Check Table:** ZCOURSE_DEV331.
4. Nhấn **Generate proposal**.
5. **Short text:** "Link to Course Table".
6. Nhấn **Copy**.

2.3. Tạo Foreign Key cho SEMESTER_ID

1. Chọn dòng SEMESTER_ID.
2. Nhấn nút **Foreign Keys**.
3. **Check Table:** ZSEMESTER_DEV331.
4. Nhấn **Generate proposal**.
5. **Short text:** "Link to Semester Table".
6. Nhấn **Copy**.

Bước 3: Hoàn tất

1. Vào **Technical Settings**, điền Data Class (USER0) và Size Category (0). Save.
2. Quay lại màn hình chính, nhấn **Activate**.

TỔNG KẾT QUAN HỆ CÁC BẢNG (ENTITY RELATIONSHIP)

Sau khi thực hiện xong, hệ thống của bạn sẽ có cấu trúc như sau:

1. **ZSTUDENT_DEV331** (Bảng cha): Chứa thông tin sinh viên.
 - Quan hệ: 1 Sinh viên có nhiều bản ghi Đăng ký trong bảng ZENROLL.
2. **ZCOURSE_DEV331** (Bảng cha): Chứa thông tin môn học.
 - Quan hệ: 1 Môn học xuất hiện nhiều lần trong bảng ZENROLL.
3. **ZSEMESTER_DEV331** (Bảng cha): Chứa thông tin học kỳ.
 - Quan hệ: 1 Học kỳ có nhiều bản ghi Đăng ký.
4. **ZENROLL_DEV331** (Bảng con):
 - Là bảng trung tâm kết nối 3 bảng trên.
 - Đảm bảo toàn vẹn dữ liệu: Bạn không thể nhập điểm cho một Sinh viên chưa tồn tại (nhờ Foreign Key Check).

Hoàn thành việc thiết kế Database theo cách "cổ điển". Giờ các em có thể dùng T-Code **SE16N** để nhập dữ liệu thử hoặc viết chương trình tạo dữ liệu mẫu.

BUỔI 15: LẬP TRÌNH DIALOG (DYNPRO) - THÊM MỚI DỮ LIỆU

A. THÔNG TIN & QUY TẮC

- **Loại chương trình:** Module Pool (Type M).
- **Tên chương trình:** SAPMZDEV006_MASTER_ENTRY
- **T-Code:** ZDEV006_MASTER
- **Cấu trúc Màn hình:**
 - **Screen 0100 (Student Master):** Lưới danh sách Sinh viên. Nút "Thêm Sinh viên" để tạo dòng trống.
 - **Screen 0200 (Course Enrollment):** Click chọn 1 SV -> Mở màn hình lưới Môn học & Điểm của SV đó. Nút "Đăng ký môn" để thêm dòng, kết hợp F4 Search Help chọn môn.

● B. REAL-WORLD TIPS (BÍ QUYẾT XỬ LÝ TABLE CONTROL)

1. Bản chất của nút "Thêm Dòng":

Table Control trên màn hình chỉ là cái "gương phản chiếu" của Internal Table (bảng nội bộ) trong RAM.

Muốn thêm 1 dòng trên màn hình -> Bạn phải APPEND INITIAL LINE (thêm dòng rỗng) vào Internal Table -> Sau đó cộng biến $TC-LINES = TC-LINES + 1$.

2. Lấy tên tự động sau khi dùng F4 (Search Help):

Khi dùng F4 để chọn Mã Môn, làm sao Tên Môn tự động hiện ra bên cạnh? Ta phải xử lý trong vòng lặp PAI (Process After Input) của lưới: Lấy Mã Môn vừa nhập -> SELECT Tên Môn -> Cập nhật lại Internal Table.

3. Lệnh MODIFY thần thánh:

Lệnh MODIFY ztable FROM TABLE itab sẽ tự động:

- Dòng nào chưa có trong Database (Khóa chính mới) -> Nó tự INSERT.
- Dòng nào có rồi -> Nó tự UPDATE.

C. NỘI DUNG THỰC HÀNH (STEP-BY-STEP)

GIẢI ĐOẠN 1: KHỞI TẠO & KHAI BÁO BIẾN (TOP INCLUDE)

1. Tạo Program SAPMZDEV006_MASTER_ENTRY (Type M, With TOP Include).
2. Code trong file TOP Include:

```
code Abap
*&-----*
*& Include MZDEV006_MASTER_ENTRYTOP
*&-----*
PROGRAM sapmzdev006_master_entry.

TABLES: zstudent_dev006, zenroll_dev006, zcourse_dev006.

DATA: ok_code TYPE sy-ucomm,
      save_ok TYPE sy-ucomm.

" 1. Cấu trúc cho Lưới Sinh viên
TYPES: BEGIN OF ty_student,
        mark      TYPE c, " Checkbox
        student_id TYPE zstudent_dev006-student_id,
        full_name  TYPE zstudent_dev006-full_name,
        dob        TYPE zstudent_dev006-dob,
        course     TYPE zstudent_dev006-course,
      END OF ty_student.

DATA: it_students TYPE TABLE OF ty_student,
      wa_student  TYPE ty_student.

" 2. Cấu trúc cho Lưới Đăng ký & Điểm
```

```
TYPES: BEGIN OF ty_enroll,  
    mark      TYPE c, " Checkbox  
    enroll_id  TYPE zenroll_dev006-enroll_id,  
    course_id  TYPE zenroll_dev006-course_id,  
    course_name TYPE zcourse_dev006-course_name,  
    score      TYPE zenroll_dev006-score,  
END OF ty_enroll.
```

```
DATA: it_enroll TYPE TABLE OF ty_enroll,  
      wa_enroll TYPE ty_enroll.
```

" 3. Khai báo biến Table Control

```
CONTROLS: tc_stu TYPE TABLEVIEW USING SCREEN 0100,  
          tc_enr TYPE TABLEVIEW USING SCREEN 0200.
```

" Biến lưu SV đang được chọn để sang màn hình 2

```
DATA: gv_current_student TYPE zstudent_dev006-student_id.
```

GIẢI ĐOẠN 2: THIẾT KẾ MÀN HÌNH 0100 (DANH SÁCH SINH VIÊN)

1. Tạo Screen 0100. Vẽ Giao diện (Layout):

- Các nút bấm:
 - Thêm Dòng (FctCode: **ADD_STU**)
 - Lưu DB (FctCode: **SAVE_STU**)
 - Xem Chi Tiết (FctCode: **DETAIL**)
- Vẽ Table Control TC_STU (có cột chọn dòng w/SelColumn WA_STUDENT-MARK).
- Các cột (cho phép Input): WA_STUDENT-STUDENT_ID, WA_STUDENT-FULL_NAME, WA_STUDENT-DOB, WA_STUDENT-COURSE.

2. Code Flow Logic (Tab Flow Logic của Screen 0100):

code Abap

PROCESS BEFORE OUTPUT.

MODULE status_0100.

" Load dữ liệu lên lưới

LOOP AT it_students INTO wa_student WITH CONTROL tc_stu CURSOR tc_stu-current line.

ENDLOOP.

PROCESS AFTER INPUT.

" Đọc dữ liệu user gõ từ màn hình vào RAM

LOOP AT it_students.

MODULE tc_stu_modify ON REQUEST.

ENDLOOP.

MODULE user_command_0100.

GIẢI ĐOẠN 3: THIẾT KẾ MÀN HÌNH 0200 (ĐĂNG KÝ MÔN & NHẬP ĐIỂM)

1. Tạo Screen 0200. Vẽ Layout:

- Chữ hiển thị ID SV đang chọn (kéo ZSTUDENT_DEV006-STUDENT_ID ra, set Output Only).
- Các nút:
 - Đăng ký môn mới (FctCode: **ADD_ENR**)
 - Lưu vào DB (FctCode: **SAVE_ENR**)
 - Quay lại (FctCode: **BACK**)
- Vẽ Table Control TC_ENR (cột chọn dòng WA_ENROLL-MARK).

- Cột **Course ID** (WA_ENROLL-COURSE_ID): Cho phép Input.
 **ĐỂ TẠO SEARCH HELP:** Double-click vào ruột cột này. Cuộn xuống dòng **Search Help**, gõ vào: **H_ZCOURSE** (Giả sử bạn đã có Search Help tên này, nếu chưa, có thể gõ trực tiếp tên bảng ZCOURSE_DEV006).
- Cột **Course Name** (WA_ENROLL-COURSE_NAME): Set **Output Only** (Hệ thống tự điền).
- Cột **Score** (WA_ENROLL-SCORE): Cho phép Input.

2. Code Flow Logic (Screen 0200):

```
code Abap
PROCESS BEFORE OUTPUT.
  MODULE status_0200.
  LOOP AT it_enroll INTO wa_enroll WITH CONTROL tc_enr CURSOR tc_enr-current_line.
  ENDLOOP.

PROCESS AFTER INPUT.
  LOOP AT it_enroll.
    MODULE tc_enr_modify ON REQUEST.
  ENDLOOP.
  MODULE user_command_0200.
```

GIAI ĐOẠN 4: VIẾT CODE XỬ LÝ (TRONG MAIN PROGRAM)

Mở Program SAPMZDEV006_MASTER_ENTRY và dán đoạn code logic này vào.

```
code Abap
*&-----*
*& PBO & PAI MÀN HÌNH 0100 (SINH VIÊN)
*&-----*
MODULE status_0100 OUTPUT.
  SET PF-STATUS 'GUI_100'. " Nhớ tạo pf-status có nút BACK, EXIT
  SET TITLEBAR 'TIT_100' WITH 'Quản Lý Danh Sách Sinh Viên'.
```

```
" Lần đầu tiên chạy, lấy data từ DB lên
IF it_students IS INITIAL.
    SELECT student_id full_name dob course
    FROM zstudent_dev006
    INTO CORRESPONDING FIELDS OF TABLE it_students.
ENDIF.
```

```
    DESCRIBE TABLE it_students LINES tc_stu-lines.
ENDMODULE.
```

```
MODULE tc_stu_modify INPUT.
    MODIFY it_students FROM wa_student INDEX tc_stu-current_line.
ENDMODULE.
```

```
MODULE user_command_0100 INPUT.
    save_ok = ok_code.
    CLEAR ok_code.
```

```
CASE save_ok.
    WHEN 'BACK' OR 'EXIT'.
        LEAVE PROGRAM.
```

```
    WHEN 'ADD_STU'.
        " BÍ QUYẾT THÊM DÒNG: Thêm 1 dòng rỗng vào Internal Table
        APPEND INITIAL LINE TO it_students.
        " Lưới sẽ tự động cập nhật số dòng do có lệnh DESCRIBE ở PBO
```

```
    WHEN 'SAVE_STU'.
        " Cập nhật dữ liệu từ Internal Table xuống Database
        DATA: lt_db_student TYPE TABLE OF zstudent_dev006,
        ls_db_student TYPE zstudent_dev006.
```

```
LOOP AT it_students INTO wa_student.
```

```
" Validation cơ bản
```

```
IF wa_student-student_id IS INITIAL.
```

```
  MESSAGE 'Mã SV không được để trống!' TYPE 'E'.
```

```
ENDIF.
```

```
MOVE-CORRESPONDING wa_student TO ls_db_student.
```

```
APPEND ls_db_student TO lt_db_student.
```

```
ENDLOOP.
```

```
" Dùng MODIFY để vừa Insert (nếu mới) vừa Update (nếu cũ)
```

```
MODIFY zstudent_dev006 FROM TABLE lt_db_student.
```

```
IF sy-subrc = 0.
```

```
  COMMIT WORK.
```

```
  MESSAGE 'Lưu danh sách Sinh viên thành công!' TYPE 'S'.
```

```
ENDIF.
```

```
WHEN 'DETAIL'.
```

```
  READ TABLE it_students INTO wa_student WITH KEY mark = 'X'.
```

```
  IF sy-subrc = 0.
```

```
    gv_current_student = wa_student-student_id.
```

```
    zstudent_dev006-student_id = gv_current_student. " Hiện thị trên header MH 200
```

```
    CLEAR it_enroll. " Xóa cache cũ
```

```
    CALL SCREEN 0200.
```

```
  ELSE.
```

```
    MESSAGE 'Vui lòng tích chọn 1 Sinh viên!' TYPE 'S' DISPLAY LIKE 'E'.
```

```
  ENDIF.
```

```
ENDCASE.
```

```
ENDMODULE.
```

```
*&-----*
```

```
*& PBO & PAI MÀN HÌNH 0200 (MÔN & ĐIỂM)
```

&-----

MODULE status_0200 OUTPUT.

SET PF-STATUS 'GUI_200'.

SET TITLEBAR 'TIT_200' WITH 'Quản Lý Điểm Sinh Viên'.

IF it_enroll IS INITIAL.

" Lấy danh sách môn đã đăng ký của SV này

SELECT a~enroll_id a~course_id b~course_name a~score

FROM zenroll_dev006 AS a

INNER JOIN zcourse_dev006 AS b ON a~course_id = b~course_id

INTO CORRESPONDING FIELDS OF TABLE it_enroll

WHERE a~student_id = gv_current_student.

ENDIF.

DESCRIBE TABLE it_enroll LINES tc_enr-lines.

ENDMODULE.

MODULE tc_enr_modify INPUT.

" LOGIC LẤY TÊN MÔN TỰ ĐỘNG (Sau khi chọn F4)

IF wa_enroll-course_id IS NOT INITIAL AND wa_enroll-course_name IS INITIAL.

SELECT SINGLE course_name FROM zcourse_dev006

INTO wa_enroll-course_name

WHERE course_id = wa_enroll-course_id.

ENDIF.

MODIFY it_enroll FROM wa_enroll INDEX tc_enr-current_line.

ENDMODULE.

MODULE user_command_0200 INPUT.

save_ok = ok_code.

CLEAR ok_code.

```
CASE save_ok.  
  WHEN 'BACK'.  
    LEAVE TO SCREEN 0100.  
  
  WHEN 'ADD_ENR'.  
    " Thêm 1 dòng trống để đăng ký môn  
    APPEND INITIAL LINE TO it_enroll.  
  
  WHEN 'SAVE_ENR'.  
    DATA: lt_db_enroll TYPE TABLE OF zenroll_dev006,  
           ls_db_enroll TYPE zenroll_dev006,  
           lv_max_id   TYPE numc10.  
  
    " Lấy ID lớn nhất để tạo ID mới  
    SELECT MAX( enroll_id ) FROM zenroll_dev006 INTO lv_max_id.  
  
    LOOP AT it_enroll INTO wa_enroll.  
      IF wa_enroll-course_id IS INITIAL.  
        CONTINUE. " Bỏ qua dòng trống  
      ENDIF.  
  
      MOVE-CORRESPONDING wa_enroll TO ls_db_enroll.  
      ls_db_enroll-student_id = gv_current_student.  
  
      " Nếu là dòng mới thêm (chưa có Enroll ID)  
      IF ls_db_enroll-enroll_id IS INITIAL.  
        lv_max_id = lv_max_id + 1.  
        ls_db_enroll-enroll_id = lv_max_id.  
  
        " Cập nhật lại ID vào màn hình  
        wa_enroll-enroll_id = lv_max_id.  
        MODIFY it_enroll FROM wa_enroll INDEX sy-tabix.
```

```
ENDIF.  
  
APPEND ls_db_enroll TO lt_db_enroll.  
ENDLOOP.  
  
MODIFY zenroll_dev006 FROM TABLE lt_db_enroll.  
IF sy-subrc = 0.  
    COMMIT WORK.  
    MESSAGE 'Đã lưu điểm và môn học thành công!' TYPE 'S'.  
ENDIF.  
ENDCASE.  
ENDMODULE.
```

D. THỬ THÁCH NÂNG CAO (CHALLENGES) CHO BUỔI 15

Challenge 1: Chống Trùng Môn Học (Validation)

- **Vấn đề:** Hiện tại người dùng có thể bấm "Đăng ký môn", nhập môn C++ hai lần cho cùng 1 sinh viên. Khi lưu Database sẽ bị lỗi hoặc dính rác.
- **Yêu cầu:** Trong sự kiện SAVE_ENR, hãy dùng kỹ thuật SORT và DELETE ADJACENT DUPLICATES trên it_enroll theo course_id. Hoặc dùng vòng lặp lồng nhau để kiểm tra xem có 2 dòng nào trùng Mã môn không. Nếu có, báo lỗi và dừng lưu.

Challenge 2: Nút "Xóa Dòng" (Delete Row)

- **Yêu cầu:** Thêm một nút **XÓA (DELETE_ROW)**.
- **Logic:**
 - Người dùng tích vào Checkbox của 1 dòng.
 - Bấm Xóa.
 - Chương trình sẽ thực hiện DELETE it_students WHERE mark = 'X'.

- *Nâng cao:* Xóa luôn dưới Database (DELETE FROM zstudent_dev006 WHERE student_id = ...).

Challenge 3: Cảnh báo "Chưa Lưu" (Unsaved Changes)

- **Yêu cầu:** Nếu người dùng đã gõ thêm 1 dòng mới, hoặc sửa điểm, nhưng vô tình bấm nút **BACK** mà chưa bấm SAVE.
- **Logic:** Hãy tạo một biến cờ DATA: gv_changed TYPE c. Set gv_changed = 'X' trong module tc_enr_modify. Khi bấm BACK, nếu gv_changed = 'X', hiện Popup hỏi *"Bạn có dữ liệu chưa lưu. Xác nhận thoát?"*.

Bài Lab này cung cấp cho sinh viên toàn bộ "vũ khí" để xử lý các nghiệp vụ phức tạp nhất của màn hình ERP truyền thống (Tạo mới, Sửa, Xác thực dữ liệu trên cùng 1 màn hình).

Tham khảo thêm

- ["SAP ABAP" "Table Control" "insert row" OR "add row"](#)
- [SAP Dynpro create new record table control](#)

BUỔI 16: LẬP TRÌNH DIALOG (DYNPRO) - HỆ THỐNG VÀO ĐIỂM

A. KIẾN TRÚC CHƯƠNG TRÌNH

- **Loại chương trình:** Module Pool (Type M).
- **Tên chương trình:** SAPMZDEV006_GRADING
- **T-Code:** ZDEV006_GRADE
- **Cấu trúc Màn hình (Screens):**
 - **Screen 100 (Main):** Nhập Lớp (Có Search Help F4) -> Hiện danh sách Sinh viên của lớp đó.
 - **Screen 200 (Detail):** Hiện chi tiết 1 Sinh viên -> Hiện danh sách các Môn học và Điểm số của SV đó.
 - **Screen 300 (Popup):** Màn hình nhỏ (Modal Dialog) để giáo viên nhập/sửa điểm cho 1 môn -> Lưu vào DB.

B. HƯỚNG DẪN THỰC HÀNH (STEP-BY-STEP)

GIẢI ĐOẠN 1: TẠO KHUNG CHƯƠNG TRÌNH & KHAI BÁO BIẾN

1. Vào T-Code **SE80**, chọn Package của bạn.
2. Chuột phải vào Package -> **Create -> Program**.
3. Tên: **SAPMZDEV006_GRADING**. Đánh dấu tích vào ô **"With TOP Include"**.
4. Nhấn Enter, hệ thống hỏi tạo TOP Include tên MZDEV006_GRADINGTOP -> Chọn Yes.
5. Mở file TOP Include lên và copy đoạn khai báo sau:

code Abap

```
*&-----*
```

```
*& Include MZDEV006_GRADINGTOP (Khai báo dữ liệu toàn cục)
```

&-----

PROGRAM sapmzdev006_grading.

" 1. Khai báo Tables để map trực tiếp với giao diện màn hình
TABLES: zstudent_dev006, zenroll_dev006, zcourse_dev006.

" 2. Biến điều khiển màn hình
DATA: ok_code TYPE sy-ucomm,
 save_ok TYPE sy-ucomm.

DATA: gv_class TYPE zstudent_dev006-course. " Ô nhập tên lớp

" 3. Khai báo dữ liệu cho Table Control 1 (Danh sách Sinh viên)

TYPES: BEGIN OF ty_student,
 mark TYPE c, " Cột Checkbox để chọn dòng
 student_id TYPE zstudent_dev006-student_id,
 full_name TYPE zstudent_dev006-full_name,
 END OF ty_student.

DATA: it_students TYPE TABLE OF ty_student,
 wa_student TYPE ty_student.

" 4. Khai báo dữ liệu cho Table Control 2 (Danh sách Điểm)

TYPES: BEGIN OF ty_enroll,
 mark TYPE c, " Cột Checkbox
 enroll_id TYPE zenroll_dev006-enroll_id,
 course_id TYPE zenroll_dev006-course_id,
 course_name TYPE zcourse_dev006-course_name,
 score TYPE zenroll_dev006-score,
 END OF ty_enroll.

DATA: it_enroll TYPE TABLE OF ty_enroll,
 wa_enroll TYPE ty_enroll.

" 5. Khai báo 2 biến điều khiển Table Control trên màn hình (BẮT BUỘC)

CONTROLS: tc_stu TYPE TABLEVIEW USING SCREEN 100,

tc_enr TYPE TABLEVIEW USING SCREEN 200.

GIẢI ĐOẠN 2: THIẾT KẾ SCREEN 100 (DANH SÁCH SINH VIÊN)

1. Chuột phải vào tên Program (SAPMZDEV006_GRADING) -> **Create** -> **Screen**.
2. Screen Number: **0100**. Description: Student List. Save.
3. Chuyển sang tab **Element List**, tại ô OK_CODE, gõ chữ **OK_CODE**.
4. Nhấn nút **Layout** (Cái thước kẻ) để mở Screen Painter.

Vẽ Giao diện Screen 100:

- Tạo 1 **Text Field**: Text là Nhập Lớp/Khóa:.
- Tạo 1 **Input/Output Field**: Name (Tên biến) phải là **GV_CLASS**.
- Tạo 1 **Pushbutton**: Name là BTN_SEARCH, Text là Tìm Kiếm, **FctCode** là SEARCH.
- Tạo 1 **Pushbutton**: Name là BTN_DETAIL, Text là Xem Chi Tiết, **FctCode** là DETAIL.
- Tạo 1 **Table Control** (Nút có hình cái bảng kẻ ô bên thanh công cụ trái). Kéo thả ra màn hình.
 - Double-click vào cái bảng vừa vẽ. Đặt tên (Name) là: **TC_STU**.
 - Tích vào ô **w/SelColumn** (With Selection Column), điền tên biến là **WA_STUDENT-MARK**.
 - Kéo 2 ô Text thả lên thanh tiêu đề cột để đặt tên: Mã SV, Họ Tên.
 - Kéo 2 ô Input/Output thả vào phần ruột của 2 cột. Đặt tên lần lượt là: **WA_STUDENT-STUDENT_ID** và **WA_STUDENT-FULL_NAME**.
 - Đổi thuộc tính 2 ô ruột này thành **Output Only** (Chỉ xem, không cho nhập).

Viết Flow Logic cho Screen 100 (Chuyển sang tab Flow Logic):

```

code Abap
PROCESS BEFORE OUTPUT.
  MODULE status_0100.
  " Vòng lặp đẩy dữ liệu từ Internal Table ra màn hình
  LOOP AT it_students INTO wa_student WITH CONTROL tc_stu CURSOR tc_stu-current_line.
  ENDLOOP.

PROCESS AFTER INPUT.
  " Vòng lặp nhận dữ liệu (checkbox) từ màn hình vào Internal Table
  LOOP AT it_students.
    MODULE tc_stu_modify ON REQUEST.
  ENDLOOP.

  MODULE user_command_0100.

  " Cấu hình Search Help (F4) cho ô Lớp
PROCESS ON VALUE-REQUEST.
  FIELD gv_class MODULE f4_class_help.

```

GIAI ĐOẠN 3: THIẾT KẾ SCREEN 200 (CHI TIẾT MÔN & ĐIỂM)

1. Tạo Screen **0200**. Description: Student Details. Đặt OK_CODE là OK_CODE.
2. Mở **Layout**.

Vẽ Giao diện Screen 200:

- Kéo 2 ô Text: Mã SV: và Họ Tên:.
- Kéo 2 ô Input (Lấy từ Dictionary F6 -> Bảng ZSTUDENT_DEV006): Kéo STUDENT_ID và FULL_NAME ra. Set chúng thành **Output Only**.
- Tạo 1 **Pushbutton**: Name là BTN_GRADE, Text là Nhập Điểm, **FctCode** là GRADE.

- Tạo 1 **Pushbutton**: Name là BTN_BACK, Text là Quay Lại, **FctCode** là BACK.
- Tạo 1 **Table Control**. Name: TC_ENR.
 - SelColumn: **WA_ENROLL-MARK**.
 - Các cột Text Header: Mã Đăng Ký, Môn Học, Tên Môn, Điểm.
 - Các ô ruột (Output Only): WA_ENROLL-ENROLL_ID, WA_ENROLL-COURSE_ID, WA_ENROLL-COURSE_NAME, WA_ENROLL-SCORE.

Viết Flow Logic cho Screen 200:

```
code Abap
PROCESS BEFORE OUTPUT.
  MODULE status_0200.
  LOOP AT it_enroll INTO wa_enroll WITH CONTROL tc_enr CURSOR tc_enr-current_line.
  ENDLOOP.

PROCESS AFTER INPUT.
  LOOP AT it_enroll.
    MODULE tc_enr_modify ON REQUEST.
  ENDLOOP.
  MODULE user_command_0200.
```

GIAI ĐOẠN 4: THIẾT KẾ SCREEN 300 (POPUP NHẬP ĐIỂM)

1. Tạo Screen **0300**. Description: Enter Score.
2. **Tab Attributes**: Đổi *Screen type* từ Normal thành **Modal Dialog Box**. OK_CODE là OK_CODE.
3. Mở **Layout**.

Vẽ Giao diện Screen 300 (Nhỏ gọn):

- Text: Mã Môn: -> Kéo biến Dictionary ZENROLL_DEV006-COURSE_ID (Output Only).

- Text: Tên Môn: -> Kéo biến Dictionary ZCOURSE_DEV006-COURSE_NAME (Output Only).
- Text: Điểm Mới: -> Kéo biến Dictionary **ZENROLL_DEV006-SCORE** (Bỏ Output Only để cho phép nhập liệu).
- Pushbutton 1: Text Lưu Điểm, **FctCode:** SAVE_SCORE.
- Pushbutton 2: Text Hủy, **FctCode:** CANCEL.

Viết Flow Logic cho Screen 300:

code Abap

downloadcontent_copy

expand_less

PROCESS BEFORE OUTPUT.

MODULE status_0300.

PROCESS AFTER INPUT.

MODULE user_command_0300.

GIẢI ĐOẠN 5: VIẾT ABAP CODE XỬ LÝ (MODULES)

Nháy đúp vào các tên MODULE trong Flow Logic của cả 3 màn hình để hệ thống tự tạo form code trong Main Program (hoặc tạo Include riêng).

Dưới đây là toàn bộ code xử lý logic:

code Abap

&-----

*& XỬ LÝ SEARCH HELP (F4) CHO Ô LỚP

&-----

```
MODULE f4_class_help INPUT.
```

```
DATA: lt_class TYPE TABLE OF zstudent_dev006,  
      lt_return TYPE TABLE OF ddshretval.
```

```
" Tìm các lớp hiện có trong hệ thống
```

```
SELECT DISTINCT course FROM zstudent_dev006 INTO CORRESPONDING FIELDS OF TABLE lt_class.
```

```
" Hiện thị Popup Search Help
```

```
CALL FUNCTION 'F4IF_INT_TABLE_VALUE_REQUEST'
```

```
EXPORTING
```

```
retfield      = 'COURSE'
```

```
dynpprog      = sy-repid
```

```
dynpnr        = sy-dynnr
```

```
dynprofield   = 'GV_CLASS'
```

```
value_org     = 'S'
```

```
TABLES
```

```
value_tab     = lt_class
```

```
return_tab    = lt_return.
```

```
ENDMODULE.
```

```
*&-----*
```

```
*& PBO & PAI MÀN HÌNH 100
```

```
*&-----*
```

```
MODULE status_0100 OUTPUT.
```

```
SET PF-STATUS 'GUI_100'. " Nháy đúp tạo GUI, thêm nút BACK, EXIT, CANCEL
```

```
SET TITLEBAR 'TIT_100' WITH 'Quản Lý Lớp Học'.
```

```
" Cập nhật số dòng cho Table Control
```

```
DESCRIBE TABLE it_students LINES tc_stu-lines.
```

```
ENDMODULE.
```

```
MODULE tc_stu_modify INPUT.
```

```
" Khi tick chọn checkbox, update lại Internal Table
MODIFY it_students FROM wa_student INDEX tc_stu-current_line.
ENDMODULE.
```

```
MODULE user_command_0100 INPUT.
```

```
save_ok = ok_code.
```

```
CLEAR ok_code.
```

```
CASE save_ok.
```

```
WHEN 'BACK' OR 'EXIT' OR 'CANCEL'.
```

```
LEAVE PROGRAM.
```

```
WHEN 'SEARCH'.
```

```
" Lấy danh sách SV theo lớp
```

```
SELECT student_id full_name
```

```
FROM zstudent_dev006
```

```
INTO CORRESPONDING FIELDS OF TABLE it_students
```

```
WHERE course = gv_class.
```

```
IF sy-subrc <> 0.
```

```
MESSAGE 'Không tìm thấy sinh viên lớp này!' TYPE 'S' DISPLAY LIKE 'E'.
```

```
ENDIF.
```

```
WHEN 'DETAIL'.
```

```
" Tìm sinh viên được tick chọn (mark = 'X')
```

```
READ TABLE it_students INTO wa_student WITH KEY mark = 'X'.
```

```
IF sy-subrc = 0.
```

```
" Đồ dữ liệu Header
```

```
zstudent_dev006-student_id = wa_student-student_id.
```

```
zstudent_dev006-full_name = wa_student-full_name.
```

```
" Lấy danh sách Môn & Điểm của SV đó
```

```
SELECT a~enroll_id, a~course_id, b~course_name, a~score
```

```
FROM zenroll_dev006 AS a
INNER JOIN zcourse_dev006 AS b ON a~course_id = b~course_id
INTO CORRESPONDING FIELDS OF TABLE @it_enroll
WHERE a~student_id = @wa_student-student_id.
```

```
CALL SCREEN 200. " Mở màn hình 200
ELSE.
MESSAGE 'Vui lòng chọn 1 sinh viên!' TYPE 'S' DISPLAY LIKE 'E'.
ENDIF.
ENDCASE.
ENDMODULE.
```

```
*&-----*
*& PBO & PAI MÀN HÌNH 200
*&-----*
```

```
MODULE status_0200 OUTPUT.
SET PF-STATUS 'GUI_200'. " Tạo GUI, thêm nút BACK
SET TITLEBAR 'TIT_200' WITH 'Chi Tiết Sinh Viên'.
DESCRIBE TABLE it_enroll LINES tc_enr-lines.
ENDMODULE.
```

```
MODULE tc_enr_modify INPUT.
MODIFY it_enroll FROM wa_enroll INDEX tc_enr-current_line.
ENDMODULE.
```

```
MODULE user_command_0200 INPUT.
save_ok = ok_code.
CLEAR ok_code.
```

```
CASE save_ok.
WHEN 'BACK'.
LEAVE TO SCREEN 100.
```

```

WHEN 'GRADE'.
    " Tìm môn học được tick chọn
    READ TABLE it_enroll INTO wa_enroll WITH KEY mark = 'X'.
    IF sy-subrc = 0.
        " Nạp dữ liệu vào các biến trên màn hình 300
        zenroll_dev006-enroll_id = wa_enroll-enroll_id.
        zenroll_dev006-course_id = wa_enroll-course_id.
        zcourse_dev006-course_name = wa_enroll-course_name.
        zenroll_dev006-score = wa_enroll-score.

        " Gọi Popup
        CALL SCREEN 300 STARTING AT 10 10 ENDING AT 50 20.
    ELSE.
        MESSAGE 'Vui lòng chọn 1 môn học để nhập điểm!' TYPE 'S' DISPLAY LIKE 'E'.
    ENDIF.
ENDCASE.
ENDMODULE.

*&-----*
*& PBO & PAI MÀN HÌNH 300 (POPUP)
*&-----*
MODULE status_0300 OUTPUT.
    SET TITLEBAR 'TIT_300' WITH 'Nhập Điểm'.
ENDMODULE.

MODULE user_command_0300 INPUT.
    save_ok = ok_code.
    CLEAR ok_code.

    CASE save_ok.
        WHEN 'CANCEL'.

```

```

LEAVE TO SCREEN 200. " Đóng popup

WHEN 'SAVE_SCORE'.
  " Validate điểm
  IF zenroll_dev006-score < 0 OR zenroll_dev006-score > 10.
    MESSAGE 'Điểm phải từ 0 đến 10!' TYPE 'E'.
  ENDIF.

  " Update vào Database
  UPDATE zenroll_dev006
    SET score = zenroll_dev006-score
    WHERE enroll_id = zenroll_dev006-enroll_id.

  IF sy-subrc = 0.
    COMMIT WORK.
    MESSAGE 'Cập nhật điểm thành công!' TYPE 'S'.

    " Cập nhật lại Internal Table để màn hình 200 hiển thị điểm mới
    wa_enroll-score = zenroll_dev006-score.
    wa_enroll-mark = ''. " Bỏ tick
    MODIFY TABLE it_enroll FROM wa_enroll.

    LEAVE TO SCREEN 200. " Đóng popup
  ELSE.
    MESSAGE 'Lỗi hệ thống, không thể lưu!' TYPE 'E'.
  ENDIF.
ENDCASE.
ENDMODULE.

```

GIAI ĐOẠN 6: TẠO T-CODE & KIỂM THỬ

1. Chuột phải vào Program -> **Create -> Transaction.**

2. Tên T-Code: ZDEV006_GRADE.
3. Start Object: **Program and screen (dialog transaction)**.
4. Nhập Program: SAPMZDEV006_GRADING | Screen: 0100.
5. Save & Activate toàn bộ.

Kịch bản Kiểm thử cho Sinh viên (Test Case):

- **Gõ T-Code ZDEV006_GRADE.**
- Tại ô "Nhập Lớp", nhấn **F4**. Một popup hiện ra danh sách các lớp (ví dụ: K18). Chọn 1 lớp.
- Nhấn **Tìm Kiếm**. Bảng bên dưới hiện ra danh sách sinh viên.
- Tích vào ô vuông ở đầu dòng của SV "Nguyen Van A". Nhấn **Xem Chi Tiết**.
- Màn hình 200 hiện ra. Chứa thông tin SV và danh sách các môn SV đó đã đăng ký.
- Tích vào ô vuông của môn "ABAP". Nhấn **Nhập Điểm**.
- Một Popup (Screen 300) hiện ra. Nhập điểm mới (Ví dụ: 9.5). Nhấn **Lưu Điểm**.
- Popup đóng lại, bảng ở màn hình 200 tự động cập nhật số 9.5.

Lời khuyên:

Table Control trong Dynpro rất nhạy cảm với tên biến. Hãy dặn sinh viên kiểm tra thật kỹ tên Name của ô nhập liệu trên Screen Painter phải **giống 100%** với tên khai báo trong bảng (Ví dụ: WA_ENROLL-SCORE). Nếu gõ sai 1 chữ, chương trình sẽ Dump ngay lập tức!

BUỔI 17: SMART FORMS - THIẾT KẾ MẪU IN BẢNG ĐIỂM

A. THÔNG TIN & QUY TẮC

- **Transaction Codes:**
 - SMARTFORMS (Thiết kế Form)
 - SMARTSTYLES (Định dạng chữ/Font)
 - SE78 (Upload hình ảnh/Logo)
- **Tên Smart Form:** ZSF_TRANSCRIPT_DEV006
- **Tên Driver Program:** ZDEV006_PRINT_TRANSCRIPT
- **Chủ đề:** In Phiếu Báo Điểm cho từng Sinh viên (Dạng PDF/In giấy).

● B. REAL-WORLD TIPS (TƯ DUY CHUYÊN GIA)

1. Sự tách biệt (Separation of Logic & Layout):

- **Driver Program (ABAP):** Làm nhiệm vụ chui xuống Database (SELECT), xử lý số liệu, tính toán phức tạp. Sau đó "đóng gói" gửi đi.
- **Smart Form (Layout):** Chỉ làm nhiệm vụ nhận "gói hàng" đó và vẽ ra tờ giấy A4 cho đẹp. Tuyệt đối hạn chế viết code SELECT Database bên trong Smart Form (Dù nó cho phép).

2. Function Module "Biến hình":

- Khi bạn Activate một Smart Form, SAP sẽ tự động sinh ra một Function Module (FM) có tên rất lạ (VD: /1BCDWB/SF00000123).
- Tên này sẽ thay đổi khi bạn chuyển code từ máy Dev sang máy Test. Do đó, KHÔNG BAO GIỜ gọi cứng (hardcode) tên FM này trong ABAP, mà phải dùng hàm SSF_FUNCTION_MODULE_NAME để tìm tên động.

3. **Cú pháp in biến:** Để in giá trị của một biến lên giấy, ta phải kẹp nó trong cặp dấu và (&). Ví dụ: &GV_NAME&.

C. CHUẨN BỊ (TẠO DDIC TYPE TRONG SE11) - 5 Phút

Vì Smart Form yêu cầu các bảng truyền vào phải có kiểu dữ liệu chuẩn từ Dictionary.

1. Vào **SE11**, tạo một **Structure** tên: ZST_GRADE_DEV006.
 - Thêm 3 trường: COURSE_ID (Type CHAR8), COURSE_NAME (Type CHAR40), SCORE (Type P Decimals 2). Activate.
2. Vào **SE11**, tạo một **Table Type** tên: ZTT_GRADE_DEV006.
 - Ô Line Type điền: ZST_GRADE_DEV006. Activate.

D. NỘI DUNG THỰC HÀNH (STEP-BY-STEP)

GAIAI ĐOẠN 1: THIẾT KẾ SMART FORM (GIAO DIỆN IN)

Bước 1: Khởi tạo Form

1. Vào T-Code **SMARTFORMS**.
2. Nhập tên Form: ZSF_TRANSCRIPT_DEV006. Nhấn **Create**.
3. Cửa sổ Form Builder hiện ra. Nhìn sang cột bên trái (Cây thư mục).

Bước 2: Định nghĩa "Cửa giao nhận" (Form Interface)

1. Double-click vào mục **Form Interface** (Nằm dưới nút Global Settings).
2. Chuyển sang tab **Import** (Dữ liệu nhận vào 1 biến đơn):
 - Nhập Parameter Name: IS_STUDENT | Type: ZSTUDENT_DEV006.
3. Chuyển sang tab **Tables** (Dữ liệu nhận vào nhiều dòng):

- Nhập Parameter Name: IT_GRADES | Type: ZTT_GRADE_DEV006.

Bước 3: Khai báo biến nội bộ (Global Definitions)

1. Double-click vào **Global Definitions**.
2. Tại tab *Global Data*, thêm 1 biến để dùng trong vòng lặp lưới:
 - Variable Name: WA_GRADE | Type: ZST_GRADE_DEV006.

Bước 4: Vẽ Layout (Cửa sổ & Văn bản)

Bật nút *Form Painter (F5)* trên thanh công cụ để xem tờ giấy A4 trực quan.

1. Tạo Header (Secondary Window):

- Chuột phải vào Pages and Windows -> %PAGE1 -> **Create -> Window**.
- Đổi tên window vừa tạo thành WIN_HEADER. (Kéo thả nó lên đầu tờ giấy A4).
- Chuột phải vào WIN_HEADER -> **Create -> Text**.
- Click vào nút **Text Editor** (Kế bên hình cây bút) ở màn hình bên phải.
- Gõ nội dung:
PHIẾU BÁO ĐIỂM SINH VIÊN
Mã SV: &IS_STUDENT-STUDENT_ID&
Họ tên: &IS_STUDENT-FULL_NAME&
(Dấu & báo cho SAP biết đây là biến, không phải chữ text thường).
- Nhấn nút Back (F3) để lưu text.

2. Tạo Bảng Điểm (Main Window):

- Chuột phải vào MAIN Window -> **Create -> Table**.
- Click vào tab **Table**. Nhấn nút *Table Painter* (Biểu tượng cái bút chì vẽ bảng). Vẽ 3 cột (Mã Môn, Tên Môn, Điểm).
- Chuyển sang tab **Data**. Đây là nơi khai báo vòng lặp:

- Đánh dấu check ô *Loop*.
- Internal table: IT_GRADES | INTO: WA_GRADE.
- **Vẽ Header của bảng (Tiêu đề cột):**
 - Mở rộng Table ra, chuột phải vào **Header** -> Create -> Table Line. Chọn Line Type là %LTYPE1 (Dòng bạn vừa vẽ).
 - Nó sẽ sinh ra 3 ô (Cell). Chuột phải vào từng Cell -> Create -> Text. Gõ lần lượt: Mã Môn, Tên Môn, Điểm Số.
- **Vẽ Dữ liệu của bảng (Main Area):**
 - Chuột phải vào **Main Area** -> Create -> Table Line. Chọn %LTYPE1.
 - Sinh ra 3 ô Cell. Chuột phải tạo Text cho từng ô. Gõ biến vào:
 - Ô 1: &WA_GRADE-COURSE_ID&
 - Ô 2: &WA_GRADE-COURSE_NAME&
 - Ô 3: &WA_GRADE-SCORE&

Bước 5: Kích hoạt Form

- Nhấn nút **Save** và nút **Activate** (Cây đũa thần).
- Nhấn nút **Test (F8)**. Nó sẽ hiện ra màn hình Function Module (Đừng chạy ở đây, ta sẽ chạy bằng Driver Program).

GIAI ĐOẠN 2: VIẾT DRIVER PROGRAM (ABAP)

Mục đích: Lấy dữ liệu từ bảng ZSTUDENT, ZENROLL, truyền vào cái Form vừa tạo để nó in ra.

1. Vào SE80, tạo Program ZDEV006_PRINT_TRANSCRIPT.
2. Copy đoạn code sau:

```
codeAbap
*&-----*
*& Report ZDEV006_PRINT_TRANSCRIPT
```

```

*&-----*
REPORT zdev006_print_transcript.

*--- 1. INPUT ---*
PARAMETERS: p_id TYPE zstudent_dev006-student_id OBLIGATORY.

*--- 2. VARIABLES ---*
DATA: ls_student TYPE zstudent_dev006,
      lt_grades TYPE ztt_grade_dev006,
      ls_grade TYPE zst_grade_dev006.

DATA: lv_fm_name TYPE rs38l_fnam. " Chứa tên Function Module của Smart Form

START-OF-SELECTION.

" --- 3. GET DATA TỪ DATABASE ---
" Lấy thông tin sinh viên
SELECT SINGLE * FROM zstudent_dev006 INTO ls_student WHERE student_id = p_id.
IF sy-subrc <> 0.
  MESSAGE 'Không tìm thấy sinh viên!' TYPE 'I'.
  RETURN.
ENDIF.

" Lấy danh sách điểm
SELECT a~course_id, b~course_name, a~score
FROM zenroll_dev006 AS a
INNER JOIN zcourse_dev006 AS b ON a~course_id = b~course_id
INTO TABLE @lt_grades
WHERE a~student_id = @p_id.

" --- 4. GỌI SMART FORM ---

```

" Bước 4.1: Đi tìm tên Function Module do Smart Form tự sinh ra

```
CALL FUNCTION 'SSF_FUNCTION_MODULE_NAME'
```

```
EXPORTING
```

```
  formname      = 'ZSF_TRANSCRIPT_DEV006' " Tên Smart Form của bạn
```

```
IMPORTING
```

```
  fm_name       = lv_fm_name           " Tên FM hệ thống trả về
```

```
EXCEPTIONS
```

```
  no_form       = 1
```

```
  no_function_module = 2
```

```
  OTHERS       = 3.
```

```
IF sy-subrc <> 0.
```

```
  MESSAGE 'Lỗi không tìm thấy Form.' TYPE 'E'.
```

```
ENDIF.
```

" Bước 4.2: Truyền dữ liệu vào FM đó để in

```
CALL FUNCTION lv_fm_name
```

```
EXPORTING
```

```
  is_student    = ls_student " Truyền Structure SV (Exporting của ABAP mapping với Importing của Form)
```

```
TABLES
```

```
  it_grades     = lt_grades " Truyền Bảng Điểm
```

```
EXCEPTIONS
```

```
  formatting_error = 1
```

```
  internal_error  = 2
```

```
  send_error      = 3
```

```
  user_canceled  = 4
```

```
  OTHERS         = 5.
```

```
IF sy-subrc <> 0.
```

```
  MESSAGE 'Lỗi trong quá trình in form.' TYPE 'E'.
```

```
ENDIF.
```

KIỂM THỬ:

1. Chạy Program bằng F8.
2. Nhập mã SE... của 1 sinh viên có điểm. Nhấn F8.
3. Một Popup hiện ra chọn máy in. Tại ô *Output Device*, gõ **LP01** (Máy in tiêu chuẩn của SAP).
4. Nhấn nút **Print Preview**.
5. **BÙM!** Bạn sẽ thấy phiếu báo điểm định dạng A4 hiện ra cực kỳ chuyên nghiệp. Từ đây có thể lưu thành file PDF.

E. THỬ THÁCH NÂNG CAO (4 CHALLENGES)

Để sinh viên giỏi thực sự làm chủ Smart Form, hãy giao các yêu cầu sau để trang trí Form. Sinh viên mở lại T-Code SMARTFORMS để làm.

Challenge 1: Thêm Logo Trường học (Graphic Node)

- **Yêu cầu:** Đưa logo FPT University (hoặc 1 ảnh bất kỳ) vào góc trên bên trái của Phiếu Báo Điểm.
- **Gợi ý giải:**
 1. Mở T-Code **SE78**. Import một ảnh Bitmap (.BMP) từ máy tính vào hệ thống SAP (Ví dụ đặt tên là ZLOGO_FPT).
 2. Vào Smart Form, chuột phải vào tờ giấy -> **Create -> Graphic**.
 3. Điền tên ZLOGO_FPT vào ô Name. Set độ phân giải. Kéo ô Graphic đó vào vị trí muốn hiển thị.

Challenge 2: Định dạng Chữ (Smart Styles)

- **Vấn đề:** Chữ in ra đang đen trắng, bằng nhau, nhìn rất phèn.
- **Yêu cầu:** Chữ "PHIẾU BÁO ĐIỂM" phải to (Size 20), in Đậm (Bold), Căn giữa.
- **Gợi ý giải:**
 1. Mở T-Code **SMARTSTYLES**. Tạo một Style mới: ZSTYLE_DEV006.

2. Tạo một Paragraph Format mới tên H1. Chọn Font Size 20, Bold, Alignment: Centered. Activate Style.
3. Quay lại Smart Form. Ở tab *Form Attributes* -> *Output Options*, gán Style ZSTYLE_DEV006 vào.
4. Vào chỗ sửa Text, gán format H1 cho dòng chữ tiêu đề.

Challenge 3: Viết Logic tính Tổng Điểm (Program Lines)

- **Yêu cầu:** Dưới cùng của bảng điểm, hiển thị thêm 1 dòng: Tổng số môn đã học: XXX.
- **Gợi ý giải:**
 1. Tại Global Definitions, tạo 1 biến đếm: GV_TOTAL_COURSES Type I.
 2. Trong cây thư mục, trước khi in Header của Bảng, tạo một node **Program Lines** (Dòng mã ABAP).
 3. Đưa IT_GRADES vào Input, GV_TOTAL_COURSES vào Output.
 4. Viết code ABAP ngay trong Smart Form: DESCRIBE TABLE it_grades LINES gv_total_courses.
 5. Sau đó vẽ 1 Text dưới bảng in ra: &GV_TOTAL_COURSES&.

Challenge 4: Ẩn hiện nội dung theo Điều kiện (Conditions)

- **Yêu cầu:** Nếu sinh viên có môn nào điểm Dưới 5.0, hãy in dòng chữ cảnh báo "Cần học cải thiện môn này!" màu đỏ ngay dưới điểm môn đó.
- **Gợi ý giải:**
 1. Tại Main Area của Table, tạo 1 Text mới chứa câu Cảnh báo.
 2. Chuyển sang tab **Conditions** (Điều kiện) của cái Text đó.
 3. Thiết lập luật: WA_GRADE-SCORE < 5.0.
 4. (Text này sẽ chỉ được in ra tờ giấy nếu điều kiện đúng tại vòng lặp đó).

BUỔI 18: TÍCH HỢP API NGOÀI - GỬI EMAIL TỰ ĐỘNG VỚI SENDGRID

Trong các hệ thống SAP truyền thống, việc gửi email thường thông qua cấu hình Basis (T-Code SCOT / SOST với máy chủ SMTP nội bộ). Tuy nhiên, trong kỷ nguyên Cloud hiện nay, tích hợp với các API bên thứ 3 chuyên gửi email như **SendGrid**, **Mailgun**, hay **AWS SES** là chuẩn mực vì tính ổn định, tốc độ, khả năng chống spam và dễ dàng thiết kế template đẹp mắt.

A. THÔNG TIN & QUY TẮC

- **Công cụ:** SE38 hoặc Eclipse ADT.
- **Package:** ZSPR26_SE1876_[AccountID]
- **Tên chương trình:** Z[AccountID]_LAB15_SENDGRID
- **Chủ đề:** Viết chương trình ABAP gọi REST API của SendGrid để gửi thông báo điểm học kỳ cho Sinh viên.

B. REAL-WORLD TIPS (TƯ DUY CHUYÊN GIA)

1. Tại sao dùng API thay vì SAP SCOT?

- Cấu hình SCOT cần quyền Basis admin, rất phức tạp.
- API (SendGrid) cung cấp khả năng theo dõi email (Ai đã mở? Ai đã click link? Email có bị rớt vào mục Spam không?). Đây là những tính năng hệ thống lõi SAP không làm tốt bằng.

2. Khái niệm HTTP POST & JSON:

- Để gửi email qua API, SAP đóng vai trò là **Client** (Người gửi yêu cầu). Nó gửi một gói tin HTTP POST chứa dữ liệu dạng JSON đến máy chủ SendGrid.

3. Bảo mật API Key:

- Trong bài Lab, ta gán cứng (Hardcode) API Key vào code cho nhanh. Nhưng ở dự án thật, API Key phải được lưu trong các bảng bảo mật an toàn (Secure Store) hoặc cấu hình trong SM59.

4. Lỗi SSL Certificate (Lỗi kinh điển):

- Lập trình viên ABAP hay khóc ròng vì lỗi HTTPIO_ERROR. 99% nguyên nhân là do SAP Server chưa cài chứng chỉ bảo mật (SSL Certificate) của SendGrid trong T-Code STRUST.

C. CHUẨN BỊ (GIẢNG VIÊN / SINH VIÊN)

1. **Tài khoản SendGrid:** Sinh viên tạo 1 tài khoản SendGrid miễn phí (hoặc Giảng viên cung cấp chung 1 API Key cho cả lớp để tiết kiệm thời gian).
2. **API Key:** Lấy chuỗi API Key (Ví dụ: SG.xxxxxxxxxxxxxxx).
3. **Verified Sender:** Email gửi đi (Từ ai?) bắt buộc phải được xác thực trên SendGrid (Ví dụ: admin@fpt.edu.vn). Trong bài Lab, dùng email bạn đã đăng ký SendGrid làm email gửi.

D. HƯỚNG DẪN XỬ LÝ LỖI SSL CERTIFICATE VỚI API NGOÀI (STRUST)

GIẢI ĐOẠN 1: TẢI CHỨNG CHỈ BẢO MẬT (SSL) TỪ SENDGRID

Chúng ta không cần vào tài khoản SendGrid để tải cái này. Chúng ta dùng chính **Trình duyệt Web** (Google Chrome hoặc Microsoft Edge) để lấy "Căn cước" của SendGrid.

Bước 1: Truy cập URL của API

1. Mở Google Chrome hoặc Edge.
2. Gõ chính xác đường dẫn API của SendGrid vào thanh địa chỉ: <https://api.sendgrid.com> và nhấn Enter.
(Trình duyệt có thể hiện thông báo lỗi 404 hoặc {"errors":...}, kệ nó, điều quan trọng là trình duyệt đã thiết lập kết nối bảo mật thành công).

Bước 2: Xem Chứng chỉ (Certificate)

1. Nhìn lên thanh địa chỉ của trình duyệt, nhấp chuột vào **Biểu tượng ổ khóa** (View site information) nằm bên trái chữ https.
2. Chọn dòng **"Connection is secure"** (Kết nối an toàn).

3. Bấm vào biểu tượng cái chứng chỉ có chữ **"Certificate is valid"** (Chứng chỉ hợp lệ). Một cửa sổ Certificate sẽ bật lên.

Bước 3: Lấy chứng chỉ Gốc (Root / Intermediate CA)

Mẹo chuyên gia: Không bao giờ tải chứng chỉ của lớp ngoài cùng (leaf certificate) vì nó hết hạn rất nhanh (vài tháng). Hãy tải chứng chỉ của Tổ chức phát hành (Intermediate hoặc Root) để nó sống được nhiều năm.

1. Trong cửa sổ Certificate, chuyển sang tab **"Certification Path"** (Đường dẫn chứng chỉ).
2. Bạn sẽ thấy một sơ đồ cây gồm 3 cấp. Ví dụ:
 - DigiCert Global Root G2 (Ông nội)
 - DigiCert Global G2 TLS RSA SHA256 2020 CA1 (Cha)
 - *.api.sendgrid.com (Con - Đừng chọn cái này).
3. Hãy click chọn cái ở giữa (Lớp Cha - Intermediate CA).

Bước 4: Xuất file (Export)

1. Sau khi chọn lớp Cha, nhấn nút **"View Certificate"** (Xem chứng chỉ). Một cửa sổ mới lại mở ra.
2. Chuyển sang tab **"Details"** (Chi tiết).
3. Nhấn nút **"Copy to File..."** (Sao chép sang tệp).
4. Cửa sổ Export Wizard hiện ra -> Nhấn **Next**.
5. **[CỰC KỲ QUAN TRỌNG]** Tại bước chọn định dạng, **BẮT BUỘC** chọn: **Base-64 encoded X.509 (.CER)**. (SAP chỉ đọc được định dạng Base64 này).
6. Nhấn Next -> Lưu file ra Desktop với tên: SendGrid_Cert.cer. Nhấn Finish.

GIẢI ĐOẠN 2: CÀI ĐẶT CHỨNG CHỈ VÀO SAP (T-CODE STRUST)

Bây giờ ta đem cái file .cer vừa tải về "nộp" cho ông bảo vệ SAP.

Bước 1: Mở trình quản lý chứng chỉ SAP

1. Vào SAP GUI, gõ T-Code: **STRUST** (Trust Manager).
2. Nhấn nút **Display <-> Change** (Biểu tượng cái kính/cây bút) trên thanh công cụ để chuyển sang chế độ Sửa (Edit mode).

Bước 2: Chọn thư mục lưu trữ (SSL Client)

1. Nhìn sang cột bên trái (cây thư mục).
2. Tìm đến mục **SSL client SSL Client (Anonymous)**. *(Đây là nơi SAP dùng để kết nối với các trang web không yêu cầu đăng nhập bằng chứng chỉ client, trường hợp của SendGrid API Key chính là cái này).*
3. **Nhấp đúp chuột (Double-click)** vào chữ SSL client SSL Client (Anonymous).
4. Nếu hệ thống hỏi "Create?" (Tạo mới) thì nhấn Yes.
5. Nhìn sang nửa màn hình bên phải, bạn sẽ thấy ô "Certificate List" (Danh sách chứng chỉ).

Bước 3: Nhập file vào SAP (Import)

1. Nhìn xuống góc dưới cùng bên trái của màn hình (Khu vực Certificate), nhấn vào biểu tượng **"Import certificate"** (Mũi tên xanh lá chỉ vào tờ giấy).
2. Chọn đường dẫn đến file SendGrid_Cert.cer ngoài Desktop mà bạn vừa tải ở Giai đoạn 1. Nhấn Open (Tích xanh).
3. Thông tin của chứng chỉ SendGrid (DigiCert) sẽ hiện lên ở khu vực dưới cùng.

Bước 4: Thêm vào danh sách tin cậy (Add to Certificate List)

1. Nhấn vào nút **"Add to Certificate List"** (Biểu tượng dấu cộng xanh lá + hoặc nút Add to list ở khu vực dưới).
2. Bạn sẽ thấy tên chứng chỉ DigiCert vừa nhảy lên khung danh sách (Certificate List) ở phía trên.
3. Nhấn nút **Save** (Hình đĩa mềm) trên cùng.

GIAI ĐOẠN 3: KHỞI ĐỘNG LẠI DỊCH VỤ MẠNG (RESTART ICM)

Lỗi sinh viên hay gặp nhất: Cài xong, Save rồi, chạy code vẫn Dump HTTPIO_ERROR. Lý do: SAP lưu cache mạng (ICM). Bạn phải bảo SAP load lại danh sách.

1. Ngay tại màn hình STRUST, nhìn lên thanh Menu trên cùng.
2. Chọn **Environment** -> **ICM Monitor**. (Hoặc ra ngoài gõ T-Code SMICM).
3. Màn hình ICM Monitor hiện ra. Nhìn lên thanh Menu.
4. Chọn **Administration** -> **ICM** -> **Exit Soft** -> **Local**.
(Thao tác này chỉ khởi động lại cổng mạng nội bộ của SAP, mất khoảng 2 giây, không làm sập hệ thống của người khác).
5. Nhấn **Yes** để xác nhận. Chờ 3 giây.
6. Thoát khỏi màn hình này.

GIAI ĐOẠN 4: KIỂM THỬ (TESTING) VÀ LỜI GIẢI

Bây giờ ông bảo vệ SAP đã nhận diện được SendGrid.

1. Quay lại chương trình ABAP ZDEV006_LAB15_SENDGRID của bạn.
2. Chạy lại chương trình (F8).
3. **Kết quả mong đợi:**
 - Chương trình KHÔNG còn bị Dump hay báo lỗi HTTPIO_ERROR.
 - HTTP Status trả về là 202 (Thành công từ SendGrid) hoặc 401/400 (Lỗi do API Key sai, nhưng ít nhất là ĐÃ KẾT NỐI ĐƯỢC ra mạng internet ngoài).

Câu hỏi Thử thách cho Sinh viên:

- **Câu hỏi:** Giả sử công ty bạn đổi hệ thống gửi email từ SendGrid sang api.mailgun.net. Bạn chạy code và lại bị lỗi HTTPIO_ERROR. Bạn sẽ làm gì?

- **Lời giải:** Em sẽ mở trình duyệt, vào <https://api.mailgun.net>, tải chứng chỉ Root/Intermediate của Mailgun (Base64 .cer). Sau đó nhờ Basis Admin hoặc tự vào STRUST, import vào mục SSL Client (Anonymous), nhấn Add to List, Save, và cuối cùng vào SMICM restart lại ICM cache.

Việc hiểu và làm chủ được T-Code STRUST sẽ biến một lập trình viên ABAP bình thường thành một chuyên gia Integration (Tích hợp thực thụ!

E. NỘI DUNG THỰC HÀNH (STEP-BY-STEP)

Kịch bản: Phòng đào tạo muốn nhập email của sinh viên, gõ tiêu đề, nội dung và bấm nút là hệ thống tự gửi email đi.

BƯỚC 1: KHỞI TẠO CHƯƠNG TRÌNH & GIAO DIỆN

1. Vào SE80, tạo chương trình ZDEV006_LAB15_SENDGRID.
2. Tạo màn hình nhập liệu (Selection Screen):

code Abap

```
*&-----*
```

```
*& Report ZDEV006_LAB15_SENDGRID
```

```
*&-----*
```

```
REPORT zdev006_lab15_sendgrid.
```

```
*--- CẤU HÌNH GIAO DIỆN NHẬP LIỆU ---*
```

```
SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE text-001.
```

```
  PARAMETERS: p_to   TYPE string LOWER CASE DEFAULT 'sinhvien_test@gmail.com', " Email người nhận
```

```
               p_sub  TYPE string LOWER CASE DEFAULT 'FPT LMS - Thông báo',    " Tiêu đề
```

```
               p_body TYPE string LOWER CASE DEFAULT 'Điểm thi của bạn đã được cập nhật.' " Nội dung
```

```
SELECTION-SCREEN END OF BLOCK b1.
```

```
*--- KHAI BÁO HẰNG SỐ BẢO MẬT ---*
```

```
" Thay chuỗi bên dưới bằng API Key thật của bạn
```

```
CONSTANTS: gc_api_key TYPE string VALUE 'SG.Day_La_API_Key_Cua_Ban.XXXXXXXXXXXXXXXXXXXXXX',
```

```
gc_api_url TYPE string VALUE 'https://api.sendgrid.com/v3/mail/send',  
gc_sender TYPE string VALUE 'email_da_xac_thuc@domain.com'. " Email gửi của bạn
```

BUỐC 2: XÂY DỰNG CHUỖI JSON PAYLOAD

SendGrid yêu cầu dữ liệu gửi lên phải tuân thủ đúng cấu trúc JSON của họ. Trong ABAP, ta dùng **String Template** (cặp dấu |...|) để dựng chuỗi này.

- Lưu ý cực kỳ quan trọng:* Trong String Template ABAP, dấu ngoặc nhọn { và } dùng để chứa biến. Vì JSON cũng dùng dấu ngoặc nhọn, ta phải dùng dấu gạch chéo ngược \ để "thoát" (Escape) nó: \{ và \}.

code Abap

START-OF-SELECTION.

" 1. Tạo chuỗi JSON theo chuẩn SendGrid

" Lưu ý: Dùng \{ và \} để ABAP hiểu đó là ký tự ngoặc nhọn của JSON, không phải biến.

DATA(lv_json_payload) =

```
| \{ | &&  
| "personalizations": [ | &&  
|   \{ | &&  
|     "to": [ \{ "email": "{ p_to }" \} ] | &&  
|   \} | &&  
| ], | &&  
| "from": \{ "email": "{ gc_sender }" \}, | &&  
| "subject": "{ p_sub }", | &&  
| "content": [ | &&  
|   \{ | &&  
|     "type": "text/plain", | &&  
|     "value": "{ p_body }" | &&  
|   \} | &&  
| ] | &&  
| \} |.
```

BUỐC 3: GỌI HTTP CLIENT ĐỂ GỬI API POST

Đây là kỹ thuật cốt lõi để ABAP giao tiếp với thế giới bên ngoài.

code Abap

" 2. Khởi tạo HTTP Client

DATA: lo_client TYPE REF TO if_http_client.

CALL METHOD cl_http_client=>create_by_url

EXPORTING

url = gc_api_url

IMPORTING

client = lo_client

EXCEPTIONS

argument_not_found = 1

plugin_not_active = 2

internal_error = 3

OTHERS = 4.

IF sy-subrc <> 0.

MESSAGE 'Lỗi khởi tạo HTTP Client' TYPE 'E'.

ENDIF.

" 3. Cấu hình HTTP Request (Method, Headers)

lo_client->request->set_method('POST').

" Khai báo gửi JSON

lo_client->request->set_header_field(name = 'Content-Type'
value = 'application/json').

" Khai báo quyền truy cập (Bearer Token)

lo_client->request->set_header_field(name = 'Authorization'
value = |Bearer { gc_api_key }|).

" Nhúng cục dữ liệu JSON vào Body của Request

lo_client->request->set_cdata(lv_json_payload).

" 4. Gửi Request đi (SEND)

```
lo_client->send(  
    EXCEPTIONS  
    http_communication_failure = 1  
    http_invalid_state         = 2 ).
```

IF sy-subrc <> 0.

MESSAGE 'Lỗi kết nối mạng khi gửi dữ liệu (Check STRUST/VPN)' TYPE 'E'.
ENDIF.

" 5. Nhận kết quả phản hồi từ SendGrid (RECEIVE)

```
lo_client->receive(  
    EXCEPTIONS  
    http_communication_failure = 1  
    http_invalid_state         = 2  
    http_processing_failed     = 3 ).
```

" 6. Xử lý trạng thái HTTP (202 là thành công đối với SendGrid)

DATA: lv_status_code TYPE i,
 lv_status_text TYPE string.

```
lo_client->response->get_status( IMPORTING code = lv_status_code  
                                reason = lv_status_text ).
```

" Lấy chi tiết lỗi (nếu có) từ SendGrid trả về

DATA(lv_response_string) = lo_client->response->get_cdata().

" Đóng kết nối để giải phóng RAM

```
lo_client->close( ).
```

" 7. Hiển thị kết quả cho người dùng

```
IF lv_status_code = 202. " SendGrid dùng mã 202 Accepted cho việc gửi email thành công  
WRITE: / '✅ THÀNH CÔNG: Email đã được SendGrid tiếp nhận và đang gửi đi!'.  
ELSE.  
WRITE: / '❌ THẤT BẠI: Lỗi từ SendGrid.' COLOR 6.  
WRITE: / 'HTTP Status Code:', lv_status_code.  
WRITE: / 'Chi tiết lỗi:', lv_response_string.  
ENDIF.
```

BUƯỚC 4: KIỂM THỬ (TESTING)

1. Nhập địa chỉ email thật của bạn vào ô người nhận (p_to).
2. Nhấn **F8**.
3. Nếu màn hình hiện ✅ THÀNH CÔNG (Status code 202): Mở hòm thư Gmail/Outlook của bạn ra kiểm tra. Email sẽ đến trong vòng vài giây.
4. Nếu báo lỗi: Đọc dòng "Chi tiết lỗi" in ra màn hình. (Ví dụ: Lỗi 401 Unauthorized do API Key sai, hoặc 400 Bad Request do chuỗi JSON bị thiếu ngoặc kép).

E. THỬ THÁCH NÂNG CAO (4 CHALLENGES)

Yêu cầu sinh viên vận dụng tư duy để nâng cấp Tool gửi mail này thành một hệ thống tự động.

Challenge 1: Rich Text (Gửi Email HTML)

- **Vấn đề:** Email hiện tại chỉ có chữ đen trắng, không xuống dòng được.
- **Yêu cầu:** Sửa chuỗi JSON Payload để email gửi đi có định dạng HTML (Ví dụ: Chữ "FPT LMS" in đậm, có màu sắc, có dòng kẻ ngang).
- **Gợi ý:** Đổi "type": "text/plain" thành "type": "text/html". Trong biến p_body hãy nhập mã HTML như Chào bạn
Điểm của bạn là...

Challenge 2: Tự động hóa lấy dữ liệu (Dynamic Data)

- **Yêu cầu:** Giao diện không cho nhập Email/Tiêu đề/Nội dung nữa. Chỉ cho phép nhập **Mã Sinh Viên (p_id)**.
- **Logic:**
 1. SELECT email và tên sinh viên từ bảng ZSTUDENT_DEV006 dựa vào p_id.
 2. SELECT điểm GPA.
 3. Code tự động nối chuỗi tạo nội dung: *"Chào [Tên SV], Điểm trung bình của bạn là [GPA]."*
 4. Gọi API gửi đi.

Challenge 3: Mass Mailing (Gửi hàng loạt - Gửi thư cảnh báo)

- **Kịch bản:** Gửi email cảnh báo cho TOÀN BỘ sinh viên có điểm < 5.0.
- **Yêu cầu:**
 1. Không dùng tham số đầu vào. Khi bấm F8 chương trình tự chạy.
 2. SELECT danh sách tất cả sinh viên bị điểm < 5.0 từ bảng ZENROLL_DEV006.
 3. Đặt đoạn code gọi HTTP Client (từ Bước 2 đến 6) vào trong một vòng lặp LOOP.
 4. *Lưu ý hiệu năng:* Nhớ gọi lo_client->close() ở cuối mỗi vòng lặp để không bị sập bộ nhớ máy chủ SAP. In ra màn hình log danh sách các email gửi thành công/thất bại.

Challenge 4: ABAP JSON Serialization (Lập trình chuẩn mực)

- **Vấn đề:** Việc nối chuỗi JSON bằng String Template `|\{...\}|` rất dễ sai cú pháp (thiếu phẩy, thiếu ngoặc).
- **Yêu cầu:** Không dùng String Template nữa. Hãy dùng thư viện chuẩn `/UI2/CL_JSON` của SAP để chuyển đổi từ ABAP Structure sang JSON.
- **Gợi ý:** Khai báo các TYPES lồng nhau khớp với cấu trúc SendGrid. Điền dữ liệu vào cấu trúc đó, rồi gọi `/ui2/cl_json=>serialize( data = ls_payload )`.

💡 F. LỜI GIẢI THAM KHẢO (SNIPPETS)

Giải Challenge 2: Tự động ghép tên sinh viên vào nội dung

```
code Abap
DATA: ls_student TYPE zstudent_dev006,
      lv_body    TYPE string.

" Lấy thông tin SV (Giả sử bảng ZSTUDENT đã được thêm cột EMAIL)
SELECT SINGLE * FROM zstudent_dev006 INTO ls_student WHERE student_id = p_id.

IF sy-subrc = 0.
  " Tự động sinh nội dung
  lv_body = |Kính gửi sinh viên <b>{ ls_student-full_name }</b>,<br><br>| &&
            |Đây là email thông báo tự động từ hệ thống LMS FPT.<br>| &&
            |Trạng thái học tập của bạn đang được cập nhật.|.

  " Thay p_body bằng lv_body trong chuỗi JSON (Và nhớ đổi type thành text/html)
ELSE.
  MESSAGE 'Không tìm thấy sinh viên!' TYPE 'S' DISPLAY LIKE 'E'.
  RETURN.
ENDIF.
```

Giải Challenge 3: Khung sườn Gửi hàng loạt (Mass Mail)

```
code Abap
" Lấy danh sách SV tụt ngòi
SELECT a~student_id, b~email, a~score
  FROM zenroll_dev006 AS a
 INNER JOIN zstudent_dev006 AS b ON a~student_id = b~student_id
 INTO TABLE @DATA(lt_failed_students)
 WHERE a~score < 5.

WRITE: / '--- BẮT ĐẦU GỬI EMAIL HÀNG LOẠT ---' COLOR 1.
```

```
LOOP AT lt_failed_students INTO DATA(ls_fail).
```

```
" --- 1. Tạo JSON cho TỪNG người ---
```

```
DATA(lv_json) = |\{ "personalizations": [\{"to": [\{"email": "{ ls_fail-email }"\}], "from": \{ "email": "{ gc_sender }" \}, "subject":  
"Cảnh báo học tập", "content":[\{"type": "text/plain", "value": "Điểm của bạn là { ls_fail-score }. Đề nghị lên phòng đào tạo." \}]\}|.
```

```
" --- 2. Mở kết nối, Set Header, Cdata ---
```

```
CALL METHOD cl_http_client=>create_by_url ...
```

```
" ... (như code mẫu) ...
```

```
" --- 3. Send & Receive ---
```

```
lo_client->send( ).
```

```
lo_client->receive( ).
```

```
" --- 4. Ghi Log ---
```

```
lo_client->response->get_status( IMPORTING code = lv_status ).
```

```
IF lv_status = 202.
```

```
  WRITE: / '  Đã gửi email tới:', ls_fail-email.
```

```
ELSE.
```

```
  WRITE: / '  Lỗi gửi tới:', ls_fail-email COLOR 6.
```

```
ENDIF.
```

```
" --- 5. BẮT BUỘC: ĐÓNG KẾT NỐI TRONG VÒNG LẶP ---
```

```
lo_client->close( ).
```

```
" Tùy chọn: Nghỉ 1 giây để không bị API SendGrid khóa vì spam quá nhanh
```

```
WAIT UP TO 1 SECONDS.
```

```
ENDLOOP.
```

TỔNG KẾT:

Qua bài Lab này, sinh viên đã tự tay "nối mạng" hệ thống SAP của mình với thế giới bên ngoài. Nắm vững kỹ năng tạo HTTP Request, cấu hình Header (Authorization Token), và xử lý JSON là những vũ khí tối quan trọng của lập trình viên ABAP thời đại Cloud tích hợp (SAP BTP, CPI).